

© 2025 Robin Bräck
Det här verket är skyddat av upphovsrätt. Se första sidan

NÄTVERK

Av Robin Bräck



Utgåva
2026-1

Licensinformation

© 2025 Robin Bräck. Detta verk, inklusive text och illustrationer, är skyddat enligt lag om upphovsrätt. Illustrationerna i boken är framtagna med hjälp av artificiell intelligens (AI) under författarens redaktionella överseende och utgör en integrerad del av det samlade verket.

Verket tillhandahålls under licensen Creative Commons Erkännande-Ickekommersiell-DelaLika 4.0 Internationell (CC BY-NC-SA 4.0). Detta innebär att du har frihet att dela och anpassa innehållet så länge du anger källan på ett korrekt sätt, inte använder materialet för kommersiella ändamål och sprider eventuella bearbetningar under samma licensvillkor. Fullständiga licensvillkor finns tillgängliga på <https://creativecommons.org/licenses/by-nc-sa/4.0/>.

Index

Innan du börjar läsa.....	4
Terminologi: WLAN vs “Wi-Fi”	4
1 Vad är ett nätverk?.....	4
1.1 Vad är ett nätverk?.....	4
1.2 Nätverkskomponenter och topologier.....	12
1.3 Protokoll och adressering.....	24
1.4 IP: Nätverkets Adressering och Logik.....	25
1.5 Kommunikation i nätverk – Ethernet och WLAN.....	40
2 Planering och design av nätverk.....	55
2.1 Mål och krav.....	55
2.2 VLAN och segmentering.....	76
2.3 Routing.....	93
2.4 DHCP, DNS och NAT.....	111
2.5 Vanliga nätverksangrepp och hur vi försvarar oss.....	128
3 Nätverksadministration.....	148
3.1 Grunder i nätverksadministration.....	148
3.2 Verktyg: ping, traceroute, ipconfig, netstat etc.....	174
3.3 Dokumentation, etikett, felsökning.....	194
3.4 Introduktion till CLI (inkl. Cisco-kommandon).....	213
3.5 Windows Server – Domäner och AD.....	232
3.6 Ubuntu Server och Linux i nätverk.....	253
3.7 MDM och klienthantering.....	272
4 Säkerhet och övervakning.....	287
4.1 Vad är säkerhet.....	287
4.2 Åtkomstkontroll och behörigheter.....	305
4.3 Övervakning och loggning.....	314
4.4 Incidenthantering och backup.....	324
5 Fördjupning.....	336
5.1 Virtualisering – plattformar och gemensamma mönster.....	336
5.2 Fördjupning: Windows Server och GPO.....	350
5.3 Fördjupning: Linux-server och policy-styrning.....	361
5.4 MDM i praktiken.....	370
5.5 Framtiden för nätverk.....	382
Appendix I – Begrepp.....	390
Appendix II - CLI-REFERENS.....	400

Innan du börjar läsa

Terminologi: WLAN vs “Wi-Fi”

I den här boken använder vi **WLAN** som samlingsnamn för trådlösa lokala nät enligt **IEEE 802.11** (radiogränssnitt, SSID, autentisering, kanaler, m.m.). Begreppet **“Wi-Fi”** är egentligen ett **certifieringsprogram och varumärke** från **Wi-Fi Alliance** som garanterar kompatibilitet mellan produkter som implementerar 802.11. Många operativsystem och menyer skriver ändå “Wi-Fi” om trådlöst nätverk, men vi använder konsekvent **WLAN** för själva tekniken för att vara tydliga – inte minst för att undvika förväxling med **VLAN** (virtuella LAN), som är något helt annat.

Som orientering: **Wi-Fi 4/5/6/6E/7** motsvarar **802.11n/ac/ax/ax (6 GHz)/be**. Att en enhet är “Wi-Fi-certifierad” betyder att den klarar interoperabilitetstesterna för vissa 802.11-funktioner, inte att “Wi-Fi” är ett annat protokoll än 802.11.

1 Vad är ett nätverk?

1.1 Vad är ett nätverk?

Ett nätverk är när två eller flera enheter kopplas samman för att kunna dela information eller resurser, till exempel en skrivare, en server eller en internetuppkoppling. Det kan handla om något så enkelt som två datorer i ett hem, eller något så avancerat som ett globalt nätverk av servrar som driver tjänster som Google, YouTube eller Microsoft Teams.

I dagens samhälle är nätverk en grundläggande del av vår infrastruktur – de finns i skolor, hem, företag, sjukhus, fabriker och nästan överallt där teknik används. Vi använder nätverk när vi skickar ett mejl, surfar på webben, jobbar i molntjänster eller spelar spel online.

1.1.2 Nätverkstyper

Ett nätverk kan se ut på många olika sätt beroende på var det används, hur stort det är och vad det ska göra. För att kunna planera, förstå och felsöka nätverk behöver vi känna till de vanligaste nätverkstyperna.

LAN – Local Area Network

Ett LAN är ett lokalt nätverk som kopplar ihop enheter i ett begränsat område, ofta inom en och samma byggnad. Det kan till exempel vara datorer i ett klassrum, skrivare i ett kontor eller en server i skolans teknikrum. LAN är snabba, stabila och har låg fördröjning.

LAN används ofta med switchar och kabeldragning, men kan även använda trådlösa tekniker (WLAN).

Exempel: *I en skola har varje klassrum flera datorer anslutna till ett LAN. Datorerna kommunicerar med en central server som hanterar inloggningar och filer.*

WAN – Wide Area Network

När ett nätverk sträcker sig över stora geografiska avstånd, t.ex. mellan städer, länder eller kontinenter, kallas det ett WAN. Internet är världens största WAN – ett globalt nätverk som kopplar ihop miljoner LAN.

WAN bygger ofta på hyrda fiberförbindelser, satellit, radiolänkar eller mobilnät. Kommunikation över WAN är långsammare och dyrare än inom LAN, men möjliggör fjärrarbete, molntjänster och globala företagssystem.

Exempel: *En bank har kontor i olika länder som är ihopkopplade via ett WAN. När du loggar in på din internetbank är det detta nätverk som gör att du når din kontoinformation, oavsett var servern står.*

MAN – Metropolitan Area Network

MAN är en typ av nätverk som används i en hel stad eller ett större samhälle. Det är vanligast inom offentlig sektor, t.ex. när skolor, vårdcentraler eller myndigheter kopplas samman via ett stadsnät. MAN byggs ofta ut av kommunala eller kommersiella operatörer och kan fungera som bas för WAN-anslutningar.

Exempel: *En kommun har ett eget stadsnät där skolor och äldreboenden är ihopkopplade. All trafik går i hög hastighet genom detta MAN innan det går vidare ut till Internet.*

PAN – Personal Area Network

PAN är det minsta nätverket och används av en enskild person – ofta för att koppla samman mobila enheter. Det kan bestå av din mobil, surfplatta,

smartwatch och trådlösa hörlurar. Kommunikation sker ofta via Bluetooth eller WLAN i ad-hoc-läge.

Exempel: När du delar internet från din mobil till en laptop via en personlig hotspot skapar du ett PAN.

1.1.3 Peer-to-peer vs klient-server

Hur enheter är organiserade i ett nätverk har stor betydelse för hur effektivt och säkert det fungerar. De två vanligaste modellerna är peer-to-peer (P2P) och klient-server.

Peer-to-peer (P2P)

I ett P2P-nätverk är alla datorer jämbördiga. Det finns ingen server, utan varje enhet kan både skicka och ta emot filer direkt från andra datorer. Det här är enkelt att sätta upp och kräver ingen särskild serverprogramvara.

Det är vanligt i små hemmiljöer eller för tillfälliga delningar – som när man kopplar ihop datorer via Bluetooth eller delar ut en mapp i Windows.

Men P2P har nackdelar:

- Det blir snabbt rörigt i större nätverk
- Det är svårt att hantera användarrättigheter
- Det är ofta osäkert, eftersom det saknas central styrning

Tänk dig att varje dator är både chef och anställd – ingen har överblick.

Klient-server

I klient-server-nätverk finns en eller flera servrar som tillhandahåller olika tjänster: inloggning, filhantering, utskrift, e-post, databaser, och så vidare. Alla andra datorer fungerar som klienter – de begär resurser från servern.

Denna modell används i nästan alla professionella nätverk – från skolor och företag till sjukhus och universitet.

Fördelar:

- Central administration (konton, behörigheter, säkerhet)
- Lätt att skapa backup och underhålla systemen

- Bättre prestanda och skalbarhet

Tänk dig en server som en bibliotekarie – klienterna frågar efter resurser, och servern bestämmer vem som får vad.

1.1.4 OSI- och TCP/IP-modellen – en introduktion

När du öppnar en webbläsare och surfar till en hemsida sker flera dolda processer i bakgrunden. För att förstå hur dessa fungerar använder vi modeller som OSI-modellen och TCP/IP-modellen. De hjälper oss att se hur data går från en applikation till en annan – genom kablar, routrar och switchar – och hela vägen till mottagaren.

OSI-modellen

OSI-modellen är en teoretisk referensmodell som delar upp kommunikation i sju lager. Varje lager ansvarar för en specifik funktion i dataöverföringen.

Lager	Namn	Funktion
7	Applikation	Programmet du använder (t.ex. webbläsare)
6	Presentation	Kodning, kryptering, formatering
5	Session	Startar och hanterar sessioner
4	Transport	Säkerställer att data kommer fram korrekt
3	Nätverk	Sköter adressering och routing (IP)
2	Datalänk	Hanterar MAC-adresser, paketering inom LAN
1	Fysiskt	Kablar, signaler, elektriska pulser

OSI är särskilt användbar vid felsökning, eftersom den gör det möjligt att isolera problem. Om nätverkskabeln är urkopplad är det lager 1 (fysiskt) som är felkällan. Om DNS inte fungerar är det lager 7 (applikation).

TCP/IP-modellen

TCP/IP-modellen används praktiskt i dagens Internet och datanätverk. Den är enklare än OSI och innehåller fyra lager:

1. **Applikation** – Appar och protokoll (HTTP, SMTP, DNS)
2. **Transport** – Kontrollerar flöde och felhantering (TCP/UDP)
3. **Internet** – Ansvarar för routing och adresser (IP, ICMP)
4. **Nätverksåtkomst** – All fysisk och elektrisk överföring (Ethernet, WLAN)

TCP/IP är grunden för all modern kommunikation online och är därför extremt viktig att förstå i praktiken.

Kontrollfrågor

1. Vad är ett nätverk?
2. Vad står LAN för, och var används det?
3. Ge ett exempel på när ett PAN används.
4. Vad skiljer peer-to-peer från klient-server?
5. Varför är klient-server säkrare än P2P?
6. Vad heter lagret som hanterar IP-adresser i OSI-modellen?
7. Vad är TCP/IP-modellen främst anpassad för?
8. Nämn två fördelar med att använda servrar.
9. Vilket lager i OSI hanterar fysiska signaler?
10. Vad är den största skillnaden mellan OSI och TCP/IP?

Fördjupningslänkar

Cloudflare (The OSI Model Explained) –

<https://www.cloudflare.com/learning/ddos/glossary/open-systems-interconnection-model-osi/>

GeeksforGeeks (Types of Computer Network) –

<https://www.geeksforgeeks.org/types-of-computer-network/>

Cisco (What is a Network?) –

<https://www.cisco.com/c/en/us/products/security/what-is-a-network.html>

IBM (Networking: A Complete Guide) –

<https://www.ibm.com/cloud/learn/networking-a-complete-guide>

Wikipedia (Internet Protocol Suite / TCP/IP model) –

https://en.wikipedia.org/wiki/TCP/IP_model

Egna anteckningar

1.2 Nätverkskomponenter och topologier

1.2.1 Routrar, switchar, accesspunkter och brandväggar

För att ett nätverk ska fungera behövs rätt komponenter – alltså fysisk utrustning som kopplar samman enheter och styr hur trafiken rör sig. Det här kapitlet ger dig en överblick över de viktigaste enheterna du kommer att arbeta med i ett nätverk.

Router

En router kopplar samman olika nätverk med varandra. Den används ofta för att ansluta ett LAN till ett WAN, till exempel när ett hemnätverk kopplas till Internet. Routern ansvarar för att hitta den bästa vägen för varje datapaket att ta, baserat på IP-adresser.

En router kan vara både en så kallad hårdvarurouter, som är speciellt anpassad för ändamålet. Det kan likväl vara en så kallad mjukvarurouter, ett operativsystem som du kan installera på vilken dator som helst, som agerar router.

I skol- eller företagsnätverk används ofta mer avancerade routrar eller brandväggssystem (som OPNsense eller Cisco ISR), medan hemnätverk ofta använder enklare kombinerade enheter som innehåller router, switch, accesspunkt och brandvägg i samma låda.

Switch

En switch används för att koppla samman flera enheter inom ett lokalt nätverk (LAN). Den tar emot datapaket och skickar dem vidare till rätt mottagare genom att läsa av varje enhets MAC-adress. Switchar bygger upp en MAC-tabell för att hålla reda på vilka enheter som finns kopplade till vilka portar.

En switch arbetar på lager 2 i OSI-modellen (datalänklagret), men det finns även Layer 3-switchar som har vissa routingfunktioner.

Switchar kan vara:

- **Ohanterade (unmanaged)** – enklare, billigare och används i små nätverk.

- **Hanterade (managed)** – ger administratören kontroll över VLAN, hastigheter, portövervakning och mer.

I en modern arkitektur används ofta **Layer 3-switchar** som kombinerar denna snabba lager 2-hantering med förmågan att utföra routing mellan de olika virtuella nätverken. Istället för att skicka all trafik till en extern router kan switchen själv flytta data mellan exempelvis VLAN 10 och VLAN 20 direkt i hårdvaran, vilket minimerar latensen och avlastar nätverkets kärna.

Accesspunkt (WLAN AP)

En accesspunkt gör det möjligt för trådlösa enheter att ansluta till ett LAN genom ett WLAN (Wireless LAN). Accesspunkten tar emot data via radio och skickar vidare den till det trådbundna nätverket – eller tvärtom.

I mindre nätverk är accesspunkten ofta inbyggd i routern. I större installationer, som skolor, används istället fristående accesspunkter som är monterade i tak eller väggar och kopplade till switchar via Ethernet.

Det är viktigt att accesspunkter är rätt placerade för att ge jämn täckning, särskilt i miljöer där många klienter rör sig över stora ytor.

Brandvägg

En brandvägg är en komponent – fysisk eller mjukvarubaserad – som filtrerar trafiken till och från nätverket. Den kan blockera eller tillåta trafik baserat på regler som administratören definierar.

Brandväggar kan arbeta med:

- IP-adresser
- Portnummer
- Protokoll (t.ex. TCP, UDP, ICMP)
- Innehåll (t.ex. webbfilter, antivirus, IDS/IPS)

En brandvägg är ett grundläggande skydd och används både på nätverksnivå (gateway-brandvägg) och direkt i operativsystem (lokal brandvägg).

1.2.2 Nätverkskort, kablar och portar

För att kunna kommunicera i ett nätverk behöver varje enhet ett sätt att ansluta till infrastrukturen – det vill säga ett nätverkskort och en fysisk koppling via kabel eller trådlös förbindelse. Här tittar vi på grunderna.

Nätverkskort (NIC)

Ett nätverkskort (Network Interface Card) är hårdvaran som gör att en dator, server eller annan enhet kan kommunicera över ett nätverk.

Nätverkskort finns i två huvudtyper:

- **Ethernetkort** – används för trådbundna anslutningar via RJ-45 och switch.
- **WLAN-kort** – används för trådlös anslutning till en accesspunkt (WLAN).

Varje nätverkskort har en inbyggd **MAC-adress** – en unik identifierare som används i LAN-kommunikation. MAC-adressen är fast programmerad och används av switchar för att avgöra vart data ska skickas.

I servermiljöer används ofta nätverkskort med flera portar eller stöd för 1/10/40/100 Gbit/s och VLAN-tagging.

Kablar

Kabeln utgör det fysiska mediet som bär den elektriska eller optiska signalen mellan nätverkets olika enheter. Valet av kabeltyp direkt påverkar nätverkets prestanda gällande hastighet, räckvidd och motståndskraft mot yttre störningar. De vanligaste kabeltyperna som används i moderna installationer är:

Twisted Pair (TP)

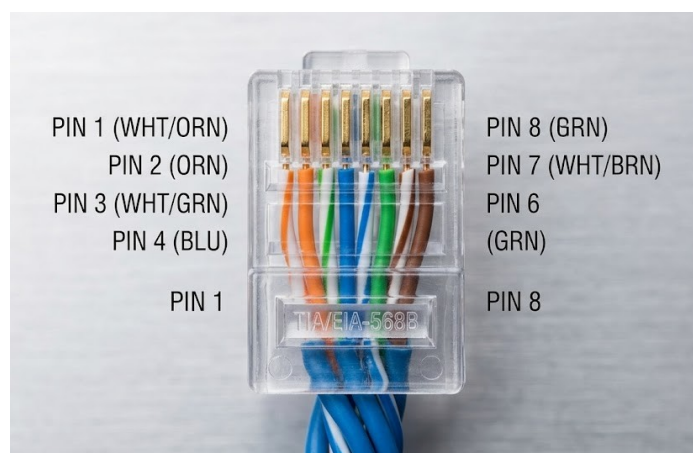
Denna kabeltyp används i nästan alla lokala nätverk (LAN). Den består av åtta ledare organiserade i fyra par, vilka är tvinnade kring varandra för att minimera elektromagnetiska störningar (crosstalk).

Beroende på kraven används kategorier som Cat5e, Cat6 eller Cat6a. För att en TP-kabel ska fungera korrekt krävs en noggrann kontaktering (termination) i en RJ45-kontakt eller i ett uttag. Denna process följer standardiserade färgscheman enligt **TIA/EIA-568A** eller **TIA/EIA-568B**, där den senare är den mest använda standarden i nya installationer.

Vid kontaktering av en kontakt används ett skalverktyg (wire stripper) för att försiktigt avlägsna det yttre höljet utan att skada ledarna, varpå ledarna rätas ut och placeras i rätt ordning:

- Orange-vit
- Orange
- Grön-vit
- Blå
- Blå-vit
- Grön
- Brun-vit
- Brun.

Efter att ledarna klippts till rätt längd och förts in i kontakten används ett pressverktyg (crimping tool) för att fixera ledarna mot kontaktens bleck.



En tekniskt korrekt kontaktering kräver att kabelns yttre hölje kläms fast inne i kontakten för att ge dragavlastning, samt att ledarnas tvinning bibehålls så nära kontaktytan som möjligt för att bibehålla kabelns kategoriklassificering.

Som ett sista steg valideras anslutningen alltid med en kabeltestare (cable tester) för att verifiera att alla ledare har elektrisk kontakt och sitter i rätt ordning.

Kapacitet och räckvidd för TP-kabel

När man väljer kategori på nätverkskabeln är det främst den önskade överföringshastigheten (throughput) och det fysiska avståndet som styr valet. En gyllene regel inom Ethernet-standarderna för kopparkabel är den maximala segmentlängden på **100 meter**. Detta avstånd inkluderar vanligtvis 90 meter fast installation i väggar och totalt 10 meter patchkablar i båda ändar. Om detta

avstånd överskrids ökar risken för signalförsvagning (attenuation) och paketförluster, vilket sänker nätverkets prestanda drastiskt.

För att ge en tydlig överblick över de vanligaste kategoriernas förmåga kan följande tabell användas som referens:

Kategori	Max hastighet	Max längd vid max hastighet	Typisk användning
Cat5e	1 Gbps	100 meter	Standard i äldre installationer och hemmanätverk.
Cat6	1 Gbps / 10 Gbps	100 m (1 Gbps) / 37–55 m (10 Gbps)	Klarar 10 Gbps över korta avstånd, bra för mindre kontor.
Cat6a	10 Gbps	100 meter	Standard för moderna professionella installationer och serverrum.
Cat7/8	25–40 Gbps	30 meter	Används främst i datacenter för korta, extremt snabba länk

Det är värt att notera att även om en Cat6-kabel tekniskt sett kan hantera 10 Gbps, så är den inte optimerad för detta över full längd. För installationer där man planerar för framtida 10 Gbps-hastigheter i hela fastigheten är därför Cat6a det rekommenderade valet, då den har en tätare tvinning och bättre skydd mot överhörning (crosstalk), vilket möjliggör full hastighet upp till den maximala längden på 100 meter.

Fiberoptisk kabel (fiber optics)

Används främst för ryggradsnät (backbone), anslutningar mellan byggnader eller miljöer som kräver extremt hög bandbredd och låg latens. Informationen överförs i form av ljuspulser genom kärnor av ultrarent glas eller plast. Den största tekniska fördelen med fiberoptik är att den är helt immun mot elektromagnetiska störningar (EMI), vilket gör den oumbärlig i industriella miljöer och vid dragning parallellt med högspänningskablar. Dessutom har

fiber betydligt lägre dämpning (attenuation) än kopparkabel, vilket möjliggör kommunikation över mycket långa avstånd utan behov av signalförstärkare.

Fiberoptik delas in i två huvudkategorier beroende på hur ljuset färdas i kärnan:

- **Multimode (MMF):** Har en relativt bred kärna som tillåter flera ljusstrålar (moder) att färdas samtidigt. Detta orsakar dock tidsdispersion över längre sträckor, vilket begränsar räckvidden. Multimode är standardvalet för kortare avstånd inom byggnader eller datacenter tack vare billigare sändarhårdvara (LED eller VCSEL).
- **Singlemode (SMF):** Har en extremt tunn kärna där endast en ljusstråle kan färdas. Detta eliminerar dispersion och gör det möjligt att skicka data tiotals mil med mycket hög precision med hjälp av lasrar. Singlemode är standard för stadsnät, operatörsnät och campus-anslutningar mellan geografiskt skilda byggnader.

Till skillnad från kopparkabel, där kategorin (t.ex. Cat6) sätter ett hårt stopp vid 100 meter, styrs fiberoptikens räckvidd av fiberkvaliteten (t.ex. OM-klasser för multimode) och vilken typ av sändtagare (transceiver) som används.

Typ	Standard	Max hastighet	Typisk räckvidd	Användningsområde
Multimode	OM3	10 Gbps	300 meter	Äldre LAN och korta länkar.
Multimode	OM4 / OM5	40 / 100 Gbps	100–400 meter	Moderna datacenter och backbone.
Singlemode	OS2	10–400 Gbps	10–40 km+	Stadsnät och långdistanslänkar.

Vid installation av fiber är även valet av kontakt (connector) avgörande för att minimera dämpning. De vanligaste typerna i lokala nätverk är **LC** (Lucent Connector), som är liten och kompakt för hög portdensitet i switchar, samt **SC** (Subscriber Connector), som ofta återfinns i äldre installationer eller vid avlämningspunkter från operatörer.

Koaxialkabel (BNC)

Denna kabeltyp förekommer numera sällan i nya Ethernet-installationer men används fortfarande inom specifika områden som kabel-TV (DOCSIS), äldre övervakningssystem (CCTV) och vissa typer av stadsnät.

En kabels kvalitet och skärmning (shielding) är avgörande faktorer, särskilt vid längre kabeldragningar. Medan oskärmad kabel (UTP) är standard i de flesta kontorsmiljöer, ställer miljöer med hög elektrisk störningsnivå krav på skärmade kablar (STP/FTP) för att garantera signalintegriteten.

Portar

En port är det fysiska gränssnittet där du kopplar in en kabel i en enhet. Vanliga porttyper i nätverk är:

- **RJ-45** – används för TP-kablar. Nästan alla nätverkskort, switchar och routrar har denna port.
- **SFP/SFP+** – moduler för fiberoptiska anslutningar. Vanliga i servrar, switchar och nätverksbackbones.
- **USB-C / Lightning** – används ibland för att dela internetanslutningar eller skapa personliga nätverk (PAN), men inte i traditionella LAN.

Portar kan stödja olika hastigheter, till exempel 10/100/1000 Mbit/s (Gigabit Ethernet), och kan ha stöd för PoE (Power over Ethernet) – vilket gör det möjligt att driva t.ex. accesspunkter via nätverkskabeln.

1.2.3 Vanliga topologier

Topologi beskriver hur nätverksenheterna är organiserade – både fysiskt och logiskt. En topologi kan påverka prestanda, skalbarhet, tillförlitlighet och kostnad. Det finns flera klassiska topologier som är bra att känna till.

Stjärntopologi

I en stjärntopologi är alla enheter anslutna till en central punkt, vanligtvis en switch. Detta är den vanligaste topologin i moderna nätverk, eftersom den är lätt att bygga ut och felsöka.

Om en kabel eller en klient går sönder påverkas inte övriga enheter. Men om switchen går ner slutar hela nätverket att fungera.

Stjärntopologi är grunden för alla moderna Ethernet-baserade LAN.

Bustopologi

Bustopologi innebär att alla enheter är kopplade längs en enda gemensam kabel (bussen). Denna topologi användes ofta med koaxialkabel i äldre nätverk, där varje enhet behövde en T-koppling och en terminator i vardera ände.

Den är billig, men har många nackdelar: dålig skalbarhet, svår felsökning och känslighet för avbrott.

Bustopologi är idag föråldrad och används endast i vissa specialmiljöer.

Ringtopologi

I en ringtopologi kopplas varje enhet till två andra, vilket bildar en sluten slinga. Data skickas runt i ett bestämt varv tills det når rätt mottagare. I vissa ringnät användes **token passing** för att bestämma vem som fick skicka.

Ringtopologi var vanliga i äldre nätverk (t.ex. Token Ring), men används inte längre i moderna datanät. Dock lever idén vidare i vissa fiber- och transportnät där redundans är viktig.

Mesh-topologi

I en mesh-topologi är varje nod kopplad till flera andra. Detta ger hög redundans och tillgänglighet. Mesh används bland annat i trådlösa mesh-nätverk i hem och företag, i backbone-nät, och i datacenter.

Det finns:

- **Full mesh** – alla noder kopplade till alla andra (dyrt och komplext)
- **Partial mesh** – utvalda noder har flera förbindelser

Mesh-topologi ger god tolerans mot fel men är dyr att implementera i större skala.

Hybridtopologi

De flesta riktiga nätverk använder en blandning av flera topologier – t.ex. stjärna internt och mesh i överliggande struktur. En hybridtopologi

kombinerar fördelar från flera olika modeller och används för att skapa flexibla och skalbara nät.

Kontrollfrågor

1. Vad är skillnaden mellan en switch och en router?
2. Vilken uppgift har en accesspunkt i ett WLAN?
3. Vad är MAC-adress och varför är den viktig?
4. Vilken typ av kabel används mest i moderna LAN?
5. När används fiberoptik istället för koppar?
6. Vad är en brandvägg och vad används den till?
7. Nämn en fördel och en nackdel med stjärntopologi.
8. Varför används inte bustopologi längre i moderna nätverk?
9. Vad är skillnaden mellan full mesh och partial mesh?
10. Vad menas med hybridtopologi?

Fördjupningslänkar

1. GeeksforGeeks (Network Devices: Hub, Switch, Router, Gateway, Bridge) – <https://www.geeksforgeeks.org/network-devices-hub-switch-router-gateway-bridge/>
2. CompTIA (Network Topologies Explained) – <https://www.comptia.org/content/articles/network-topologies>
3. Diffen (Router vs. Switch: Key Differences) – https://www.diffen.com/difference/Router_vs_Switch
4. Cisco (What is a Network Switch?) – <https://www.cisco.com/c/en/us/products/switches/what-is-a-network-switch.html>
5. Wikipedia (Network Topology) – https://en.wikipedia.org/wiki/Network_topology

Egna anteckningar

1.3 Protokoll och adressering

1.3.1 Varför protokoll?

Protokoll är **regler** som gör att olika system kan förstå varandra. De definierar hur data ska paketeras, adresseras, överförs och kontrolleras för fel. I praktiken arbetar flera protokoll tillsammans i en **protokollstack** (t.ex. TCP/IP), där varje lager har sin uppgift: från fysisk överföring till applikationer som webbläsare och e-post.

Ett vanligt felsökningsgrepp är att "gå neråt i stacken" tills du hittar lägsta lagret där felet uppstår.

Exempel: En webbplats laddar inte. Du kontrollerar först om applikationen (HTTP) svarar, sedan om transport (TCP/443) etablerar förbindelse, därefter om Internetlagret (IP) ruttar rätt, och slutligen om nätverksåtkomst (Ethernet/WLAN) har länk.

1.3.2 TCP och UDP (transportlagret)

TCP (Transmission Control Protocol) är **förbindelseorienterat**: trevägshandskakning (SYN-SYN/ACK-ACK), kvittenser och omsändning vid paketförlust. Det passar där **tillförlitlighet** är viktigt: webben (HTTP/HTTPS), e-post (SMTP/IMAP), filöverföring (SFTP/FTPS).

UDP (User Datagram Protocol) är **förbindelselöst**: inget handskak, inga kvittenser. Fördel: låg latens och mindre overhead. Vanligt för realtidsapplikationer som VoIP, vissa spel och DNS-frågor.

Portnummer identifierar vilken tjänst som ska ta emot datan. "Välkända portar" (0-1023) används av standardtjänster, "registrerade" (1024-49151) av applikationer, och "dynamiska/privata" (49152-65535) väljs oftast som källport av klienter.

Exempel: Din klient öppnar en HTTPS-anlutning från källport 51234 till servers port 443. Svaret går tillbaka till 51234.

1.3.3 Vanliga infrastrukturprotokoll

Se även 2.4 DHCP, DNS och NAT för detaljer och felsökning.

- **ARP (Address Resolution Protocol)** – översätter **IP → MAC** inom LAN. Nödvändigt för att skicka Ethernet-ramar till rätt nätverkskort.
- **ICMP (Internet Control Message Protocol)** – skickar kontrollmeddelanden (t.ex. *ping*). Bra för diagnos, inte för applikationsdata.
- **DHCP (Dynamic Host Configuration Protocol)** – tilldelar automatiskt IP-adress, nätmask, gateway och DNS till klienter.
- **DNS (Domain Name System)** – översätter **namn → IP** (t.ex. *example.se → 93.184.216.34*). Kritisk för nästan all applikationstrafik.
- **NTP (Network Time Protocol)** – synkroniserar tid mellan system; viktigt för loggar, Kerberos och certifikat.

Blockeras DNS eller DHCP i fel segment får klienter snabbt "mystiska" problem (ingen adress, eller kan inte nå resurser trots nätverkslänk).

1.4 IP: Nätverkets Adressering och Logik

En IP-adress identifierar ett nätverksinterface så att paket kan levereras rätt, ungefär som gata + postnummer för post. Det finns två versioner i bruk: IPv4 och IPv6. IPv4 har $2^{32} = 4\,294\,967\,296$ möjliga adresser; i praktiken färre för vanliga värdar eftersom många intervall är reserverade (t.ex. privatnät, loopback, multicast). Med fler enheter per person, servrar och sensorer tog IPv4-utrymmet i praktiken slut, vilket ledde till tekniker som CIDR (effektivare uppdelning), NAT (många interna delar på en publik adress) och hårdare återanvändning. IPv6 löser bristen med $2^{128} \approx 3,4 \times 10^{38}$ adresser och en hierarkisk modell som skalar – vanligtvis /64 per VLAN i lokala nät.

Vi skiljer också på publika och privata adresser (vilka får "synas" på internet). Publika adresser är globalt routbara och tilldelas via internetleverantörer. Privata IPv4-intervall (10/8, 172.16/12, 192.168/16) används inom organisationer och översätts via NAT när de ska ut på internet. I IPv6 motsvaras detta av Global Unicast (publikt, 2000::/3) och ULA (privat, fc00::/7) som inte ska routas på internet; i IPv6 ersätts NAT normalt av tydliga brandväggspolicies. På lagnivå finns dessutom link-local-adresser för lokal kommunikation: 169.254.0.0/16 i IPv4 (APIPA) och fe80::/10 i IPv6.

För att lyckas i resten av boken behöver du förstå tre saker:

1. Se att en adress tillhör ett interface, inte "hela datorn".
2. Kunna läsa prefixlängden (t.ex. /24) och därmed skilja Net ID och Host ID.
3. Förstå publik vs privat och när NAT respektive brandväggspolicy används.

I IPv6 får förvisso klienterna globala adresser (2001:db8:...), men skyddas av brandväggsregler – ingen NAT krävs.

1.4.1 Vad är en IP-adress? (Post och smeknamn)

En IP-adress identifierar ett **nätverksinterface**, inte nödvändigtvis hela datorn. Om din laptop har både Wi-Fi och en nätverkskabel inkopplad, har den två olika interface och därmed två olika IP-adresser.

För att förstå skillnaden mellan hur vi pratar på internet och i vårt lokala nätverk kan vi använda liknelsen om smeknamn:

- **Privat IP-adress:** Detta är som ett smeknamn. Hemma kanske din familj kallar dig "snuttegubben", men det är inget namn du använder officiellt på banken. På samma sätt används privata adresser (som 192.168.1.15) bara inom ditt lokala nätverk.
- **Publik IP-adress:** Detta är ditt officiella namn och adress som fungerar överallt. Det är den adressen som routern använder när den pratar med omvärlden.

1.4.2 IP-adressen som tekniskt ID-kort (32 bitar)

Även om vi skriver IPv4-adresser som fyra siffergrupper (oktetter) separerade av punkter, t.ex. 192.168.10.15, så ser datorn bara ett och nollor. En IPv4-adress består av totalt **32 bitar**.

Den binära uppställningen för 192.168.10.15:

Varje oktett består av 8 bitar. Här ser du hur adressen ser ut för nätverkskortet:

- **192** = 11000000
- **168** = 10101000

- **10** = 00001010
- **15** = 00001111

Hela adressen: 11000000.10101000.00001010.00001111

1.4.3 Speciella adresser du måste känna till

I din vardag som tekniker kommer du stöta på vissa adresser som har specifika regler:

- **Privata adresser (RFC1918):** Dessa får användas fritt internt men kan inte "synas" på internet.
 - 10.0.0.0 till 10.255.255.255
 - 172.16.0.0 till 172.31.255.255
 - 192.168.0.0 till 192.168.255.255
- **Loopback (127.0.0.1):** Datorns egen "spegel". Om du kan pinga 127.0.0.1 vet du att nätverksstacken i ditt operativsystem fungerar.
- **APIPA (169.254.x.x):** En "nöd-adress". Om din dator inte får kontakt med en DHCP-server ger den sig själv en adress i detta intervall. Ser du en 169-adress vet du direkt att det är problem med adressutdelningen.

1.4.4 Net ID och Host ID (Att dra gränser)

Varje IPv4-adress delas logiskt i två delar: **Net ID** (vilken gata/nätverk du tillhör) och **Host ID** (vilket husnummer/enhet du är).

- **Net ID:** Identifierar själva nätverket.
- **Host ID:** Identifierar den specifika enheten i det nätverket.

Inom varje subnät finns två reserverade adresser:

1. **Nätadressen:** Den första adressen (alla hostbitar är 0). Kan inte ges till en dator.
2. **Broadcastadressen:** Den sista adressen (alla hostbitar är 1). Används för att ropa till ALLA enheter i nätet.

Exempel 192.168.1.0/24:

- **Net ID:** 192.168.1.0
- **Broadcast:** 192.168.1.255
- **Användbara adresser:** 192.168.1.1 till 192.168.1.254 (Totalt 254 stycken).

1.4.5 Subnätmasken – Nätverkets mall

Subnätmasken är mallen som bestämmer exakt var gränsen mellan Net ID och Host ID går. Idag använder vi nästan uteslutande **CIDR-notation** (prefix), vilket skrivs som ett snedstreck följt av antalet ettor i masken, t.ex. /24.

Hur masken styr storleken:

- **/24** (255.255.255.0): 24 ettor. Ger plats för 254 enheter.
- **/16** (255.255.0.0): 16 ettor. Ger plats för över 65 000 enheter.
- **/8** (255.0.0.0): 8 ettor. Ger plats för över 16 miljoner enheter.

1.4.6 Klass A, B och C (Historik)

Förr delades nätverk in i fasta klasser. Även om vi idag använder CIDR (klasslös routing) är det bra att känna till "defaultmaskerna":

- **Klass A:** 1.0.0.0 – 126.0.0.0 (Default /8)
- **Klass B:** 128.0.0.0 – 191.255.0.0 (Default /16)
- **Klass C:** 192.0.0.0 – 223.255.255.0 (Default /24)

1.4.7 Räkna ut rätt mask – Steg för steg

Som tekniker "subnätar" du ett nätverk för att öka säkerheten (separera gäster från servrar) och prestandan (minska broadcast-trafik).

Metodik:

1. **Bestäm behov:** Hur många klienter behöver du per klassrum/avdelning?
2. **Hitta bitarna:** Använd formeln $2^h - 2 \geq \text{antal hostar}$ (där h är antalet nollor i slutet av masken).

3. **Blockstorlek:** Beräkna avståndet mellan nätverken: $2^{256} - \text{maskvärdet}$.

Exempel: Du har 20 datorer i ett labbrum.

- En /27 ger $2^5 - 2 = 30$ adresser. Det räcker!
- Masken för /27 är 255.255.255.224.
- Blockstorleken är $2^{256} - 224 = 32$.
- Dina subnät hoppar alltså med 32: .0, .32, .64, .96...

1.4.8 CIDR-lathund för /24

Här är en snabbreferens när du delar upp ett vanligt Klass C-nät:

Prefix	Mask	Antal subnät	Användbara hostar
/25	255.255.255.128	2	126
/26	255.255.255.192	4	62
/27	255.255.255.224	8	30
/28	255.255.255.240	16	14
/30	255.255.255.252	64	2 (Perfekt för länk mellan två routrar)

1.4.9 NAT och PAT: Tolken mellan privat och publikt

Som vi lärde oss tidigare finns det speciella IP-adresser (RFC1918) som bara får användas internt. Men hur kan en dator med adressen 192.168.1.10 surfa på webben om den adressen inte får synas på internet? Svaret är **NAT** (Network Address Translation).

NAT – En adressändring i farten

NAT fungerar som en mellanhand. När din dator skickar ett paket till internet, byter din router ut din privata avsändaradress mot routerns egen **publika IP-adress**. När svaret kommer tillbaka från internet, kommer

routern ihåg vem som bad om informationen och byter tillbaka adressen så att paketet hittar hem till din dator igen.

PAT – Många delar på en adress (Overload)

I ett vanligt hem eller på en skola finns det hundratals enheter, men ofta bara en enda publik IP-adress. Hur vet routern vilken mobil eller dator som ska ha vilket svar? Här använder vi **PAT** (Port Address Translation), även kallat "NAT Overload".

Tänk dig ett stort företag med ett enda officiellt telefonnummer. För att nå rätt person internt använder man **anknytningar**. PAT fungerar exakt likadant:

1. Din dator skickar en förfrågan från port 5001.
2. Routern tar paketet, ändrar din privata IP till sin publika, men ger paketet en unik "anknytning" (en port, t.ex. 10001).
3. När svaret kommer tillbaka till routern på port 10001, vet den direkt: "Aha, det här ska till datorn som använde port 5001 internt".

Det är denna teknik som gör att hela din familj eller klass kan surfa samtidigt på samma internetuppkoppling utan att paketen hamnar fel.

Port Forwarding (DNAT) – Att öppna dörren utifrån

Normalt sett blockerar en brandvägg all trafik som kommer utifrån om ingen på insidan har bett om den. Men ibland vill du att folk utifrån ska kunna nå en tjänst inne i ditt nätverk, till exempel en webbserver eller en spelserver. Då använder vi **Port Forwarding** (även kallat Destination NAT).

Det innebär att vi instruerar routern: "Om det kommer trafik på port 443 (webbtrafik), skicka den direkt vidare till vår interna webbserver".

Exempel från verkligheten: Tänk dig att du administrerar en **OPNsense**-brandvägg (en vanlig mjukvarubrandvägg).

- Din brandvägg har den publika adressen 203.0.113.10.
- Du har en webbserver i ett säkert område (DMZ) med den privata adressen 192.168.10.20.

För att världen ska kunna se din hemsida ställer du in en regel för Port Forwarding:

- **Inkommande trafik:** Publik IP 203.0.113.10 på port 443 (HTTPS).
- **Mål:** Skickas vidare till den interna adressen 192.168.10.20 på port 443.

Nu kan vem som helst i världen skriva in din publika adress i sin webbläsare, och din brandvägg kommer automatiskt att slussa in dem till rätt server på insidan.

Tips: När du felsöker nätverk och märker att du kan "pinga" en server men inte surfa till den, är det ofta en Port Forwarding-regel eller en felaktig PAT-inställning som spökar. Det är nätverksteknikerns vardag!

1.4.10 IPv4 vs IPv6: Ett arkitektoniskt skifte

För att förstå varför vi byter protokoll räcker det inte med att säga att adresserna tog slut. Vi måste titta på hur nätverkets själva logik har förändrats. IPv4 skapades i en tid när man trodde att några miljoner adresser skulle räcka för hela världen. IPv6 skapades för en framtid där varje glödlampa, sensor och rymdsond behöver en egen identitet.

1.4.10.1 Den stora skillnaden: Brist mot överflöd

I IPv4 har vi levt i en "bristekonomi". Eftersom det bara finns ca 4,3 miljarder adresser har vi blivit experter på att snåla.

- **IPv4:** Liknar en trångbodd stad där många måste dela på samma officiella postadress (NAT).
- **IPv6:** Liknar att varje individ på planeten får ett eget privat höghus med adresser. Det finns ingen anledning att snåla längre.

1.4.10.2 NAT och PAT – En nödlösning som försvinner

Den största skillnaden i din vardag som tekniker är synen på **NAT (Network Address Translation)**.

- **I IPv4-världen:** NAT är standard. Vi "gömmer" tusentals privata adresser bakom en enda publik adress. Detta skapar säkerhet genom att dölja insidan, men det gör också nätverket komplext. Det bryter den så kallade

"End-to-End"-principen (att två enheter ska kunna prata direkt med varandra).

- **I IPv6-världen:** NAT är (i de flesta fall) dött. Varje enhet får en globalt unik adress. Detta återställer internets grundtanke: direkt kommunikation. Säkerheten flyttas från "att vara gömd bakom NAT" till "att vara skyddad av en intelligent brandvägg".

1.4.10.3 Hur enheterna hittar hem: DHCP vs SLAAC

I IPv4 är vi helt beroende av en DHCP-server för att enheter ska få adresser automatiskt. Om servern dör, stannar nätverket. IPv6 introducerar en mycket smartare metod:

- **SLAAC (Stateless Address Autoconfiguration):** En enhet i ett IPv6-nätverk behöver inte be om lov för att få en adress. Den lyssnar på routern för att få veta vilket nätverk (prefix) den tillhör, och sedan bygger den resten av adressen själv (ofta baserat på sitt eget MAC-nummer). Det är äkta plug-and-play.
- **DHCPv6:** Finns fortfarande som ett komplement om man vill ha mer kontroll, men det är inte längre ett absolut krav för att nätverket ska fungera.

1.4.10.4 Effektivitet under huven (Header-design)

När en router ska skicka ett paket i IPv4 måste den läsa igenom ett ganska stökigt "kuvert" (header) med massor av information som ofta är onödig.

IPv6 har en mycket renare design. Trots att adresserna är mycket längre, är själva kuvertet enklare för routrarna att läsa. Det gör att moderna routrar kan skicka IPv6-paket snabbare och med mindre ansträngning än IPv4-paket.

1.4.10.5 Sammanfattning av de tekniska skillnaderna

Funktion	IPv4	IPv6
Adresslängd	32 bitar (Decimalt)	128 bitar (Hexadecimalt)
Konfiguration	Manuellt eller DHCP	Självkonfigurerande (SLAAC) eller DHCPv6

Funktion	IPv4	IPv6
Säkerhet	Tillägg (IPsec är valfritt)	Inbyggt stöd för kryptering (IPsec)
NAT	Nödvändigt för att spara adresser	Behövs normalt inte
Broadcast	Ja (skickar till alla i onödan)	Nej (använder effektiv Multicast istället)

Teknikerns reflektion: Den svåraste biten med IPv6 är inte matematiken, det är att sluta tänka att man måste "snåla". I IPv4 försöker vi alltid göra subnäten så små som möjligt för att inte slösa. I IPv6 ger vi bort miljarder adresser till ett enda VLAN utan att blinka. Det är ett mentalt skifte som du måste bemästra.

1.4.11 Hexadecimal logik: Varför bokstäver?

I IPv4 använde vi decimaltal (0–255), men i IPv6 räcker inte det. För att skriva ner 128 bitar på ett effektivt sätt använder vi **hexadecimala tal** (basen 16).

- Ett hexadecimalt tecken kan vara 0–9 eller A–F.
- Varje tecken motsvarar exakt **4 bitar** (en så kallad *nibble*).
- Fyra tecken bildar ett block på 16 bitar (t.ex. 2001).

Genom att använda hex kan vi trycka ihop massor av information på kort yta. Ett block som abcd ser kanske ut som slumpmässiga bokstäver, men för nätverkskortet är det en specifik ström av 16 ettor och nollor.

1.4.11.1 Adressens anatomi: 64/64-delningen

En IPv6-adress är inte bara en lång rad tecken; den är strikt hierarkisk. Den gyllene regeln för oss tekniker är att vi nästan alltid delar adressen på mitten, vid **64-bitarsgränsen**.

1. **Network Prefix (De första 64 bitarna):** Detta är nätverkets identitet. Det talar om för routarna i världen vilket nätverk paketet ska till. Det delas ofta upp i:

- *Global Routing Prefix*: Tilldelas av din internetleverantör (ISP).
- *Subnet ID*: Den del du som tekniker styr över för att skapa olika VLAN (t.ex. elev, personal, gäst).

2. Interface ID (De sista 64 bitarna): Detta är enhetens unika "namn" inom det lokala nätverket.

Varför är Interface ID så stort? Det kan verka som slöseri att ha 64 bitar för enheter i ett enda rum. Men det finns en genial tanke: med 64 bitar är risken för att två enheter väljer samma adress i princip noll. Detta gör att tekniker som **SLAAC** fungerar felfritt – enheten kan generera sitt eget Interface ID utan att fråga någon om lov.

1.4.11.2 Att planera med prefix (Nibble-boundary)

Som nätverkstekniker är din viktigaste uppgift att planera hur adresserna ska fördelas. I IPv6 räknar vi inte enstaka adresser, vi räknar nätverk.

Här är den vanliga hierarkin:

- **/48 (Site Prefix)**: Standard för ett helt företag eller ett campus. Ger dig 65 536 stycken /64-nät.
- **/56 (Customer Assignment)**: Ofta vad en mindre skola eller ett mindre kontor får. Ger 256 stycken /64-nät.
- **/64 (VLAN Prefix)**: Slutstationen. Det minsta prefixet vi använder för ett vanligt lokalt nätverk.

Exempel på planering: Om skolan får prefixet 2001:db8:aaaa::/48, kan du dela upp det så här:

- 2001:db8:aaaa:0001::/64 – Skolans servrar
- 2001:db8:aaaa:0010::/64 – Elev-WLAN
- 2001:db8:aaaa:0020::/64 – Personal-WLAN

Ser du hur enkelt det är? Du behöver bara ändra siffrorna i det fjärde blocket för att byta nätverk. Ingen binär matte behövs vid köksbordet som i IPv4!

1.4.11.3 Neighbor Discovery: Hej då, ARP!

I IPv4 använde vi ARP (Address Resolution Protocol) för att koppla ihop en IP med en MAC-adress. ARP skickade ut "broadcasts" som störde alla

enheter i onödan. IPv6 använder istället **Neighbor Discovery Protocol (NDP)** via ICMPv6.

När din dator vill prata med en annan på samma switch skickar den en **Neighbor Solicitation**. Men istället för att skrika till alla (broadcast), skickar den meddelandet till en mycket specifik **Multicast-adress** som bara den enhet som faktiskt har adressen lyssnar på. Det är som att visa i ett rum istället för att skrika i en megafon – det blir mycket tystare och effektivare i nätverket.

1.4.11.4 SLAAC – Hur enheten skapar sin egen adress

SLAAC är magin i IPv6. När du kopplar in en dator händer följande:

1. Datorn skickar en **Router Solicitation** ("Finns det någon router här?").
2. Routern svarar med en **Router Advertisement** ("Ja, jag finns! Du tillhör nätverket 2001:db8:cafe:1::/64").
3. Datorn tar nätverksdelen och kombinerar den med ett Interface ID den hittar på själv (ofta en slumpmässig siffra för att skydda din integritet).
4. **Klart!** Enheten har en global adress och kan surfa direkt. Inget krångel med DHCP-pooler som tar slut.

1.4.12 Gateways och grundläggande routing

Default gateway krävs för att nå andra nät. Finns ingen väg (route) till ett nät, kommer trafiken att skickas mot **defaulten**. På routrar kan du skapa **statiska rutter** (manuellt) eller använda **dynamiska protokoll** (t.ex. RIP, OSPF) som lär sig vägar automatiskt.

I små miljöer räcker ofta statisk routing; i större nät blir dynamisk routing en nödvändighet för skalbarhet och redundans.

1.4.13 Vanliga portar och tjänster

- **20/21** FTP (filöverföring – legacy, ersätt ofta med SFTP/FTPS)
- **22** SSH (säker fjärranslutning)
- **23** Telnet (osäkert, lämnas av historiska skäl)

- **25/587/465** SMTP (e-post ut)
- **53** DNS (UDP/TCP)
- **67/68** DHCP (server/klient)
- **80** HTTP (webb)
- **123** NTP (tid)
- **139/445** SMB (fildelning i Windows)
- **389/636** LDAP/LDAPS (katalogtjänster)
- **443** HTTPS (säker webb)
- **3389** RDP (fjärrskrivbord)

*I brandväggsdesign öppnas bara nödvändiga portar **inifrån → ut** och **utifrån → in**; allt annat blockeras som standard.*

Kontrollfrågor

1. Hur många bitar består en IPv4-adress respektive en IPv6-adress av?
2. Du kör kommandot `ip addr` och ser att ditt interface har adressen 169.254.44.12. Vad har hänt tekniskt och varför kommer du inte ut på internet?
3. I ett IPv4-nätverk med prefixet /26, hur många användbara adresser kan du dela ut till datorer?
4. Varför kan du aldrig ge den första och den sista adressen i ett IPv4-subnät till en vanlig dator?
5. Förklara med egna ord varför vi använder PAT (Port Address Translation) i nästan alla hemmanätverk.
6. Förkorta denna adress så mycket som möjligt:
2001:0db8:0000:0000:0000:0000:abc0:0001.
7. Beskriv kort hur en dator i ett IPv6-nätverk kan få en adress utan att prata med en DHCP-server.
8. Vilket portnummer och vilket protokoll (TCP/UDP) används normalt för DNS-uppslag?
9. Varför anses Telnet (port 23) vara osäkert jämfört med SSH (port 22)?
10. Vad händer med ett paket om en router tar emot det och inte hittar någon specifik rutt till måladressen i sin routningstabell?

Fördjupningslänkar

Cloudflare (The OSI Model vs TCP/IP) –

<https://www.cloudflare.com/learning/ddos/glossary/open-systems-interconnection-model-osi/>

Practical Networking (Subnetting Mastery) –

<https://www.practicalnetworking.net/index/subnetting/>

RIPE NCC (IPv6 for Beginners) – <https://www.ripe.net/about-us/press-centre/understanding-ipv6/>

Cisco (IPv4 Addressing and Subnetting Guide) –

<https://www.cisco.com/c/en/us/support/docs/ip/routing-information-protocol-rip/13788-3.html>

SpeedGuide (Common Ports Reference) –

<https://www.speedguide.net/ports.php>

IANA (IPv4 Special-Purpose Address Registry) –

<https://www.iana.org/assignments/iana-ipv4-special-registry/>

IPv6.com (SLAAC and DHCPv6 Explained) – <https://ipv6.com/guides/slaac-vs-dhcpv6/>

Wireshark (Understanding the IPv6 Header) –

<https://wiki.wireshark.org/IPv6>

Network World (Why NAT is still alive in IPv6) –

<https://www.networkworld.com/article/2223363/why-nat-is-still-alive-in-ipv6.html>

GeeksforGeeks (Introduction to Routing) –

<https://www.geeksforgeeks.org/routing-in-computer-networks/>

Egna anteckningar

1.5 Kommunikation i nätverk – Ethernet och WLAN

Kommunikation i ett lokalt nätverk bygger på gemensamma regler för hur data paketeras och skickas över det fysiska mediet. För att enheter ska kunna förstå varandra används standarder som definierar allt från spänningsnivåer i kablar till hur adresser läses av i nätverkskortet.

1.5.1 Ethernet på datalänk- och fysiskt lager

Ethernet är den dominerande standarden för trådbundna nätverk. Den beskriver hur data delas upp i ramar (frames) för att transporteras på datalänk-lagret (lager 2) i OSI-modellen. En typisk Ethernet-ram är uppbyggd av flera fält som fyller specifika funktioner för att säkerställa att informationen når rätt mottagare och är intakt:

- **Destination och Källa (MAC-adresser):** Varje nätverkskort har en unik fysisk adress (MAC-adress) som används för att identifiera sändare och mottagare inom samma lokala nätverk.
- **EtherType:** Anger vilket protokoll som ligger paketerat i ramen, oftast IPv4 eller IPv6.
- **Payload (nyttodata):** Själva informationen som ska skickas, exempelvis en del av en bild eller ett dokument.
- **FCS/CRC (Frame Check Sequence):** En kontrollsumma som används för felkontroll. Om mottagaren beräknar en annan summa än den som står i ramen, anses datan vara skadad och kastas.

Inom Ethernet används tre huvudsakliga adresseringsmetoder: **Unicast** (till en specifik mottagare), **Multicast** (till en grupp av mottagare) och **Broadcast** (till samtliga enheter i nätverket). En vanlig process där dessa samverkar är när en klient behöver nå en IP-adress men saknar mottagarens fysiska adress. Klienten skickar då en förfrågan (ARP request) som en broadcast till adressen ff:ff:ff:ff:ff:ff. Den enhet som äger IP-adressen svarar med sin MAC-adress, varpå den fortsatta kommunikationen kan ske som unicast.

Standardstorleken för en Ethernet-ram, känd som **MTU (Maximum Transmission Unit)**, är 1500 byte. I specialiserade miljöer som datacenter används ibland större ramar, så kallade **Jumbo Frames** på

upp till 9000 byte, för att effektivisera transporten av stora mängder data. Detta kräver dock att samtliga enheter i kedjan har stöd för samma MTU-inställning.

Modern Ethernet inkluderar även möjligheten att strömförsörja enheter direkt via nätverkskabeln genom tekniken **PoE (Power over Ethernet)**. Beroende på enhetens behov används olika standarder: **802.3af** (upp till 15,4 W), **802.3at** (PoE+, upp till 30 W) och den kraftfullare **802.3bt** (PoE++, upp till 60–90 W). Vid planering är det kritiskt att kontrollera att switchens totala effektbudget (PoE budget) räcker för samtliga anslutna accesspunkter och kameror.

1.5.2 Mediedelning, duplex och kollisioner

En av de mest fundamentala skillnaderna mellan Ethernet och WLAN är hur de hanterar tillgången till det fysiska mediet. Detta påverkar både nätverkets latens (ping) och dess totala genomströmning.

CSMA/CD i trådbundna nätverk

I nätverkets barndom, när enheter delade samma fysiska kabel via hubbar, användes metoden **CSMA/CD (Carrier Sense Multiple Access with Collision Detection)**. Enheten lyssnar på kabeln (Carrier Sense) och om ingen annan pratar, börjar den skicka data. Om två enheter pratar samtidigt uppstår en kollision (collision). Enheterna upptäcker detta (Collision Detection), avbryter sändningen och väntar en slumpmässig tid innan de försöker igen.

I moderna nätverk med switchar och full duplex (full-duplex) har varje enhet en dedikerad länk till switchen där den kan skicka och ta emot data samtidigt. Detta eliminerar i praktiken behovet av CSMA/CD och tar bort risken för kollisioner, vilket ger en mycket låg och stabil latens.

CSMA/CA i trådlösa nätverk (WLAN)

Trådlös kommunikation fungerar annorlunda eftersom mediet (luften) delas av alla enheter på samma kanal. WLAN är till sin natur **halv duplex (half-duplex)**, vilket innebär att endast en enhet kan prata åt gången på en given kanal. Istället för att upptäcka kollisioner i efterhand, försöker WLAN att undvika dem helt genom **CSMA/CA (Carrier Sense Multiple Access with Collision Avoidance)**.

Innan en trådlös enhet skickar data lyssnar den på kanalen. Om den är upptagen väntar enheten. Även om kanalen verkar ledig, väntar enheten en kort slumpmässig tid för att minska risken att krocka med en annan enhet som också väntar. Dessutom kräver nästan varje skickat datapaket en bekräftelse (ACK) från mottagaren. Denna extra väntetid och de administrativa bekräftelserna (overhead) är den främsta anledningen till att svarstider (ping) alltid är högre i ett trådlöst nätverk jämfört med Ethernet. Faktorer som radiostörningar, hinder och att många enheter konkurrerar om samma sändningstid (airtime) förstärker denna effekt ytterligare.

Autonegotiation och felmatchning

För att underlätta drift används **autonegotiation**, där två sammankopplade enheter automatiskt kommer överens om högsta möjliga hastighet och duplex. Om enheterna misslyckas med detta, eller om en tekniker manuellt låser ena änden till en specifik hastighet (t.ex. 100/Full) medan den andra står på auto, uppstår en **duplex-mismatch**. Detta leder till att den enhet som står på halv duplex tror att det sker kollisioner vid normal trafik, vilket orsakar kraftiga paketförluster och prestandaproblem som kan vara svåra att diagnosticera.

1.5.3 VLAN och trunkar i korthet

Detta avsnitt är en översikt. Detaljer som native VLAN, DTP med mera behandlas i avsnitt 2.2.

VLAN (802.1Q) skapar logiska, separata nät på samma fysiska switch. En **accessport** tillhör ett VLAN. En **trunk** bär flera VLAN mellan switchar/routrar med hjälp av 802.1Q-taggar. VLAN-ID 1–4094 (praktiskt: undvik "native VLAN" exponerat på trunkar).

*För trafik mellan VLAN krävs **routing** (t.ex. L3-switch eller "router-on-a-stick"). Segmentering minskar broadcastdomäner och höjer säkerheten.*

1.5.4 WLAN – begrepp och arkitektur

WLAN (IEEE 802.11) ger trådlös åtkomst till LAN. En **accesspunkt (AP)** sänder ett **SSID** (nätverksnamn). Varje AP-radio har en **BSSID** (MAC-adress). En **BSS** är cellen runt en AP; flera BSS som använder samma SSID bildar ett **ESS** (samma logiska nät över större yta).

Band och kanaler

- **2,4 GHz (802.11/b, 802.11/g, 802.11/n)**: Lång räckvidd, färre icke-överlappande kanaler, mer störningskänsligt.
- **5 GHz (802.11/ac)**: Fler kanaler, högre kapacitet, kortare räckvidd; vissa kanaler kräver DFS.
- **6 GHz (802.11ax i 6 GHz-bandet)**: Många rena kanaler (där det är tillåtet), kortare räckvidd.

I 2,4 GHz används i praktiken 1/6/11 som icke-överlappande kanalval (i vissa länder finns kanal 13, men överlapp måste beaktas).

Kanaltäthet (20/40/80/160 MHz) påverkar kapacitet och räckvidd: bredare kanal ⇒ högre topphastighet men större risk för störningar och sämre frekvensåteranvändning.

Exempel: *I en skola med många klassrum ger 5 GHz på 20 eller 40 MHz ofta bäst total kapacitet (mer frekvensåteranvändning) jämfört med några få breda 80/160-kanaler.*

Viktigt: *802.11ax finns även i 2,4/5 GHz-bandet; här avses specifikt varianten i 6 GHz.*

1.5.5 Medietillgång och prestanda i WLAN

WLAN använder **CSMA/CA** (collision avoidance): klienter lyssnar innan de sänder. Mekanismer som **RTS/CTS** kan användas för att minska problem med "hidden nodes" (klienter som inte hör varandra).

Moderna WLAN använder **OFDM/OFDMA** och **MIMO/MU-MIMO** för att utnyttja luften effektivt och serva flera klienter samtidigt. Verklig kapacitet styrs av **SNR**, **interferens**, kanaltäthet, antal **spatial streams** och hur många klienter som konkurrerar om **airtime**.

Placering av AP, sändareffekt och dämpning (väggar, metall, glas) avgör täckning. För stark effekt kan försämma roaming och skapa mer kanalstörning.

1.5.6 Säkerhet i WLAN: Från håll till kryptering i världsklass

Att skicka data via radiovågor är i praktiken som att stå och ropa i ett klassrum fullt med folk – vem som helst som befinner sig inom hörhåll kan

lyssna på vad du säger. I nätverkssammanhang kallas detta för att etern är ett delat medium. För att förhindra att obehöriga läser vår privata trafik måste vi använda kryptering. Under de senaste 25 åren har vi gått från system som var så svaga att de kunde knäckas på sekunder till moderna protokoll som kräver enorm datorkraft för att ens ha en chans att komprometteras.

WEP – Det första, misslyckade försöket

När Wi-Fi först lanserades 1997 skapades **WEP (Wired Equivalent Privacy)**. Tanken var god: man ville att ett trådlöst nätverk skulle vara lika säkert som ett trådbundet. Men tekniskt sett var det en katastrof. WEP förlitade sig på en algoritm som heter RC4 och en statisk nyckel som aldrig ändrades.

Det stora problemet var den så kallade "initieringsvektorn" (IV). Det är en liten del av krypteringen som ska vara unik för varje paket. I WEP var denna alldeles för kort. När ett nätverk har mycket trafik börjar IV-värdena återanvändas. En angripare som lyssnar på nätverket kan samla in dessa kollisioner och med enkel mjukvara räkna ut själva huvudlösenordet.

- **Hur det fungerar:** Använder en statisk nyckel (ett lösenord) som delas av alla. Data krypteras med en algoritm som heter RC4.
- **Fördelar:** Kräver i princip noll processorkraft. Fungerar på antik hårdvara.
- **Nackdelar:** Extremt osäkert. Den korta startvektorn (IV) gör att krypteringen återanvänds för ofta.
- **Varför det är så dåligt:** En modern laptop kan knäcka ett WEP-lösenord på under en minut genom att bara lyssna på trafiken.

WPA och TKIP – En tillfällig lösning

När säkerhetshålen i WEP blev kända behövde branschen en snabb lösning som inte krävde att alla köpte nya routrar. Resultatet blev **WPA (Wi-Fi Protected Access)** med protokollet **TKIP**. TKIP fungerade som ett "plåster" på RC4-algoritmen genom att byta ut krypteringsnyckeln för varje enskilt paket. Det var säkrare än WEP, men eftersom det fortfarande byggde på samma grundläggande matematik hittade hackare snart nya vägar in. Dessutom blev TKIP en flaskhals för prestandan i takt med att nätverken blev snabbare.

- **Hur det fungerar:** Behåller RC4-krypteringen men lägger till TKIP (Temporal Key Integrity Protocol) som byter nyckel löpande.
- **Fördelar:** Gick att installera via mjukvaruuppdateringar på gammal WEP-hårdvara.
- **Nackdelar:** TKIP har egna sårbarheter och begränsar hastigheten på moderna nätverk drastiskt (ofta till 54 Mbit/s).

WPA2-PSK och den berömda 4-vägshandskakningen

Med **WPA2** kom den stora förändringen: **AES (Advanced Encryption Standard)**. Detta är samma kryptering som banker och myndigheter använder. För hemmet och mindre kontor använder vi oftast **PSK (Pre-Shared Key)**, där alla använder samma lösenord.

Det som gör WPA2 intressant under huven är hur en enhet faktiskt ansluter till en accesspunkt. Detta sker via en **4-vägshandskakning (4-way handshake)**. Syftet med handskakningen är att bevisa att både klienten och accesspunkten kan lösenordet utan att de faktiskt skickar lösenordet i klartext över luften.

Så här fungerar 4-vägshandskakningen förenklat:

1. **ANonce:** Accesspunkten skickar ett slumpmässigt tal (en "nonce") till klienten.
2. **SNonce + MIC:** Klienten skapar ett eget slumpmässigt tal och kombinerar det med lösenordet för att skapa en unik sessionsnyckel. Sedan skickas klientens "nonce" plus en kontrollsumma (MIC) tillbaka.
3. **GTK:** Accesspunkten skapar nu samma sessionsnyckel på sin sida. Om allt stämmer skickar den ut gruppnyckeln (för broadcast-trafik).
4. **ACK:** Klienten bekräftar att allt är klart och krypteringen startar.

Sårbarheten här är att en hackare kan "spela in" denna handskakning och sedan sitta hemma i lugn och ro och testa miljarder lösenord mot den (en offline-attack).

- **Hur det fungerar:** Introducerade AES-kryptering och 4-vägshandskakning.
- **Fördelar:** Mycket stark kryptering som stöds av nästan alla moderna enheter.

- **Nackdelar:** Sårbart för "brute force"-attacker om man använder ett svagt lösenord. Om en person lämnar organisationen måste lösenordet bytas för alla.

WPA2-Enterprise – För den professionella miljön

I skolor och på stora företag räcker det inte med ett gemensamt lösenord. Där använder vi **WPA2-Enterprise**. Skillnaden är att inloggningen sker mot en central databas, oftast via protokollet **802.1X**. Istället för ett lösenord i routern skickas dina inloggningsuppgifter till en **RADIUS-server**.

Detta ger oss som tekniker full kontroll. Om en elev slutar eller tappar bort sin dator kan vi spärra just det kontot direkt utan att påverka någon annan. Det är skillnaden mellan att ha en huvudnyckel till ett helt hus (PSK) och att ha personliga passerkort (Enterprise).

- **Hur det fungerar:** Kräver unik inloggning per användare via en RADIUS-server.
- **Fördelar:** Unika nycklar, hög spårbarhet och enkel administration av många användare.
- **Nackdelar:** Betydligt mer komplext att konfigurera och kräver extra serverresurser.

WPA3 – Framtiden är här

WPA3 lanserades för att täppa till de sista hålen i WPA2. Den största nyheten för privatpersoner är **SAE (Simultaneous Authentication of Equals)**. Det ersätter den gamla handskakningen med en metod som gör det omöjligt för hackare att spela in trafiken och knäcka lösenordet i efterhand. Även om du har ett ganska svagt lösenord är WPA3 extremt svårt att forcera.

För företag innebär **WPA3-Enterprise** ännu starkare krypteringsalgoritmer (192-bitars läge) för att skydda mot framtida hot, inklusive kvantdatorer.

- **Hur det fungerar (SAE):** Använder en metod där lösenordet aldrig kan gissas genom att analysera inspelad trafik.
- **Fördelar:** Skyddar mot de vanligaste attackerna mot WPA2. Tvingar fram skyddade ledningsramar (PMF).

- **Nackdelar:** Kräver modern hårdvara; många äldre enheter (som skrivare eller gamla mobiler) kan inte ansluta till ett rent WPA3-nätverk.

Hur sätter man upp ett bra WLAN?

Som it-tekniker är din uppgift att balansera säkerhet och användarvänlighet. Här är de gyllene reglerna:

- **Välj rätt säkerhetsnivå:** Använd **WPA3** där det går. För att stödja äldre enheter kan du använda "WPA2/WPA3 Mixed Mode", men aktivera aldrig WEP eller WPA1.
- **Segmentera med VLAN:** Detta är avgörande. Skapa olika trådlösa nätverk (SSID) för olika syften. Gäster ska aldrig vara på samma nätverk som ekonomiavdelningen eller skolans servrar.
 - *Exempel:* **SSID "Personal"** (VLAN 10, WPA2-Enterprise), **SSID "Gäster"** (VLAN 30, WPA2-PSK med isolering).
- **Sluta dölja SSID:** Att dölja nätverksnamnet (Hidden SSID) ger ingen säkerhet alls, eftersom enheterna ändå måste ropa efter nätverket högt i etern. Det gör bara anslutningen instabilare.
- **Använd RADIUS för kontroll:** Så fort du hanterar mer än ett tiotal användare är RADIUS-baserad inloggning (Enterprise) det enda ansvarsfulla valet.

Hänvisning: Vi kommer att titta närmare på hur man sätter upp och konfigurerar en **RADIUS-server** (exempelvis FreeRADIUS eller Windows NPS) i **Kapitel 4: Nätverkstjänster och Autentisering**. Där lär vi oss hur servern pratar med accesspunkterna för att släppa in rätt användare på rätt VLAN.

1.5.7 Roaming och den osynliga upplevelsen

I ett litet hem räcker det ofta med en router, men på en skola eller ett kontor har vi dussintals eller hundratals accesspunkter (AP). När du går genom korridoren med din laptop vill du att den sömlöst ska flytta över från en AP till en annan utan att videosamtalet bryts. Detta kallas för **roaming**.

Det viktigaste att förstå som tekniker är att det nästan alltid är **klienten** (mobilen eller datorn) som bestämmer när det är dags att byta AP. Men nätverket kan hjälpa till på traven genom några viktiga tekniker:

- **802.11r (Fast Transition):** När vi använder WPA2-Enterprise krävs en lång handskakning mot RADIUS-servern varje gång vi byter AP. Det tar tid. 802.11r gör att delar av handskakningen sker i förväg, vilket sparar tiden för bytet från sekunder till millisekunder.
- **802.11k (Neighbor Reports):** Istället för att din mobil ska behöva skanna alla radiokanaler för att hitta nästa AP, skickar nätverket en lista på "grannar" som finns i närheten. Det sparar både tid och batteri.
- **802.11v (BSS Transition):** Här kan nätverket faktiskt ge klienten ett förslag: "Jag ser att du har svag signal hos mig, men AP:n i taket bredvid dig har jättebra signal. Flytta dit!".

Utmaningar för teknikern: Sticky Clients och Airtime

Ett vanligt problem är "**Sticky Clients**" – enheter som envist håller kvar vid en svag AP trots att de står precis under en ny. För att lösa detta kan vi ställa in en **Minimum RSSI**. Det innebär att vi instruerar accesspunkten att sparka ut klienter om deras signalstyrka blir för låg, vilket tvingar dem att leta efter en bättre signal.

En annan fälla är att skapa för många trådlösa nätverk (**SSID**). Varje SSID kräver att accesspunkten skickar ut små "här är jag"-paket (beacons) flera gånger i sekunden. Om du har 10 olika SSID på samma ställe kommer luften vara så full av dessa administrativa paket att det knappt finns plats för riktig data. Håll dig till 2–4 SSID och använd VLAN för att separera trafiken istället.

1.5.8 Ethernet vs WLAN – Varför behöver vi båda?

Som it-tekniker får man ofta frågan: "Varför ska vi dra kablar när Wi-Fi är så snabbt?". Svaret ligger i skillnaden mellan ett **delat medium** och ett **dedikerat medium**.

Ethernet: Den stabila motorvägen

När du kopplar in en nätverkskabel har du en egen "fil" rakt in i switchen. Modern Ethernet körs i **Full Duplex**, vilket betyder att du kan skicka och

ta emot data samtidigt med full hastighet utan att krocka med någon annan.

- **Fördelar:** Ingen jitter (variation i fördröjning), maximal säkerhet (du måste fysiskt komma åt kabeln) och stabil kapacitet.
- **När väljer vi det?** Alltid för servrar, stationära datorer, skrivare och förstås som "backbone" – kablarna som matar accesspunkterna med ström (PoE) och data.

WLAN: Friheten med kompromisser

Wi-Fi fungerar i **Half Duplex**. Det är som en walkie-talkie: bara en person kan prata åt gången på samma kanal. Om tio elever laddar ner stora filer samtidigt på samma accesspunkt, måste de dela på den totala hastigheten, och de måste "vänta på sin tur". Dessutom kan mikrovågsugnar, betongväggar och grannens nätverk störa signalen.

- **Fördelar:** Extrem rörlighet och enkel skalbarhet för många enheter utan att behöva borra hål i väggarna.
- **Nackdelar:** Varierande latens (ping) och känslighet för störningar.

Den moderna designen

I ett modernt nätverk bygger vi en hybrid. Vi använder fiber och kopparkabel (Ethernet) i väggarna för att skapa ett stabilt fundament med hög kapacitet (10 Gbps eller mer). Sedan placerar vi ut välplanerade accesspunkter som ger användarna den rörlighet de behöver.

En gyllene regel för oss tekniker är: **"Om det har en fast plats och har ett nätverksuttag - använd kabel"**. Genom att flytta bort tunga laster (som en stationär dator som streamar video eller en server) från det trådlösa nätverket, lämnar vi mer "airtime" ledig för de mobila enheter som verkligen behöver den.

Tänk på: När du designar ett nätverk, tänk på att Wi-Fi bara är så bra som kablarna bakom det. Om du har en modern Wi-Fi 6-accesspunkt men kopplar in den med en gammal sliten Cat5-kabel, kommer kabeln att bli flaskhalsen oavsett hur snabb radiotekniken är.

1.5.9 802.11-standarder

Nedan är de vanligaste standarderna som nämns i specifikationer för klienter och accesspunkter. (Vi använder **IEEE-namnen** konsekvent.)

Standard	Band	Teknik i korthet	Typiska kanalbredder	Max teoretisk länkhastighet*	Notering
802.11b	2,4 GHz	DSSS/CCK	20 MHz	11 Mbit/s	Legacy, undvik i nya miljöer
802.11g	2,4 GHz	OFDM	20 MHz	54 Mbit/s	Bakåtkompatibel med b
802.11n	2,4 & 5 GHz	OFDM, MIMO	20/40 MHz	upp till 600 Mbit/s (4×4)	Vanlig, men äldre; stöd för både band
802.11ac	5 GHz	MU-MIMO (DL), högre QAM	20/40/80/160 MHz	>1 Gbit/s (beroende på streams/bredd)	Endast 5 GHz
802.11ax	2,4/5/6 GHz	OFDMA , MU-MIMO (UL/DL), TWT	20/40/80/160 MHz	flera Gbit/s (beroende på streams/bredd)	Effektivare airtime, stöd i 6 GHz där tillåtet

*Max "länk-rate" är teoretisk och förutsätter optimala förhållanden; **verklig throughput är lägre.**

Snabba köptips för elever

- Satsa på **802.11ax** för framtidssäkerhet (gärna stöd för både 2,4 och 5 GHz; 6 GHz där det är tillåtet).
- Välj minst **2×2 MIMO** på klienter; fler streams på AP ger bättre kapacitet för många klienter.
- Kontrollera stöd för **WPA3** och regelbundna firmware-uppdateringar.

- I skolmiljö: AP med **PoE-stöd**, **DFS** (för fler 5 GHz-kanaler) och möjlig **controller/central hantering**.
- Titta på **kanalbredd** (20/40 MHz räcker ofta bättre i miljöer med många celler än 80/160 MHz).

Exempel: *En elev köper en laptop med 802.11ax (2×2) och får stabil prestanda i 5 GHz-bandet i skolan, även när många är uppkopplade samtidigt.*

Kontrollfrågor

1. Vilken funktion fyller FCS (Frame Check Sequence) i en Ethernet-ram?
2. Hur används en MAC-adress på datalänk-lagret (lager 2) för att leverera data till rätt enhet?
3. Beskriv processen när en enhet skickar en ARP-förfrågan (ARP request).
4. Vad är standard-MTU för en Ethernet-ram och vad händer om en enhet i nätverket inte stödjer "Jumbo Frames"?
5. Vilken PoE-standard (802.3) bör användas om en accesspunkt kräver 25 W effekt för att fungera?
6. Förklara skillnaden mellan CSMA/CD och CSMA/CA. Varför behövs olika metoder för trådbunden respektive trådlös kommunikation?
7. Varför uppstår det i praktiken inga kollisioner i ett modernt switchat nätverk som kör full duplex (full-duplex)?
8. Nämn två specifika orsaker till att svarstiden (ping) ofta är högre i ett WLAN jämfört med Ethernet.
9. Vad är en "duplex-mismatch" och vilka symptom kan det ge i nätverket?
10. Vad innebär det att en trådlös kanal är ett halv duplex-medium (half-duplex)?

Fördjupningslänkar

IEEE 802.3 Working Group (Officiell webbplats för utveckling av Ethernet-standarder) - <https://www.ieee802.org/3/>

Wi-Fi Alliance (Information om certifieringar och tekniska framsteg inom Wi-Fi) - <https://www.wi-fi.org/>

Cisco: Ethernet Technologies (Teknisk djupdykning i Ethernet-ramar och fysiska lager) - <https://www.cisco.com/c/en/us/support/docs/lan-switching/ethernet/10561-ethernet-tech.html>

PoE Standards Overview (Detaljerad jämförelse mellan 802.3af, at och bt) - <https://www.electronics-notes.com/articles/connectivity/ethernet-ieee-802-3/poe-power-over-ethernet-standards.php>

Wireshark Wiki: Ethernet (Praktisk genomgång av hur Ethernet-trafik analyseras i paketsnivå) - <https://wiki.wireshark.org/Ethernet>

Cloudflare: What is Network Latency? (Förklaring av latens, ping och faktorer som påverkar hastighet) - <https://www.cloudflare.com/learning/network-layer/what-is-latency/>

Computer History Museum: The History of Ethernet (Berättelsen om hur Ethernet skapades vid Xerox PARC) - <https://www.computerhistory.org/revolution/networking/19/374>

GeeksforGeeks: CSMA/CD vs CSMA/CA (Tydlig jämförelse mellan metoderna för mediedelning [medium access]) - <https://www.geeksforgeeks.org/difference-between-csma-ca-and-csma-cd/>

Practical Networking: How ARP Works (Visuell och pedagogisk guide till Address Resolution Protocol) - <https://www.practicalnetworking.net/series/arp/how-arp-works/>

Troubleshooting Duplex Issues (Teknisk guide för att identifiera och åtgärda felmatchning av duplex) - <https://community.cisco.com/t5/networking-knowledge-base/troubleshooting-ethernet-collisions-and-duplex-issues/ta-p/3116172>

Egna anteckningar

2 Planering och design av nätverk

Innan ett nät byggs, fysiskt, är det alltid av största vikt att ha en plan. Denna plan fyller funktionen av Vad som skall göras, Hur det skall göras och Varför det skall utformas på det sätt. Genom att ha en bra och stabil plan blir det lätt att se fördelar och brister, felsöka nätverket och se vilka svårigheter som finns vid en eventuell förändring av nätverket. Många nätverk är bra och välstrukturerade inledningsvis, och anpassade efter behovet som finns när de byggs. Men i och med att strukturer förändras, kundflöden varierar, anställda ökar och minskar och mycket annat, så behöver även nätverket vara möjligt att anpassa utefter nuvarande och framtida behov, detta genom att behöva göra så små ingrepp i infrastrukturen som möjligt.

2.1 Mål och krav

Designen av ett robust nätverk inleds med en grundlig behovsanalys (requirements analysis) för att fastställa nätverkets syfte och användningsområden. Processen innebär en kartläggning av användarnas beteendemönster, vilket inkluderar vilka applikationer som prioriteras, antalet samtidiga klienter (concurrent clients) och från vilka fysiska platser åtkomst krävs. Denna kravbild utgör fundamentet för tekniska beslut rörande:

Topologi (topology): den fysiska och logiska utformningen av nätverket.

Adressering och segmentering (segmentation): planering av IP-adresser och hur nätverkstrafiken fördelas och isoleras.

Brandväggspolicys (firewall policies): definitioner av vilka tjänster som ska tillåtas och hur åtkomstbegränsningar ska implementeras.

Trådlös kapacitet (wireless capacity): säkerställande av god täckning och metoder för att hantera klientbelastning utan prestandaförlust.

Genom en tydlig kravspecifikation minimeras risken för kostsamma ändringar i ett senare skede, samtidigt som det blir möjligt att verifiera att

den slutgiltiga lösningen uppfyller verksamhetens förväntningar. Kraven delas tekniskt in i följande kategorier:

Funktionella krav (functional requirements): omfattar konkreta tjänster såsom röstsamtal via nätverket (VoIP), videoströmning, DHCP och DNS, samt behovet av segmentering via virtuella lokala nätverk (VLAN).

Icke-funktionella krav (non-functional requirements): fokuserar på nätverkets inneboende egenskaper, såsom bandbredd (bandwidth), latens (latency), tillgänglighet (availability), säkerhetsnivå, skalbarhet (scalability) och budgetramar.

Gränssnitt (interfaces): rör externa anslutningar och villkor, vilket inkluderar internetkapacitet, tillgång till publika IP-adresser, behov av krypterade tunnlar mellan platser (site-to-site VPN) samt leverantörsavtal (SLA).

Ett konkret exempel på en kravbild för en gymnasieskola kan innefatta stöd för 600 samtidiga klienter fördelade på strikt separerade segment för elever, personal och gäster. Tekniska mål kan i detta scenario innefatta en latens under 10 ms inom campusområdet, central loggning av trafik för spårbarhet samt en garanterad tillgänglighet på 99,5 % under schemalagd skoltid.

2.1.2 Arkitektur och topologi

Vid utformningen av nätverksarkitekturen eftersträvas en modell som är begriplig, robust och underhållsvänlig. En vedertagen metod för större miljöer, såsom skolor och campusområden, är den hierarkiska modellen (hierarchical network design). Genom att dela upp nätverket i tre logiska lager underlättas både felsökning och framtida expansion. Denna arkitektur styrs av följande huvudkomponenter:

Core (kärnlager): nätverkets ryggrad (backbone) som ansvarar för snabb transport av stora datamängder mellan olika byggnader och zoner. Här prioriteras hastighet och tillgänglighet framför avancerad paketfiltrering.

Distribution (distributionslager): fungerar som en länk mellan kärnlageret och användarna. Här hanteras routing mellan olika virtuella nätverk (VLAN) samt implementering av säkerhetspolicys.

Access (accesslager): den del där klienter, skrivare och accesspunkter ansluts till nätverket. Logiskt används oftast en stjärntopologi (star topology) med redundanta uppkopplingar (uplinks) uppåt i hierarkin.

För att säkerställa driftsäkerhet implementeras ofta redundans genom dubbla länkar eller enheter. Detta medför ett behov av protokoll för loopskydd (loop protection), såsom STP, RSTP eller MSTP, för att förhindra att trafik cirkulerar okontrollerat i redundanta slingor.

Beslutet om var routing (L3) ska ske påverkar nätverkets skalbarhet. Genom att använda L3-switchar i distributionslagret och konfigurera virtuella gränssnitt (SVI - Switched Virtual Interface) kan routing ske lokalt och snabbt. Alternativt kan routing skötas centralt i en brandvägg eller router om all trafik kräver strikt granskning.

Vid designval bör följande principer beaktas:

Hierarki: konsekvent användning av core, distribution och access för att skapa en förutsägbar trafikmiljö.

Redundans: dubblering av kritiska enheter och länkar där avbrott innebär stora konsekvenser för verksamheten.

Routing-placering: val av L3-placering i distribution eller centralt beroende på krav på prestanda och säkerhetskontroll.

Undvikande av kedjekoppling (daisy chaining): långa kedjor av sammankopplade accessswitchar bör undvikas, då ett fel tidigt i kedjan isolerar alla bakomliggande enheter. Trafik bör istället samlas upp direkt i distributionslagret.

Ett praktiskt exempel på denna arkitektur är en fastighet med två distributionsswitchar (konfigurerade som en logisk enhet via stack eller vPC), där accessswitchar på respektive våningsplan ansluts med dubbla fiberlänkar för att eliminera enskilda felkällor (single point of failure).

2.1.3 Hierarkisk design och nätverksnivåer (Tiers)

Den hierarkiska designen delas tekniskt upp i tre nivåer, ofta benämnda som "tiers". Varje nivå har specifika uppgifter och ställer olika krav på hårdvarans prestanda och funktionalitet. Genom att separera funktionerna skapas ett nätverk som är enklare att hantera, skala och felsöka. Denna

modell är universell och används för att beskriva allt från lokala företagsnätverk till hur de största internetleverantörerna samverkar globalt.

Nivå 3: Accesslagret (Access Tier) Accesslagret utgör nätverkets ytterkant (edge) och är den nivå där slutanvändare och kringutrustning ansluter. Fokus ligger på att erbjuda hög portdensitet och grundläggande säkerhetsfunktioner för att skydda nätverket mot felaktiga anslutningar. Ett större lokalt nätverk (LAN) på ett våningsplan, där hundratals enheter ansluter via vägguttag eller trådlöst, är ett typiskt exempel på en miljö på nivå 3.

- **Typisk hårdvara:** Switchar med 24 eller 48 portar och stöd för strömförsörjning (PoE/PoE+) för att driva enheter utan extra eluttag.
- **Säkerhet:** Implementering av portskydd (port security) för att begränsa antalet tillåtna MAC-adresser per port, samt 802.1X-autentisering för identitetskontroll.
- **Exempel:** En accessswitch i ett klassrum som strömförsörjer trådlösa accesspunkter och ansluter stationära elevdatorer till rätt virtuellt nätverk (VLAN).

Nivå 2: Distributionslagret (Distribution Tier) Distributionslagret fungerar som en länk mellan accesslagret och kärnan. Här aggregeras trafik från ett flertal accessswitchar, och det är på denna nivå nätverkets logiska regler och styrning implementeras. Nivån utgör ofta gränsen för routing (Layer 3 boundary). En skola med en välplanerad arkitektur fungerar som en typisk nivå 2-miljö, där distributionslagret samlar upp trafiken från samtliga klassrum och byggnader för vidare befordran.

- **Trafikstyrning:** Hantering av kommunikation mellan olika virtuella nätverk genom inter-VLAN routing och virtuella gränssnitt (SVI).
- **Policyimplementering:** Användning av åtkomstlistor (ACL - Access Control Lists) för att styra vilken trafik som får flöda mellan segment, samt prioritering av trafik genom tjänstekvalitet (QoS - Quality of Service).
- **Exempel:** En central switch i en byggnad som samlar upp fiberlänkar från alla våningsplan, sköter DHCP-vidarebefordran (DHCP Relay) till centrala servrar och ser till att gästtrafik isoleras från personalnätet.

Nivå 1: Kärnlagret (Core Tier) Kärnlagret utgör nätverkets centrala ryggrad (backbone). Dess enda syfte är att transportera stora datamängder mellan olika nätverksdelar eller distributionspunkter med minsta möjliga fördröjning (latency). Här undviks komplexa processer som kan sänka hastigheten. På den globala nivån representeras nivå 1 av de stora internetleverantörerna (ISP), såsom Telia (Arelion), Tele2 och Telenor, som äger och driver de ryggradsnät som kopplar samman länder och kontinenter.

- **Prestanda:** Användning av höghastighetsanslutningar, ofta via fiberoptik med kapaciteter på 10, 40 eller 100 Gbps.
- **Tillgänglighet:** Extremt hög redundans genom dubblerade enheter och länkar som möjliggör drift även vid hårdvarufel.
- **Exempel:** Två kraftfulla kärnswitchar placerade i ett centralt serverrum som kopplar samman olika skolbyggnader i ett campusnätverk. Kärnlagret ser till att trafik från biblioteket når skolans centrala lagringsserver utan flaskhalsar.

I mindre nätverksmiljöer används ibland en förenklad modell som kallas för en sammanvuxen kärna (collapsed core). I denna design slås kärnlagret och distributionslagret samman till en gemensam enhet för att reducera kostnader och komplexitet, samtidigt som den hierarkiska fördelen mot accesslagret bibehålls.

2.1.4 IP-plan, VLAN och namnstandard

2.1.4 IP-planering, VLAN-struktur och namnstandard

En välstrukturerad plan för IP-adresser och virtuella nätverk (VLAN) utgör fundamentet för en skalbar och lätthanterlig nätverksmiljö. Utan en konsekvent metodik ökar risken för adresskollisioner, säkerhetsbrister och svåröverskådlig dokumentation. Arbetet inleds med att reservera en privat adressrymd (private address space) enligt RFC1918, exempelvis 10.0.0.0/8, vilken sedan delas upp i logiska enheter baserat på geografisk plats, såsom byggnader, eller specifika funktioner.

Varje virtuellt nätverk (VLAN) fungerar som en isolerad broadcastdomän, och för att möjliggöra kommunikation krävs att varje VLAN knyts till ett unikt subnät (subnet). Genom att använda en konsekvent IP-plan kan

tekniker utläsa en enhets funktion eller placering direkt genom dess IP-adress. Följande principer bör ligga till grund för utformningen:

VLAN-konventioner (VLAN conventions): Varje roll i nätverket tilldelas ett unikt VLAN-ID. Det är god praxis att använda ett logiskt numreringsschema som är lätt att komma ihåg och dokumentera. Exempelvis kan VLAN 10 reserveras för personalens administrativa trafik, VLAN 20 för elever, VLAN 30 för gäståtkomst, VLAN 40 för servrar och VLAN 50 för hantering av accesspunkter.

Dimensionering och subnettering (dimensioning): Varje subnät måste dimensioneras utifrån antalet samtidiga klienter med en inbyggd marginal för framtida tillväxt. I miljöer med hög klienttäthet, såsom trådlösa nätverk för elever, krävs ofta större prefix (exempelvis /22 som ger 1022 adresser) för att säkerställa att adresserna i DHCP-poolen inte tar slut under perioder med hög belastning.

Adresshantering och tjänster (IPAM & services): För varje subnät definieras en fast gateway-adress, vilket vanligtvis är den första eller sista användbara adressen i intervallet. Vidare konfigureras DHCP-inställningar (DHCP scopes) med relevanta optioner. Dessa optioner instruerar klienten om vilken router som ska användas, vilka DNS-servrar som ska sköta namnuppslagning samt vilken NTP-server som används för tidssynkronisering.

Namnstandard (naming convention): En enhetlig namnstandard för nätverksutrustning som switchar, accesspunkter och servrar är avgörande för effektiv övervakning och felsökning. Namnet bör koda information om enhetens fysiska placering, dess roll i den hierarkiska modellen och ett unikt löpnummer. Detta möjliggör snabb identifiering i loggfiler och vid larmhantering utan att tekniker behöver konsultera externa ritningar.

Exempel: Vid konfigurationen av elevsegmentet på en skola. Här kan VLAN 20 tilldelas subnätet 10.20.0.0/22. Gateway-adressen sätts till 10.20.0.1 och DHCP-intervallen konfigureras mellan 10.20.0.50 och 10.20.3.200. DNS-optionerna pekar i detta fall mot nätverkets Active Directory-servrar (AD) på 10.40.0.10 och 10.40.0.11 för att säkerställa korrekt funktionalitet inom domänen.

Enligt namnstandarderna kan en accessswitch placerad på tredje våningen i hus 1 då ges namnet HUS1-SW-A3.

2.1.5 Fysisk infrastruktur

2.1.5.1 Hårdvara

Den fysiska infrastrukturen utgör fundamentet för en tillförlitlig nätverksdrift. Utformningen av serverrum och korskopplingsskåp (wiring closets) påverkar direkt nätverkets livslängd, säkerhet och underhållsvänlighet. En oorganiserad miljö försvårar felsökning och ökar risken för mänskliga fel vid servicearbeten. För att säkerställa en stabil miljö bör följande tekniska aspekter beaktas vid planeringen:

Organisering och märkning (patching and labeling):

Racksystem och patchpaneler ska vara logiskt märkta och strukturerade. Genom att separera koppar- och fiberkablage samt använda horisontella och vertikala kabelhanterare (cable managers) skapas en överskådlig miljö där service kan utföras effektivt utan att störa intilliggande utrustning.

Märkningen bör vara hierarkisk och utgå från den fysiska platsen. En vedertagen metod är att använda ett koordinatsystem som identifierar rum, rack, panel och port.

- **Format för uttagsmärkning:** [Rack-ID] – [Panel-ID] : [Portnummer]
 - **Exempel:** A1-P02:24
 - **Betydelse:** Uttaget leder till rack A1, patchpanel 02, port nummer 24.
- **Märkning i båda ändar:** Varje patchkabel (patch cord) märks i båda ändar med information om var den andra änden är ansluten. Detta eliminerar behovet av att manuellt spåra kablar genom trånga kabelkanaler.
- **Uttag i vägg:** Uttag vid användarens arbetsplats märks med exakt samma beteckning som motsvarande port i korskopplingsskåpet.

Strukturen i ett rack bör planeras för att minimera korsande kablage och optimera luftflödet för kylning.

Interleaving (Varvning): En effektiv metod är att placera en switch mellan två patchpaneler. Detta möjliggör användning av mycket korta patchkablar (ofta 15–20 cm) som går direkt från panelen till switchen, vilket minskar behovet av stora horisontella kabelhanterare och ger en extremt ren installation.

Kabelhanterare (Cable managers):

- **Horisontella:** Används för att leda kablar från en switchport till rackets sidor.
- **Vertikala:** Stora kanaler på sidorna av racket där överskottskabel förvaras och leds uppåt eller nedåt.

Färgkodning (Color coding): Genom att använda olika färger på patchkablarna kan olika tjänster identifieras visuellt på långt håll:

- **Blå:** Standard datatrafik (klienter).
- **Röd:** Kritiska anslutningar (servrar/uplinks).
- **Gul:** Trådlösa accesspunkter (PoE-enheter).
- **Grön:** Övervakningskameror eller säkerhetssystem.

Den fysiska märkningen är endast effektiv om den åtföljs av en uppdaterad dokumentation (rack diagram). Detta inkluderar:

Rackritning: En visuell översikt som visar exakt vilken utrustning som är monterad på vilken höjdenhet (U - Unit).

Patch-schema: En förteckning eller databas som visar förbindelsen mellan varje vägguttag och switchport, inklusive vilket VLAN som är konfigurerat på porten.

Exempel: *En felsökningsprocess i en välmärkt miljö är när en användare rapporterar fel på uttag C1-P01:12. Teknikern kan då gå direkt till rack C1, hitta panel 1 och port 12, se på färgen på kabeln att det rör sig om en standardklient, och omedelbart identifiera vilken switchport som är kopplad till uttaget utan att behöva gissa eller dra i kablar.*

2.1.5.2 Kablage

Kablageval (cabling): Valet av kablage styrs av krav på överföringshastighet, fysiska avstånd och den omgivande miljöns elektriska egenskaper.

Access-lagret: I anslutningar till klienter och accesspunkter används vanligtvis partvinnad kopparkabel (twisted pair) av kategorin Cat6 eller Cat6a, vilket möjliggör hastigheter upp till 10 Gbps över begränsade sträckor.

Skärmning (shielding): Utöver kategorivalet ställs krav på hur väl kabeln ska skyddas mot yttre störningar. Här skiljer man på två huvudtyper:

- **UTP (Unshielded Twisted Pair):** Oskärmad kabel som är standard i de flesta kontors- och skolmiljöer. Den är flexibel, kostnadseffektiv och kräver ingen särskild jordning i nätverksutrustningen för att fungera korrekt.
- **STP (Shielded Twisted Pair):** Skärmad kabel utrustad med metallfolie eller fläta för att skydda signalen mot elektromagnetiska störningar (EMI - Electromagnetic Interference). Denna typ är nödvändig i industriella miljöer eller där nätverkskablage dras tätt intill kraftiga elkablar och motorer. Användning av STP kräver att hela infrastrukturen, inklusive kontakter och patchpaneler, är korrekt jordad för att skärmningen inte ska fungera som en antenn för störningar.

Backbone: För anslutningar mellan switchar och byggnader används fiberoptik (fiber optics). Eftersom fiber överför data med ljus istället för elektricitet är den helt immun mot elektromagnetiska störningar, vilket gör den idealisk för ryggradsnätet.

Multimode-fiber (MMF): Använder en bredare kärna och är optimerad för kortare avstånd (upp till ca 500 meter) inom byggnader eller mellan närliggande skåp.

Singlemode-fiber (SMF): Har en mycket tunn kärna och kräver laser som ljuskälla. Den används för långa sträckor (flera mil) eller där extremt hög kapacitet krävs mellan geografiskt skilda noder.

2.1.5.3 Övrigt

Strömförsörjning och PoE-budget: Vid dimensionering av switchar i accesslagret krävs en noggrann beräkning av den totala effektförbrukningen för anslutna enheter (PoE budget). Accesspunkter, övervakningskameror och IP-telefoner belastar switchens nätaggregat olika mycket beroende på deras klassificering (802.3af/at/bt). För kritisk utrustning bör avbrottsfri kraftförsörjning (UPS - Uninterruptible Power Supply) installeras för att skydda mot spänningsspikar och korta strömbavbrott.

Driftmiljö och skydd: Serverrum och apparatskåp kräver adekvat kylning (cooling) för att transportera bort den värme som aktiv utrustning genererar. Dessutom ska fysisk låsbarhet och skyddade kabelvägar (conduits/cable trays) planeras för att förhindra obehörig åtkomst eller oavsiktlig skada på infrastrukturen.

Vid nyinstallation är det en ekonomisk och teknisk fördel att planera för en expansionsmarginal på 10–30 %. Att installera extra fiberpar eller lämna tomma platser i patchpaneler vid det initiala arbetet är betydligt mer kostnadseffektivt än att utföra kompletteringar i ett senare skede.

Exempel: Vid en fysisk planering på ett våningsplan med 40 arbetsplatser och 5 accesspunkter. Här installeras en switch med en PoE-budget på minst 180 W (för att täcka AP-last och framtida behov) i ett låsbart väggskåp. Skåpet utrustas med en mindre UPS och kylfläktar, och kopplas till byggnadens centrala kärnswitch via en redundant multimode-fiberlänk för att säkerställa högsta möjliga tillgänglighet.

2.1.6 WLAN-design (kanal, täckning, kapacitet)

Vid utformningen av trådlösa lokala nätverk (WLAN) prioriteras kapacitet och användarupplevelse framför enbart täckning. En modern design kräver en djup förståelse för hur radiovågor interagerar med den fysiska miljön och hur tillgänglig bandbredd fördelas mellan klienterna. Målet är att skapa en miljö där anslutningen är stabil även vid hög klienttätet.

2.1.6.1 Platsundersökning och miljöanalys

Grundbulten i en trådlös design är en noggrann platsundersökning (site survey). Genom denna kartläggs de fysiska lokalerna för att identifiera faktorer som påverkar signalstyrka och kvalitet:

Väggmaterial och dämpning (attenuation): Olika material som betong, glas och metall påverkar radiovågornas förmåga att tränga igenom eller reflekteras.

Störkällor (interference): Identifiering av icke-nätverksrelaterade störningar, exempelvis mikrovågsugnar eller Bluetooth-enheter, samt närliggande trådlösa nätverk.

Användartäthet: Analys av var användare förväntas befinna sig under belastningstoppar, vilket styr placeringen och antalet accesspunkter (AP).

2.1.6.2 Frekvensband och kanalstrategi

Valet av frekvensband och kanaler är avgörande för nätverkets totala prestanda. Genom att använda olika band kan nätverket optimeras för både räckvidd (range) och hastighet:

- **2,4 GHz-bandet (802.11b/g/n):** Erbjuder lång räckvidd och god genomträngningsförmåga men har ett begränsat antal icke-överlappande kanaler (1, 6 och 11). Bandet är ofta hårt belastat och mer känsligt för störningar (interference). Detta band bör främst användas för IoT-enheter (Internet of Things) och inte för ordinarie datatrafik. En realistisk genomströmning i detta band ligger ofta mellan 30–60 Mbit/s.
- **5 GHz-bandet (802.11ac/ax):** Ger betydligt fler kanaler och högre kapacitet, men med kortare räckvidd jämfört med 2,4 GHz. Vissa kanaler inom detta band kräver dynamiskt frekvensval (DFS – Dynamic Frequency Selection) för att undvika störningar med exempelvis väderradar. Detta band är mer anpassat för ordinarie datatrafik, där en realistisk hastighet ligger mellan 300–600 Mbit/s (teoretiskt maximalt 866 Mbit/s eller 1,7 Gbit/s beroende på standard och antal dataströmmar).
- **6 GHz-bandet (802.11ax/Wi-Fi 6E):** Introducerar ett stort antal breda och helt rena kanaler, vilket möjliggör extremt hög kapacitet. Räckvidden är dock kortast av de tre banden, vilket kräver en tätare

placering av accesspunkter (access points). Bandet är optimerat för ordinarie datatrafik med en förväntad hastighet omkring 1,2 Gbit/s.

- **Wi-Fi 7 (802.11be):** Denna nya standard på 6 GHz-bandet kan använda ännu bredare kanaler (320 MHz) och nå teoretiska hastigheter på över 5 Gbit/s till en enskild enhet, samt uppåt 40 Gbit/s för en hel accesspunkt genom aggregering av flera band.

Kanalbredden (channel width) i miljöer med många accesspunkter, såsom skolor, bör begränsas till 20 eller 40 MHz. Detta möjliggör en mer effektiv frekvensåteranvändning (frequency reuse) och minskar risken för ko-kanalsstörning (co-channel interference), där accesspunkter på samma kanal saktar ner varandra.

2.1.6.3 SSID-hantering och säkerhet

För att minimera administrativ overhead bör antalet trådlösa nätverksnamn (SSID) hållas lågt, vanligtvis mellan två och fyra. Varje SSID skickar ut kontrollpaket (beacons) flera gånger per sekund, vilket vid ett stort antal SSID kan konsumera en betydande del av den tillgängliga sändningstiden (airtime).

Trafiken separeras logiskt genom att mappa varje SSID till ett specifikt virtuellt nätverk (VLAN). En vanlig strategi innefattar separata SSID för personal, elever och gäster. Säkerhetsnivån bör alltid vara den högsta möjliga som enheterna stödjer, företrädesvis WPA3 eller WPA2-Enterprise, medan osäkra protokoll som WEP eller TKIP konsekvent bör undvikas.

2.1.6.4 Roaming och prestandaoptimering

För en sömlös användarupplevelse krävs att klienter kan flytta mellan accesspunkter utan avbrott, en process som kallas roaming. Detta kräver en viss cellöverlappning (ofta 15–20 %) så att klienten kan identifiera en ny AP innan anslutningen till den gamla bryts.

- **Min-RSSI:** Genom att konfigurera en lägsta tillåtna signalstyrka (Minimum RSSI) kan nätverket tvinga klienter att släppa en svag anslutning och istället söka efter en starkare accesspunkt i närheten.
- **Band Steering:** En teknik som aktivt uppmuntrar klienter som stödjer 5 GHz eller 6 GHz att använda dessa band istället för det mer belastade 2,4 GHz-bandet.

Exempel: En genomtänkt konfiguration är en skola där personalens SSID (VLAN 10) använder certifikatbaserad inloggning, medan elevernas SSID (VLAN 20) prioriteras i 5 GHz-bandet med 40 MHz kanalbredd för att balansera hastighet och stabilitet.

2.1.7 Säkerhetsdesign och zoner

Säkerhet bör betraktas som en integrerad del av nätverksdesignen från projektstart (security by design). Genom att dela upp nätverket i logiska zoner begränsas rörelsefriheten för en potentiell angripare och risken för spridning vid en säkerhetsincident minimeras. Detta koncept, ofta kallat segmentering (segmentation), innebär att trafik mellan olika delar av nätverket måste passera en kontrollpunkt, vanligtvis en brandvägg, där strikta regler tillämpas.

2.1.7.1 Zonindelning och brandväggspolicys

En effektiv säkerhetsarkitektur bygger på att definiera zoner baserat på tillit och funktion. För varje zon definieras brandväggspolicys enligt principen om minsta möjliga åtkomst (principle of least privilege). Vanliga zoner inkluderar:

LAN (klientnätverk): Denna zon består ofta av ett eller flera lokala nätverk (LAN/VLAN). Syftet är att gruppera betrodda interna enheter, såsom personalens datorer eller elevernas laptops, i separata segment för att kontrollera trafikflödet mellan dem.

Serverzon: Ett skyddat område för kritiska resurser som databaser och filservrar. Åtkomst hit begränsas till specifika portar och tjänster (t.ex. TCP 445 för fildelning).

DMZ (Demilitarized Zone): En zon för tjänster som måste vara nåbara från internet, såsom webbserverar eller externa e-postserverar. Enheterna i DMZ har normalt ingen tillåtelse att initiera anslutningar inåt mot det interna nätverket.

Gästnätverk: Ett helt isolerat segment för externa besökare. Trafik tillåts vanligtvis endast mot internet, medan all kommunikation mot interna zoner (LAN/Servrar) blockeras.

Managementzon (hanteringszon): En dedikerad zon för nätverksutrustningens administrativa gränssnitt (switchar, routrar,

brandväggar). Denna zon ska isoleras från vanliga klientnätverk och endast vara nåbar från specifika administrativa subnät eller via krypterad fjärranslutning (VPN).

2.1.7.3 Övervakning och loggning

En säkerhetsdesign är inte komplett utan förmågan att upptäcka och analysera avvikelser i realtid.

Central loggning (Syslog): All nätverksutrustning bör konfigureras för att skicka händelseloggar till en central server (Syslog server). Vanliga verktyg för detta är **rsyslog** eller **Graylog**. En loggfil kan exempelvis visa exakt när en switchport gick ner eller när en användare misslyckades med att logga in i administrationsgränssnittet.

- För en djupare genomgång av hur operativsystemet genererar och hanterar interna loggfiler, se boken **Datorteknik**.

SIEM (Security Information and Event Management): I mer avancerade miljöer används system som samlar in och korrelerar data från flera källor. Ett populärt verktyg inom öppen källkod är **Wazuh**, medan **Splunk** ofta används i större företagsmiljöer. En SIEM kan exempelvis upptäcka ett mönster där tio misslyckade inloggningsförsök följs av en lyckad inloggning från en ovanlig IP-adress, och omedelbart larma teknikern om ett pågående intrångsförsök.

2.1.8 Drift, övervakning och livscykel

Ett nätverk bör designas för effektiv drift (operations) från dag ett. En tekniskt avancerad lösning tappar snabbt sitt värde om den är svår att övervaka, underhålla eller felsöka. Genom att implementera fasta rutiner och automatiserade verktyg säkerställs nätverkets tillgänglighet och livslängd.

2.1.8.1 Standardisering och grundkonfiguration

För att minimera komplexiteten i en driftsmiljö krävs en standardiserad grundkonfiguration (base configuration) för all nätverksutrustning. Detta innebär att samtliga enheter, oavsett placering, följer samma regelverk för:

Tidssynkronisering (NTP - Network Time Protocol):

Säkerställer att alla loggar har korrekta tidsstämplar, vilket är kritiskt vid felsökning och säkerhetsanalys.

Fjärråtkomst och säkerhet: Implementering av säkra protokoll som SSH istället för Telnet, samt central autentisering via AAA (Authentication, Authorization, and Accounting).

Information och banners: Standardiserade meddelanden som visas vid inloggning för att informera om nätverkets ägare och gällande policys.

2.1.8.2 Övervakning och larmhantering

Övervakning (monitoring) handlar om att proaktivt identifiera problem innan de påverkar användarna. Detta sker ofta via protokoll som **SNMP (Simple Network Management Protocol)** eller modernare strömmande telemetri (telemetry). Systemet bör konfigureras för att övervaka kritiska parametrar och generera larm vid överskridna gränsvärden:

Fysisk status: Kontroll av fläktar, temperatur och nätaggregatens hälsa.

Kapacitet: Övervakning av processorns belastning (CPU), minnesanvändning (RAM) och tillgänglig PoE-budget.

Nätverksprestanda: Analys av bandbreddsutnyttjande på upplänkar (uplinks), paketförluster och trådlös sändningstid (airtime).

Exempel på populära programvaror för övervakning är **Zabbix**, **LibreNMS** och **PRTG**. Dessa verktyg kan visualisera nätverkets status i realtid och skicka aviseringar via e-post eller meddelandetjänster vid incidenter.

2.1.8.3 Underhåll, backup och ändringshantering

Livscykelhantering innebär att nätverket kontinuerligt uppdateras och säkras.

Konfigurationsbackup: Automatiserade backuper av enheternas konfigurationsfiler bör ske nattligt. Verktyg som **Oxidized** eller **Rancid** kan användas för att inte bara spara kopior, utan även

versionshantera ändringar så att tekniker kan se exakt vad som ändrats mellan två tidpunkter.

Firmware-uppgraderingar: Regelbundna uppdateringar av nätverksutrustningens mjukvara är nödvändiga för att täppa till säkerhetshål och introducera ny funktionalitet.

Ändringshantering (Change Management): Varje större ändring i nätverket bör följa en fastställd process med en tydlig dokumentation av syftet, ett tidsfönster för genomförandet och en plan för att återställa nätverket (rollback plan) om något går fel.

2.1.8.4 Dokumentation och tillgångshantering

En uppdaterad dokumentation är nätverksteknikerns viktigaste verktyg. Detta inkluderar en logisk topologiritning, en aktuell IP- och VLAN-plan samt en matris över brandväggsregler (ACL matrix). För att hålla ordning på hårdvaran används ofta en enkel **CMDB (Configuration Management Database)** eller assetlista som innehåller:

- Fysisk plats och rackenhet.
- Serienummer och inköpsdatum.
- IP-adress och management-VLAN.
- Supportstatus och garantitid (End-of-Life/End-of-Support).

Ett kraftfullt verktyg för att kombinera dokumentation av IP-planering och fysisk infrastruktur är **NetBox**, vilket fungerar som en central källa för sanning (Source of Truth) för hela nätverksmiljön.

För en djupare förståelse av hur serverhårdvara och operativsystemets livscykel hanteras, se boken **Datorteknik**.

2.1.9 Test, validering och acceptans

Innan ett nätverk övergår i full drift krävs en systematisk process för testning och validering. Syftet är att objektivt bevisa att den tekniska lösningen uppfyller de kravspecifikationer (requirements specifications) som fastställdes i nätverkets planeringsfas. Genom att genomföra ett strukturerat acceptanstest (acceptance testing) säkerställs att alla funktioner arbetar korrekt under både normal belastning och vid simulerade felscenarier.

2.1.9.1 Funktionstester och policyverifiering

Det första steget i valideringen är att kontrollera de grundläggande nätverkstjänsterna. Detta innefattar verifiering av adressering, namnupplösning och åtkomstkontroll:

Connectivity och VLAN-reachability: Kontroll av att enheter i rätt segment kan nå varandra och att kommunikation mellan olika virtuella nätverk (inter-VLAN routing) fungerar enligt designen.

DHCP och DNS: Test av adressutdelning och kontroll av att korrekta inställningar, såsom gateway och DNS-servrar, levereras till klienterna.

Brandväggspolicys: Verifiering av att trafik som ska blockeras nekas, och att endast tillåten trafik når skyddade zoner. Här används ofta verktyg som **Nmap** för att skanna portar och säkerställa att inga oavsiktliga öppningar finns i brandväggsreglerna.

2.1.9.2 Prestandamätning och kapacitet

När funktionaliteten är bekräftad genomförs mätningar av nätverkets faktiska prestanda (performance measurement). Detta görs för att säkerställa att bandbredd, latens (latency) och jitter (variation i fördröjning) ligger inom fastställda gränsvärden.

Genomströmning (Throughput): Mätning av nätverkets förmåga att transportera data under hög belastning. Programvaran **iPerf3** är en branschstandard för att mäta maximal TCP- och UDP-genomströmning mellan två noder.

Latens och Jitter: Detta är särskilt kritiskt för realtidstjänster som röstsamtal (VoIP) och videokonferenser. Verktyg som **NetBeez** eller inbyggda funktioner i nätverksutrustningen kan användas för att kontinuerligt mäta svarstider och variationer i nätverket.

2.1.9.3 Redundanstester och failover

För att validera nätverkets robusthet måste redundansmekanismerna utsättas för stresstester. Det handlar om att simulera hårdvaru- eller länkfel och mäta hur snabbt nätverket återhämtar sig (convergence time).

STP/RSTP-konvergens: Verifiering av att loopskyddet aktiverar alternativa vägar vid ett kabelbrott utan att orsaka broadcast-stormar.

Gateway-redundans: Om protokoll för redundanta gateways som **VRRP** (Virtual Router Redundancy Protocol) eller **HSRP** (Hot Standby Router Protocol) används, testas växling (failover) mellan routrar för att säkerställa att klienterna bibehåller sin internetanslutning vid ett maskinfel.

Uplink-failover: Simulering av fiberbrott mellan distributions- och kärnswitchar för att bekräfta att trafiken automatiskt omdirigeras via sekundära länkar.

2.1.9.4 WLAN-validering under belastning

Trådlösa nätverk kräver specifik validering för att säkerställa täckning och kapacitet i hela den planerade miljön.

Roaming-test: Verifiering av att en rörlig klient kan flytta mellan accesspunkter utan märkbara avbrott i pågående sessioner. Verktyg som **WiFiman** eller **Ekahau** kan användas för att visualisera signalstyrka och roaming-beteende.

Lasttest (Stress test): Genomförande av mätningar under realistisk belastning, exempelvis under en lektion där många klienter streamar video samtidigt, för att observera sändningstid (airtime) och eventuell kanalinterferens.

2.1.9.5 Acceptansprotokoll och dokumentation

Samtliga testresultat sammanställs i ett formellt acceptansprotokoll (acceptance protocol). Detta dokument fungerar som ett kvitto på att installationen uppfyller de funktionella och icke-funktionella kraven. Dokumentationen bör även inkludera instruktioner för återställning (rollback plan) vid framtida uppgraderingar av systemet.

Exempel: Företaget Roynes Garn & IT simulerar ett fiberbrott mellan en distributionsswitch och kärnnätet. Om loggarna i övervakningssystemet visar att failover skedde på under en sekund och pågående applikationer förblev opåverkade, anses redundanskravet vara uppfyllt.

Kontrollfrågor

1. Vad är skillnaden mellan funktionella och icke-funktionella krav i en behovsanalys (requirements analysis)?
2. Vilka är de tre huvudlagren i den hierarkiska nätverksmodellen (hierarchical network design)?
3. Vilken specifik funktion har distributionslagret (distribution tier) i förhållande till accesslagret?
4. Varför beskrivs kärnlagret (core tier) som nätverkets ryggrad (backbone) och vad prioriteras där?
5. Vad innebär begreppet "collapsed core" och i vilka miljöer är det lämpligt?
6. Varför är en genomtänkt namnstandard (naming convention) avgörande vid felsökning i större nätverk?
7. Vad är den tekniska skillnaden mellan UTP- och STP-kablage gällande störningsskydd och installation?
8. Varför rekommenderas 5 GHz- eller 6 GHz-banden för ordinarie datatrafik snarare än 2,4 GHz-bandet?
9. Vilket syfte tjänar en DMZ (demilitarized zone) i en säkerhetsdesign med avseende på trafikflöden?
10. Vad är målet med att genomföra ett redundanstest (failover test) som en del av ett acceptansprotokoll?

Fördjupningslänkar

Cisco Design Guides (Övergripande genomgång av hierarkisk nätverksdesign [hierarchical network design]) - <https://www.cisco.com/c/en/us/td/docs/solutions/Enterprise/Campus/campover.html>

IEEE 802.3 (Standarder och tekniska specifikationer för Ethernet) - https://standards.ieee.org/standard/802_3-2022.html

Wi-Fi Alliance (Viktiga aspekter och nyheter kring Wi-Fi 6 och Wi-Fi 6E) - <https://www.wi-fi.org/discover-wi-fi/wi-fi-certified-6>

NIST (Guide för nätverkssegmentering [network segmentation] och säkerhet i industriella kontrollsystem) - <https://csrc.nist.gov/publications/detail/sp/800-82/rev-2/final>

IETF RFC 1918 (Adressallokering för privata nätverk [private address space]) - <https://datatracker.ietf.org/doc/html/rfc1918>

SNMP (Protokollspecifikationer för nätverkshantering [network management]) - <http://www.snmp.com/protocol/>

NetBox (Dokumentation för central källa för sanning [source of truth] gällande infrastruktur) - <https://docs.netbox.dev/en/stable/>

iPerf3 (Användardokumentation och mätning av genomströmning [throughput testing]) - <https://iperf.fr/iperf-doc.php>

ANSI/TIA-606-B (Standard för administration av infrastruktur för telekommunikation) - <https://www.tiaonline.org/>

RIPE NCC (Grunderna i IP-planering för IPv6) - <https://www.ripe.net/manage-ips-and-asns/ipv6/ipv6-address-planning-basics/>

Egna anteckningar

2.2 VLAN och segmentering

Segmentering innebär att ett fysiskt nätverk delas upp i mindre, logiskt åtskilda domäner. Syftet är att begränsa spridningen av nätverkstrafik, tekniska fel och säkerhetshot, vilket resulterar i en infrastruktur som är både effektivare och säkrare. I ett traditionellt, ofragmenterat nätverk – ofta kallat ett platt nätverk (flat network) – ingår samtliga anslutna enheter i en och samma broadcastdomän. Det innebär att varje broadcast-paket som skickas ut, exempelvis en ARP-förfrågan eller DHCP-begäran, måste tas emot och behandlas av precis alla enheter i nätverket. I en miljö med ett stort antal klienter skapas då en betydande mängd onödig trafik som konsumerar både bandbredd och processorkraft, vilket sänker nätverkets totala prestanda.

2.2.1 Varför segmentera nät?

I praktiken genomförs segmentering oftast genom användning av virtuella lokala nätverk (VLAN), vilka kompletteras med brandväggspolicys för att kontrollera trafiken mellan de olika delarna. Denna metodik medför tre huvudsakliga fördelar för nätverksdriften:

Prestandaoptimering genom mindre broadcastdomäner minskar den mängd onödig trafik som når klienternas nätverkskort. Genom att begränsa räckvidden för broadcast- och multicast-trafik säkerställs att nätverksresurserna används mer effektivt, vilket leder till en snabbare och mer stabil användarupplevelse.

Säkerhetsmässiga fördelar är ofta den primära drivkraften bakom segmentering. Genom att isolera olika användargrupper och tjänster i egna segment begränsas möjligheten till lateral förflyttning (lateral movement) för en potentiell angripare eller skadlig kod. Om en infekterad enhet identifieras i ett segment, förhindrar segmenteringen att hotet sprids fritt till kritiska delar av nätverket. Brandväggspolicys mellan VLAN-gränserna gör det möjligt att definiera exakt vilka flöden som är tillåtna, vilket skapar ett starkt skydd för känslig data.

Felsökning och hantering förenklas avsevärt när nätverket har tydliga logiska gränser. Om ett nätverksfel uppstår kan en tekniker snabbare ringa in orsaken genom att observera vilket segment som

påverkas. Det gör det också möjligt att utföra underhåll eller ändringar i ett segment utan att riskera oavsiktlig påverkan på hela nätverket.

Exempel: En skola väljer att segmentera sitt nätverk genom att separera trafik för personal, elever, gäster, servrar och administrativ hantering (management) i helt egna VLAN. Genom att endast tillåta specifikt definierade flöden mellan dessa zoner skapas en robust miljö där exempelvis en gäst på skolan inte har någon teknisk möjlighet att nå personalens interna filservrar eller nätverkets administrativa gränssnitt.

2.2.2 VLAN-grunder: 802.1Q, access och trunk

Ett virtuellt lokalt nätverk (VLAN) är en logisk indelning av ett fysiskt nätverk som gör det möjligt att separera enheter och trafik oberoende av deras fysiska anslutningspunkt. För att en switch ska kunna hantera denna logik delas dess portar in i olika lägen beroende på vilken typ av utrustning som ansluts och hur trafiken ska transporteras genom nätverket.

Den mest grundläggande porttypen är **accessporten (access port)**. En accessport är konfigurerad att tillhöra ett enda specifikt VLAN och används vanligtvis för att ansluta slutanvändarnas utrustning, såsom datorer, IP-telefoner eller skrivare. När trafik lämnar en accessport mot en klient tas all VLAN-information bort; trafiken är då otaggad (untagged). Detta innebär att klienten inte behöver ha någon kännedom om nätverkets virtuella struktur för att kunna kommunicera.

När trafik från flera olika VLAN behöver transporteras över en och samma fysiska länk, exempelvis mellan två switchar eller mellan en switch och en router, används istället en **trunkport (trunk port)**. För att mottagaren ska veta vilket VLAN en specifik dataram tillhör används industristandarden **IEEE 802.1Q**. Genom denna standard läggs en unik identifikation, en tagg (tag), till i Ethernet-ramens header. Denna tagg innehåller ett VLAN-ID, vilket är ett nummer mellan 1 och 4094 som definierar det logiska nätverket. Tidigare fanns tillverkarspecifika protokoll för detta ändamål, såsom Ciscos **ISL (Inter-Switch Link)**, men i modern nätverksdesign betraktas dessa som föråldrade (legacy) då 802.1Q erbjuder universell kompatibilitet mellan olika tillverkare.

Ett centralt begrepp vid konfiguration av trunkar är **Native VLAN**. Detta definierar vilket VLAN som ska hantera trafik som av någon anledning anländer till trunken utan en 802.1Q-tag. I en säker nätverksdesign är det god praxis att ändra Native VLAN från standardinställningen (ofta VLAN 1) till ett unikt och oanvänt VLAN-ID. Genom att isolera otaggad kontrolltrafik från användarnas datatrafik minskas risken för specifika säkerhetshot, såsom VLAN-hoppning (VLAN hopping).

För att automatisera uppsättningen av trunkar använder vissa tillverkare protokoll som **DTP (Dynamic Trunking Protocol)**. DTP försöker automatiskt förhandla med motparten för att avgöra om en länk ska bli en trunk eller en accessport. Även om detta kan förenkla installationen, medför det risker i form av oförutsägbarhet och potentiella säkerhetshål. En fackmässig och säker design innebär därför att man konsekvent stänger av DTP och istället statiskt konfigurerar portläget som antingen access eller trunk. Detta säkerställer att nätverkets topologi förblir exakt så som teknikern har planerat den.

Exempel: Två switchar, SW1 och SW2. Dessa kopplas samman med en trunklänk som är konfigurerad att bära trafik för VLAN 10 (Personal), 20 (Elev), 30 (Gäst) och 40 (Servrar). Om en nätverksskrivare ansluts till port 12 på SW1, konfigureras den specifika porten som en accessport i VLAN 20. Skrivaren kan då kommunicera med elevernas datorer i samma VLAN över hela nätverket, trots att den bara ser vanlig, otaggad trafik vid sitt fysiska uttag.

Switchar i en VLAN-miljö

I sin enklaste form fungerar en switch genom att bygga upp en **MAC-tabell** (MAC address table), även kallad en **CAM-tabell** (Content Addressable Memory). Denna tabell mappar fysiska MAC-adresser till specifika portar på switchen. När en switch tar emot en datagram läser den av avsändarens MAC-adress och sparar informationen om vilken port den anlände på. Nästa gång trafik ska skickas till den adressen vet switchen exakt vilken port den ska använda istället för att skicka ut trafiken på samtliga portar.

När VLAN introduceras förändras switchens interna logik genom att den skapar vad som kan liknas vid flera oberoende switchar i samma fysiska hårdvara. Detta hanteras tekniskt genom att lägga till en kolumn för

VLAN-ID i MAC-tabellen. Varje MAC-adress knyts nu inte bara till en port, utan även till ett specifikt virtuellt nätverk.

Denna logiska isolering (logical isolation) innebär att switchen utför en strikt filtrering:

- **Inläarning:** När en dator på en accessport i VLAN 10 skickar data, lär sig switchen dess MAC-adress enbart för VLAN 10.
- **Vidarebefordran:** Om en annan enhet i VLAN 20 försöker skicka data till samma MAC-adress, kommer switchen att neka detta, eftersom den enheten inte existerar i VLAN 20:s logiska tabell.
- **Översvämning (Flooding):** Om destinationsadressen är okänd, eller om det rör sig om en broadcast (ff:ff:ff:ff:ff:ff), skickar switchen endast ut kopior av ramen på de portar som tillhör samma VLAN. Detta är grunden för hur broadcastdomäner begränsas.

Det är här skillnaden mellan en **ohanterad switch** (unmanaged switch) och en **hanterad switch** (managed switch) blir tydlig. En ohanterad switch saknar förmågan att läsa av 802.1Q-taggar eller dela upp sin MAC-tabell; den ser hela nätverket som en enda stor domän. En hanterad switch ger däremot administratören full kontroll över hur dessa logiska gränser dras, vilket möjliggör övervakning, hastighetsbegränsning och avancerad säkerhetsfunktionalitet för varje enskilt VLAN.

2.2.3 Inter-VLAN-routing: L3-switch eller “router-on-a-stick”

Eftersom virtuella nätverk (VLAN) fungerar som helt isolerade broadcastdomäner, kan enheter i olika VLAN som standard inte kommunicera med varandra, även om de är anslutna till samma fysiska switch. För att möjliggöra trafikflöden mellan dessa segment krävs en enhet som arbetar på nätverkslagret (lager 3), en process som kallas **Inter-VLAN-routing**. I modern nätverksdesign löses detta huvudsakligen på två sätt: genom en router eller brandvägg med en trunkanslutning, eller direkt i en lager 3-switch (Layer 3 switch).

Router-on-a-stick

Den traditionella metoden, ofta kallad **router-on-a-stick**, innebär att en router ansluts till switchen via en enda fysisk trunklänk. På routern skapas sedan logiska **subinterfaces** (subinterfaces), där varje subinterface

tilldelas en 802.1Q-taggar som motsvarar ett specifikt VLAN. Varje sådant virtuellt gränssnitt ges en unik IP-adress som fungerar som **standard-gateway** (default gateway) för klienterna i det aktuella VLAN-segmentet. Fördelen med denna metod är att den ofta används i kombination med en brandvägg, vilket gör det enkelt att centralisera säkerhetspolicys och granska all trafik som rör sig mellan segmenten. Nackdelen är att all trafik måste färdas upp till routern och tillbaka över samma länk, vilket kan skapa en flaskhals om mängden intern datatrafik är mycket stor.

L3-routing

I större och mer prestandakrävande nätverk används istället en **lager 3-switch**. Här sker routingen direkt i switchens hårdvara med hjälp av specialiserade kretsar (ASIC), vilket ger en extremt hög genomströmning (throughput) och minimal fördröjning. Tekniskt åstadkoms detta genom att konfigurera ett **virtuellt switchgränssnitt** (SVI - Switch Virtual Interface) för varje VLAN. Ett SVI fungerar som en virtuell routerport inuti switchen och agerar gateway för segmentet. Genom att flytta routingen till distributions- eller kärnlagret avlastas nätverkets externa router eller brandvägg, som då kan fokusera helt på trafik mot internet eller andra externa nätverk.

Valet av placering för nätverkets gateway styrs av behovet av skalbarhet och säkerhetskontroll. I en miljö där trafik mellan VLAN kräver djupgående paketinspektion (DPI) eller strikta brandväggsregler är en centraliserad placering i en brandvägg att föredra. Om målet däremot är högsta möjliga hastighet för intern trafik, exempelvis mellan användare och lokala servrar, är en lager 3-switch det mest effektiva valet.

Exempel: En skola där elevernas VLAN 20 har tilldelats subnätet 10.20.0.0/22. Genom att skapa ett SVI för VLAN 20 med IP-adressen 10.20.0.1 på en central lager 3-switch, får alla elever en väg ut ur sitt eget segment. Switchen kan sedan programmeras med åtkomstlistor (ACL) eller skicka trafiken vidare till en brandvägg som tillämpar specifika policys. I detta scenario kan brandväggen konfigureras för att tillåta trafik från elevernas VLAN mot internet, men samtidigt blockera alla försök att initiera anslutningar mot serversegmentet i VLAN 40, vilket säkerställer att känslig data förblir skyddad trots att routing är aktiverad.

2.2.4 Designmönster: Vanliga VLAN och segmenteringsprofiler

En välplanerad segmenteringsstrategi utgår från funktionella zoner där varje zon tilldelas ett eget VLAN-ID och subnät. Följande mönster är vanligt förekommande i moderna nätverksinstallationer:

- **Personal (VLAN 10):** Detta segment är avsett för administrativa medarbetare och lärare. Här tillåts fullständig åtkomst till interna resurser såsom Active Directory (AD), filservrar och administrativa stödsystem. Eftersom enheterna i detta segment hanterar känsliga data krävs ofta högre säkerhetskrav på klientsidan.
- **Elev (VLAN 20):** Ett segment med hög klienttäthet men begränsad behörighet. Elever ges åtkomst till internet och nödvändiga lärplattformar, men nekas konsekvent tillträde till administrativa resurser eller nätverkets infrastrukturkomponenter.
- **Gäst (VLAN 30):** En isolerad zon för externa besökare. Här tillämpas ofta strikt klientisolering (client isolation), vilket innebär att trådlösa klienter i samma VLAN inte kan kommunicera med varandra. Trafiken styrs via hårda utgångsregler (egress rules) i brandväggen som endast tillåter trafik ut mot internet.
- **Servrar (VLAN 40):** En skyddad zon för nätverkets centrala tjänster, inklusive DNS, DHCP, filservrar och domänkontrollanter. Åtkomst till denna zon styrs av strikta åtkomstlistor (ACL) baserat på vilka tjänster (portar) som de olika klientsegmenten behöver nå.

Förutom de användarfokuserade segmenten krävs dedikerade nätverk för nätverkets egen infrastruktur och specialiserade tjänster:

- **Infrastruktur och WLAN-AP (VLAN 50):** Används uteslutande för hantering av accesspunkter och tunnling av trådlös trafik. Genom att separera accesspunkternas egen trafik från klienternas data försvåras försök att manipulera nätverksutrustningen.
- **Voice (VLAN 60):** Dedikerat för IP-telefoni och realtidskommunikation. Eftersom rösttrafik är extremt känslig för fördröjning och jitter, implementeras tjänstekvalitet (QoS - Quality of Service) med hjälp av prioriteringstaggar (DSCP eller 802.1p). I switchen definieras ofta en förtroendegräns (trust boundary) så nära

telefonen som möjligt, vilket säkerställer att rösttrafiken prioriteras genom hela nätverket.

- **Management (VLAN 99):** Ett kritiskt segment för nätverksutrustningens administrativa gränssnitt. Detta nätverk bör aldrig vara nåbart direkt från klientsegmenten. Åtkomst tillåts endast från specifika administrativa arbetsstationer eller via krypterad fjärranslutning (VPN).

Genom att konsekvent tillämpa dessa designmönster skapas en förutsägbar miljö där säkerhetspolicys kan appliceras på gruppnivå istället för på enskilda enheter. Ett gästnätverk på en skola kan exempelvis konfigureras så att all trafik mot interna adresser (RFC1918) automatiskt blockeras i brandväggen, medan personalsegmentet ges de öppningar som krävs för det dagliga arbetet.

2.2.5 Lager-2-skydd: "hårda portar" och kontroll av giftig trafik

Grunden för nätverkstopologins stabilitet vilar på **Spanning Tree Protocol (STP)**. För att nätverket ska kunna upptäcka loopar och bestämma vilka redundanta vägar som ska blockeras eller vara aktiva, skickar switchar kontrollmeddelanden till varandra som kallas **BPDUs (Bridge Protocol Data Units)**. En BPDU fungerar som nätverkets "hjärtslag" och innehåller information om nätverkets struktur och vilken switch som agerar central punkt, så kallad Root Bridge. Genom att förstå och kontrollera dessa meddelanden kan vi implementera avancerade skydd.

En primär säkerhetsåtgärd är att alltid konfigurera portlägen explicit. Genom att stänga av automatisk förhandling (DTP/auto trunk) och låsa accessportar till ett fast läge, förhindras försök att olovligen etablera en trunkanslutning för att få åtkomst till flera VLAN. Vidare används **Port Security** för att begränsa antalet tillåtna fysiska adresser (MAC-adresser) per port. Genom att använda funktionen "sticky" kan switchen lära in den första enheten som ansluts och därefter blockera alla andra, vilket effektivt stoppar obehöriga från att koppla in egen utrustning i nätverksuttag.

För att skydda nätverkets logiska struktur används funktioner som **BPDUs Guard** och **Root Guard**. BPDU Guard konfigureras på accessportar där

endast slutanvändare ska finnas; om porten tar emot ett BPDU-meddelande innebär det att någon har anslutit en annan switch eller enhet som försöker påverka nätverkets topologi. Porten stängs då omedelbart av för att skydda nätverket. Root Guard används istället för att säkerställa att ingen annan switch i nätverket kan utse sig själv till Root Bridge, vilket håller nätverkets hierarki intakt och förutsägbar.

Ytterligare lager av skydd hanterar hot relaterade till adressering och identitet:

- **DHCP Snooping:** Denna funktion innebär att switchen endast litar på legitima DHCP-servrar anslutna till specifika, betrodda portar. Det förhindrar att en angripare sätter upp en falsk DHCP-server för att styra användarnas trafik till sig själv (Man-in-the-Middle).
- **Dynamic ARP Inspection (DAI):** Genom att validera ARP-tabellen mot den databas som DHCP Snooping bygger upp, kan switchen stoppa försök till ARP-spoofing, där en enhet försöker utge sig för att vara en annan (exempelvis nätverkets gateway).
- **IP Source Guard:** Denna funktion kontrollerar att den käll-IP som en klient använder faktiskt matchar den adress som tilldelats via DHCP, vilket blockerar försök att använda falska IP-adresser på nätverket.
- **Storm Control:** Fungerar som en säkerhetsventil som stryper mängden broadcast- eller multicast-trafik om den överskrider en viss nivå. Detta skyddar nätverket från att bli helt paralyserat vid hårdvarufel eller loopar som skapar extrem trafikbelastning.

Exempel: Vikten av dessa skydd visas tydligt i en skolmiljö där en elev ansluter en medtagen "smart-switch" till ett vägguttag i ett klassrum. Utan skydd skulle denna lilla switch kunna börja skicka ut BPDUs och potentiellt orsaka en omräkning av hela skolans nätverkstopologi, med avbrott som följd. Med BPDU Guard aktiverat på porten identifieras den främmande switchen omedelbart, porten slår ifrån och resten av nätverket förblir opåverkat.

2.2.6 VLAN i WLAN – SSID, mappning och gästisolering

I ett modernt företagsnätverk fungerar det trådlösa nätverket som en naturlig förlängning av den trådbundna infrastrukturen. För att

upprätthålla samma säkerhets- och segmenteringsmodell oavsett hur en användare ansluter, används teknik för att mappa ett **SSID** (Service Set Identifier) direkt till ett specifikt **VLAN**. Detta innebär att när en accesspunkt (access point - AP) tar emot trafik från en trådlös klient, märker den ramen med rätt VLAN-id innan den skickas vidare över nätverkets trunk-anslutningar. På så sätt kan brandväggar och routrar tillämpa samma regler för en elev eller anställd, oavsett om de sitter vid en fast dator eller använder en laptop via Wi-Fi.

En av de viktigaste tillämpningarna av denna teknik är hanteringen av gästnätverk (guest networks). Genom att tilldela gäster ett eget SSID som är kopplat till ett isolerat VLAN (exempelvis VLAN 30), kan man säkerställa att besökare aldrig får tillgång till interna resurser som servrar eller skrivare. I dessa segment är det fackmässig standard att aktivera **klientisolering** (client isolation / client-to-client blocking). Denna funktion förhindrar trådlösa klienter i samma VLAN från att kommunicera med varandra, vilket effektivt stoppar spridning av skadlig kod mellan gästers enheter. För att ytterligare skydda nätverket används ofta strikta **utgångsfilter** (egress filters) i brandväggen som endast tillåter vanlig webbftrafik (TCP/UDP) ut mot internet.

Vid design av trådlösa nätverk bör man sträva efter att hålla antalet SSID på en låg nivå, normalt mellan två och fyra per accesspunkt. Varje extra SSID skapar administrativ overhead i form av kontrollmeddelanden (beacons) som tar upp värdefull sändningstid i luften. För känsliga nätverk där personal eller elever hanterar inloggningsuppgifter används **802.1X/Enterprise-autentisering**, vilket ger varje användare en unik identitet och möjliggör dynamisk tilldelning av VLAN baserat på vem som loggar in. För gästnätverk kan en **captive portal** (inloggningssida) användas om det finns behov av att användare accepterar ordningsregler, men det bör kombineras med hastighetsbegränsning (**rate limit**) för att förhindra att gäster konsumerar all tillgänglig bandbredd.

Förtydligande (sammanfattning):

- **SSID-mappning:** Genom att koppla SSID till VLAN förlängs nätverkets segmentering ut i luften.
- **Optimering:** Begränsa antalet SSID till 2–4 per accesspunkt för att spara bandbredd.

- **Säkerhet:** Använd 802.1X/Enterprise för personliga nät och klientisolering (client isolation) för gäster.
- **Gästhantering:** Internet-only-åtkomst via strikta filter och eventuellt en captive portal för inloggning.

Exempel: På en skola mappas det trådlösa nätverket så att SSID "Elev" leder till VLAN 20 och SSID "Personal" till VLAN 10. Ett tredje SSID, "Gäst", är kopplat till VLAN 30. En elev som ansluter till det trådlösa nätverket hamnar därmed i samma logiska segment som de stationära datorerna i datorsalarna. Gäster som ansluter till sitt SSID isoleras helt från varandra på accesspunkten och deras trafik styrs via brandväggen direkt ut till internet, utan att de ens kan se att det finns andra servrar eller enheter i skolans nätverk.

2.2.7 STP, RSTP och MSTP i redundanta VLAN-miljöer

Spanning Tree Protocol (STP), definierat i standarden IEEE 802.1D, fungerar som nätverkets automatiska loopskydd. Protokollet identifierar redundanta länkar och försätter dem i ett blockerande läge (blocking state) så att endast en aktiv logisk väg existerar mellan två punkter, vilket skapar en trädstruktur i topologin. Om en aktiv länk går ner, upptäcker protokollet detta och öppnar automatiskt en av de blockerade vägarna. Den ursprungliga STP-standardens är dock relativt långsam, med en konvergenstid (convergence time) på upp till 50 sekunder innan trafiken kan flöda igen.

För att möta kraven i moderna nätverk utvecklades **Rapid STP (RSTP)**, standarden 802.1w. RSTP fungerar enligt samma grundprinciper men använder en betydligt snabbare metod för att förhandla fram topologin. Vid ett länkfel kan RSTP återställa anslutningen på bråkdelar av en sekund eller upp till några sekunder, vilket minimerar påverkan på pågående sessioner och applikationer.

När nätverket innehåller ett stort antal virtuella nätverk (VLAN) kan hanteringen av Spanning Tree bli komplex. I en miljö med hundratals VLAN skulle körandet av en separat STP-instans per VLAN (PVST+) konsumera onödigt mycket processorkraft i switcharna. Här introduceras **Multiple STP (MSTP)**, standarden 802.1s. MSTP gör det möjligt att gruppera flera VLAN i ett fåtal logiska instanser (instances). Detta

förenklar driften och möjliggör effektiv lastbalansering (load balancing) genom att olika instanser kan ha olika switchar som centralpunkt, så kallad **Root Bridge**.

En kritisk aspekt i designen är just kontrollen över nätverkets Root Bridge. I en oplanerad miljö kan den äldsta eller sämsta switchen i nätverket råka bli Root Bridge baserat på sitt MAC-nummer, vilket tvingar trafiken att ta ineffektiva vägar. En fackmässig konfiguration innebär att man manuellt låser rollen som Root Bridge till nätverkets mest centrala och kraftfulla switchar, vanligtvis i distributions- eller kärnlagret (core). Detta säkerställer en förutsägbar och optimal trafikväg.

För att ytterligare optimera prestandan tillämpas **VLAN-pruning** på trunklänkar. Detta innebär att man begränsar vilka VLAN som tillåts passera en specifik trunk (allowed-list). Genom att endast tillåta de VLAN som faktiskt behövs i motstående switch minskas mängden onödig broadcast-trafik och säkerheten ökar. Det är härvid av yttersta vikt att listorna över tillåtna VLAN matchar i båda ändar av länken för att undvika svårdiagnostiserade anslutningsproblem. Liksom i tidigare kapitel betonas vikten av ett konsekvent konfigurerat **Native VLAN** som hålls separat från produktionstrafik för att minimera risker för otaggad trafik som når fel segment.

Exempel: En redundant miljö är två kärnswitchar som är sammankopplade med RSTP. Genom att konfigurera den ena som primär Root Bridge för hälften av nätverkets VLAN, och den andra som primär för den resterande hälften, utnyttjas nätverkets totala bandbredd effektivt samtidigt som full redundans bibehålls vid ett maskinfel.

2.2.8 Dokumentation och felsökning

En nätverksdesign är aldrig starkare än sin dokumentation. I en segmenterad miljö, där logiska gränser korsar den fysiska infrastrukturen, är en uppdaterad dokumentation teknikerens viktigaste resurs för både drift och felsökning. Utan en tydlig karta över VLAN-id, subnät och gateways blir det nästan omöjligt att snabbt identifiera varför en specifik tjänst inte fungerar.

Dokumentationsmodeller för olika arkitekturer

Beroende på om nätverket använder en central router eller en distribuerad lager 3-switch ser dokumentationen olika ut. Nedan följer två exempel på hur informationen bör struktureras:

Exempel 1: Router-on-a-Stick (RoaS)

Här ligger fokus på routerns subinterfaces och de 802.1Q-taggar som används för att separera trafiken över trunken.

VLAN-ID	Namn	Subinterface	802.1Q Tag	Gateway (IP/Mask)	Syfte
10	Personal	gi0/0.10	10	10.10.0.1/24	Administrativ trafik
20	Elev	gi0/0.20	20	10.20.0.1/22	Elevdatorer/ Laptops
99	Management	gi0/0.99	99	10.99.0.1/24	Hantering av switchar/AP

Exempel 2: Lager 3-switch (L3)

Här dokumenteras istället de virtuella gränssnitten (SVI) direkt i switchen, samt vilka fysiska portar eller trunkar som bär respektive VLAN.

VLAN-ID	SVI Namn	IP-adress	Nätmask	DHCP-Scope	Primär trunk-rutt
40	Vlan40_Serv	10.40.0.1	255.255.255.0	10.40.0.10–50	Core-SW1 <-> Dist-SW1
60	Vlan60_Voice	10.60.0.1	255.255.255.0	10.60.0.100–250	Samtliga access-trunkar

Strukturerad felsökning (Troubleshooting)

När ett fel rapporteras i ett segmenterat nätverk bör felsökningen ske systematiskt, med start i accesslagret (bottom-up). Genom att verifiera varje lager i ordning undviks onödigt arbete med komplexa routing-inställningar om felet i själva verket är fysiskt.

Felsökningsschema i sex steg:

1. **Fysisk länk och portläge:** Kontrollera att kabeln är hel och att porten är i rätt läge. Är det en accessport till en klient eller en trunk till en annan switch? Är rätt VLAN tilldelat på porten?
2. **Trunk-validering:** Om trafiken passerar mellan switchar, kontrollera att det aktuella VLAN-id:t finns med i listan över tillåtna VLAN (allowed list). Verifiera även att Native VLAN matchar i båda ändar av trunken för att undvika trafikläckage.
3. **Lager 3-status (Gateway):** Kontrollera att det virtuella gränssnittet (SVI) eller subinterfacet är i läge "up/up". Svara gateway på ping? Har klienten rätt IP-adress och nätmask i förhållande till sitt VLAN?
4. **Infrastrukturstjänster:** Verifierar att klienten faktiskt får ett DHCP-lease och att DNS-inställningarna pekar mot rätt servrar. Kan klienten göra namnuppslagningar?
5. **Spanning Tree (STP):** Undersök nätverkstopologin via switchen. Finns det en blockerad länk där du förväntar dig trafik? Är nätverkets Root Bridge placerad på den planerade enheten (Core/Distribution)?
6. **Tabellanalys:** Granska switchens MAC-tabell och routerns ARP-tabell. Ser du klientens fysiska adress i rätt VLAN-segment?

Exempel: En användare i personalstyrkan rapporterar att de inte når filservern. Teknikern börjar med att kontrollera switchen och upptäcker att uttaget vid användarens plats av misstag har blivit konfigurerat som access i VLAN 20 (Elev) istället för VLAN 10 (Personal). Eftersom brandväggspolicyn blockerar all trafik från elevsegmentet mot serversegmentet, nekades

användaren åtkomst. Genom att enkelt flytta porten till rätt VLAN i konfigurationen återställs förbindelsen omedelbart.

Kontrollfrågor

1. Vilka är de tre främsta anledningarna till att man väljer att segmentera ett platt nätverk (flat network)?
2. Förklara den tekniska skillnaden på hur en Ethernet-ram hanteras när den lämnar en accessport (access port) jämfört med en trunkport (trunk port).
3. Vad är syftet med ett Native VLAN och varför anses det vara en säkerhetsrisk att låta det vara kvar på standardinställningen?
4. Beskriv för- och nackdelar med metoden "Router-on-a-stick" jämfört med att använda en lager 3-switch för Inter-VLAN-routing.
5. Hur förändras en switchs MAC-tabell (CAM-tabell) när VLAN introduceras i nätverket?
6. Varför är BPDU Guard en kritisk funktion att aktivera på accessportar i en skolmiljö?
7. Förklara sambandet mellan DHCP Snooping och Dynamic ARP Inspection (DAI). Varför kräver DAI att snooping-funktionen är aktiverad?
8. Vilket problem löser Multiple STP (MSTP) som inte den vanliga Rapid STP (RSTP) kan hantera lika effektivt i miljöer med hundratals VLAN?
9. Vad innebär "VLAN-pruning" och vilken effekt har det på nätverkets prestanda?
10. Vid felsökning enligt "bottom-up"-modellen, vilka är de två första sakerna du bör kontrollera om en klient i ett specifikt VLAN saknar kontakt med sin gateway?

Fördjupningslänkar

Cisco Networking Academy (Grundläggande och avancerad genomgång av VLAN-koncept) - <https://www.netacad.com/>

Practical Networking: Inter-VLAN Routing (Router-on-a-Stick och SVI) - <https://www.practicalnetworking.net/featured/inter-vlan-routing/>

IEEE Xplore: 802.1Q Standard (Den officiella tekniska specifikationen för VLAN-tagging [tagging]) - <https://ieeexplore.ieee.org/document/8403927>

PacketLife.net: STP Cheat Sheet (Spanning Tree-protokollen och deras tillstånd) - <https://packetlife.net/library/cheat-sheets/>

Network World (Artikel om hur man säkrar accesslagret mot vanliga attacker) - <https://www.networkworld.com/article/711202/layer-2-security-best-practices.html>

Wireshark Wiki: 802.1Q (Hur man analyserar taggad trafik i ett paketanalysverktyg [packet sniffer]) - <https://wiki.wireshark.org/VLAN>

NIST: Guide to Network Segmentation and Security (Myndighetsrekommendationer för segmentering av kritiska nätverk) - <https://csrc.nist.gov/publications/detail/sp/800-125b/final>

NetBox Documentation (Guide för hur man dokumenterar VLAN och IP-planer i en central källa för sanning [source of truth]) - <https://docs.netbox.dev/>

Proxmox VE Documentation: VLAN Network Configuration (Relevant för virtualiserade miljöer och Proxmox-servrar) - https://pve.proxmox.com/wiki/Network_Configuration#_vlan_802_1q

Red Hat: Configuring VLAN Tagging in Linux (Hantera VLAN direkt i moderna Linux-miljöer, exempelvis RHEL 9) - https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/9/html/configuring_and_managing_networking/configuring-vlan-tagging_configuring-and-managing-networking

Egna anteckningar

2.3 Routing

För att kunna fatta beslut om vidarebefordran (forwarding) innehåller routingtabellen information om tillgängliga destinationsnät (prefix), nästa hopp (next hop) och vilket utgående gränssnitt (exit interface) som ska användas. När ett paket anländer till routern jämförs dess destinations-IP-adress mot tabellen enligt en strikt prioriteringsordning.

2.3.1 Grunder: hur routrar väljer väg

Den mest fundamentala regeln för vägval är **längsta prefix-matchning** (longest prefix match - LPM). Om en destinationsadress matchar flera rutter i tabellen väljer routern alltid den mest specifika rutten, det vill säga den med längst nätmask (prefixlängd). Ett praktiskt exempel på detta är om ett paket ska skickas till adressen 10.20.1.15. Om routingtabellen innehåller både en standardrutt (default route) på 0.0.0.0/0 och en specifik rutt för 10.20.0.0/22, kommer routern att välja /22-rutten eftersom den är mer exakt och därmed anses vara en bättre matchning.

När information om vägar samlas in från olika källor, såsom statiska rutter (static routes) eller olika dynamiska routingprotokoll som OSPF och RIP, uppstår behovet av att värdera källornas tillförlitlighet. Detta hanteras genom **administrativ distans** (administrative distance - AD). Varje källa tilldelas ett numeriskt värde där ett lägre värde innebär högre prioritet. En statisk rutt (AD 1) anses till exempel mer pålitlig än en rutt inlärd via OSPF (AD 110). Om två olika protokoll föreslår en väg till exakt samma nätverk, installeras den rutt som har lägst AD i den aktiva routingtabellen.

Om flera vägar till samma mål har lärt sig via samma protokoll, används istället en **metrik** (metric) för att jämföra dem. Metriken är en form av kostnadsskala som varierar beroende på protokoll; RIP använder till exempel antal hopp (hop count) medan OSPF använder en kostnad baserad på länkens bandbredd (bandwidth). Den väg som har lägst metrik väljs som den primära vägen. I situationer där två eller flera vägar har exakt samma kostnad kan routern använda **ECMP** (Equal-Cost Multi-Path) för att fördela trafiken över de olika länkarna, vilket ökar den totala kapaciteten och ger en form av lastbalansering (load balancing).

Genom att kombinera dessa mekanismer säkerställer routern att paketen alltid tar den mest logiska vägen genom nätverket, samtidigt som den snabbt kan ställa om ifall en länk går ner och en alternativ väg med högre metrik eller administrativ distans blir aktuell.

2.3.2 Statisk routing – enkelhet och kontroll

För att förstå statisk routing kan vi jämföra det med hur vi navigerar i vardagen. Tänk dig att du ska ge en vän en vägbeskrivning till ett köpcentrum. Du skriver ner exakt vilka gator hen ska svänga på: "Kör rakt fram i två kilometer, sväng höger vid kyrkan, ta sedan andra avfarten i rondellen". Detta är en statisk instruktion. Så länge vägen är fri fungerar det ypperligt och är mycket lätt att följa.

Problemet uppstår om det plötsligt pågår ett vägarbete eller om en bro är avstängd längs den utstakade rutten. En statisk rutt har ingen förmåga att känna av hinder eller anpassa vägbeskrivningen efter rådande trafikläge. Trafiken kommer envist att köra fram till hindret, inse att det inte går att passera, vända tillbaka till utgångspunkten och sedan försöka exakt samma väg igen och igen. Utan en mänsklig administratör som ändrar instruktionen kommer trafiken aldrig fram. Detta skiljer sig markant från moderna GPS-tjänster som Google Maps, vilka automatiskt räknar om rutten vid köer eller olyckor – en funktion som i nätverksvärlden motsvaras av dynamisk routing.

I professionell nätverksutrustning, exempelvis i Ciscos kommandoradsgränssnitt (CLI - Command Line Interface), konfigureras en statisk rutt med ett enkelt kommando. Om vi vill att vår router ska hitta till labbnätverket 10.50.0.0/16 genom att skicka trafiken till en annan router med IP-adressen 192.168.1.2, skriver vi:

```
Router(config)# ip route 10.50.0.0 255.255.0.0 192.168.1.2
```

Här definierar vi först destinationsnätet, följt av dess nätmask (subnet mask) och slutligen nästa hopp (next hop), vilket är adressen till den router som faktiskt "äger" vägen vidare. Om vi istället vill skapa en standardrutt (default route) – den rutt som tar hand om allt som inte finns i vår lokala tabell, såsom trafik mot internet – använder vi nollor för att representera "vilket nät som helst":

```
Router(config)# ip route 0.0.0.0 0.0.0.0 192.168.1.1
```


För den som arbetar med brandväggar som har ett grafiskt gränssnitt (GUI), till exempel **OPNsense**, är funktionen för statisk routing inbyggd direkt i webbgränssnittet. Under menyn för systeminställningar och rutter kan administratören enkelt lägga till rutter genom att fylla i destinationsnät och gateway i tydliga formulärfält. Det bakomliggande konceptet är exakt detsamma som i Ciscos CLI, men gränssnittet gör det mer överskådligt för den som föredrar att klicka fram inställningarna framför att skriva kod.

Trots sin enkelhet kräver statisk routing att man alltid tänker på returvägen (return path). Eftersom routern bara följer den exakta vägbeskrivningen du gett den, kommer svarspaket från destinationsnätet inte att hitta tillbaka om inte även den mottagande routern har en statisk rutt (eller standardrutt) som pekar tillbaka till källan. Utan denna ömsesidiga vägbeskrivning uppstår asymmetrisk routing, vilket i praktiken innebär att kommunikationen bryts.

Fördelar:

- *Enkelt och förutsägbart* – lätt att förstå och felsöka.
- *Ingen protokollchatt* – noll kontrolltrafik och minimal CPU-belastning.
- *Fin kontroll* – du bestämmer exakt vägval.
- *Bra som backup* – “floating static” tar över först vid fel.

Nackdelar:

- *Ingen självläkning* – topologiändringar kräver manuell uppdatering.
- *Skalar dåligt* – många nät = många rader att underhålla.
- *Risk för asymmetri/blackholes* om returvägar missas.

Användningsområden:

- *Små nät / labbar / edge mot internet* (default route).
- *Enkla hub-and-spoke* (spoke pekar mot hub med statik).
- *Backup-vägar* (floating static bakom dynamik).

Exempel: Router A har en standardrutt (default route) mot internet (0.0.0.0/0) och behöver en manuell statisk rutt (static route) som pekar ut nätet bakom Labbroutern för att trafiken ska hitta dit. Labbroutern måste i sin tur ha en returväg, oftast en egen

standardrutt som pekar tillbaka mot Router A, för att svarstrafik och internetåtkomst ska fungera. Detta skapar en fast och förutsägbär vägbeskrivning mellan näten som fungerar utmärkt så länge infrastrukturen är stabil, men den saknar helt förmåga att självläka eller hitta omvägar om en länk går ner.

2.3.3 Dynamisk routing – när nätet ska anpassa sig själv

Grunden för dynamisk routing är förmågan att skapa grannrelationer (adjacencies). Routrarna skickar kontinuerligt ut små kontrollmeddelanden, så kallade hello-paket (hello packets), för att bekräfta att deras grannar fortfarande är aktiva och nåbara. Så länge dessa paket tas emot betraktas länken som stabil. Om en router slutar svara eller om en ny länk kopplas in, sprids denna information genom hela nätverket. Processen där alla routrar uppdaterar sina tabeller för att nå en gemensam bild av den nya topologin kallas för konvergens (convergence). Ett nätverk som har konvergerat snabbt minimerar risken för avbrott och säkerställer att datatrafiken alltid tar den bästa vägen.

Dessa protokoll delas grovt in i två huvudfamiljer beroende på hur de beräknar och sprider information om vägar. Den första familjen är distansvektor-protokoll (distance-vector), där RIP (Routing Information Protocol) är ett klassiskt exempel. Dessa fungerar enligt principen "routing genom rykten". Varje router skickar sin kompletta routingtabell till sina grannar och berättar i vilken riktning (vector) och hur långt bort (distance) ett nätverk ligger. Den andra familjen är länktillstånds-protokoll (link-state), med OSPF (Open Shortest Path First) som det mest kända exemplet. Istället för att bara sprida rykten delar dessa routrar detaljerad information om statusen på sina egna länkar. Detta gör att varje router kan bygga upp en exakt och komplett karta över hela nätverkets topologi och själv räkna ut den bästa vägen med hjälp av matematiska algoritmer.

Den främsta fördelen med att använda dynamiska protokoll är nätverkets förmåga till självläkning (self-healing). Om en primär länk går ner upptäcker protokollet detta omedelbart och dirigerar om trafiken till en reservväg utan att en tekniker behöver ingripa. Detta gör tekniken extremt skalbar, då nya nätverk och platser kan läggas till utan att dussintals statiska rutter behöver uppdateras manuellt på varje enskild

router. Det minskar risken för mänskliga fel och gör driften av stora campusrät eller företagskoncerner möjlig.

Det finns dock en ökad komplexitet som en nätverkstekniker måste ta hänsyn till. Dynamiska protokoll introducerar administrativ trafik (overhead) i form av de meddelanden som skickas mellan routrarna, vilket konsumerar en del av nätverkets bandbredd och routerns processorstyrka (CPU). Dessutom krävs en djupare förståelse för protokollspecifika beteenden såsom timers, mätvärden (metrics) och filtrering. En felkonfiguration av dessa parametrar kan i värsta fall leda till att nätverket väljer ineffektiva vägar eller hamnar i ett instabilt läge där routingtabellerna ständigt ändras. Trots detta är dynamisk routing den absoluta standarden i alla nätverk som kräver hög tillgänglighet och enkel tillväxt.

Fördelar (generellt):

- *Självläkning* – anpassar sig vid fel utan manuell insats.
- *Skalbarhet* – enklare när nät och sajter växer.
- *Mindre manuellt pyssel* – färre statiska rader att underhålla.

Nackdelar (generellt):

- *Ökad komplexitet* – grannskap, timers, filtrering, autentisering.
- *Protokolltrafik* – viss overhead och potentiella angreppsytor.
- *Felkonfiguration kostar* – felaktiga parametrar kan ge sämre vägar.

2.3.4 RIP v2 – enkelt och pedagogiskt

RIP v2 (Routing Information Protocol version 2) är ett klassiskt distansvektor-protokoll (distance-vector protocol) som fungerar som en utmärkt introduktion till dynamisk routing. Det bygger på en enkel logik där varje router delar sin kunskap om tillgängliga nätverk med sina närmaste grannar genom periodiska uppdateringar, vanligtvis var 30:e sekund. Genom att "byta kartor" med varandra bygger routrarna upp en gemensam förståelse för nätverkets struktur. Denna beräkningsmodell bygger på **Bellman-Fords algoritm** (Bellman-Ford algorithm), vilket innebär att varje router fattar sina beslut baserat på den ackumulerade kostnaden – i det här fallet hopp – som rapporteras från dess grannar.

Inom RIP v2 används hoppräknning (hop count) som den enda metriken (metric). Det innebär att den bästa vägen till ett mål alltid anses vara den väg som passerar genom lägst antal routrar. En stor begränsning i protokollet är dock dess skalbarhet; det stöder maximalt 15 hopp. Ett nätverk som ligger 16 hopp bort betraktas som oåtkomligt (unreachable), vilket gör att protokollet inte kan användas i stora eller komplexa nätverksmiljöer.

Till skillnad från den äldre version 1 har RIP v2 stöd för VLSM (Variable Length Subnet Masking) och CIDR (Classless Inter-Domain Routing), vilket innebär att information om nätmasker skickas med i uppdateringarna. För att hantera risker med loopar och felaktig information använder protokollet flera tekniska säkerhetsmekanismer:

Split horizon: En regel som förhindrar en router från att skicka tillbaka information om ett nätverk på samma gränssnitt som den lärde sig informationen ifrån.

Poison reverse: När en rutt går ner markeras den omedelbart med en metrik på 16 (oändlighet) för att informera grannarna om att vägen är bruten.

Hold-down timers: En väntetid som förhindrar att en router för snabbt accepterar en förändring av en rutt som nyss har markerats som osäker, vilket stabiliserar nätverket vid tillfälliga länkfel.

Fördelar och nackdelar med RIP v2

Den främsta fördelen med RIP v2 är dess extrema enkelhet. Det är snabbt att konfigurera och ställa in i mindre miljöer eller labbnätverk, och det kräver minimalt med resurser i form av processorkraft (CPU) och minne. Dessutom har protokollet en mycket bred kompatibilitet och finns tillgängligt på nästan alla typer av nätverksutrustning, oavsett tillverkare.

Nackdelarna är dock betydande i moderna produktionsmiljöer. Eftersom metriken endast baseras på hopp, tar protokollet ingen hänsyn till länkarnas faktiska kvalitet eller hastighet. En router kan därför välja en långsam 10 Mbps-länk över ett hopp istället för en stabil 10 Gbps-länk som kräver två hopp. Vidare lider RIP v2 av långsam konvergens (convergence); vid ett länkfel kan det ta flera minuter innan hela nätverket har uppdaterat sina tabeller, vilket i dagens uppkopplade värld ofta betraktas som ett oacceptabelt avbrott. Det finns även en risk för

"count-to-infinity"-problem där felaktiga rutter räknas upp steg för steg innan de når maxgränsen.

Användningsområden och exempel

Idag rekommenderas RIP v2 sällan för skarp produktion där snabb återhämtning och hög prestanda krävs. Istället används det främst inom utbildning för att lära ut grundläggande routingprinciper, eller i mycket små och stabila miljöer där enkelhet prioriteras högre än effektivitet. Det kan även fungera som en temporär lösning i ett "proof-of-concept" där man snabbt behöver få kommunikation att fungera mellan några få enheter utan komplex konfiguration.

- **Fördelar:**

- Mycket enkelt – snabbt att få igång i labb/mindre nät.
- Stöd för VLSM/CIDR (till skillnad från v1).
- Bred kompatibilitet – finns på många plattformar.

- **Nackdelar:**

- Skalar dåligt – 15 hopp-begränsning.
- Långsam konvergens – periodiska uppdateringar; "count-to-infinity"-problem.
- Grovt metric – tar inte hänsyn till bandbredd/kvalitet.

- **Användningsområden:**

- Utbildning och små labbnät (visa principer).
- Mycket små, statiska miljöer där enkelhet > prestanda.
- Temporära proof-of-concept där snabb uppstart är viktig.

Exempel: Tre routrar kopplade i en triangelform kör RIP v2. Om den direkta länken mellan två av routrarna bryts, kommer de inte omedelbart att hitta omvägen via den tredje routern. De måste först vänta på att deras interna tidtagare (timers) löper ut och att nästa periodiska uppdatering skickas. Efter en stund kommer hoppräknningen att uppdateras och trafiken hittar den alternativa vägen, men användarna i nätverket kommer att uppleva en tydlig fördröjning innan förbindelsen återställs.

2.3.5 OSPF – snabbt och skalbart

OSPF (Open Shortest Path First) är ett länktillstånds-protokoll (link-state protocol) som bygger på att varje router i nätverket har en komplett och identisk karta över topologin. Istället för att bara lita på vad grannen säger, annonserar varje enhet sin egen status via **LSA** (Link-State Advertisements). Denna information lagras i en databas (Link-State Database - LSDB), och med hjälp av **Dijkstras algoritm** beräknar varje router den absolut mest effektiva vägen till varje nätverk. Denna metod ger en betydligt snabbare konvergens (convergence) än äldre protokoll, vilket innebär att nätverket kan återhämta sig från ett länkfel på under en sekund.

Metrik och kostnad (Cost)

Till skillnad från RIP, som bara räknar antal hopp, använder OSPF en metrik som kallas **kostnad** (cost). Kostnaden beräknas som standard genom att dela en referensbandbredd (reference bandwidth) med länkens faktiska bandbredd. Detta innebär att en snabb fiberlänk får en mycket låg kostnad, medan en långsammare länk får en hög kostnad. Genom att justera dessa värden kan en tekniker finjustera exakt hur trafiken ska flöda genom nätverket för att undvika flaskhalsar och maximera genomströmningen (throughput).

Hierarkisk design med Areas

För att OSPF ska kunna skala till mycket stora nätverk används ett system med områden (areas). Om alla routrar i ett stort campusnätverk skulle dela all information med varandra, skulle routingtabellerna och de matematiska beräkningarna snabbt konsumera för mycket minne och processorkraft (CPU). Lösningen är att dela upp nätverket i logiska zoner.

Hjärtat i denna hierarki är **Area 0**, även kallad ryggradsområdet (backbone area). Alla andra områden måste vara fysiskt eller logiskt anslutna till Area 0. Genom denna uppdelning begränsas spridningen av detaljerad topologi-information; en router i Area 1 behöver inte känna till varje enskild länkändring i Area 2. Vid gränserna mellan områden används **summering** (summarization) för att aggregera många specifika rutter till ett fåtal mer generella, vilket håller routingtabellerna kompakta och lättlästa. För miljöer med begränsade resurser kan man även använda specialiserade areatyper som **Stub** eller **NSSA** (Not-So-Stubby Area), vilka

filtrerar bort onödig extern information för att ytterligare minska nätverkets brus.

DR och BDR i Ethernet-segment

I nätverkstyper där många enheter delar samma medium, såsom ett vanligt Ethernet-segment (multi-access), skulle mängden grannrelationer (adjacencies) snabbt bli ohanterlig om alla routrar pratade direkt med alla. För att optimera detta väljer OSPF en **DR** (Designated Router) och en **BDR** (Backup Designated Router). Istället för ett nät av anslutningar skickar alla routrar sina uppdateringar enbart till DR, som sedan ansvarar för att distribuera informationen till resten av segmentet. Detta minskar mängden kontrolltrafik och säkerställer en stabil och förutsägbar kommunikation.

Säkerhet och prestanda

För att skydda nätverket mot felkonfigurationer eller illasinnade angrepp stödjer OSPF **autentisering** (authentication). Detta innebär att routrar måste bevisa sin identitet med ett lösenord eller en kryptografisk nyckel innan de tillåts utbyta routinginformation. Genom att kombinera detta med avancerad filtrering och summering kan en administratör skapa en miljö som inte bara är snabb och självläkande, utan också mycket robust och säker.

I professionella miljöer där redundans är ett krav är OSPF det självklara valet. Dess förmåga att reagera blixtsnabbt på topologiförändringar, tillsammans med en skalbar arkitektur och ett intelligent vägval baserat på bandbredd, gör det lämpligt för allt från medelstora företag till stora campusnätverk. Den enda egentliga nackdelen är att det kräver en djupare teknisk förståelse för hur LSAs, timers och olika areatyper samverkar för att undvika komplexa felsökningsscenarier.

Fördelar:

- *Snabb konvergens* – reagerar snabbt på fel.
- *Skalbar arkitektur* – areas minskar databas och uppdateringar.
- *Finare metric* – bandbreddsbaserad kostnad ger bättre vägval.
- *Många verktyg* – summering, filtrering, autentisering.

Nackdelar:

- *Mer komplex* – kräver förståelse för areas, LSAs, timers och adjacency-tillstånd.
- *Kräver korrekt design* – fel DR/BDR-placering eller area-gränser kan skapa problem.
- *Lite mer overhead* än statik/RIP.

Användningsområden:

- *Campus/enterprise* (medel-stora till stora nät).
- *Miljöer med redundans* där snabb återhämtning behövs.
- *Nät med många VLAN/subnät* där summering/areas ger ordning.

Exempel: En skola med distribution i två separata byggnader, som båda tillhör OSPF Area 0. Där konfigureras varje enskilt våningsplan som egna områden (areas). Om en fiberkabel mellan två våningar går av, skickas omedelbart en LSA ut. Samtliga routrar i nätverket uppdaterar sina kartor och räknar om de bästa vägarna. Inom loppet av en bråkdel av en sekund har trafiken styrts om via en sekundär väg, utan att användarna ens märker av avbrottet.

2.3.6 Första-hopp-redundans – VRRP/HSRP

Tekniken bygger på att två eller flera fysiska routrar samarbetar för att presentera en gemensam **virtuell IP-adress** (VIP) och en **virtuell MAC-adress** för klienterna i ett VLAN. En av enheterna utses till aktiv (Active/Master) och ansvarar för att vidarebefordra all trafik som skickas till den virtuella adressen. Den andra enheten befinner sig i vänteläge (Standby/Backup) och övervakar kontinuerligt den aktiva enhetens status genom kontrollmeddelanden (hellos).

Om den aktiva enheten slutar svara, tar standby-enheten omedelbart över rollen som aktiv. För att informera nätverkets switchar om att den virtuella MAC-adressen nu finns på en annan fysisk port, skickar den nya aktiva enheten ut ett så kallat **gratuitous ARP** (G-ARP). Detta meddelande uppdaterar switcharnas MAC-tabeller utan att klienterna behöver ändra sin konfiguration eller ens märka att ett hårdvarufel har inträffat.

För att skapa en robust och intelligent redundans används ofta funktionerna **preemption** och **tracking**:

- **Preemption:** Gör det möjligt för en router med högre prioritet att automatiskt återta rollen som aktiv när den kommer online igen efter ett avbrott eller underhåll.
- **Tracking:** Tillåter en gateway att övervaka statusen på sina egna upplänkar (uplinks). Om en kritisk länk mot nätverkets kärna går ner, kan routern automatiskt sänka sin egen prioritet så att en annan router, som har fungerande vägar vidare, kan ta över rollen som aktiv gateway.

Fördelar och nackdelar

Den främsta fördelen med FHRP är att redundansen är helt transparent för slutanvändarna. Eftersom den virtuella gateway-adressen förblir densamma, krävs inga manuella ändringar på klientnivå vid ett haveri. Konceptet är dessutom enkelt att dokumentera och testa, vilket gör det till en hörnsten i professionell nätverksdesign.

Det finns dock vissa begränsningar. Implementeringen innebär ytterligare protokoll att förvalta, där parametrar som prioriteringar, tidtagare (timers) och tracking-objekt måste konfigureras korrekt för att undvika instabilitet. Det är också viktigt att notera att VRRP och HSRP i sitt grundutförande inte erbjuder automatisk lastdelning (load balancing); endast en enhet är aktiv åt gången per virtuell adress. För att utnyttja båda enheternas kapacitet samtidigt krävs mer komplexa uppsättningar, såsom att låta olika enheter vara aktiva för olika VLAN. Slutligen löser FHRP endast redundansen för klienternas första hopp; det skyddar inte mot fel som ligger djupare i nätverkets routing- eller brandväggslogik.

Användningsområden och exempel

FHRP används primärt för VLAN-gateways i nätverkets distributions- eller kärnlager där avbrott i produktionen inte kan accepteras. Det är även ett outhärligt verktyg vid planerat underhåll, då tekniker kan flytta trafikflöden mellan enheter utan att användarna tappar sina sessioner.

Fördelar:

- *Transparent för klienter* – samma gateway-IP lever vidare vid fel.
- *Enkelt koncept* – lätt att förstå, testa och dokumentera.

- *Flexibelt* – preemption/track ger snabb och rätt failover.

Nackdelar:

- *Ytterligare protokoll att hantera* – prioritet, timers, tracking måste hållas rätt.
- *Ingen automatisk lastdelning* (GLBP kan, men är ovanligt/leverantörsbundet).
- *Löser bara gateway-resiliens* – inte bakomliggande routing- eller brandväggsproblem.

Användningsområden:

- *VLAN-gateways i distribution/core* där hög tillgänglighet krävs.
- *Migrering/underhåll* – byt hårdvara utan klientstopp.

Exempel: En nätverksdesign för VLAN 10 (Personal) använder den virtuella IP-adressen 10.10.0.1 som standard-gateway. I nätverkets distributionslager finns två switchar, Dist-SW-A och Dist-SW-B. Under normal drift är Dist-SW-A konfigurerad som aktiv och hanterar all trafik för personalstyrkan. Vid ett planerat servicearbete på Dist-SW-A, eller vid ett plötsligt strömavbrott, upptäcker Dist-SW-B bortfallet av kontrollmeddelanden och övertar rollen som aktiv gateway på några få sekunder. Genom att skicka ett gratuitous ARP informeras övriga switchar om förändringen, och de anställda kan fortsätta sitt arbete mot den virtuella adressen 10.10.0.1 utan märkbara störningar.

2.3.7 Summering och PBR – kontroll och städning

I takt med att ett nätverk expanderar växer även routingtabellerna, vilket kan leda till ökad resursförbrukning och mer komplex felsökning. För att hantera detta används **prefix-summering** (route summarization / prefix aggregation). Detta innebär att en router annonserar ett enda stort, sammanhållet nätverk (ett prefix) istället för att skicka ut information om varje enskilt litet subnät. Genom att "städa" routingtabellen på detta sätt minskas belastningen på enheternas processorer (CPU) och mängden administrativ trafik som skickas mellan olika geografiska platser eller areor begränsas. En länkändring i ett litet subnät behöver då inte tvinga

hela nätverket att räkna om sina kartor, så länge det summerade prefixet fortfarande är nåbart.

Medan summering handlar om att förenkla nätverkskartan, används **Policy-Based Routing (PBR)** för att skapa specifika undantag från den vanliga routinglogiken. Normalt väljer en router väg enbart baserat på paketets destinationsadress, men med PBR kan vi styra trafik baserat på källadress (source address), protokoll eller applikationstyp. Det kan exempelvis handla om att låta vanlig surftrafik gå via en billigare internetlänk, medan kritisk produktionstrafik eller specifika servrar styrs via en mer stabil fiberanslutning eller en specifik proxy. Eftersom PBR åsidosätter den ordinarie "bästa vägen"-logiken krävs noggrann dokumentation och ofta användning av övervakning via **SLA/tracking**. Om den väg som policyn pekar ut slutar fungera, måste routern kunna upptäcka detta för att inte skicka trafiken i ett "svart hål" (black hole).

Användningen av dessa tekniker innebär en avvägning. Summering kräver en välplanerad adressplan där nätverken ligger logiskt efter varandra, men det ger i gengäld ett mer stabilt och skalbart nätverk. PBR ger en granulär kontroll som är ovärderlig i komplexa miljöer för att uppfylla krav på säkerhet eller prestanda, men det ökar också den logiska komplexiteten och kan göra felsökning mer tidskrävande om man inte har full koll på vilka regler som faktiskt styr trafiken.

Fördelar:

- *Summering*: mindre tabeller och mindre brus mellan sajter.
- *PBR*: granular kontroll för särskilda flöden (compliance/prestanda).

Nackdelar:

- *Summering*: kräver planerad adressering; risk att "gömma" fel under en bred annons.
- *PBR*: ökar komplexitet; kan bryta "bästa-väg"-logiken och försvåra felsökning.

Användningsområden:

- *Större nät med OSPF* – summera per site/area-gräns.
- *Speciella trafikfall* – dirigera vissa källor/appar via annan länk/proxy.

2.3.8 Sammanfattning, Best Practice och felsökning

När vi ser på routing som helhet handlar det om att hitta balansen mellan enkelhet och automation. Valet av metod beror helt på nätverkets storlek och krav på tillgänglighet.

Metod	Lämplig miljö	Främsta fördel	Främsta nackdel
Statisk routing	Små nätverk, labbar, enskilda länkar	Full kontroll, ingen overhead	Ingen självläkning, svårskalat
RIP v2	Utbildning, mycket små stabila miljöer	Enkel konfiguration	Långsam konvergens, 15 hopp
OSPF	Campus, företag, datacenter	Snabb konvergens, skalbarhet	Högre komplexitet, CPU-last

En fackmässig nätverksdesign (best practice) innebär att man konsekvent använder standardrutter (default routes) mot internet och summerar interna nätverk vid strategiska gränser. Man bör även sträva efter att hålla den administrativa distansen (administrative distance) och metriken (metric) så förutsägbar som möjligt för att undvika asymmetriska flöden.

Vid felsökning av routingproblem bör man arbeta systematiskt lager för lager. Processen inleds med att verifiera länkstatus och den lokala adresseringen (ARP/ND) för att säkerställa att enheten kan prata med sin standard-gateway (default gateway). Därefter granskas routingtabellen för att se om det finns en matchande rutt och att nästa hopp (next hop) är korrekt. Om dynamiska protokoll används kontrolleras status för grannrelationer (adjacencies) och eventuella timers. Först när den logiska vägen är bekräftad bör man kontrollera brandväggspolicys längs vägen och slutligen själva applikationen.

De vanligaste felen som en tekniker möter är ofta relaterade till saknade standardrutter, felaktiga nästa hopp eller överlappande nätverksprefix. Ett effektivt sätt att lokalisera var trafiken stannar är att använda verktyg som **ping** och **tracert** (tracert i Windows och mtr i Linuxsystem). På professionell

utrustning ger kommandon som `show ip route` och `show ip ospf neighbor` en direkt inblick i routerns beslutsunderlag.

Summering:

- *Grundverktyg:* `ping`, `tracroute`/`tracert`, `arp -a`, `ip neigh`, `ip route`/`route print`.
- *På nätutrustning:* `show ip route`, `show ip ospf neighbor`, `show ip protocols`, `show interfaces`.
- *Vanliga fel:* saknad default, fel nästa hopp, överlappande prefix, asymmetrisk routing, blockerad port i brandvägg.

Exempel: *En klient kan surfa på internet men når inte en server på en annan filial. Genom att köra en `tracroute` upptäcks att trafiken stannar direkt efter den lokala brandväggen. En kontroll av routingtabellen visar att rutten till filialens nätverk (10.50.0.0/16) finns där, men brandväggens regelverk (inter-VLAN policy) saknar en tillåtelse för just den trafiken. Efter att regeln har lagts till och validerats fungerar kommunikationen igen.*

Kontrollfrågor

1. Förklara begreppet längsta-prefix-matchning (longest prefix match) och ge ett exempel på när det avgör vägvalet.
2. Vad är skillnaden mellan administrativ distans (administrative distance) och metrik (metric)?
3. Vilka är de tre största riskerna med att enbart använda statisk routing i ett växande nätverk?
4. Beskriv liknelsen mellan statisk routing och en papperskarta kontra dynamisk routing och en GPS.
5. Varför anses RIP v2 vara föråldrat i moderna produktionsmiljöer, trots att det är ett dynamiskt protokoll?
6. Vad är syftet med att dela upp ett OSPF-nätverk i olika areor (areas), och vilken roll spelar Area 0?
7. Hur fungerar en Designated Router (DR) i ett OSPF-nätverk och varför behövs den?
8. Förklara hur HSRP eller VRRP kan ge en klient oavbruten internetåtkomst även om en fysisk router går sönder.
9. Vad innebär prefix-summering (route summarization) och vilken teknisk fördel ger det för routerns CPU?
10. Varför är det viktigt att kontrollera returvägen (return path) när man felsöker routingproblem?

Fördjupningslänkar

Cisco Networking Academy: Routing Concepts (Omfattande utbildningsmaterial om grundläggande routing) –

<https://www.netacad.com/>

Practical Networking: Routing Table Logic (Tydlig genomgång av hur en router fattar beslut) –

<https://www.practicalnetworking.net/featured/routing-table-logic/>

OSPF Tutorial - Flight-Sustained (Djupdykning i OSPF areas, LSAs och algoritmer) – <https://www.flightsustained.com/ospf-tutorial/>

RFC 2328: OSPF Version 2 (Den officiella specifikationen för OSPF) –

<https://datatracker.ietf.org/doc/html/rfc2328>

PacketLife.net: Routing Cheat Sheets (Samling av snabbguider för RIP, OSPF och BGP) – <https://packetlife.net/library/cheat-sheets/>

Cisco: HSRP Configuration Guide (Teknisk vägledning för konfiguration av första-hopp-redundans) –

https://www.cisco.com/c/en/us/td/docs/switches/lan/catalyst3750/software/release/12-2_55_se/configuration/guide/scg3750/swhsrp.html

OPNsense Documentation: Static Routes (Guide för hur man hanterar rutter i ett grafiskt gränssnitt) –

<https://docs.opnsense.org/manual/routes.html>

Network Lessons: Route Summarization (Pedagogiska exempel på hur man räknar ut summerade prefix) –

<https://networklessons.com/cisco/ccna-routing-switching/route-summarization-explained>

Red Hat: Configuring Static Routes in Linux (Hur man hanterar routingtabellen i en Linux-miljö) –

https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/9/html/configuring_and_managing_networking/configuring-static-routes_configuring-and-managing-networking

Wireshark Wiki: OSPF (Analysera OSPF-paket och LSAs i realtid) –

<https://wiki.wireshark.org/OSPF>

Egna anteckningar

...

2.4 DHCP, DNS och NAT

För att ett nätverk ska vara praktiskt användbart i en modern produktionsmiljö räcker det inte med enbart fungerande switchar och routrar. Det krävs även en uppsättning stödtjänster som automatiserar adressering, förenklar navigation och hanterar bristen på publika adresser. Kapitel 2.4 fokuserar på de tre hörnstenarna **DHCP**, **DNS** och **NAT**, vilka tillsammans utgör det logiska ramverket för hur enheter identifieras och kommunicerar både lokalt och över internet.

Dessa tjänster fungerar som nätverkets infrastruktur för användarvänlighet och skalbarhet. Genom att implementera robusta lösningar för namnkonvertering (name resolution) och dynamisk tilldelning av nätverksparametrar (dynamic allocation) minskas det administrativa arbetet avsevärt samtidigt som risken för mänskliga fel, såsom dubblade IP-adresser, minimeras. Att förstå samspelet mellan dessa protokoll är avgörande för att kunna bygga nätverk som inte bara är tekniskt korrekta, utan också driftsäkra och lätta att underhålla i en professionell miljö.

2.4.1 DHCP – grunder

DHCP (Dynamic Host Configuration Protocol) är den tjänst som automatiserar tilldelningen av nätverksparametrar (network parameters) till klienter. Istället för att en tekniker manuellt behöver konfigurera varje enskild enhet med en statisk adress, ser DHCP till att enheter får all nödvändig information för att kunna kommunicera så snart de ansluter till nätverket. Detta inkluderar inte bara en unik **IP-adress** (IP address), utan även **nätmask** eller **prefix** (subnet mask/prefix), **standard-gateway** (default gateway) och adresser till **DNS-servrar** (DNS servers).

När en klient startar upp och saknar en giltig nätverkskonfiguration inleds en utbyte av meddelanden mellan klienten och servern. Denna sekvens kallas ofta för **DORA-processen** (DORA process), baserat på de fyra stegen i protokollet:

1. **Discover (Upptäckt):** Klienten skickar ut ett broadcast-meddelande (DHCPDISCOVER) på det lokala nätverket för att hitta en tillgänglig DHCP-server. Eftersom klienten ännu inte har en IP-

adress, används adressen 0.0.0.0 som avsändare och 255.255.255.255 som mottagare.

2. **Offer (Erbjudande):** En eller flera DHCP-servrar som tar emot förfrågan svarar med ett DHCPOFFER. Detta meddelande innehåller ett förslag på en ledig IP-adress samt övriga nätverksinställningar.
3. **Request (Begäran):** Klienten tar emot erbjudandet och svarar med ett DHCPREQUEST. Genom detta meddelande meddelar klienten formellt att den vill använda den föreslagna adressen och informerar samtidigt eventuella andra servrar om att deras erbjudanden kan dras tillbaka.
4. **Acknowledgement (Bekräftelse):** Servern skickar ett slutgiltigt DHCPACK som bekräftar att klienten nu har rätt att använda adressen. I detta skede registreras även kopplingen mellan klientens fysiska MAC-adress och den tilldelade IP-adressen i serverns databas.

Den tilldelade adressen ägs inte permanent av klienten, utan tilldelas under en begränsad tidsperiod som kallas för en **lease** (lease). Under denna period behåller klienten adressen, men för att undvika att anslutningen bryts påbörjas en automatisk förnyelseprocess (renewal) när hälften av tiden har passerat. Om klienten lämnar nätverket utan att förnya sin lease, blir adressen efter en tid återigen tillgänglig för andra enheter i serverns adresspool (address pool).

Utöver de grundläggande parametrarna kan DHCP-servern skicka med ytterligare instruktioner via så kallade **DHCP-options** (DHCP options). Dessa numeriska koder tillåter administratören att distribuera specifika inställningar som krävs för nätverkets funktion. Vanliga exempel inkluderar **Option 3** för att definiera routern (gateway), **Option 6** för DNS-servrar, **Option 15** för domänsuffix (domain suffix) och **Option 42** för tidsservrar via NTP (Network Time Protocol). För enheter som kräver en fast adress, såsom nätverksskrivare eller servrar, kan en **reservation** (reservation) skapas i servern. Detta innebär att en specifik IP-adress permanent låses till en unik MAC-adress, vilket kombinerar enkelheten hos DHCP med stabiliteten hos statisk adressering.

Exempel: En dator i VLAN 20 startas upp och ropar efter en adress. Eftersom DHCP-servern ofta är centraliserad, används en **relay**

(DHCP relay) i nätverkets gateway för att vidarebefordra förfrågan från elevens lokala segment till servern. I detta scenario får klienten exempelvis adressen 10.20.1.57/22, med 10.20.0.1 som sin gateway och 10.40.0.10–11 som DNS-servrar. Allt detta sker på några millisekunder, vilket gör att användaren kan börja arbeta omedelbart utan manuell handpåläggning från IT-avdelningen.

2.4.2 DHCP-design: scopes, relay och robusthet

När ett nätverk växer och delas upp i flera VLAN (virtual local area networks) blir det opraktiskt och kostsamt att placera en fysisk DHCP-server i varje enskilt segment. Istället används en centraliserad design där en eller två kraftfulla servrar hanterar adressutdelningen för hela organisationen. För att detta ska fungera tekniskt krävs en **DHCP-relay** (DHCP relay), ofta konfigurerad som en **IP helper-address** (IP helper address) på nätverkets gateway. Eftersom DHCP-processen inleds med en broadcast-förfrågan som normalt inte passerar en router, agerar gatewayen som en mellanhand. Den fångar upp klientens förfrågan, paketerar om den till ett unicast-meddelande och skickar den direkt till den centrala servern.

På den centrala servern skapas unika **adressomfång** (scopes) för varje VLAN som ska servas. Ett scope innehåller det specifika subnätets nätverksadress, nätmask och den pool av adresser som är tillgängliga för utdelning. En viktig del av designen är att definiera **undantag** (exclusions) för de adresser som används statiskt i nätverket, såsom adresser till switchar, skrivare, gateways och brandväggar. Genom att exkludera dessa från den dynamiska utdelningen förhindras IP-adresskonflikter (IP address conflicts) som annars skulle kunna sänka kritiska infrastrukturkomponenter.

En strategisk parameter i nätverksdesignen är valet av **giltighetstid** (lease time). Denna bör anpassas efter hur rörlig användargruppen är i det specifika segmentet. I ett **gästnät** (guest network) med hög omsättning av enheter är det klokt att använda en kort giltighetstid, exempelvis 8 timmar. Detta säkerställer att adresser snabbt återgår till poolen när gästerna lämnar byggnaden, vilket förhindrar att adresserna tar slut. För **personalsegment** (staff networks) eller fasta arbetsplatser där samma datorer är anslutna dag efter dag, är en längre tid på 3–7

dagar mer lämplig. Det minskar mängden kontrolltrafik i nätverket och ger en stabilare miljö för användarna.

För att säkerställa hög tillgänglighet (high availability) och robusthet implementeras ofta **DHCP-failover** (DHCP failover). Genom att köra två servrar i ett kluster kan de dela på lasten (load share) eller fungera i ett läge där den ena står i beredskap (hot-standby). Om den primära servern går ner tar reserven omedelbart över hanteringen av alla scopes utan att klienterna märker något avbrott. Detta är kritiskt i miljöer där nätverksåtkomst är direkt beroende av att snabbt kunna få en fungerande IP-adress.

Säkerheten kring DHCP hanteras primärt i accesslagret genom **DHCP-snooping** (DHCP snooping). Denna funktion förhindrar att en obehörig eller felkonfigurerad enhet agerar som en falsk DHCP-server (rogue DHCP server). Switchen konfigureras så att endast specifika portar, exempelvis de som leder mot nätverkets kärna och den legitima servern, markeras som **betrodda** (trusted). Alla DHCP-svar som anländer på en obetrodd port blockeras omedelbart, vilket skyddar användarna från att få felaktiga gateways eller DNS-inställningar som skulle kunna användas för avlyssning.

Förtydligande (sammanfattning):

- **DHCP-relay (DHCP relay):** En central server når och servar klienter i många olika VLAN via gatewayens hjälp.
- **Giltighetstid (lease time):** Konfigureras kort i gästnät för att spara adresser och längre i personalnät för stabilitet.
- **Redundans (failover):** Användning av två servrar för att säkerställa hög tillgänglighet (high availability).
- **Säkerhet:** Implementering av DHCP-snooping (DHCP snooping) i accesslagret för att stoppa obehöriga servrar.

Exempel: I en skolmiljö har VLAN 30 (Gäst) en kort giltighetstid på 8 timmar för att hantera det stora flödet av mobiltelefoner och besökare, medan VLAN 10 (Personal) har en inställning på 5 dagar för att skapa stabilitet för lärarnas datorer. DHCP-snooping är aktivt på samtliga access-switchar, där endast portarna mot nätverkets kärna är betrodda. Om en elev av misstag skulle koppla in en egen

router i ett klassrum, kommer dess DHCP-svar att stoppas av switchen innan de hinner störa andra användare på skolan.

2.4.3 DNS – Grundläggande funktion och namnupplösning

DNS (Domain Name System) fungerar som nätverkets och internetets telefonbok. Dess primära uppgift är att översätta mänskligt läsbara namn, som `skolan.se`, till de maskinläsbara IP-adresser som routrar och datorer använder för att hitta varandra. Utan DNS skulle vi tvingas memorera komplexa sifferkombinationer för varje tjänst och webbplats vi vill använda. Processen fungerar även i motsatt riktning genom **PTR-poster** (PTR records), där en IP-adress kan översättas tillbaka till ett namn (reverse DNS), vilket ofta används för loggning och säkerhetsverifiering.

Systemet är uppbyggt i en strikt hierarki som börjar vid **rotnivån** (root), följt av **toppdomäner** (TLD - Top Level Domains) som `.se`, `.com` eller `.local`, och slutligen specifika **zoner** (zones) som ägs av organisationer. När en enhet behöver slå upp ett namn vänder den sig till en **rekursiv resolver** (recursive resolver). Denna fungerar som en förmedlare som letar upp svaret genom att fråga sig fram i hierarkin tills den når den **autoritativa namnservern** (authoritative name server) som faktiskt äger informationen för den specifika domänen.

För att göra nätverket effektivare används **cachelagring** (caching). När en resolver väl har fått ett svar sparar den det lokalt under en viss tid så att nästa klient som frågar efter samma namn kan få ett svar omedelbart. Hur länge informationen får sparas i cachen styrs av ett värde som kallas **TTL** (Time to Live). Ett lågt TTL-värde gör att ändringar sprids snabbt men ökar belastningen på servrarna, medan ett högt värde sparar resurser men gör att gamla adresser lever kvar längre vid en flytt eller förändring.

Inom DNS-zonen finns olika typer av poster för olika ändamål:

- **A-post** (A record) och **AAAA-post** (AAAA record): Kopplar ett namn till en IPv4- respektive IPv6-adress.
- **CNAME-post** (CNAME record): Fungerar som ett alias (alias) där ett namn pekar på ett annat namn.
- **MX-post** (MX record): Anger vilken server som ansvarar för att ta emot e-post (mail) för domänen.

- **TXT-post** (TXT record): Innehåller textinformation, ofta använd för säkerhetsinställningar som SPF och DKIM för att verifiera e-postavsändare.
- **SRV-post** (SRV record): Används för att identifiera specifika tjänster (services) i nätverket, såsom LDAP eller Kerberos. I en miljö med Active Directory (AD) är dessa poster helt kritiska för att klienter ska kunna hitta domänkontrollanter (domain controllers).
- **NS-post** (NS record): Pekar ut vilka namnservrar (name servers) som har rätten att svara auktoritativt för zonen.

I en företagsmiljö används ofta en **AD-integrerad DNS** (AD-integrated DNS). Detta innebär att DNS-tjänsten körs direkt på domänkontrollanterna och att zondata replikeras säkert mellan dem. Detta är nödvändigt för att de tidigare nämnda SRV-posterna ska kunna uppdateras dynamiskt när servrar och tjänster startar upp i nätverket.

***Exempel:** En klient vill ansluta till servern `files.skolan.local`. Den skickar en förfrågan till nätverkets lokala resolver. Eftersom resolvern är auktoritativ för den interna zonen (tack vare AD-integreringen), hittar den omedelbart A-posten `10.40.0.50` i sin databas. Svaret returneras till klienten, vars webbläsare eller filhanterare då kan kontakta rätt IP-adress direkt utan att användaren någonsin behövt känna till den bakomliggande adresseringen.*

Förtydligande (sammanfattning):

- **Posttyper:** A/AAAA, CNAME, MX, TXT, SRV och NS är de vanligaste och viktigaste posterna i en zon.
- **Rekursiv resolver (recursive resolver):** Den enhet som tar emot klientens fråga, letar upp svaret och sparar det i sin cache (cache).
- **Giltighetstid (TTL):** Ett värde som bestämmer hur länge ett svar får cachelagras innan det måste hämtas på nytt.
- **AD-integrerad DNS:** En förutsättning i moderna Windows-miljöer för att hantera SRV-poster och intern namnlösning (name resolution) på ett säkert och automatiserat sätt.

2.4.4 DNS-design: split-horizon, forwarders och redundans

I en professionell nätverksmiljö, särskilt inom skola och förvaltning, räcker det sällan med en enkel standardkonfiguration av DNS. För att balansera behovet av intern säkerhet med snabb åtkomst till internetresurser krävs en genomtänkt design som hanterar olika namnrymder och säkerställer att tjänsten alltid är tillgänglig, även vid hårdvarufel.

Ett av de vanligaste designmönstren i större nätverk är **split-horizon DNS** (split-horizon DNS). Detta innebär att organisationen hanterar två olika versioner av sin DNS-information beroende på varifrån förfrågan kommer. Interna klienter får svar från en lokal namnserver som har kännedom om interna resurser och domäner, exempelvis

*.skolan.local, vilka aldrig exponeras mot internet. Externa användare som försöker nå skolans publika webbplats på *.skolan.se får istället svar från en publik namnserver i en DMZ eller hos en extern leverantör. Denna uppdelning ökar säkerheten dramatiskt genom att dölja nätverkets interna struktur och IP-adresser för omvärlden, samtidigt som interna användare får snabb och korrekt namnupplösning (name resolution) för sina lokala tjänster.

För att optimera hastigheten och kontrollen över externa uppslagningar använder nätverkets rekursiva resolvers ofta **vidarebefordrare** (forwarders). Istället för att den lokala servern själv ska gå hela vägen till rot-servrarna för varje ny förfrågan, skickas frågan vidare till en förvald kraftfull DNS-tjänst, såsom internetoperatörens (ISP) servrar eller en filtrerande säkerhetstjänst (t.ex. Cisco Umbrella eller Cloudflare). Detta snabbar inte bara upp svarstiderna tack vare stora befintliga cacher (caches), utan tillåter även administratören att tillämpa policys och blockera skadliga domäner redan på nätverksnivå innan de når användarens webbläsare.

Redundans (redundancy) är en kritisk faktor i DNS-designen. Eftersom nästan ingen modern nätverkskommunikation fungerar utan fungerande namnupplösning, bör en miljö alltid ha minst två aktiva resolvers. I en virtualiserad miljö är det god praxis att placera dessa på olika fysiska värdar (hosts) och gärna separera dem i olika VLAN eller säkerhetszoner för att minimera risken att båda slås ut samtidigt vid ett nätverksfel. Genom att tilldela båda dessa adresser till klienterna via DHCP, kan

klienten automatiskt växla till den sekundära servern om den primära skulle sluta svara.

Förvaltningen av DNS-zonerna kräver också en medveten strategi kring **giltighetstid** (TTL - Time to Live). För stabila resurser som filservrar eller gateways kan ett längre TTL-värde på exempelvis 24 timmar användas för att minska belastningen. Om en tjänst däremot planeras att flyttas eller om man befinner sig i en testfas, bör TTL sänkas till några minuter för att säkerställa att ändringar sprids snabbt i nätverket. Slutligen ger en aktiv loggning (logging) och övervakning av svarstider en ovärderlig insyn i nätverkshälsan. Onormala mönster i DNS-trafiken, såsom en plötslig ökning av uppslagningar mot okända domäner, kan vara ett tidigt tecken på en pågående cyberattack eller infekterade klienter.

Exempel: En skola använder två AD-integrerade DNS-servrar på adresserna 10.40.0.10 och 10.40.0.11 för att hantera den interna domänen skolan.local. För alla externa frågor är dessa konfigurerade att använda operatörens forwarders för maximal prestanda. För att ytterligare stärka säkerheten finns en tredje, läsande resolver placerad i en isolerad DMZ. Denna server loggar all utgående trafik och skickar omedelbart ett larm till IT-avdelningen om den upptäcker att klienter försöker kommunicera med domäner som är kända för att sprida skadlig kod.

Förtydligande (sammanfattning):

- **Split-horizon:** Separerar interna och externa zoner (zones) för att dölja intern infrastruktur och öka säkerheten.
- **Forwarders:** Delegerar externa uppslagningar till kraftfulla servrar för att öka hastigheten och möjliggöra filtrering.
- **Redundans (redundancy):** Kräver minst två oberoende resolvers för att garantera att namnupplösningen alltid fungerar.
- **Giltighetstid (TTL):** Justeras baserat på hur ofta en post (record) förväntas ändras; kort för rörliga tjänster och lång för stabila.

2.4.5 NAT och PAT – grunder och begrepp

Eftersom antalet tillgängliga publika IPv4-adresser är begränsat, bygger nästan all modern kommunikation mot internet på adressöversättning

(address translation). Denna teknik gör det möjligt för ett nätverk att använda privata adresser (private addresses) enligt standarden **RFC1918** internt, men ändå kommunicera med omvärlden genom en eller ett fåtal publika adresser. Den mest grundläggande formen av detta kallas **NAT44** (Network Address Translation för IPv4).

I praktiken är den absolut vanligaste varianten av adressöversättning **PAT** (Port Address Translation), ibland även kallat "NAT overload". Här delar ett stort antal interna klienter på en enda publik IP-adress. För att nätverksutrustningen ska veta vilket svarspaket som hör till vilken intern dator används unika portnummer (port numbers) för att skilja de olika flödena åt. När trafiken lämnar nätverket kallas processen för **SNAT** (Source NAT), då källadressen (source address) i paketet skrivs om från en privat adress till den publika adressen på brandväggens utsida.

När vi däremot vill att någon från internet ska kunna nå en specifik tjänst inne i vårt nätverk, exempelvis en webbserver eller ett fjärrskrivbord, används **DNAT** (Destination NAT), mer känt som **portvidarebefordran** (port forwarding). Här skriver brandväggen om destinationsadressen (destination address) i det inkommande paketet så att det styrs till rätt server på det interna nätverket. Det är viktigt att förstå att även om NAT döljer den interna nätverksstrukturen för omvärlden, utgör det i sig inget fullständigt skydd. Som tekniker bör man alltid se NAT som en ren adresseringsteknik som kräver kompletterande brandväggsregler (firewall rules) för att faktiskt kontrollera och filtrera trafiken.

I den modernare standarden **IPv6** har behovet av NAT i princip försvunnit. Eftersom det finns ett nästan oändligt antal adresser tilldelas enheter i stället **globala unicast-adresser** (global unicast addresses) som är unika i hela världen. Säkerheten hanteras här uteslutande genom brandväggspolicys och segmentering (segmentation) i stället för adressöversättning. Även om tekniker som **NPTv6** (Network Prefix Translation) existerar för specifika specialfall, är målet i IPv6-miljöer en rak och obruten kommunikation från källa till mål (end-to-end connectivity).

Exempel: En skola förfogar över den publika IP-adressen 203.0.113.10 på sin brandväggs yttre gränssnitt. När elever på VLAN 20 (10.20.0.0/22) surfar ut på internet används SNAT (PAT) för att översätta deras interna adresser till skolans gemensamma

publika adress. För att externa användare ska kunna nå skolans interna webbserver i DMZ-zonen har en DNAT (port forwarding) konfigurerats för TCP-port 443. All trafik som anländer till brandväggens utsida på port 443 skickas då automatiskt vidare till servern på den interna adressen 192.0.2.20.

Förtydligande (sammanfattning):

- **SNAT/PAT:** Möjliggör att många interna klienter delar på en publik adress för utgående trafik genom att använda unika portnummer.
- **DNAT/Port forwarding:** Dirigerar trafik från en publik port (port) till en specifik intern tjänst eller server.
- **NAT ≠ säkerhet:** Adressöversättning måste alltid kompletteras med en aktiv brandväggspolicy (firewall policy) för att nätverket ska vara skyddat.
- **IPv6:** I normalfallet används ingen NAT; säkerhet och ordning upprätthålls genom brandväggar och logisk segmentering (segmentation).

2.4.6 Vanliga tillämpningar och fallgropar

Adressöversättning (NAT) och namnupplösning (DNS) fungerar ofta sömlöst, men i mer komplexa nätverksmiljöer uppstår specifika utmaningar när dessa tekniker interagerar. En av de vanligaste fallgroparna är behovet av **Hairpin NAT**, även kallat **NAT Loopback**. Detta fenomen uppstår när en intern klient försöker nå en intern server genom att använda serverns publika IP-adress eller domännamn. Eftersom trafik som skickas mot den publika adressen normalt förväntas komma utifrån, kan brandväggen misslyckas med att dirigera om paketet tillbaka in i nätverket utan en specifik regel för detta ändamål.

En annan betydande utmaning är **Double NAT**, vilket ofta uppstår när en egen brandvägg placeras bakom internetleverantörens (ISP) router. Detta skapar två lager av adressöversättning som komplicerar både portvidarebefordran (port forwarding) och uppsättning av VPN-tunnlar. I värsta fall kan internetleverantören själv använda **CGNAT** (Carrier-Grade NAT), vilket innebär att flera av deras kunder delar en enda publik adress redan i operatörens nät. Detta kan i praktiken omöjliggöra inkommande

anslutningar till nätverket helt och hållet, då slutanvändaren inte har kontroll över den yttersta adressöversättningen.

Vissa äldre eller mer specialiserade protokoll är dessutom mycket känsliga för adressöversättning. **SIP** (Session Initiation Protocol), som används för IP-telefoni, och äldre **FTP** i aktivt läge (Active FTP) bäddar in IP-adresser direkt i datapaketets innehåll. Detta kräver att brandväggen utför en djupare paketinspektion genom en **ALG** (Application Layer Gateway) för att manuellt skriva om dessa adresser, eller att man går över till modernare och mer NAT-vänliga lägen.

Inom adressering (DHCP) och namnhantering (DNS) är de vanligaste problemen ofta relaterade till grundläggande konfigurationsmissar. Om en klient inte får någon IP-adress alls beror det ofta på att porten tillhör fel VLAN, att en nödvändig **DHCP-relay** (IP helper) saknas på gatewayen, eller att det saknas lediga adresser i rätt adressomfång (scope). För DNS kan felaktiga vidarebefordrare (forwarders) eller missar i **split-horizon**-konfigurationen leda till att interna tjänster inte hittas, medan för långa **TTL-värden** kan orsaka mystiska fel där gamla adresser lever kvar långt efter att en server har flyttats.

***Exempel:** Elever på en skola upptäcker att de kan nå intranätet via adressen `intranet.skolan.se` när de sitter hemma, men inte när de är uppkopplade i klassrummet. Orsaken är att **hairpin NAT** saknas i brandväggen; den befintliga DNAT-regeln är endast konfigurerad att fungera för trafik som anländer på brandväggens yttre gränssnitt. För att lösa problemet skapas antingen en specifik loopback-regel i NAT-tabellen, eller så läggs en intern A-post in i den lokala DNS-servern som pekar namnet direkt på serverns interna IP-adress i DMZ-zonen.*

Förtydligande (sammanfattning):

- **Hairpin NAT:** Nödvändigt för att interna klienter ska nå interna tjänster via nätverkets publika IP-adress.
- **Double NAT/CGNAT:** Skapar problem för inkommande trafik, portöppningar och stabilitet i VPN-anslutningar.
- **Protokollkänslighet:** Tjänster som SIP och FTP kan kräva specifik inspektion (ALG) för att fungera genom en brandvägg.

- **Felsökning (DHCP/DNS):** Vanliga felkällor inkluderar saknade relays, felaktiga VLAN-medlemskap, fel i forwarders eller utdaterad cache på grund av långa TTL-värden.

2.4.7 Felsökning: metodik och verktyg

Vid felsökning av nätverkstjänster är den mest effektiva metoden att arbeta systematiskt från klienten och utåt, lager för lager. Genom att isolera varje komponent i kedjan kan man snabbt avgöra om problemet ligger i den fysiska anslutningen, i adresseringen (DHCP), i namnupplösningen (DNS) eller i själva trafikstyrningen (NAT/brandvägg).

När en klient har problem med att få en IP-adress (DHCP) inleds felsökningen med att kontrollera den fysiska länken (link light) och verifiera att porten i switchen tillhör rätt VLAN. Om nätverket använder säkerhetsfunktioner som **DHCP-snooping** bör loggarna i switchen granskas för att se om förfrågningar har blockerats. På klientsidan kan man tvinga fram en ny adressförfrågan; i Windows görs detta med kommandona `ipconfig /release` följt av `ipconfig /renew`, medan man i Linux-miljöer ofta använder `dhclient -r` för att släppa adressen och `dhclient` för att begära en ny. Om problemet kvarstår bör administratören kontrollera status för **DHCP-relay** (IP-helper) på gatewayen och se efter att det finns lediga adresser kvar i serverns adressomfång (scope).

För problem rörande namnupplösning (DNS) används verktyg som **nslookup** eller **dig** för att testa om klienten kan kommunicera med sin tilldelade namnserver (resolver). Det är viktigt att kontrollera vilket namn som faktiskt efterfrågas och om det finns komplicerade kedjor av alias (**CNAME-poster**). Om en ändring nyligen har gjorts men inte verkar ha slagit igenom, kan det vara nödvändigt att rensa klientens lokala cache (cache) med kommandot `ipconfig /flushdns` i Windows eller motsvarande tjänst i Linux. En viktig del i metodiken är att alltid testa anslutningen direkt mot en IP-adress; om det går att nå en server via IP men inte via namn, har man framgångsfullt isolerat felet till DNS-lagret.

Felsökning av adressöversättning (NAT) kräver ofta insyn i brandväggens eller routerns interna **NAT-tabell** (translation table). Här verifieras att översättningen faktiskt skapas och att trafiken tillåts flöda i rätt riktning enligt brandväggens regelverk. Ett kraftfullt sätt att gå djupare är att

utföra **paketfångst** (packet capture/tcpdump) på både det inre och yttre gränssnittet samtidigt. Om paketet anländer på insidan men aldrig lämnar utsidan med en översatt adress, ligger felet i NAT-konfigurationen. I fall där interna tjänster inte nås inifrån nätverket via sitt publika namn, bör man särskilt kontrollera om **hairpin NAT** (loopback) är korrekt implementerat.

Exempel: En klient kan nå en server via dess IP-adress `10.40.0.50`, men misslyckas med att ansluta via namnet `files.skolan.local`. Vid kontroll med `nslookup` visar det sig att klienten försöker ställa frågan direkt till en publik DNS-server (`8.8.8.8`) istället för nätverkets interna resolver. Detta är en policyavvikelse som ofta beror på manuella inställningar på klienten eller felaktiga parametrar i DHCP. Åtgärden blir att tvinga klienten att använda skolans godkända resolvers genom att justera **DHCP-option 6** och samtidigt lägga till brandväggsregler som blockerar utgående DNS-trafik till andra servrar än de som är godkända.

Förtydligande (sammanfattning):

- **DHCP:** Verifiera rätt VLAN och fungerande relay; granska lease-status på servern och klientens kommandon (`ipconfig/dhclient`).
- **DNS:** Använd `nslookup` eller `dig` för att testa uppslag; kontrollera TTL och rensa lokal cache (`flushdns`) vid behov.
- **NAT:** Kontrollera översättningstabellen (translation table), trafikriktning och eventuellt behov av hairpin NAT.
- **Avgränsa:** Testa alltid förbindelse med både IP och namn för att snabbt isolera om felet ligger i namnupplösningen eller i själva anslutningen.

Checklista för felsökning av nätverkstjänster:

1. Fysisk länk och VLAN-tillhörighet?
2. IP-adress erhållen via DHCP? (Kolla `ipconfig / ifconfig`)
3. Kan jag pinga min default gateway?
4. Fungerar namnupplösning lokalt? (`nslookup files.skolan.local`)
5. Fungerar namnupplösning externt? (`nslookup google.se`)

6. Finns det en NAT-översättning i brandväggen?

Viktiga portnummer att komma ihåg:

- DNS: Port 53 (UDP/TCP)
- DHCP: Port 67 (Server) och 68 (Klient) (UDP)
- HTTP/HTTPS: Port 80 / 443 (TCP)
- NTP: Port 123 (UDP)

Kontrollfrågor

1. Beskriv de fyra stegen i DORA-processen (DORA process) och förklara vad som händer i varje steg.
2. Varför är en DHCP-relay (IP helper-address) nödvändig i ett segmenterat nätverk med flera VLAN?
3. Diskutera för- och nackdelar med korta respektive långa giltighetstider (lease times) för olika typer av nätverk.
4. Hur fungerar DHCP-snooping (DHCP snooping) och varför är det viktigt att definiera betrodda portar (trusted ports)?
5. Förklara skillnaden mellan en rekursiv resolver (recursive resolver) och en auktoritativ namnserver (authoritative name server).
6. Vilken roll spelar SRV-poster (SRV records) i en miljö med Active Directory?
7. Vad är syftet med split-horizon DNS och vilken säkerhetsfördel ger det organisationen?
8. Förklara tekniken PAT (Port Address Translation) och hur den gör det möjligt för hundratals enheter att dela på en enda publik IP-adress.
9. Beskriv ett scenario där hairpin NAT (NAT loopback) krävs och vad som händer om funktionen saknas.
10. Om en klient når en server via IP-adress men inte via dess domännamn, vilka tre steg skulle du ta för att felsöka problemet?

Fördjupningslänkar

Cisco: Understanding and Troubleshooting DHCP in Switched Networks (En djupdykning i DHCP-processer och relay) - [cisco.com](https://www.cisco.com)

Internet Systems Consortium (ISC): DHCP Failover Design (Teknisk vägledning för redundanta DHCP-lösningar) - [isc.org](https://www.isc.org)

Cloudflare: What is DNS? (En mycket pedagogisk genomgång av DNS-hierarkin och uppslag) - cloudflare.com/learning/dns/what-is-dns/

Microsoft Learn: AD-Integrated DNS Zones (Hur DNS samverkar med Active Directory i Windows Server) - learn.microsoft.com

Practical Networking: NAT and PAT Explained (Tydliga visualiseringar av hur portar och adresser översätts) - practicalnetworking.net

RFC 1918: Address Allocation for Private Internets (Standarddokumentet för de privata adressrymderna) - datatracker.ietf.org/doc/html/rfc1918

DNS Checker: Propagation Tool (Ett verktyg för att se hur DNS-ändringar sprids över världen) - dnschecker.org

OPNsense Documentation: NAT Port Forwarding (Praktisk guide för DNAT och Hairpin NAT) - docs.opnsense.org

Wireshark Wiki: DHCP Analysis (Hur man läser och analyserar DHCP-paket i en paketfångst) - wiki.wireshark.org/DHCP

Google Public DNS: Security and Privacy (Information om hur publika resolvers hanterar förfrågningar och hot) - developers.google.com/speed/public-dns

Egna anteckningar

2.5 Vanliga nätverksangrepp och hur vi försvarar oss

Många angrepp lyckas inte för att de är särskilt "smarta" eller tekniskt sofistikerade, utan snarare för att nätverket drivs med osäkra standardinställningar (default settings), saknar tillräcklig segmentering (segmentation) eller lider av bristande övervakning (monitoring). Det är ofta i de små gliporna – en glömd portkonfiguration eller ett obevakat broadcast-domän – som hotaktörer hittar sitt fotfäste. Genom att förstå angreppens anatomi kan en tekniker gå från att vara reaktiv till att bli en proaktiv arkitekt som bygger säkerhet från grunden, istället för att bara lägga till den som ett efterhandsfilter.

Ett robust försvar bygger på principen om försvar på djupet (defense in depth), där varje lager i nätverksstacken bidrar till den totala motståndskraften. Här spelar de mekanismer som tidigare behandlats, såsom VLAN och segmentering (segmentation) i kapitel 2.2 samt DHCP, DNS och NAT i kapitel 2.4, en avgörande roll. Dessa är inte bara funktionella verktyg för att få trafiken att flyta, utan utgör fundamentet för att isolera hot och skydda nätverkets integritet (integrity). I det här avsnittet undersöks sex specifika attacker som är vanligt förekommande i verkliga miljöer, hur de manifesterar sig i trafiken och vilka motåtgärder som krävs för att göra försvaret robust.

2.5.1 DDoS – överbelastning av länk, protokoll eller applikation

En DDoS-attack (**Distributed Denial-of-Service**) är ett koordinerat försök att göra en tjänst, en server eller en hel nätverksinfrastruktur otillgänglig för legitima användare genom att dränka målet i trafik. Till skillnad från en vanlig DoS-attack, som kommer från en enda källa, använder en DDoS-attack tusentals eller miljontals kapade enheter – ofta kallat ett **botnät** (botnet) – för att rikta trafiken mot målet.

Dessa angrepp kan delas in i tre huvudkategorier beroende på vad de försöker slå ut. **Volymetriska attacker** (volumetric attacks) fokuserar på att fylla upp nätverkets bandbredd så att ingen annan trafik kommer fram. **Protokollattacker** (protocol attacks), såsom en **SYN-flood**, utnyttjar svagheter i nätverksprotokoll som TCP för att förbruka serverns resurser genom att lämna tusentals anslutningar halvöppna. Slutligen finns **applikationsattacker** (application layer attacks) som riktar in sig

på specifika funktioner, exempelvis genom att skicka en enorm mängd HTTP-förfrågningar som ser legitima ut men som tvingar webbservern att utföra tunga databasberäkningar tills den kraschar.

För att visualisera detta kan man tänka sig att "hela stan" plötsligt försöker kliva på exakt samma buss samtidigt. Bussen har fysiskt inte plats för alla, dörrarna kan inte stängas och bussen kommer ingenstans. Resultatet blir att de vanliga resenärerna, som faktiskt har biljetter och behöver komma till jobbet, blir stående kvar på hållplatsen utan möjlighet att resa.

Försvar och motåtgärder

När en attack väl är igång handlar försvaret om att så snabbt som möjligt identifiera och filtrera bort den skadliga trafiken. Detta görs ofta genom att omdirigera trafiken till ett **skrubbningscenter** (scrubbing center) hos en operatör eller molnleverantör, där avancerade algoritmer rensar bort angreppstrafiken och bara släpper igenom legitima paket. På lokal nivå kan man aktivera **SYN-cookies** för att skydda TCP-stacken mot öppna anslutningsförsök och använda **hastighetsbegränsning** (rate-limiting) för att strypa trafikflöden som avviker från det normala.

För att skydda sig proaktivt krävs en arkitektur som sprider riskerna. Genom att använda **Anycast** eller ett **CDN** (Content Delivery Network) kan man sprida ut belastningen på servrar över hela världen, vilket gör det mycket svårare för en angripare att slå ut hela tjänsten samtidigt. Det är också avgörande att ha kontinuerlig övervakning av nätverksflöden med protokoll som **NetFlow** eller **sFlow**. Genom att sätta upp larm på avvikande mönster kan man upptäcka en attack i sin linda och aktivera skyddsmekanismer innan tjänsten går ner. En välkonfigurerad brandvägg med strikta **åtkomstlistor** (ACL - Access Control Lists) kan också stoppa kända skadliga protokoll och adresser långt ute i nätverkets utkant.

Historiskt exempel och fördjupning

Ett av de mest kända fallen av en storskalig DDoS-attack är botnätet **Mirai** som under 2016 slog mot DNS-leverantören **Dyn**. Angreppet utfördes genom att kapa miljontals osäkrade IoT-enheter (Internet of Things), såsom webbkameror och routrar med standardlösenord. Genom att slå ut Dyn, som agerade namnserver åt många av världens största tjänster, blev sajter som Twitter, Spotify och Netflix oåtkomliga för användare i både USA och Europa under flera timmar.

Du kan läsa mer om den fascinerande historien bakom Mirai och de tre unga hackarna som skapade det hos WIRED: [The Mirai Botnet: The Untold Story](#)

2.5.2 ARP-förgiftning – Man-in-the-Middle på LAN

ARP-förgiftning (ARP poisoning eller ARP spoofing) utnyttjar en grundläggande svaghet i hur enheter på ett lokalt nätverk (LAN) kopplar samman IP-adresser med fysiska MAC-adresser. Eftersom ARP-protokollet (Address Resolution Protocol) i sitt grundutförande är tillitsbaserat och saknar inbyggd autentisering, litar enheter blint på de svar de får. En angripare kan därför skicka falska ARP-svar (gratuitous ARP) till både en klient och dess nätverks-gateway. Genom att påstå att angriparens egen MAC-adress hör till gatewayens IP-adress luras klienten att skicka all sin trafik till angriparen istället för till routern. Samtidigt luras gatewayen att tro att angriparen är klienten. Resultatet blir en **Man-in-the-Middle (MitM)**-situation där angriparen kan avlyssna, ändra eller helt blockera trafiken utan att användaren märker något.

Detta kan liknas vid att någon sätter upp en falsk namnskylt på din brevlåda. Postbäraren börjar leverera din post till fel person, som kan läsa dina brev innan de eventuellt lämnas vidare till dig. Tekniskt sett kan man upptäcka detta genom att titta i enhetens ARP-tabell (ARP table). Om man ser konstiga bindningar, till exempel att flera olika IP-adresser pekar på exakt samma MAC-adress, eller om man ser dubbla svar på samma fråga, är det ett tydligt tecken på en pågående attack.

Försvar och motåtgärder

När en attack väl är igång handlar det om att snabbt identifiera källan och isolera den. Genom att granska switchens MAC-tabell kan man se vilken fysisk port som skickar ut de falska svaren och stänga ner den. För att skydda sig proaktivt krävs ett aktivt försvar i nätverkets switchar, där det främsta verktyget är **Dynamic ARP Inspection (DAI)**. DAI fungerar som en dörrvakt som kontrollerar varje ARP-paket som passerar genom switchen. Switchen jämför informationen i ARP-paketet med en betrodd databas – oftast den bindingsdatabas som skapas av **DHCP-snooping**. Om IP-adressen och MAC-adressen i ARP-paketet inte matchar det som finns i den betrodda databasen, kastas paketet omedelbart och attacken stoppas innan den hinner göra skada.

För att göra försvaret ännu mer robust bör DAI kombineras med **IP Source Guard** på accessportarna, vilket förhindrar enheter från att skicka trafik med en förfalskad käll-IP. Det är även rekommenderat att implementera nätverksåtkomstkontroll via **802.1X** för att säkerställa att endast kända och autentiserade enheter överhuvudtaget får ansluta till nätverket. En annan viktig säkerhetsåtgärd är att alltid placera nätverksutrustningens hanteringsgränssnitt i ett eget, isolerat **management-VLAN** som vanliga användare inte har åtkomst till. Genom att se till att nätverket fungerar som en "låst brevlåda med rätt namn" minimeras risken för att interna angripare ska kunna manipulera trafikflödena.

Exempel och fördjupning

Ett typiskt scenario för en utförd attack är i ett oskyddat kontorsnätverk där en angripare ansluter en laptop och kör ett automatiserat verktyg som *Ettercap* eller *Bettercap*. Inom några sekunder börjar trafiken från grannens dator flyta genom angriparens maskin. Angriparen kan då se okrypterade lösenord eller injicera skadlig kod i webbsidor som användaren besöker. Utan DAI i switchen är detta angrepp nästintill osynligt för den vanliga användaren.

Du kan läsa mer om de tekniska detaljerna kring hur man konfigurerar Dynamic ARP Inspection och hur det skyddar nätverket i Ciscos officiella dokumentation: [Configuring Dynamic ARP Inspection \(Cisco\)](#)

2.5.3 Rogue/Evil-Twin-AP – falsk accesspunkt med ditt SSID

En **Rogue AP** (otillåten accesspunkt) eller en **Evil Twin** (falsk tvilling-AP) är ett angrepp där en obehörig person placerar en egen trådlös sändare i eller i närheten av nätverket. Genom att använda exakt samma nätverksnamn (SSID – Service Set Identifier) som det legitima nätverket luras klienternas enheter att ansluta till den falska punkten. Angriparen konfigurerar ofta sin utrustning för att sända med starkare signal eller använder störsändningstekniker för att tvinga bort klienter från det riktiga nätverket. När enheten väl är ansluten till "tvillingen" kan angriparen utföra en Man-in-the-Middle-attack, avlyssna okrypterad trafik eller omdirigera användaren till falska inloggningssidor för att stjäla referenser (credentials).

Detta kan liknas vid att en falsk buss med exakt samma linjenummer som din vanliga buss kör fram till hållplatsen. Du kliver på i god tro, men istället för att komma till din slutstation hamnar du i händerna på någon som kan registrera allt du gör under resan. Fenomenet är särskilt farligt eftersom moderna smartphones och datorer ofta är inställda på att automatiskt ansluta till kända nätverksnamn utan att fråga användaren om lov.

Försvar och motåtgärder

Om en attack upptäcks när den är igång är det primära motdraget att använda ett **WIDS/WIPS** (Wireless Intrusion Detection/Prevention System). Dessa system finns inbyggda i professionella trådlösa lösningar och skannar kontinuerligt efter sändare som utger sig för att tillhöra nätverket men som inte är godkända av kontrollenheten (controller). Vid en identifierad attack kan systemet skicka ut avautentiseringsramar (deauthentication frames) som klipper kopplingen mellan klienterna och den falska accesspunkten, vilket effektivt oskadliggör angreppet medan tekniker letar efter den fysiska hårdvaran.

För att skydda sig proaktivt bör man implementera **WPA2/3-Enterprise** (802.1X) istället för att använda ett gemensamt lösenord (PSK – Pre-Shared Key). Med 802.1X krävs att varje användare autentiserar sig individuellt, och viktigast av allt: klienten kan konfigureras att verifiera ett **servercertifikat** (server certificate) från nätverket. Om den falska accesspunkten inte kan visa upp ett giltigt certifikat som är signerat av organisationens betrodda utfärdare, kommer klienten att vägra ansluta. Man bör även aktivera **PMF** (Protected Management Frames / 802.11w) för att förhindra att angripare kan sparka ut klienter från det riktiga nätverket, samt använda **klientisolering** (client isolation) i gästnätverk för att minimera risken att en infekterad enhet kan skada andra användare i samma VLAN.

Exempel och fördjupning

Ett välkänt exempel på detta är angrepp i publika miljöer som flygplatser eller caféer, där angripare sätter upp en sändare med namn som "Free_Airport_WiFi". Användare som ansluter tror att de använder en officiell tjänst, men i själva verket passerar all deras banktrafik och sociala medier-inloggningar genom angriparens dator. I en skolmiljö skulle en

elev kunna göra samma sak genom att sända ut skolans officiella SSID från en mobiltelefon eller en liten bärbar router gömd i ett skåp.

Forskningen kring att upptäcka och motverka dessa "tvillingar" är omfattande och använder ofta statistiska metoder för att se skillnad på radioegenskaper mellan äkta och falska sändare. Du kan läsa mer om avancerade metoder för att upptäcka dessa angrepp i denna forskningsartikel från MDPI: [Detection of Evil Twin Access Points \(MDPI\)](#)

2.5.4 Deauth/Disassoc-DoS i WLAN

En **Deauthentication-** eller **Disassociation-attack** är en form av överbelastningsangrepp (DoS) som riktar sig specifikt mot de trådlösa kontrollramarna i ett Wi-Fi-nätverk. I standardutförandet av äldre trådlösa protokoll är hanteringsramar (management frames), såsom kommandon för att koppla ner en klient, okrypterade och saknar autentisering. Detta innebär att en angripare med enkla verktyg kan förfälska en accesspunkts (AP) MAC-adress och skicka ut avautentiseringsramar (deauthentication frames) till en specifik klient eller till alla enheter på ett SSID. Mottagaren kan inte avgöra om kommandot kommer från den legitima accesspunkten eller från angriparen och klipper därför omedelbart anslutningen.

Denna attack kan liknas vid att en person klär ut sig till konduktör och går genom ett tåg och ropar att alla passagerare måste kliva av omedelbart på grund av ett tekniskt fel. Passagerarna, som litar på uniformen, lämnar tåget och kvar står en tom vagn. På samma sätt töms en accesspunkt på sina användare. Angreppet används ofta som ett första steg i en mer avancerad attack, till exempel för att tvinga en klient att koppla ner från det riktiga nätverket och istället ansluta till en **Evil Twin** (falsk tvilling-AP), eller för att fånga upp de första paketen i en handskakning för att knäcka lösenord.

Försvar och motåtgärder

Om en attack pågår i realtid är det primära verktyget för upptäckt ett **WIDS/WIPS** (Wireless Intrusion Detection/Prevention System). Systemet genererar larm när det ser en onormal mängd hanteringsramar som inte kommer från nätverkets egna enheter. Eftersom angriparen måste befinna sig inom radoräckvidd för att utföra attacken, kan tekniker använda signalstyrkemätning för att fysiskt lokalisera och avlägsna störningskällan. Det är dock viktigt att skilja på protokollattacker och ren

radiostörning (jamming). Jamming är en fysisk attack där angriparen sänder brus på samma frekvenser som nätverket använder, vilket dränker den legitima trafiken. Detta kan endast lösas genom att fysiskt ta bort sändaren eller genom att byta till andra kanaler och frekvensband.

För att skydda sig proaktivt är den mest effektiva metoden att aktivera **PMF** (Protected Management Frames), även känt som standarden **802.11w**. Med PMF aktiverat signeras hanteringsramarna kryptografiskt, vilket gör att klienten kan verifiera att ett "koppla ner"-kommando faktiskt kommer från den riktiga accesspunkten. Om angriparen försöker skicka en förfalskad ram kommer klienten att ignorera den eftersom signaturen saknas eller är felaktig. I den modernare säkerhetsstandarden **WPA3** är PMF ett krav och alltid aktiverat, men i WPA2-miljöer måste det ofta slås på manuellt i accesspunktens konfiguration. Genom att göra "konduktörens röst" igenkännbar genom kryptering förhindras massutloggningar och nätverket blir betydligt mer stabilt.

Exempel och fördjupning

Ett vanligt exempel på en utförd attack är i en skolmiljö där en elev använder en liten mikrokontroller (exempelvis en ESP8266 med mjukvaran "Wi-Fi Deauther") för att störa ut trådlösa projektorer eller elevdatorer under en lektion. Utan PMF aktiverat kan en sådan billig enhet störa ut hela klassrummets trådlösa kommunikation på några sekunder. Attacken är extremt svår att upptäcka för vanliga användare, som bara upplever att "nätet dog" eller att de ständigt kastas ut.

För en djupare teknisk genomgång av hur man konfigurerar och validerar skyddet för hanteringsramar på professionell utrustning, rekommenderas Ciscos officiella vägledning: [Configure 802.11w Management Frame Protection \(Cisco\)](#)

2.5.5 VLAN-hopping – att smita mellan segment

VLAN-hopping (VLAN hopping) är en attackmetod där en angripare försöker skicka trafik direkt från ett VLAN till ett annat utan att passera de säkerhetskontroller som finns i en router eller brandvägg. I ett välkonfigurerat nätverk ska segmenteringen vara absolut, men genom att utnyttja svagheter i hur switchar hanterar trunking-protokoll (trunking protocols) och taggning kan en angripare "smita" förbi dessa logiska gränser. Det finns främst två sätt detta sker på: genom att lura switchen

att förhandla fram en trunk-anslutning (**switch spoofing**) eller genom att manipulera nätverksramarna med dubbla taggar (**double tagging**).

Vid **double tagging** utnyttjar angriparen hur switchar hanterar det så kallade ursprungliga VLAN:et (native VLAN). Angriparen skickar en nätverksram med två 802.1Q-tagggar. Den första switchen ser den yttre taggen, som matchar det native VLAN som porten tillhör, och tar bort den (stripping) innan ramen skickas vidare över trunken. Den andra switchen ser då den inre taggen och tror att ramen hör hemma i det målvlan som angriparen vill nå. Detta kan liknas vid felmärkta paket på ett sorteringsband; paketet har en yttre etikett som gör att det passerar första kontrollen, men väl inne på bandet avslöjar en inre etikett att paketet hör hemma på ett helt annat ställe och glider in i fel lastbil.

Försvar och motåtgärder

Att stoppa en pågående VLAN-hopping-attack i realtid är tekniskt svårt då trafiken ofta ser legitim ut för switchens hårdvara. Det handlar därför främst om att genomföra en korrekt och säker grundkonfiguration (hardening) av nätverksutrustningen för att omöjliggöra angreppet innan det startar. Den viktigaste åtgärden är att inaktivera den automatiska förhandlingen av trunks, känd som **DTP** (Dynamic Trunking Protocol). Genom att hårdkoda (hard-coding) varje port till att antingen vara en ren accessport (access port) eller en statisk trunkport (trunk port) med kommandot `switchport nonegotiate`, förhindras angriparen från att utföra switch spoofing.

För att motverka double tagging är det ett fackmässigt krav att aldrig använda standard-VLAN:et (VLAN 1) som native VLAN. Istället bör man tilldela native VLAN till ett unikt och helt oanvänt ID (till exempel VLAN 999) på alla trunks. Man bör dessutom konfigurera switchen att uttryckligen tagga även trafiken i native VLAN för att förhindra att otaggade ramar accepteras. Slutligen bör man tillämpa principen om minsta behörighet genom att använda funktionen för tillåtna VLAN (**allowed VLANs**) på alla trunk-anslutningar. Genom att begränsa vilka VLAN som faktiskt får passera mellan switcharna minskas angreppsytan avsevärt. Tillsammans skapar dessa åtgärder en miljö som liknar ett sorteringsband med strikta kontroller och tydlig märkning.

Exempel och fördjupning

Ett klassiskt exempel på en utförd attack är när en angripare ansluten till en accessport som råkar ha kvar standardinställningen "Dynamic Auto" skickar DTP-paket för att lura switchen att porten ska vara en trunk. När trunken väl är etablerad får angriparen tillgång till all trafik som passerar mellan switcharna och kan fritt kommunicera med servrar i känsliga segment som annars skulle ha varit dolda bakom en brandvägg.

Du kan läsa mer om de tekniska detaljerna kring riskerna med VLAN 1 och hur man säkrar sina switchar mot hopping-attacker på Cisco Learning Network: [VLAN 1 and VLAN Hopping Attack \(Cisco Learning Network\)](#)

Det här är två av de mest klassiska attackerna i ett lokalt nätverk som direkt utnyttjar hur DHCP-protokollet är konstruerat för att vara hjälpsamt och snabbt, snarare än säkert. De fungerar ofta bäst i kombination: först rensar man spelplanen (Starvation) och sedan kliver man in som den falska hjälparen (Spoofing).

2.5.6 DHCP-spoofing och Starvation – Attacken mot adressutdelningen

En **DHCP Starvation-attack** går ut på att en angripare försöker tömma nätverkets DHCP-server på alla tillgängliga IP-adresser. Genom att använda automatiserade verktyg skickar angriparen tusentals DHCP-förfrågningar (DHCPDISCOVER) med förfalskade MAC-adresser (MAC spoofing). För DHCP-servern ser det ut som om tusentals nya unika enheter plötsligt har anslutit till nätverket och begärt en adress. Servern delar troget ut sina adresser tills dess **adresspool** (address pool) är helt tom.

Detta kan liknas vid att en person går in på ett apotek och tar samtliga nummerlappar från maskinen i entrén. När de riktiga kunderna kommer in finns det inga nummer kvar att ta, och de kan därför aldrig få hjälp, trots att personalen bakom disken faktiskt är ledig. I nätverket innebär detta en **överbelastning** (Denial-of-Service) där legitima klienter aldrig får den konfiguration de behöver för att komma ut på nätet.

När adresserna väl är slut kliver **DHCP-spoofing** in i bilden. Angriparen startar en egen, falsk DHCP-server (**rogue DHCP server**) på sin dator. Eftersom den riktiga servern är tom på adresser, är angriparens server

den enda som svarar när en ny klient ropar efter hjälp. Angriparen delar då ut en fungerande IP-adress, men sätter sin egen dator som **standard-gateway** (default gateway) och en kontrollerad server som **DNS-server**. Detta skapar en perfekt **Man-in-the-Middle**-situation där all klientens trafik passerar angriparen innan den skickas vidare.

Försvar och motåtgärder

Om en attack pågår är det första steget att identifiera den fysiska porten där den onormala mängden DHCP-trafik eller de falska svaren kommer ifrån och stänga ner den. Man kan också rensa DHCP-serverns **bindningsdatabas** (binding database) för att frigöra adresser, men om angriparen fortfarande är aktiv kommer poolen att tömmas igen omedelbart.

För att skydda sig proaktivt är det absoluta standardförsvaret **DHCP-snooping** (DHCP snooping). Som vi gick igenom i kapitel 2.4 innebär detta att switchen delar upp sina portar i **betrodda** (trusted) och **obetrodda** (untrusted). Endast porten som leder till den riktiga DHCP-servern markeras som betrodd. Om switchen ser ett DHCP-svar (DHCP OFFER eller DHCP ACK) komma in på en obetrodd port, blockeras det direkt. För att stoppa Starvation-attacker kombineras detta ofta med **port-säkerhet** (port security), där man begränsar hur många unika MAC-adresser som får kommunicera via en enskild accessport. Om en elevdator plötsligt försöker använda 500 olika MAC-adresser stängs porten ner automatiskt.

Exempel och fördjupning

Ett vanligt scenario i en skolmiljö är en elev som kör ett verktyg som *Yersinia* från sin bärbara dator. På bara några sekunder kan verktyget generera tillräckligt med trafik för att tömma DHCP-poolen för ett helt VLAN. Eleverna i klassrummet bredvid tappar då sin anslutning så snart deras **giltighetstid** (lease time) går ut, och angriparen kan börja fiska efter inloggningsuppgifter genom att dirigera om deras trafik till en falsk inloggningsportal.

Du kan läsa mer om hur man sätter upp ett effektivt försvar mot dessa attacker och hur man konfigurerar DHCP-snooping i detalj hos NetworkLessons: [DHCP Snooping Configuration and Explained](#)

2.5.7 CAM-tabells-översvämning (MAC-flood)

En **CAM-tabells-översvämning** (CAM table overflow), mer känd som en **MAC-flood**, riktar in sig på själva "hjärnan" i en nätverksswitch. En switch använder en tabell för innehållsadresserbart minne (CAM – Content Addressable Memory) för att hålla reda på vilken MAC-adress som finns på vilken fysisk port. Detta minne är dock begränsat. En angripare utnyttjar detta genom att skicka tusentals nätverksramar med slumpmässigt genererade käll-MAC-adresser (source MAC addresses) till switchen. Inom några sekunder fylls CAM-tabellen med falsk information och når sin maxkapacitet. När tabellen är full kan switchen inte längre lära sig nya, legitima adresser och går in i ett läge som kallas **fail-open**, där den börjar bete sig som en enkel **hub**. Det innebär att all inkommande trafik skickas ut på samtliga portar i samma VLAN (flooding), vilket tillåter angriparen att avlyssna trafik som egentligen var menad för andra enheter genom en enkel paketfångst (packet capture).

Detta kan liknas vid ett adressregister i en stor postsorteringscentral. Om registret plötsligt fylls med tusentals påhittade namn och adresser kan personalen inte längre slå upp var paketen faktiskt ska ta vägen. För att paketen inte ska bli liggande väljer man att skicka ut en kopia av varje paket till varje enskilt utlämningsställe i hopp om att det hamnar rätt någonstans. På så sätt börjar man broadcasta informationen istället för att sortera den, vilket gör att vem som helst på nätverket kan läsa post som inte är adresserad till dem.

Försvar och motåtgärder

Om en attack pågår är det mest effektiva motdraget att omedelbart stänga ner den port som pumpar in de ogiltiga adresserna och rensa switchens dynamiska MAC-tabell för att återställa normal drift. För att skydda sig proaktivt är standardförsvaret i professionella campus-switchar att aktivera **portsäkerhet** (port security). Genom att begränsat det maximala antalet MAC-adresser (max-MAC) som tillåts på en enskild accessport kan man effektivt stoppa attacken innan tabellen fylls. En vanlig rekommendation är att använda funktionen **sticky MAC**, där switchen lär sig de första adresserna som ansluts och sparar dem i konfigurationen, vilket förhindrar att okända adresser överhuvudtaget får kommunicera på samma port.

Utöver portsäkerhet bör man även implementera **stormkontroll** (storm control) på nätverkets ytterkanter (edge ports). Denna funktion övervakar mängden broadcast-, multicast- och okänd unicast-trafik på porten och stryker anslutningen om trafiken överskrider en fördefinierad tröskel. Det är också god praxis att sätta upp larm (SNMP traps eller syslog) som varnar administratören om en port uppvisar ett onormalt antal MAC-adresser. För den som vill bygga ett mer proaktivt försvar rekommenderas även att kika på avsnitt 3.1 om baslinjer och 3.2 om övervakningsverktyg för att lära sig hur man upptäcker avvikelser i tid. Genom att kombinera dessa tekniker med en tydlig **violation-policy**, där porten exempelvis stängs ner helt (shutdown) vid ett regelbrott, skapar man ett försvar som automatiskt oskadliggör angreppet.

Exempel och fördjupning

Ett typiskt exempel på en utförd attack är en elev som kör ett verktyg som *macof* (en del av dsniff-paketet) från en Linux-dator i klassrummet. Inom några ögonblick kan verktyget generera hundratusentals falska MAC-adresser, vilket räcker för att sänka de flesta enterprise-switchar och tvinga dem att börja läcka trafik till angriparens dator. Utan portsäkerhet konfigurerad är detta ett av de enklaste sätten att snabbt få tillgång till känslig information i ett lokalt nätverk utan att behöva knäcka några lösenord.

Du kan läsa mer om de tekniska detaljerna kring hur man konfigurerar och hanterar portsäkerhet på moderna switchar i Ciscos officiella dokumentation: [Configuring Port Security \(Cisco\)](#)

2.5.8 Broadcast-storm – kaos i nätverkstrafiken

En broadcast-storm uppstår när broadcast-paket (eller multicast/okänd unicast) loopas runt i ett nätverk och dupliceras i en oändlig spiral. Till skillnad från IP-paket (lager 3) som har ett TTL-värde (Time to Live) som gör att de dör efter ett visst antal hopp, saknar Ethernet-ramar (lager 2) en sådan mekanism. Om det finns en cirkulär väg mellan två eller flera switchar kommer en enda broadcast-ram att skickas runt varv efter varv. Varje gång ramen passerar en switch skickas den ut på alla portar, vilket snabbt leder till att antalet ramar i nätverket växer exponentiellt. Inom några sekunder konsumeras all tillgänglig bandbredd och switcharnas

processorer (CPU) blir så överbelastade med att hantera dessa ramar att legitim trafik inte längre kan komma fram.

Detta kan liknas vid att två personer i en ekande korridor börjar ropa efter varandra samtidigt. Om ljudet studsar perfekt mellan väggarna och förstärks för varje eko, kommer ljudnivån snabbt att bli så öronbedövande att ingen längre kan höra vad som sägs. I nätverket manifesterar det sig ofta genom att samtliga lampor på en switch börjar blinka febrilt i samma takt (en så kallad "blink-of-death"), och alla anslutna enheter tappar kontakten med sina gateways och servrar.

Försvar och motåtgärder

När en broadcast-storm väl är igång är det mest effektiva motdraget att fysiskt eller logiskt bryta loopen. Detta innebär ofta att en tekniker måste börja dra ur kablar mellan switchar tills stormen lägger sig och nätverket återhämtar sig. Det är en stressig process eftersom man under stormen ofta tappar möjligheten att logga in på switcharna via nätverket (In-band management) för att stänga ner portar mjukvarumässigt. För att skydda sig proaktivt är det absolut viktigaste försvaret att ha **Spanning Tree Protocol (STP)** korrekt konfigurerat. STP upptäcker cirkulära vägar automatiskt och försätter "överflödiga" portar i ett blockerande läge (blocking state) för att förhindra att loopar ens kan bildas.

Som ett extra skyddslager bör man även implementera **stormkontroll** (storm control) på alla accessportar. Denna funktion fungerar som en säkerhetsventil; om mängden broadcast-trafik på en enskild port överstiger en viss procent av länkens totala kapacitet, börjar switchen kasta paketen eller stänger ner porten helt. Det är även rekommenderat att använda funktioner som **BPDU Guard** på portar där användare ansluter sin utrustning. Om någon av misstag (eller avsiktligt) kopplar in en egen switch eller brygger två uttag med en kabel, känner BPDU Guard av detta och stänger omedelbart av porten innan en loop hinner sänka hela VLAN-segmentet.

Exempel och fördjupning

Ett klassiskt exempel på ett "fel" som blir till en attack mot tillgängligheten är i ett klassrum där en elev hittar en lös nätverkskabel och kopplar in båda ändarna i två olika nätverksuttag i väggen. Om inte STP eller BPDU Guard är aktivt i skolans access-switch, kommer detta att skapa en loop som omedelbart sänker nätverket för hela våningsplanet.

En angripare kan utnyttja samma princip genom att använda mjukvara för att generera en enorm mängd broadcast-trafik från en enda dator för att se om nätverkets stormkontroll är korrekt inställd.

Du kan läsa mer om hur man diagnostiserar och förebygger broadcast-stormar samt hur man optimerar STP för att undvika dessa scenarier i Ciscos tekniska vägledning: [Troubleshooting LAN Switching Environments \(Cisco\)](#)

2.5.9 DNS-förgiftning – manipulering av namnupplösningen

DNS-förgiftning, ofta kallad **cache-förgiftning** (cache poisoning), går ut på att en angripare introducerar falsk information i en DNS-resolvers minne (cache). Eftersom en resolver sparar svar för att snabba upp framtida förfrågningar, kommer alla klienter som använder den resolvern att få det förfalskade svaret så länge det ligger kvar i cachén. Angreppet kan ske genom att angriparen skickar en mängd förfalskade svar till en resolver innan det riktiga svaret från den auktoritativa servern hinner fram. Om angriparen lyckas gissa rätt transaktions-ID, accepterar resolvern det falska svaret som sanning.

Detta kan liknas vid att någon bryter sig in på ett tryckeri och ändrar i stadens officiella kartor. När folk sedan använder kartan för att hitta till banken, leder vägbeskrivningen dem istället direkt till angriparens gömälle. Eftersom användaren litar på kartan (DNS-servern), ifrågasätter de inte att de hamnat fel förrän det är för sent. Resultatet blir att användare som tror att de besöker sin bank eller myndighet i själva verket hamnar på en exakt kopia av webbplatsen som kontrolleras av angriparen.

Försvar och motåtgärder

Om en attack upptäcks när den pågår är den omedelbara åtgärden att rensa resolverns cache (flush cache) och starta om tjänsten för att tvinga fram nya, korrekta uppslagningar. Man bör även tillfälligt byta till en annan pålitlig **vidarebefordrare** (forwarder) medan man utreder källan till de falska svaren. För att skydda sig proaktivt är den absolut viktigaste tekniken **DNSSEC** (Domain Name System Security Extensions). DNSSEC lägger till digitala signaturer på DNS-posterna, vilket gör att resolvern kan verifiera att svaret faktiskt kommer från rätt källa och inte har manipulerats under vägen.

En annan viktig försvarsåtgärd är att använda moderna tekniker för krypterad DNS, såsom **DNS over HTTPS (DoH)** eller **DNS over TLS (DoT)**. Dessa protokoll förhindrar att en angripare på det lokala nätverket kan läsa eller ändra i DNS-förfrågningarna. Det är även rekommenderat att använda resolvers som implementerar **source port randomization**, vilket gör det betydligt svårare för en angripare att gissa rätt parametrar för att lyckas med en förgiftning. Genom att kombinera dessa tekniska skydd med en strikt brandväggspolicy (firewall policy) som endast tillåter DNS-trafik till och från kända, betrodda servrar, skapas ett mycket starkt försvar mot manipulering av namnupplösningen.

Exempel och fördjupning

Ett historiskt avgörande ögonblick för DNS-säkerhet var upptäckten av **Kaminsky-sårbarheten** år 2008. Säkerhetsforskaren Dan Kaminsky fann en fundamental svaghet i DNS-protokollet som gjorde det möjligt att förgifta en hel domän på bara några sekunder. Upptäckten var så allvarlig att den ledde till en hemlig, global insats där världens största teknikföretag samarbetade för att täppa till luckan innan den hann utnyttjas i stor skala. Detta blev startskottet för den breda utrullningen av DNSSEC.

Du kan läsa mer om den dramatiska historien bakom Kaminsky-sårbarheten och hur den förändrade internet i den här artikeln från Wired: [The Day the Internet Almost Broke \(Wired\)](#)

2.5.10 Sammanfattning och Best Practice

Att säkra ett nätverk handlar sällan om att hitta en enskild "silverkula", utan om att bygga en arkitektur som är motståndskraftig genom flera samverkande lager. De nio angreppstyper vi har gått igenom – allt från storskaliga **DDoS-attacker** till interna **ARP-förgiftningar** och **VLAN-hopping** – har en gemensam nämnare: de utnyttjar antingen protokoll som är byggda på tillit eller nätverksutrustning som körs med osäkra standardinställningar (default settings). Genom att systematiskt stänga dessa dörrar skapas en miljö där ett enskilt misstag eller en infekterad klient inte tillåts sänka hela infrastrukturen.

Grunden för ett säkert nätverk börjar i accesslagret (access layer), där användare och enheter först ansluter. En av de mest kritiska åtgärderna är att implementera **portsäkerhet** (port security) och **DHCP-snooping**.

Genom att begränsa antalet MAC-adresser per port och endast tillåta DHCP-svar från betrodda källor elimineras risken för både **MAC-flooding** och **DHCP-spoofing** omedelbart. När DHCP-snooping är aktivt skapas dessutom en bindingsdatabas som är en förutsättning för att kunna köra **Dynamic ARP Inspection (DAI)**, vilket i sin tur sätter stopp för **ARP-förgiftning** och Man-in-the-Middle-attacker på det lokala nätverket.

För att hantera logisk segmentering och förhindra att trafik "läcker" mellan olika VLAN via **VLAN-hopping**, krävs en strikt hantering av trunkar. Det är ett fackmässigt krav att inaktivera dynamisk förhandling via **DTP** och istället hårdkoda (hard-code) varje ports roll. Genom att dessutom byta ut det ursprungliga VLAN:et (**native VLAN**) från standardvärdet VLAN 1 till ett oanvänt ID och se till att all trafik taggas, tas de tekniska förutsättningarna för **double-tagging** bort. För att skydda nätverkets tillgänglighet mot både mänskliga fel och medvetna försök att skapa **broadcast-stormar**, måste **Spanning Tree Protocol (STP)** vara korrekt konfigurerat tillsammans med skyddsfunktioner som **BPDUGuard** på alla användarportar.

I den trådlösa miljön är det tydligt att gamla metoder med gemensamma lösenord (PSK) inte längre räcker till i professionella sammanhang. För att motverka **Evil Twin-attacker** och avlyssning är övergången till **WPA2/3-Enterprise** (802.1X) nödvändig, då det tillåter klienten att verifiera nätverkets identitet via servercertifikat. Samtidigt är aktivering av **PMF** (Protected Management Frames / 802.11w) den enda effektiva metoden för att stoppa de störande **Deauth-attackerna** som annars enkelt kan kasta ut användare från nätet. På tjänstenivå måste tilliten till namnupplösningen stärkas genom implementering av **DNSSEC**, vilket förhindrar **DNS-förgiftning** genom att signera svaren kryptografiskt.

Slutligen krävs ett försvar mot de yttre hoten i form av **DDoS-attacker**. Här handlar det om att flytta försvaret så långt ut i kedjan som möjligt, ofta genom att använda molnbaserade skyddstjänster, **CDN-nätverk** och tekniker som **Anycast** för att sprida ut belastningen. Genom att kombinera dessa tekniska spärrar med en kontinuerlig övervakning via **NetFlow** eller **sFlow**, kan administratören etablera en baslinje (baseline) för vad som är normal trafik. Det gör det möjligt att upptäcka och agera på avvikelser innan de hinner utvecklas till fullskaliga avbrott. Att säkra nätverket är en pågående process, men genom att följa dessa

branschstandarder (best practices) skapas en miljö som är både professionell, förutsägbar och motståndskraftig.

Kommandon och verktyg för validering:

- Visa ARP-tabellen: `arp -a` (Windows) / `ip neigh` (Linux)
- Visa MAC-tabellen i switch: `show mac address-table`
- Kontrollera DHCP-snooping status: `show ip dhcp snooping binding`
- Testa namnupplösning: `nslookup` / `dig`

Kontrollfrågor

1. Förklara skillnaden mellan en volymetrisk DDoS-attack (volumetric attack) och en applikationsattack (application layer attack). Vilka olika resurser försöker de överbelasta?
2. Beskriv hur en angripare kan använda ARP-förgiftning (ARP poisoning) för att utföra en Man-in-the-Middle-attack. Varför litar klienterna på de falska svaren?
3. Vilken teknisk funktion i en switch är nödvändig för att Dynamic ARP Inspection (DAI) ska fungera, och varför behövs den kopplingen?
4. Hur kan användandet av WPA2/3-Enterprise (802.1X) och servercertifikat förhindra att en användare ansluter till en Evil Twin-accesspunkt?
5. Varför är hanteringsramar (management frames) sårbara för Deauth-attacker i äldre trådlösa nätverk, och hur löser PMF (802.11w) detta?
6. Beskriv mekanismen bakom "double tagging" vid VLAN-hopping. Vilken roll spelar det ursprungliga VLAN:et (native VLAN) i detta angrepp?
7. Förklara sambandet mellan DHCP Starvation och DHCP Spoofing. Varför utförs de ofta tillsammans?
8. Vad händer rent tekniskt med en switch när dess CAM-tabell (CAM table) blir full under en MAC-flood, och vilken säkerhetsrisk innebär detta för trafiken i VLAN-segmentet?
9. Hur skiljer sig en broadcast-storm från andra attacker när det gäller dess ursprung, och vilka två tekniker är viktigast för att förhindra att den sänker nätverket?
10. På vilket sätt skyddar DNSSEC mot cache-förgiftning (cache poisoning), och vad är den största skillnaden mellan DNSSEC och krypterad DNS (som DoH eller DoT)?

Fördjupningslänkar

Cisco: Configuring Dynamic ARP Inspection (Teknisk vägledning för att säkra lager 2-trafik) - cisco.com

Wired: The Mirai Botnet Story (En djupdykning i hur osäkra IoT-enheter skapade ett globalt kaos) - wired.com

Cloudflare Learning: What is a DDoS Attack? (Pedagogiska förklaringar av olika typer av överbelastningsangrepp) - cloudflare.com

MDPI Research: Detection of Evil Twin APs (Vetenskaplig artikel om avancerade metoder för att upptäcka falska accesspunkter) - mdpi.com

NetworkLessons: DHCP Snooping Explained (Praktiska exempel på hur man konfigurerar skydd mot DHCP-attacker) - networklessons.com

Cisco Learning Network: VLAN Hopping (Detaljerad genomgång av switch-spoofing och double tagging) - learningnetwork.cisco.com

Wired: The Kaminsky DNS Bug (Historien om sårbarheten som nästan förstörde internet och ledde till DNSSEC) - wired.com

OWASP: Denial of Service (Information om applikationsspecifika överbelastningsattacker) - owasp.org

Cisco: Port Security Guide (Hur man begränsar MAC-adresser och hanterar violations på accessportar) - cisco.com

SANS Institute: Securing Wireless Networks (Best practice och ramverk för säkra trådlösa implementationer) - sans.org

Egna anteckningar

...

3 Nätverksadministration

När de tekniska fundamenten för routing, switching och nätverkstjänster väl är på plats, skiftar fokus från konstruktion till långsiktig förvaltning och nätverksadministration (network administration). Att administrera ett nätverk handlar om betydligt mer än att bara hålla igång trafiken; det är en disciplin som kombinerar teknisk expertis med metodiskt tänkande för att säkerställa hög tillgänglighet (high availability), god prestanda och en hög säkerhetsnivå. En skicklig administratör ser nätverket som en levande organism som ständigt kräver tillsyn, optimering och anpassning efter verksamhetens föränderliga behov.

Detta kapitel fungerar som en brygga mellan de logiska protokollen och den praktiska vardagen i ett serverrum eller ett nätverkscenter (NOC - Network Operations Center). Här utforskas de verktyg och metoder som används för att övervaka systemens hälsa, hantera identiteter i komplexa katalogtjänster (directory services) och konfigurera både servrar och klienter på ett strukturerat sätt. Målet med nätverksadministration är att skapa en miljö som är både förutsägbar och skalbar (scalable), där problem upptäcks och åtgärdas innan de hinner påverka slutanvändarna. Genom att kombinera beprövade administrativa rutiner med moderna verktyg för automatisering läggs grunden för en stabil och professionell IT-infrastruktur.

3.1 Grunder i nätverksadministration

Inom ramen för de administrativa grunderna ligger fokus på det systematiska arbetet och de verktyg som utgör en nätverksteknikers dagliga hantverk. Grundläggande nätverksadministration (fundamentals of network administration) handlar om att etablera fungerande arbetssätt (workflows) som minimerar risker vid förändringar och maximerar effektiviteten vid felsökning. Det handlar om att gå från ett reaktivt läge, där man släcker bränder när de uppstår, till ett proaktivt läge där dokumentation, baslinjer (baselines) och standardiserade konfigurationer skapar stabilitet över tid.

Att behärska dessa grunder innebär att förstå hela kedjan av händelser i ett nätverk, från det att en klient begär en adress via DHCP till dess att en säker fjärranslutning (remote access) etableras för underhåll. Det

innefattar även vikten av att hantera ändringar (change management) och säkerhetsuppdateringar (patch management) på ett ansvarsfullt sätt. Genom att tillämpa en tydlig metodik och använda rätt diagnostiska verktyg kan administrativa uppgifter utföras med precision. Detta avsnitt ger den nödvändiga verktygslådan för att hantera driften av moderna nätverksmiljöer, oavsett om det rör sig om små lokala nät eller stora distribuerade system med hundratals noder.

3.1.1 Syfte, roller och arbetssätt

Nätverksadministration (network administration) utgör den disciplin som krävs för att planera, drifta, övervaka (monitoring) och ständigt förbättra nätverksmiljön. Det övergripande syftet är att säkerställa att användare, servrar och tjänster får den tillgänglighet (availability), prestanda (performance) och säkerhet (security) som verksamheten kräver för att fungera effektivt. Arbetet är omfattande och spänner över allt från den fysiska infrastrukturen (physical infrastructure) – såsom kablage, switchar och brandväggar – till det logiska lagret (logical layer) där konfiguration av VLAN, routing, DNS, DHCP och autentisering (authentication) sker. Utöver de tekniska aspekterna innefattar administrationen även de processer som skapar spårbarhet (traceability) och stabilitet, däribland dokumentation, ändringshantering (change management) och etablerade incidentrutiner (incident response routines).

För att upprätthålla en professionell driftnivå underlättar det att tänka i termer av olika roller, även i mindre miljöer där en och samma person ibland förväntas bära alla hattar samtidigt. Genom att särskilja rollen som designar och arkitekter (network architect) nätet från den roll som sköter den dagliga driften och övervakningen (network operations), skapas en naturlig struktur för hur beslut fattas och genomförs. En viktig del i arbetssättet är att ha en tydlig metodik för vem som godkänner förändringar i nätet, vilket minskar risken för oplanerade avbrott. Att arbeta strukturerat innebär att varje teknisk justering ses som en del av en större livscykel där planering och validering är lika viktiga som själva utförandet.

Förtydligande (sammanfattning):

- **Mål:** Att uppnå och bibehålla tillgänglighet (availability), prestanda (performance), säkerhet (security) och fullständig spårbarhet (traceability).
- **Omfång:** Hantering av både den fysiska infrastrukturen (physical infrastructure), de logiska tjänsterna och de administrativa processerna.
- **Roller:** Uppdelning mellan design och arkitektur (design/architecture), löpande drift och övervakning (operations/monitoring) samt hantering av ändringar och incidenter (change/incident management).

Exempel: En elevdator i skolan kan plötsligt inte nå det interna intranätet. Nätverksadministratören agerar då i sin driftsroll och följer en etablerad standardmetodik (se avsnitt 3.1.5) för att isolera felet. Alla observationer och de åtgärder som vidtas dokumenteras löpande, och när felet är avhjälpt sker en formell återkoppling enligt skolans incidentrutin för att säkerställa att liknande problem kan förebyggas i framtiden.

3.1.2 Basverktyg och kommandon

Att behärska ett kärnbibliotek av verktyg för kontroll, mätning och felsökning (troubleshooting) är en grundförutsättning för att kunna arbeta effektivt i moderna nätverksmiljöer. En skicklig administratör växlar sömlöst mellan olika operativsystem och plattformar, vilket gör det nödvändigt att förstå hur samma logiska funktioner uttrycks i exempelvis Cisco IOS kontra Linux eller Windows. I det dagliga arbetet används dessa verktyg för att verifiera konfigurationen på både klientsidan, serversidan och direkt på nätutrustningens kommandoradsgränssnitt (CLI - Command Line Interface). Tabellen nedan sammanställer de vanligaste kommandona för att inspektera nätverkets tillstånd.

Funktion	Cisco IOS	Linux (iproute2/Bash)	Windows (CMD/PowerShell)
Visa IP-adresser	show ip interface brief	ip -brief addr	ipconfig / Get-NetIPAddress
Gränssnittsstatus	show interfaces status	ip link show	ipconfig /all / Get-NetAdapter
Nåbarhet (Ping)	ping [IP/namn]	ping [IP/namn]	ping [IP/namn]
Vägval (Traceroute)	tracert [IP/namn]	tracert / mtr [IP/namn]	tracert [IP/namn]
Routingtabell	show ip route	ip route	route print / Get-NetRoute
ARP-tabell	show arp	ip neigh	arp -a / Get-NetNeighbor
Aktiva portar/flöden	show tcp brief	ss -tuln	netstat -ano / Get-NetTCPConnection
DNS-uppslag	<i>(Begränsat i IOS)</i>	dig / nslookup	nslookup / Resolve-DnsName

Genom att kombinera dessa verktyg kan en administratör snabbt isolera var i nätverksstacken ett problem uppstått. En systematisk metodik innebär att man jämför resultaten från olika lager för att se var kommunikationen bryts.

Särskilt i Linux-miljöer är mtr ett överlägset alternativ vid analys av instabila anslutningar, då den till skillnad från vanlig traceroute skickar kontinuerliga paket och visar statistik över latens (latency) och paketförlust (packet loss) för varje hopp över tid. För djupare analys av själva trafikflödena används paketfångst (packet capture); här är **Wireshark** den ledande grafiska lösningen (GUI) för att inspektera protokoll i detalj, medan **tcpdump** är standardverktyget för analys direkt via terminalen. Vid trådlös felsökning används specifika kommandon som **iw** för att granska kanaler och signalstyrka (SNR - Signal-to-Noise Ratio).

Att regelbundet använda dessa verktyg under normal drift är avgörande för att etablera nätverkets **baslinje** (baseline). Genom att känna till vad som är normala svarstider, hur en felfri routingtabell ser ut eller vilka portar som förväntas vara öppna, kan administratören betydligt snabbare identifiera avvikelser som tyder på begynnande hårdvarufel eller nätverksangrepp.

Exempel: Du utför en **ping** mot IP-adressen `10.40.0.50` och får ett lyckat svar (OK), men när du försöker göra ett uppslag med **nslookup** mot namnet `files.skolan.local` misslyckas det. Slutsatsen blir då att den underliggande IP-anslutningen (IP connectivity) fungerar korrekt och att problemet är isolerat till namnupplösningen (DNS), vilket behandlas mer ingående i avsnitt 3.1.4.

3.1.3 Standardkonfigurationer och baslinjer

Se även kap. 8 i *Datorteknik*, för djupare information om loggning och säkerhet i Linux och Windows.

Starka grundinställningar gör nätverket förutsägbart och säkert. Genom att etablera en tydlig **baslinje** (baseline) för alla nätverkskomponenter säkerställs att varje switch, router och brandvägg i infrastrukturen uppträder enhetligt. Detta underlättar inte bara den dagliga driften utan är även en förutsättning för att snabbt kunna rulla ut nya enheter eller återställa system efter ett maskinfel. En väl genomarbetad baslinje adresserar allt från tidssynkronisering och loggning till strikt åtkomstkontroll och säkerhetsinställningar i accesslagret.

Kärnan i en professionell baslinje är tidssynkronisering via **nätverkstidsprotokollet** (NTP - Network Time Protocol) och centraliserad loggning (syslog). För att händelser i nätverket ska kunna korreleras vid en incident krävs att alla enheter har exakt samma tidskälla; utan synkroniserade klockor blir loggfiler från olika servrar och switchar nästintill omöjliga att analysera tillsammans. Loggarna bör skickas till en central loggserver eller en **Event Viewer**-samlare för att skydda dem mot lokal manipulation och för att möjliggöra automatiserade larm vid kritiska händelser.

Säkerheten kring själva administrationen är en annan pelare i baslinjen. Detta innebär att osäkra protokoll som Telnet omedelbart inaktiveras till förmån för **SSH** (Secure Shell) för alla terminalanslutningar. Istället för delade lösenord används unika administratörskonton (unique admin accounts), gärna kopplade till en central autentiseringstjänst (AAA - Authentication, Authorization, and Accounting) med stöd för **flerfaktorsautentisering** (MFA - Multi-Factor Authentication) där det är tekniskt möjligt. All administration bör ske i ett dedikerat och isolerat **administrations-VLAN** (management VLAN) som skyddas av strikta **åtkomstlistor** (ACL - Access Control Lists). Dessa listor säkerställer att endast betrodda administratörsverktyg och kända IP-adresser tillåts kommunicera med nätverksutrustningens hanteringsgränssnitt.

På hårdvarunivå innefattar baslinjen även att stänga ner onödiga och osäkra funktioner som **DTP** (Dynamic Trunking Protocol) och att aktivera skyddsmekanismer som **stormkontroll** (storm control) och **portsäkerhet** (port security). Slutligen är en fungerande rutin för konfigurationsbackup (config backup) och versionshantering (version control) livsviktig. Genom att spara historik över alla ändringar kan administratören snabbt utföra en jämförelse (diff) mellan den nuvarande konfigurationen och en tidigare känd fungerande version, vilket är ovärderligt vid felsökning efter en misslyckad uppdatering.

Förtydligande (sammanfattning):

- **Tid och loggning:** Konfigurera NTP mot en gemensam, pålitlig källa och skicka alla loggar till en central server för analys och arkivering.
- **Åtkomst och identitet:** Använd uteslutande SSH, tillämpa unika administratörskonton och implementera MFA där det är möjligt för att höja säkerheten.
- **Management-design:** Isolera nätverksutrustningen i ett eget management-VLAN och begränsa åtkomsten med brandväggsregler eller ACL:er.
- **Konfigurationshantering:** Etablera en **standardkonfiguration** (golden config), utför regelbundna backuper och använd versionshantering för att kunna följa historik och snabbt återställa tjänster.

Exempel: Alla switchar i nätverket har konfigurerats med NTP-servern 10.40.0.12 som tidskälla och skickar sin syslog-trafik till 10.40.0.14. All fjärråtkomst sker via SSH, och administrationsgränssnitten har flyttats till VLAN 99 med en ACL som endast släpper in trafik från IT-avdelningens specifika administratörsverktyg.

3.1.4 DNS, DHCP och adressering i drift

Se även kap. 6.7 i boken *Datorteknik för mer information om Active Directory*

I vardagen utgör DNS och DHCP själva navet för användarupplevelsen i nätverket. Om dessa tjänster inte fungerar optimalt spelar det ingen roll hur snabba routrarna är; användarna kommer uppleva att "nätet ligger nere" så fort de inte kan få en IP-adress eller slå upp ett domännamn. För en administratör handlar driften om att proaktivt hantera adressomfång (scopes), hantera reservationer (reservations) för fast utrustning och säkerställa att förfrågningar når fram via rätt relay-inställningar. Samtidigt krävs en stabil namnarkitektur där interna zoner, **split-horizon** och effektiva vidarebefordrare (forwarders) samverkar för att ge snabba svarstider.

När nätverket är i produktion är administratörens primära uppgift i DHCP-miljön att övervaka utnyttjandegraden av adresspoolerna. Genom att matcha giltighetstider (lease times) mot användarnas rörlighet kan man förhindra att adresser tar slut; ett gästnätverk (guest network) mår bäst av korta tider för att snabbt frigöra resurser, medan personalens fasta segment kan ha betydligt längre tider för att öka stabiliteten. Det är också här som administrativa reservationer (reservations) görs för att säkerställa att nätverksskrivare och servrar alltid har samma adress utan att behöva konfigureras statiskt på själva enheten. Allt detta kräver en noggrann dokumentation i form av en adressplan (address plan) som tydligt visar kopplingen mellan VLAN, IP-prefix, gateway och tillhörande DHCP-scope.

Inom namnhanteringen (DNS) är det ofta en **Active Directory-integrerad** lösning som utgör basen. Här hanterar administratören de interna zonerna och ser till att klienterna tilldelas minst två oberoende namnservrar (resolvers) via

DHCP-option 6. Genom att använda split-horizon DNS kan man styra så att interna användare når resurser via privata adresser medan externa besökare ser de publika, vilket ökar både säkerhet och prestanda. En annan viktig driftaspekt är hanteringen av **TTL** (Time to Live) för DNS-poster; ett väl valt värde balanserar behovet av snabb uppdatering vid förändringar mot att minska onödig trafik och belastning på namnservrarna.

För att hålla ordning på infrastrukturen är det fackmässigt att upprätthålla en aktuell tabell över nätverkets adressering. Denna fungerar som administratörens karta och kompass vid felsökning och expansion.

VLAN	Funktion	Prefix / Subnät	Gateway	DHCP Scope
10	Personal	10.10.0.0/24	10.10.0.1	10.10.0.100–250
20	Elever	10.20.0.0/22	10.20.0.1	10.20.0.100–10.20.3.250
30	Gäster	172.16.0.0/24	172.16.0.1	172.16.0.20–250
99	Management	10.99.0.0/24	10.99.0.1	Inget (Statiskt)

Förtydligande (sammanfattning):

- **DHCP:** Säkerställ fungerande relay (IP-helper) och anpassa giltighetstid (lease time) efter målgruppens behov.
- **DNS:** Använd AD-integrerade zoner med redundanta resolvers och optimera TTL för att balansera stabilitet och flexibilitet.
- **Adressplan:** Dokumentera systematiskt förhållandet mellan VLAN, prefix, gateway och adresspooler (scopes).

Exempel: Gäster i nätverket klagar på att internet upplevs som långsamt. Vid en mätning visar det sig att deras enheter är konfigurerade att gå direkt mot publika namnservrar ute på internet istället för nätverkets lokala resolvers. Detta leder till onödig latens (latency) för varje uppslagning. Lösningen blir att uppdatera policyn i brandväggen för att tvinga all DNS-trafik till skolans egna

resolvers, som i sin tur använder snabba forwarders för att hämta externa svar. Detta sänker svarstiderna dramatiskt för gästerna.

3.1.5 Felsökningsmetodik (lager för lager)

Vid felsökning av nätverk är en konsekvent och logisk metodik administratörens absolut viktigaste tillgång. Att "skjuta från höften" eller ändra flera parametrar samtidigt leder sällan till en lösning, utan snarare till ökad förvirring och potentiellt nya fel. Genom att arbeta systematiskt, antingen nerifrån och upp (från det fysiska kablaget) eller uppifrån och ner (från applikationen), kan man metodiskt utesluta felkällor lager för lager. Varje steg i processen bör grundas på faktiska mätningar, loggar och observationer för att säkerställa att man bygger sin analys på fakta snarare än antaganden.

Det första steget i all felsökning är att definiera problemets symptom och omfattning (scope). Det är avgörande att veta om felet påverkar en enskild klient, en specifik användargrupp eller en hel byggnad. När omfattningen är känd börjar den tekniska granskningen ofta vid den fysiska länken och lågnivå-inställningarna (link and low-level). Här kontrolleras att länklampor lyser, att kablar är hela och att porten i switchen är konfigurerad med rätt hastighet, duplex och framför allt rätt VLAN. Om den fysiska anslutningen är stabil går man vidare till adressering och gateway (address and gateway). Här verifieras att klienten har fått en korrekt IP-konfiguration via DHCP och att den kan kommunicera med sin standard-gateway (default gateway) genom ett enkelt ping-test.

När den grundläggande IP-anslutningen är bekräftad skiftas fokus mot namnupplösning och rutter (names and routes). Med verktyg som **nslookup**, **dig** och **traceroute** (eller **mtr**) kontrolleras att klienten kan översätta namn till adresser och att trafiken tar rätt väg genom nätverket. Om trafiken stannar halvvägs eller nekas åtkomst, är det dags att granska brandväggar och policys (firewall and policy). Genom att studera brandväggsloggar kan man se om paket kastas på grund av felaktiga regler eller om trafiken blockeras i en viss riktning. Det sista lagret i felsökningen rör själva applikationen (application layer), där man kontrollerar att rätt portar är öppna, att certifikat är giltiga och att autentiseringen (authentication) fungerar som förväntat.

Förtydligande (sammanfattning):

- **Symptom och omfattning (scope):** Identifiera exakt vad som inte fungerar och hur många användare som påverkas.
- **Länk och lågnivå:** Kontrollera fysisk anslutning, länkhastighet och VLAN-tillhörighet på portnivå.
- **Adress och gateway:** Verifiera IP-adress (ipconfig/ip addr) och anslutning till gateway.
- **Namn och rutter:** Kontrollera DNS-uppslag och trafikens väg genom nätverket (traceroute/mtr).
- **Brandvägg och policy:** Granska loggar och verifiera att brandväggsregler tillåter trafiken i båda riktningarna.
- **Applikation:** Kontrollera applikationsspecifika inställningar såsom portar, kryptering och autentisering.

Exempel: Elevernas trådlösa nätverk (VLAN 20) fungerar utan problem i skolans A-hus, men i B-huset kan ingen ansluta. Vid en systematisk kontroll av infrastrukturen upptäcks att trunk-anslutningen (trunk) mellan husens switchar saknar VLAN 20 i sin lista över tillåtna VLAN (allowed-list). Åtgärden blir att lägga till VLAN 20 på trunken, varpå funktionen omedelbart verifieras med en testklient på plats i B-huset.

3.1.6 Övervakning och loggning

Mer information finns i 11.8 Verktyg, övervakning och loggning i boken Datorteknik

Övervakning (monitoring) och loggning (logging) är två sidor av samma mynt men fyller olika funktioner i nätverksadministrationen. Man brukar säga att övervakningen talar om hur nätet mår i realtid, medan loggningen talar om vad som faktiskt hände vid en viss tidpunkt. Genom att använda protokollet för nätverkshantering (SNMP - Simple Network Management Protocol) eller modern telemetri (telemetry) kan administratören kontinuerligt samla in data om enheternas hälsa. Detta innefattar kritiska parametrar som processorutnyttjande (CPU), minnesanvändning (RAM), länkbelastning, strömmatning via nätverket (PoE - Power over Ethernet) och temperatur.

Loggning fokuserar istället på händelseförlopp och säkerhet. Genom att skicka systemhändelser till en central loggserver (syslog) eller samla in data via Windows **Event Viewer**, får man en samlad bild av brandväggsbeslut (firewall decisions), inloggningar (logins) och felmeddelanden från tjänster. För att denna enorma datamängd ska bli begriplig används visualiseringsverktyg och instrumentpaneler (dashboards) som lyfter fram viktiga nyckeltal (KPIs) såsom upptid (uptime), latens (latency) och antal incidenter. Genom att definiera larmtrösklar (alarm thresholds) kan systemet automatiskt varna när något avviker från det normala, vilket gör att administratören kan agera proaktivt istället för att bara reagera på felanmälningar.

Förtydligande (sammanfattning):

- **SNMP och telemetry:** Används för att övervaka tekniska parametrar på länkar, PoE-budget, resursutnyttjande och belastning i det trådlösa nätverkets lufttid (airtime).
- **Syslog och Event-hantering:** Fokuserar på historik och spårbarhet kring brandväggstrafik, autentisering (authentication) och kritiska systemhändelser.
- **Instrumentpaneler (dashboards):** Sammanställer data till överskådliga nyckeltal som ger en snabb blick över nätverkets status, upptid och latens.

Exempel: Varje vardag vid lunchtid uppstår en plötslig spik i lufttidsutnyttjandet (airtime spike) på ett specifikt våningsplan, vilket gör nätet instabilt. Genom att granska loggarna ser administratören att gäst-SSID (guest SSID) blir helt överbelastat när elever och personal samlas i lunchrummet. Åtgärden blir att installera en extra accesspunkt (AP), aktivera **band steering** för att flytta klienter till 5 GHz-bandet samt justera **min-RSSI** för att tvinga enheter att ansluta till den närmaste sändaren, vilket löser problemet.

3.1.7 Patchning, ändringshantering och backup

Mer information finns i 11.9 Underhåll och säkerhetskopiering i boken *Datorteknik*.

För att bibehålla en stabil och säker nätverksmiljö krävs väl fungerande rutiner för underhåll. Det handlar om att balansera behovet av nya

funktioner och säkerhetsfixar mot risken för att uppdateringar orsakar oplanerade avbrott. Genom att arbeta med strukturerad patchning (patching), formell ändringshantering (change management) och automatiserade backuper (backups) skapas en miljö där misstag minimeras och återställningstiden vid fel förkortas avsevärt. Dokumentation är här den röda tråden; varje ändring ska kunna spåras bakåt och varje system ska kunna återställas till en känd fungerande version.

Säkerhetsuppdateringar och firmware-uppdateringar (firmware updates) bör hanteras enligt en fastställd patchpolicy (patch policy). Kritiska säkerhetsfixar prioriteras högst, men det är fackmässigt att alltid testa uppdateringen i en mindre skala eller i en labbmiljö innan den rullas ut i full produktion. Vid större ingrepp tillämpas ändringshantering (change management), vilket innebär att man planerar ett servicefönster (maintenance window), dokumenterar syftet med ändringen och identifierar eventuell påverkan på användarna. En central del i planeringen är att ha en tydlig rollback-plan (rollback plan) – en instruktion för hur man snabbt återställer systemet om något går fel under uppdateringen.

När det gäller backuphantering räcker det inte med att ta manuella kopior då och då. Istället bör nätverksutrustningens konfigurationer dumpas automatiskt, till exempel varje natt, och versionshanteras (versioning) så att administratören kan jämföra skillnader (diff) mellan olika datum. För maximal säkerhet bör backuperna sparas på en fysiskt separat plats (off-site backup) och rutinmässigt testas genom återläsningstester (restore tests). Endast genom att faktiskt prova att återställa en konfiguration kan man vara säker på att backupen fungerar den dagen den verkligen behövs. All dokumentation kring ändringar och konfigurationer bör följa minimikrav på tydlighet, inklusive titel, datum, version, ansvarig person och referenser till ärendenummer.

Dokumentationsexempel för storföretag

I en större organisation räcker det inte med lösa anteckningar; dokumentationen måste vara standardiserad för att möjliggöra samarbete mellan olika tekniker och avdelningar. Ett centralt dokument är ändringsansökan (Change Request - CR), som fungerar som ett formellt

beslutsunderlag inför ett arbete. En sådan dokumentation innehåller kritiska fält för att minimera osäkerhet och risk.

Ändringsansökan (Change Request)

Fält	Beskrivning / Exempel
Ärende-ID	CR-2026-03-23-001
Ansvarig tekniker	Robin Bräck
Syfte och omfattning	Uppgradering av firmware på kärnswitchar (core switches) för att åtgärda kritisk sårbarhet (CVE-2026-X).
Påverkan (Impact)	Total nätverksnedtid i Hus B under 15 minuter per switch.
Riskbedömning	Medelhög. Risk för konfigurationsfel vid omstart.
Rollback-plan	Vid fel: återställ föregående firmware-image via TFTP och läs in nattlig konfigurationsbackup.
Verifiering	Ping-test mot gateway, kontroll av OSPF-grannskap och verifiering av elev-VLAN.

För att bibehålla ordningen över tid används en ändringslogg (change log) som ger en kronologisk översikt av allt som skett i infrastrukturen. Detta dokument är det första en tekniker läser vid felsökning för att se om ett nyligen utfört arbete kan vara orsaken till ett problem. Loggen bör vara kortfattad men innehålla referenser till djupare dokumentation eller ärenden.

Ändringslogg (Change Log)

Datum	Enhet / System	Version	Ansvarig	Sammanfattning av ändring
2026-03-10	SW-CORE-01	v17.9.2	RB	Patchning av säkerhetshål. Inga avvikelser.
2026-03-12	FW-BORDER	v7.4.1	RB	Ny ACL för att blockera utgående DNS (Policy 3.1.4).

Datum	Enhet / System	Version	Ansvarig	Sammanfattning av ändring
2026-03-20	SW-ACCESS-05	v15.2.4	AL	Utökat DHCP-pool för Gäst-VLAN (VLAN 30).

Slutligen krävs en fysisk och logisk koppling i form av portetikettering (port labeling). Varje port i en switch ska vara dokumenterad så att det är tydligt vad som är anslutet, vilket VLAN som används och var den fysiska slutpunkten finns. Detta underlättar enormt när en tekniker befinner sig fysiskt i ett serverrum och snabbt behöver identifiera en specifik länk.

Portetikettering (Switch Port Map)

Switch	Port	VLAN	Beskrivning / Syfte	Fysisk plats / Uttag
SW-ACC-01	Gi0/1	10	Personal-PC	Rum 204, Uttag A1
SW-ACC-01	Gi0/2	20	Elev-AP	Korridor vån 2, takmontage
SW-ACC-01	Gi0/48	99	Trunk till CORE	Patchpanel 1, Port 12

Förtydligande (sammanfattning):

- **Patchpolicy:** Prioritera kritiska säkerhetsfixar och utför alltid tester i begränsad skala före bred utrullning.
- **Ändringshantering (Change):** Definiera vem som gör vad, när det sker, hur effekten mäts och hur en återgång (rollback) genomförs vid fel.
- **Backup:** Implementera nattliga konfigurationsdumpar med versionshistorik, säkerställ kopior utanför den primära sajten och utför regelbundna återläsningstester.

Exempel: En planerad uppgradering av skolans centrala brandvägg genomförs klockan 18:00 för att minimera påverkan på undervisningen. Före och efter uppgraderingen genomförs mätningar av latens (latency) och genomströmning för att verifiera resultatet. En fullständig backup av konfigurationen tas automatiskt precis före start, vilket möjliggör en snabb rollback och jämförelse

mot nätverkets standardkonfiguration (golden config) om problem skulle uppstå.

3.1.8 Säker administration och åtkomst

Mer information finns i 11.6 Användarkonton och behörigheter samt 11.10 Säkerhet i boken Datorteknik.

Säker administration och åtkomst utgör fundamentet för att skydda nätverkets kontrollplan (control plane). Om en obehörig får tillgång till administrativa rättigheter spelar övriga säkerhetsinställningar ingen roll, då hela infrastrukturen kan komprometteras inifrån. Grunden för en säker hantering börjar redan vid den första konfigurationen av en enhet genom att skapa ett dedikerat lokalt administratörskonto (local admin account) och omedelbart gå över till krypterad fjärråtkomst via **SSH** (Secure Shell). För att höja säkerheten ytterligare bör lösenordsbaserad inloggning fasas ut till förmån för nyckelautentisering (key authentication/SSH keys), vilket i princip eliminerar risken för brute force-attacker mot administrativa tjänster.

En bärande princip inom nätverksadministration är tillämpandet av minsta behörighet (least privilege). Det innebär att användare och administratörer endast ges de rättigheter som krävs för att utföra specifika uppgifter, och att administrativa konton aldrig används för vardagliga sysslor. I större miljöer separeras administrativ trafik helt från vanlig användartrafik genom ett dedikerat management-VLAN (management VLAN). För att nå detta VLAN från en extern plats eller från det vanliga användarnätet används ofta en **hoppserver** (jump host) eller en bastion (bastion host). Detta är en hårt härdad mellanserver placerad i ett säkert segment; administratören loggar först in på hoppservern, och därifrån etableras anslutningar till nätverksutrustningens hanteringsgränssnitt. Denna arkitektur säkerställer att ingen direktadministration kan ske från osäkra klientnätverk.

Utöver kontroll av åtkomstvägarna krävs stark autentisering (authentication). Detta innefattar unika administratörskonton (unique admin accounts) för varje individ, vilket skapar spårbarhet (traceability) i loggarna, samt implementering av **flerfaktorsautentisering** (MFA - Multi-Factor Authentication) där det är tekniskt möjligt. På nätverkets lägre nivåer, i lager 2 (Layer 2), måste infrastrukturen härdas för att

förhindra fysiska angrepp och felkopplingar. Detta görs genom att låsa accessportarna med **portsäkerhet** (port security), aktivera **DHCP-snooping** och **Dynamic ARP Inspection (DAI)** för att säkra adresseringen, samt skydda nätverkets topologi med **BPDU Guard**. Genom att kombinera dessa skyddsmekanismer, som beskrevs mer ingående i kapitel 2.2 och 2.5, skapas en miljö där administrativa åtkomster är strikt kontrollerade och loggade.

Förtydligande (sammanfattning):

- **Administrativ åtkomstväg:** Använd säkra anslutningar via VPN eller hoppserver (jump host) för att nå ett isolerat management-VLAN skyddat av brandväggsregler eller åtkomstlistor (ACL).
- **Identitet och behörighet:** Tillämpa unika administratörskonton, använd MFA och tilldela rättigheter baserat på principen om minsta behörighet (least privilege).
- **Lager 2-skydd (L2 hardening):** Säkra fysiska portar med portsäkerhet (port security), förhindra manipulerad adressering med snooping/inspection och skydda nätverkets logik med BPDU Guard och stormkontroll (storm control).

Exempel: En administratör som arbetar hemifrån kopplar först upp sig via en säker VPN-tunnel till skolans nätverk. Därifrån loggar administratören in på en härdad bastion (jump host) placerad i ett isolerat management-VLAN. Först från denna server tillåts SSH-anslutningar vidare till switchar och brandväggar. Nätverket är konfigurerat så att inga andra IP-adresser än bastion-serverns har rätt att kommunicera med nätverksutrustningens administrativa gränssnitt.

3.1.9 Fjärråtkomst och drift på distans

Mer information finns i 10 Dataöverföring och Fjärradministration i boken Datorteknik.

Säker fjärrdrift (remote operations) är en förutsättning för modern nätverksadministration, då det sällan är praktiskt eller möjligt att befinna sig fysiskt vid varje enhet. För att detta ska fungera krävs en genomtänkt arkitektur som erbjuder åtkomst på flera olika nivåer, från den

underliggande hårdvaran till det grafiska operativsystemet. Valet av verktyg styrs av vilken typ av system som ska administreras och vilket djup av kontroll som krävs för stunden.

Fjärrinloggning sker i huvudsak på tre olika nivåer. På den mest grundläggande nivån, hårdvarunivån, används tekniker för **out-of-band management** (OOB). För servrar innebär detta användning av dedikerade styrkort som **CIMC** (Cisco Integrated Management Controller), **iLO** (Integrated Lights-Out) eller **iDRAC** (integrated Dell Remote Access Controller). Dessa fungerar som en "dator i datorn" med egen nätverksanslutning och strömförsörjning. Det gör det möjligt för en administratör att fjärrstyra servern även om operativsystemet har kraschat, ändra inställningar i BIOS/UEFI eller utföra en hård omstart (hard reset) på distans.

På mjukvarunivå, direkt i operativsystemet eller i nätverksutrustningens konfigurationsläge, är **SSH** (Secure Shell) den absoluta standarden. SSH erbjuder en krypterad textbaserad terminalanslutning som är resurssnål och mycket säker, vilket gör den idealisk för administration av Linux-servrar, switchar och routrar. För system som kräver ett visuellt gränssnitt, framför allt Windows-baserade servrar och klienter, används den grafiska nivån via **RDP** (Remote Desktop Protocol). RDP strömmar skrivbordsmiljön till administratören och tillåter interaktion med mus och tangentbord som om man satt framför skärmen.

Oavsett nivå är säkerheten kring fjärråtkomsten kritisk. Inga administrativa portar (såsom SSH 22 eller RDP 3389) bör någonsin lämnas öppna direkt mot internet. Istället etableras en säker kanal via en **VPN-tunnel** (Virtual Private Network) med stark autentisering (authentication), ofta i kombination med en **hoppserver** (jump host). För kritiska platser, såsom centrala serverrum, bör man även planera för en alternativ väg in om den primära internetförbindelsen skulle gå ner. Detta löses ofta genom att ansluta hårdvarans management-portar till ett separat nätverk som nås via ett **LTE-modem** (4G/5G). På så sätt behåller administratören kontrollen över nätet även under ett omfattande fiberavbrott.

Förtydligande (sammanfattning):

- **Protokoll:** SSH för textbaserad administration och RDP för grafiska miljöer – båda ska alltid vara krypterade.

- **Kanaler:** All fjärråtkomst sker via VPN med stark autentisering (authentication); inga öppna admin-portar direkt mot internet.
- **Hårdvarustyrning (OOB):** Användning av CIMC, iLO eller iDRAC för åtkomst på hårdvarunivå, oberoende av operativsystemets status.
- **Alternativ väg (LTE):** Planera för utomstående kanaler (out-of-band) för att nå kritisk utrustning vid primära nätverksfel.

Felsökning i nivåer – Eskaleringssteg vid anslutningsproblem

När en tjänst eller server slutar svara bör en administratör följa en logisk eskaleringsstege (escalation path) för att identifiera var felet ligger. Genom att börja på den högsta nivån och arbeta sig neråt mot hårdvaran kan man snabbt avgöra om problemet ligger i applikationen, operativsystemet eller i den fysiska utrustningen.

1. Nivå 1: Applikation och Grafik (RDP/VNC/Webb)

Du försöker nå serverns skrivbord eller webbtjänst. Om detta misslyckas men servern fortfarande svarar på **ping**, tyder det på att just den specifika tjänsten har stannat eller att grafikmotorn i operativsystemet är överbelastad.

2. Nivå 2: Operativsystem och Terminal (SSH/PowerShell)

Om den grafiska inloggningen sviker, försöker du ansluta via en textbaserad terminal. SSH kräver betydligt mindre resurser än RDP. Om du kommer in här kan du starta om tjänster eller granska loggar. Om även SSH misslyckas (Connection refused/timeout), tyder det på att hela operativsystemet har hängit sig (kernel panic/blue screen).

3. Nivå 3: Hårdvara och Out-of-Band (CIMC/iLO/iDRAC)

Detta är din "sista livlina". Om operativsystemet är helt dött loggar du in via serverns hanteringskort. Här ser du serverns faktiska skärmutgång (KVM) och kan se felmeddelanden som annars kräver en fysisk skärm i serverrummet.

Exempel på ett felsökningsflöde:

- **Problem:** Anslutningen till filservern går ner.
- **Steg 1:** Du försöker ansluta via **RDP** för att se vad som händer $\rightarrow$ **Misslyckas** (Timeout).
- **Steg 2:** Du försöker ansluta via **SSH** för att starta om tjänsten $\rightarrow$ **Misslyckas** (Inget svar).
- **Analys:** Serverns operativsystem har sannolikt hängt sig totalt.
- **Lösning:** Du loggar in via **CIMC**, ser en felkod på den virtuella skärmen och utför en hård omstart (Power Cycle).
- **Verifiering:** Du följer uppstarten via CIMC, testar att **SSH** svarar när OS:et laddat klart, och verifierar slutligen att **RDP** och filtjänsten fungerar igen.

Exempel: Den primära fiberanslutningen till skolan går plötsligt ner. Tack vare ett installerat OOB-LTE-modem i serverrummet kan administratören ändå logga in via VPN till brandväggen. Väl inne kan administratören använda CIMC för att kontrollera serverstatus och omdirigera trafiken till en reservlänk utan att behöva åka till skolan fysiskt.

3.1.10 Automatisering och skriptning

Mer information finns i 7.7.3 Bash-skriptning och 8.2.7 Linux automatisera säkerhetsuppgifter (cron, script, loggning) i boken Datorteknik.

I takt med att nätverk växer i storlek och komplexitet blir det omöjligt att hantera varje enskild enhet manuellt. Automatisering (automation) och skriptning (scripting) är verktygen som gör det möjligt för en administratör att hantera hundratals enheter lika enkelt som en enda. Genom att använda Bash-skript eller schemalagda uppgifter via **cron** i Linux kan repetitiva manuella uppgifter automatiseras, vilket minskar risken för den mänskliga faktorn (human error). Detta arbetssätt kallas ofta för **Infrastructure as Code** (IaC), där nätverkets tillstånd definieras i filer som sedan rullas ut automatiskt.

Grunden för automatisering ligger i att behärska skriptspråk som **Python**, **Bash** eller **PowerShell**. Utöver rena skript används ofta

automatiseringsverktyg som **Ansible**, **Terraform** eller **Puppet**. Dessa verktyg tillåter administratören att skapa "spelböcker" (playbooks) som beskriver hur nätverket ska se ut. Istället för att logga in på 50 switchar för att skapa ett nytt VLAN, körs ett skript som pushar ut ändringen till alla berörda enheter samtidigt.

Förtydligande (sammanfattning):

- **Syfte:** Öka effektiviteten, minska konfigurationsfel och möjliggöra storskalig drift genom att gå från manuell till kodstyrd hantering.
- **Verktyg:** Python och Bash för generell automatisering; PowerShell för Windows-miljöer; Ansible eller Terraform för orkestrering (orchestration) av infrastruktur.
- **Metodik:** Infrastructure as Code (IaC) – lagra nätverkets konfiguration i läsbara filer som enkelt kan versionshanteras och återrullas.

Exempel: En skola ska rulla ut en ny säkerhetsinställning (t.ex. BPDU Guard) på 200 switchar. Istället för att logga in på varje enskild enhet, skriver administratören ett kort skript. Skriptet körs en gång, loggar in på samtliga switchar, applicerar inställningen och genererar en rapport över att allt lyckats. Det som tidigare tog dagar att genomföra manuellt är nu klart på några minuter.

3.1.11 Praktiska exempel på automatisering

En stor fördel med att använda verktyg som Ansible är konceptet **idempotens** (idempotency). Det innebär att om du kör skriptet en gång så genomförs ändringen, men om du kör det igen och inställningen redan finns, så gör skriptet ingenting. Detta gör automatiseringen mycket säkrare än enkla rad-för-rad-skript (scripting) som bara skickar kommandon blint.

Här är ett exempel på en enkel spelbok (playbook) som säkerställer att VLAN 10 (Personal) skapas på en grupp switchar.

YAML

- name: Konfigurera VLAN på skolans switchar

```
hosts: access_switchar
gather_facts: no
tasks:
  - name: Skapa VLAN 10 med namnet Personal
    cisco.ios.ios_vlans:
      config:
        - name: Personal
          vlan_id: 10
      state: merged
```

Inventariefiler och distribution

För att kunna distribuera (deploy) konfigurationen krävs även en **inventariefil** (inventory file). Detta är en lista som talar om för verktyget vilka enheter som tillhör gruppen `access_switchar`. I listan definieras IP-adresser och ofta vilka inloggningsuppgifter som ska användas för att ansluta via **SSH** (Secure Shell).

Ini, TOML

```
[access_switchar]
10.99.0.10
10.99.0.11
10.99.0.12
```

När filerna är klara distribueras konfigurationen från administratörens dator eller en **hoppserver** (jump host) genom ett enda kommando i terminalen:

```
ansible-playbook -i inventory.ini configure_vlan.yml
```

När kommandot körs etablerar Ansible en krypterad anslutning till varje switch i listan, kontrollerar om VLAN 10 redan finns och skapar det om det saknas. Administratören får sedan en tydlig rapport i terminalen som visar vilka enheter som ändrades (changed) och vilka som redan var korrekt konfigurerade (ok). Detta arbetssätt gör att nätverket blir homogent och att risken för att en switch glöms bort vid en stor förändring elimineras helt.

Bash-skript och SSH-loopar för enkel hantering

Medan **Ansible** är det professionella valet för storskalig drift, är **Bash-skript** (Bash scripting) utmärkt för att lära sig grunderna i hur man automatiserar enklare uppgifter. Här handlar det om att använda en loop (loop) för att utföra samma kommando på flera enheter efter varandra.

Ett vanligt scenario för en nätverkstekniker är att snabbt vilja hämta konfigurationen från flera switchar för att göra en backup, eller att skicka ut en enkel ändring till alla enheter i en viss korridor. Istället för att logga in på varje switch manuellt, skapar vi en textfil med alla IP-adresser och ett skript som läser filen rad för rad.

Först skapas en fil, till exempel `switchar.txt`, med de adresser som ska hanteras:

Plaintext

```
10.99.0.10
10.99.0.11
10.99.0.12
```

Därefter skriver vi skriptet `backup_switches.sh`. Skriptet använder en `for`-loop för att gå igenom listan och utföra ett kommando via **SSH**.

Bash

```
#!/bin/bash
# Enkelt skript för att hämta backup från switchar

# Definiera listan med IP-adresser
SWITCH_LIST="switchar.txt"

# Starta loopen som läser filen rad för rad
for IP in $(cat $SWITCH_LIST); do
    echo "--- Påbörjar backup av $IP ---"

    # Använd SSH för att köra kommandot och spara utdatan till en fil
    # 'admin' är användarnamnet på switchen
    ssh admin@$IP "show running-config" > backup_$IP.txt

    echo "Klart! Backup sparad som backup_$IP.txt"
done

echo "Samtliga backuper är genomförda."
```

Distribution och körning (Deployment)

Att distribuera (deploy) ett sådant här skript är enkelt eftersom det körs lokalt från din administratörsdator (exempelvis din Linux Mint-laptop) eller från en **hoppserver** (jump host). Innan skriptet körs behöver du se till att det har rättigheter att köras som ett program med kommandot `chmod +x backup_switches.sh`.

När du sedan kör skriptet med `./backup_switches.sh`, kommer din dator att försöka ansluta till varje switch i listan i tur och ordning. Om du har konfigurerat **SSH-nycklar** (SSH keys) mellan din dator och switcharna, kommer skriptet att köra helt automatiskt utan att fråga efter lösenord. Detta är en mycket kraftfull metod som visar hur grundläggande Linux-kunskaper (från kapitel 7 och 8 i din bok) direkt kan användas för att göra nätverksadministrationen mer effektiv och mindre felbenägen.

Förtydligande (sammanfattning):

- **Logik:** Använd en textfil för att lagra IP-adresser och en for-loop i Bash för att iterera genom dem.
- **Kommando:** Använd SSH för att skicka kommandon till switcharna och omdirigera utdatan (>) till filer på din egen dator.
- **Förutsättning:** Nyckelbaserad autentisering (key-based authentication) är nästan ett krav för att automatiseringen ska flyta smidigt utan manuella avbrott.

Kontrollfrågor

1. Förklara skillnaden mellan rollerna nätverksarkitekt (network architect) och nätverksdrift (network operations). Varför är det viktigt att skilja på dessa även i mindre organisationer?
2. Du misstänker att en Windows-klient har problem med sin gateway. Vilket kommando använder du i Windows för att se routingtabellen, och vad heter motsvarande kommando i Linux?
3. Varför är NTP (Network Time Protocol) en så kritisk del av en nätverksbaslinje (baseline) när det kommer till logganalys?
4. Beskriv begreppet split-horizon DNS. I vilket scenario är det fördelaktigt att använda denna teknik?
5. Vid felsökning enligt lager-för-lager-metodiken: Vilka steg kontrollerar du efter att den fysiska länken är bekräftad men innan du börjar titta på brandvägsregler?
6. Vad är den principiella skillnaden mellan vad SNMP talar om för dig jämfört med vad syslog berättar?
7. Inom ändringshantering (change management) talar man om en rollback-plan. Vad bör en sådan plan innehålla och när ska den upprättas?
8. Beskriv arkitekturen för en hoppserver (jump host). Vilket säkerhetsproblem löser den i samband med fjärradministration?
9. Du kan inte nå en server via RDP, men du kan logga in via CIMC/iLO. Vad drar du för slutsats om serverns tillstånd och vilka åtgärder kan du vidta via CIMC?
10. Vad innebär det att ett automatiseringsverktyg som Ansible är idempotent, och varför är det säkrare än att köra ett vanligt Bash-skript mot 100 switchar?

Fördjupningslänkar

Cisco: Guide to Network Management Design (Strategier för att bygga en stabil driftorganisation) –

<https://www.cisco.com/c/en/us/td/docs/solutions/Enterprise/Campus/campover.html>

Ansible for Network Automation (Komma igång med att hantera nätverk som kod) – <https://www.ansible.com/use-cases/network-automation>

Microsoft Learn: Troubleshoot Network Connectivity (Systematisk felsökning i Windows-miljöer) – <https://learn.microsoft.com/en-us/windows-server/networking/technologies/network-subsystem/net-sub-ts-guide>

Red Hat: Introduction to mtr (Varför mtr är bättre än traceroute för nätverksdiagnostik) – <https://www.redhat.com/sysadmin/mtr-network-diagnostics>

NTP.org: Why time is important (Teknisk bakgrund till tidssynkroniseringens betydelse) –

<http://support.ntp.org/bin/view/Support/WhyIsTimeImportant>

Cisco: Integrated Management Controller (CIMC) Guide (Fjärrstyrning på hårdvarunivå för M4/M5-servrar) –

https://www.cisco.com/c/en/us/td/docs/unified_computing/ucs/c/sw/gui/config/guide/2-0/b_Cisco_UCS_C-series_GUI_Configuration_Guide_201.html

SANS Institute: Secure Jump Stack Architecture (Hur man bygger säkra bastion-lösningar) – <https://www.sans.org/white-papers/36552/>

Wireshark Foundation: Troubleshooting with Wireshark (Video-tutorials för djupgående paketanalys) – <https://www.wireshark.org/docs/>

ITIL Foundation: Change Management Process (Branschstandard för hur man hanterar ändringar i IT-miljöer) –

<https://www.axelos.com/certifications/itil-service-management>

ISC2: The Principle of Least Privilege (Djupdykning i behörighetskontroll och säker administration) – <https://www.isc2.org/Insights/2021/05/The-Principle-of-Least-Privilege>

Egna anteckningar

...

3.2 Verktyg: ping, traceroute, ipconfig, netstat etc.

Mer information om grundläggande kommandon och diagnostik finns i 7.7.2 Kommandon samt 11.8 Verktyg, övervakning och loggning i boken Datorteknik.

Att behärska diagnostikverktyg handlar om betydligt mer än att bara kunna skriva in rätt kommandon; det handlar om förmågan att tolka den information som strömmar tillbaka på skärmen och använda den för att metodiskt utesluta felkällor. Varje verktyg i en administratörs arsenal ger en unik pusselbit i nätverkets aktuella status, från den lägsta fysiska nivån till applikationslagret. Genom att kombinera resultaten från olika verktyg kan en tekniker snabbt verifiera hypoteser under felsökning: Är det kabeln som är trasig, routern som tappat vägen, eller är det en tjänst i operativsystemet som slutat svara?

I detta avsnitt fördjupar vi oss i hur dessa verktyg fungerar under ytan och hur du som administratör läser av de subtila ledtrådar som finns i deras utdata (output). Vi kommer att titta på allt från klassisk nåbarhetskontroll (reachability) och vägvalsanalys till granskning av lokala nätverksstackar och portar. Målet är att utveckla en intuitiv förståelse för nätverkets beteende så att du snabbare kan gå från symptom till lösning.

3.2.1 Varför dessa verktyg?

När en användare rapporterar att något "inte fungerar" gäller det för administratören att snabbt och systematiskt avgöra exakt var i kedjan felet ligger. Det kan röra sig om allt från en fysisk nätlänk (network link) eller felaktig adressering (addressing) till problem med namnupplösning (name resolution), routing (routing) eller själva tjänsten och dess specifika port (port). De klassiska verktygen – såsom **ping**, **traceroute/tracert**, **ipconfig** (Windows) eller **ip** (Linux), **netstat/ss/nmap**, samt **nslookup/dig**, **arp/route** och vid behov **Wireshark/tcpdump** – låter dig testa kommunikationskedjan steg för steg. Grundtanken är att bekräfta de delar som fungerar för att kunna ringa in det exakta lager där det slutar fungera.

Genom att använda dessa verktyg i en logisk ordning undviker administratören att slösa tid på gissningar. Om man exempelvis kan verifiera att den lokala IP-konfigurationen är korrekt och att kontakten

med nätverkets första hopp (hop) fungerar, kan felsökningen snabbt flyttas vidare till högre lager som namnhantering eller applikationsspecifika inställningar. För mer komplexa problem krävs en djupanalys (deep analysis) av trafiken, vilket möjliggörs genom paketfångst (packet capture) där varje enskilt protokoll kan granskas i detalj för att hitta avvikelser.

Förtydligande (sammanfattning):

- **Länk och IP:** Kontrollera adressering med **ipconfig** eller **ip** och testa anslutningen till gateway med **ping**.
- **Namnupplösning:** Verifiera DNS-funktionen med **nslookup** eller **dig**.
- **Vägval (Routing):** Följ paketens väg genom nätverket med **tracert**, **tracert** eller **mtr**.
- **Tjänst och port:** Granska anslutningar lokalt med **netstat** eller **ss**, och testa tillgänglighet för specifika tjänster med **Test-NetConnection** (Windows) eller **nc** (Linux).
- **Djupanalys:** Utför detaljerad granskning av trafikflöden med **Wireshark** eller **tcpdump**.

3.2.2 ping – när vi fram överhuvudtaget?

Kommandot **ping** är nätverksadministratörens mest använda verktyg för att snabbt verifiera grundläggande nåbarhet (reachability). Det bygger på kontrollprotokollet **ICMP** (Internet Control Message Protocol) och fungerar genom att skicka en förfrågan, en så kallad **echo request**, till en målenhet som i sin tur förväntas svara med en **echo reply**. Genom denna enkla mekanism kan administratören mäta tre kritiska faktorer: om målet är uppe, hur lång tid kommunikationen tar (latency/latens) och om några paket försvinner på vägen (packet loss/paketförlust). Vid en systematisk felsökning är det fackmässigt att börja "nära" genom att pinga den egna IP-adressen (för att kontrollera den lokala nätverksstacken), därefter sin standard-gateway (för att kontrollera den lokala länken) och slutligen en extern adress på internet.

Det är dock viktigt att komma ihåg att **ping** inte är ett ofelbart bevis på att en tjänst fungerar. Många moderna brandväggar och servrar är

konfigurerade att blockera eller ignorera ICMP-trafik av säkerhetsskäl, vilket innebär att en enhet kan framstå som "död" i ett ping-test trots att applikationer som webb eller e-post fungerar utmärkt. Ping bekräftar alltså att vägen är öppen för ICMP, men 100 % paketförlust behöver inte betyda att nätet är nere – det kan helt enkelt vara en säkerhetspolicy som stoppar just dessa kontrollpaket. Vid tolkning av resultaten tyder hög latens eller stora variationer i svarstid (**jitter**) oftast på överbelastning eller störningar i infrastrukturen, medan enstaka paketförluster kan indikera hårdvarufel eller dålig signalstyrka i trådlösa nät.

Förtydligande (sammanfattning):

Windows: Använd `ping [IP/namn]` för fyra paket, `ping -t` för en kontinuerlig ström som hjälper vid felsökning av glappkontakt, eller `ping -4/-6` för att tvinga fram ett specifikt protokoll.

Linux: Använd `ping -c [antal]` för att begränsa antalet paket (annars fortsätter det i oändlighet) eller `ping -I [gränssnitt]` för att testa nåbarhet från ett specifikt nätverkskort.

Tolkning: Hög latens eller jitter tyder på nätproblem. Om 100 % av paketen går förlorade men tjänsterna i övrigt fungerar, är det sannolikt en brandvägg som blockerar ICMP.

Exempel: Du utför ett test där **ping 10.20.0.1** (gateway) fungerar felfritt, men när du kör **ping 8.8.8.8** börjar du tappa paket. Slutsatsen av detta test är att felet sannolikt ligger "utåt", någonstans mellan din lokala gateway och internetleverantören, snarare än i det interna nätverket i byggnaden.

Metodisk felsökning med ping – stegen utåt

Det första steget är att verifiera att den lokala nätverksstacken (network stack) i operativsystemet mår bra. Detta görs genom att pinga **loopback-adressen** 127.0.0.1. Om detta misslyckas är felet mjukvarubaserat; det tyder på att drivrutiner eller nätverksprotokoll i själva operativsystemet är korrupta eller felkonfigurerade, och ingen trafik kommer kunna lämna datorn oavsett hur kablarna ser ut. När mjukvaran är bekräftad går man vidare till att pinga sin **egen IP-adress**. Detta testar kommunikationen

med det fysiska nätverkskortet (NIC - Network Interface Card) och bekräftar att hårdvaran är vaken.

Efter att den egna hårdvaran bekräftats är nästa naturliga steg att testa kontakten med en **granne i samma VLAN**. Det innebär att du pingar en annan känd enhet, till exempel en kollegas dator eller en skrivare, som har en IP-adress inom samma subnät. Detta steg är kritiskt eftersom det helt utesluter routern ur ekvationen. Om du kan pinga en granne men inte din gateway, vet du med säkerhet att din **Lager 2-anslutning** (Layer 2 connectivity) fungerar och att felet specifikt rör kommunikationen mot routern eller en felaktig konfiguration av gateway-adressen. Om du däremot inte når din granne tyder det på ett lokalt problem i switchen, såsom en felaktig VLAN-tagging (VLAN tagging), problem med adresstjänsten (ARP - Address Resolution Protocol) eller att **portsäkerhet** (port security) har blockerat din anslutning.

När den lokala kommunikationen är säkerställd går man vidare till att pinga sin **standard-gateway** (default gateway). Detta testar den fysiska och logiska förbindelsen till nätverkets router. Därefter pingas en **publik IP-adress** ute på internet, till exempel 8.8.8.8, för att bekräfta att din router lyckas skicka paket vidare genom brandväggen och ut till internetleverantören (ISP). Det sista steget i kedjan är att pinga ett **domännamn**, exempelvis google.com, vilket bekräftar att namnupplösningen (DNS - Domain Name System) fungerar korrekt.

Sammanfattning av felsökningsstegen:

- **127.0.0.1 (Loopback):** Kontrollerar nätverksprotokollen i operativsystemet.
- **Egen IP-adress:** Kontrollerar att nätverkskortet (NIC) svarar och är aktivt.
- **Granne i samma LAN/VLAN:** Kontrollerar lokal Lager 2-kommunikation och LAN/VLAN-tillhörighet utan inblandning av routern.
- **Standard-gateway:** Kontrollerar den logiska vägen ut ur det lokala delnätet.
- **Publik IP (t.ex. 8.8.8.8):** Kontrollerar internetanslutningen och routing ut ur huset.

- **Domännamn (t.ex. google.com):** Kontrollerar att namnupplösningen (DNS) fungerar.

Exempel: En elev kan inte nå skolans lärplattform. Du pingar elevens gateway (OK), men när du testar att pinga elevens bänkgranne (som sitter i samma VLAN) misslyckas det. Genom detta enkla test har du ringat in att felet inte ligger hos internetleverantören eller i skolans centrala router, utan sannolikt i switchens konfiguration eller i den fysiska kabeldragningen mellan de två datorerna.

3.2.3 traceroute / tracert – var tar trafiken vägen?

När vi med hjälp av ping har konstaterat att en destination inte går att nå, eller att svarstiderna är ovanligt höga, behöver vi veta exakt var på vägen problemet uppstår. Här använder vi verktyg för vägvalsanalys (path analysis). I Windows heter kommandot **tracert** och i Linux används **traceroute**. Dessa verktyg fungerar genom att skicka ut en serie paket med ett medvetet lågt värde på fältet **TTL** (Time to Live). Varje router (hop) som paketet passerar minskar TTL-värdet med ett. När värdet når noll kastas paketet, och routern skickar tillbaka ett kontrollmeddelande (ICMP Time Exceeded) till avsändaren. Genom att successivt öka TTL från 1 och uppåt kan administratören kartlägga varje enskilt hopp längs vägen till målet.

Det är viktigt att känna till de tekniska skillnaderna mellan plattformarna för att kunna tolka resultaten korrekt. Windows-versionen (**tracert**) använder som standard ICMP-paket för hela mätningen. Linux-versionen (**traceroute**) använder däremot ofta UDP-paket mot höga portnummer som standard. Detta kan leda till att resultaten skiljer sig åt om det finns brandväggar längs vägen som tillåter en typ av trafik men blockerar den andra. I Linux kan man vid behov tvinga verktyget att använda ICMP genom växeln `-I`. Om en rad i utdatan endast visar asterisker (`* * *`), betyder det att den specifika routern inte svarade i tid, vilket ofta beror på att administratören för den enheten har valt att prioritera ner eller helt blockera kontrolltrafik av säkerhetsskäl.

mtr – nätverksteknikerns realtidsvy

För en mer djupgående analys i Linux-miljöer är **mtr** (My Traceroute) det överlägsna valet. Till skillnad från den klassiska traceroute-funktionen,

som endast ger en ögonblicksbild av vägen, kombinerar **mtr** funktionaliteten från både ping och traceroute i en levande realtidsvy. Verktøget fortsätter att skicka paket och samla in statistik så länge det körs, vilket gör det möjligt att se trender över tid. Detta är ovärderligt för att identifiera periodiska paketförluster (intermittent packet loss) eller variationer i svarstid (jitter) på specifika hopp. Medan en vanlig traceroute kan se "ren" ut vid ett enstaka test, kan **mtr** avslöja att en router i mitten av kedjan faktiskt tappar 5 % av trafiken under belastning.

Förtydligande (sammanfattning):

- **Windows:** Använd `tracert -d [IP/namn]`. Växeln `-d` förhindrar att verktøget försöker göra DNS-uppslag på varje hopp, vilket gör att mätningen går betydligt snabbare.
- **Linux:** Använd `traceroute -I [IP/namn]` för att använda ICMP, eller kör **mtr** för att få en interaktiv vy med statistik över latens och paketförlust för varje hopp.
- **Tolkning:** Om privata IP-adresser (t.ex. 10.x.x.x) dyker upp mitt i en sökning mot internet tyder det på att trafiken passerar genom lokala nätverk, NAT-lösningar eller operatörers interna infrastruktur. Ett stopp mellan hopp N och N+1 indikerar en blockering eller ett fel på just den sträckan.

Exempel: Du kör kommandot `tracert intranet.skolan.se` och ser att trafiken flyter fint genom de första tre hoppen, men stannar helt vid det fjärde hoppet som identifieras som skolans interna brandvägg. Slutsatsen blir att anslutningen fungerar fram till brandväggen, men att antingen en brandväggsregel blockerar trafiken eller att det saknas en korrekt rutt (route) vidare mot det skyddade nätverket (DMZ).

3.2.4 ipconfig (Windows) / ip (Linux) – grunder och snabbrengöring

Mer information om nätverkskonfiguration i operativsystem finns i 7.7.2 Kommandon samt 11.2 Grundläggande arkitekturprinciper i boken Datorteknik.

För att förstå hur en specifik enhet ser på nätverket används verktyg som inspekterar det lokala gränssnittet (interface). Här ser administratören avgörande detaljer som aktuell IP-adressering, standard-gateway (default gateway), DNS-servrar, anslutningsspecifika suffix, DHCP-status och det fysiska länkläget (link state). I Windows-miljöer är det klassiska kommandot **ipconfig** standard, medan Linux-system har gått över till det mer kraftfulla verktyget **ip** (från paketet iproute2), vilket ersätter det numera föråldrade ifconfig.

När en klient har problem med anslutningen är det första steget att kontrollera om gränssnittet överhuvudtaget har tilldelats en giltig IP-adress. Om en Windows-klient visar en adress i spannet 169.254.x.x har den tilldelats en **APIPA**-adress (Automatic Private IP Addressing). Detta är ett tydligt tecken på att enheten inte har fått kontakt med en DHCP-server, trots att nätverkskortet och kabeln fungerar. I sådana lägen kan administratören utföra en "snabbrengöring" genom att tvinga fram en ny adressförfrågan eller rensa lokala cacher som kan innehålla gammal eller felaktig information.

I moderna nätverksmiljöer, särskilt sådana som använder systemd, hanteras namnupplösningen ofta av specifika tjänster snarare än bara en statisk fil. I Linux används då verktyget **resolvectl** för att granska och styra vilken DNS-server som faktiskt används för ett specifikt gränssnitt. Att kunna rensa DNS-cachens innehåll (**flush DNS**) är också en viktig del av felsökningsprocessen, då klienten annars kan fortsätta försöka ansluta till en gammal IP-adress trots att DNS-posten har uppdaterats centralt.

Syntax-exempel för Windows (PowerShell/CMD)

I Windows är ipconfig huvudverktyget för att interagera med nätverksstacken (network stack). För att se den fullständiga konfigurationen, inklusive fysiska adresser (MAC addresses) och DNS-servrar, används växeln för alla detaljer:

```
ipconfig /all
```

När en klient har fått en felaktig adress, eller om du nyligen har ändrat ett DHCP-scope i servern, kan du tvinga klienten att släppa sin nuvarande adress och begära en ny genom att köra dessa två kommandon i följd:

```
ipconfig /release
```

```
ipconfig /renew
```

För att hantera problem med namnupplösning (name resolution) där klienten minns gamla eller felaktiga IP-adresser, kan du först granska innehållet i den lokala cachen (cache) och sedan rensa den helt:

```
ipconfig /displaydns
```

```
ipconfig /flushdns
```

Syntax-exempel för Linux (Bash)

I Linux-miljöer används det moderna `ip`-kommandot för att hantera nästan all nätverkskonfiguration. Det är modulärt uppbyggt där du anger vilket objekt du vill inspektera, såsom adresser (`addr`), länkar (`link`) eller rutter (`route`). För att se alla IP-adresser på systemets gränssnitt (interfaces) används:

```
ip addr (eller kortversionen ip a)
```

Om du enbart vill se om de fysiska eller virtuella kablarna är anslutna, det vill säga länkhastighet (`link speed`) och status, används `link`-modulen:

```
ip link
```

För att se hur datorn skickar trafik vidare, alltså dess routingtabell (`routing table`) och vilken enhet som är standard-gateway (`default gateway`), skrivs följande:

```
ip route
```

I moderna distributioner som kör **systemd** (vilket är standard i exempelvis Ubuntu och Linux Mint) hanteras DNS-inställningarna ofta av en specifik tjänst. För att se vilka namnservrar (`resolvers`) som faktiskt används för varje specifikt nätverkskort används kommandot för upplösningskontroll:

```
resolvectl status
```

Genom att använda dessa kommandon direkt i din terminal kan du snabbt verifiera att de logiska inställningarna stämmer överens med din adressplan (`address plan`).

Förtydligande (sammanfattning):

- **Windows:** Använd `ipconfig /all` för att se all detaljerad information. Med `/release` följt av `/renew` begärs en ny

adress från DHCP-servern. Kommandot `/flushdns` tömmer den lokala namn-cachen, medan `/displaydns` visar dess nuvarande innehåll.

- **Linux:** Använd `ip addr` för adresser, `ip link` för fysisk länkstatus och `ip route` för att se routingtabellen. För systemd-baserade system används `resolvectl status` för att verifiera DNS-inställningar.
- **Snabbkoll:** Administratören kontrollerar systematiskt: Har gränssnittet (interface) en IP? Är det rätt VLAN? Stämmer gateway och DNS med nätverkets adressplan?

Exempel: En klient i nätverket visar adressen `169.254.12.45` (APIPA). Detta bevisar att nätverkskortet är aktivt men att DHCP-förfrågan inte når fram eller inte besvaras. Som administratör börjar du då kontrollera om **DHCP relay** (ip-helper) är korrekt konfigurerad i routern och att switchens port tillhör rätt VLAN, så att klientens förfrågan faktiskt kan nå DHCP-servern.

3.2.5 netstat / ss – lyssnar tjänsten, och vem pratar den med?

När du har bekräftat att IP-anslutningen och namnupplösningen fungerar, skiftas fokus till själva applikationslagret (application layer). Frågan är då: Har tjänsten startat korrekt och "lyssnar" den på rätt port? För att ta reda på detta används **netstat** i Windows och det modernare **ss** (Socket Statistics) i Linux. Dessa verktyg ger administratören en direkt insyn i nätverksstackens aktuella anslutningar och visar vilka portar som är öppna för inkommande trafik. Det är här du kan avgöra om ett problem beror på att en tjänst har kraschat lokalt eller om trafiken blockeras av en brandvägg (firewall) någonstans på vägen.

I Windows ger kommandot en överblick över alla aktiva anslutningar och lyssnande portar (listening ports). Genom att använda specifika växlar kan administratören se vilket process-ID (PID) som äger en viss anslutning, vilket gör det möjligt att spåra en misstänkt nätverksaktivitet till ett specifikt program. I Linux-världen har `ss` ersatt det gamla `netstat` eftersom det är avsevärt snabbare och kan visa mer detaljerad statistik om nätverksuttag (sockets). Om du ser att en tjänst har status **LISTEN** på servern, men en klient fortfarande inte kan ansluta, är det ett starkt bevis

på att problemet sannolikt ligger i en brandvägg eller felaktig routing (routing) mellan enheterna, snarare än i själva applikationen.

Nmap – Nätverksteknikerns blick utifrån

Som ett komplement till de lokala verktygen används **nmap** (Network Mapper) för att skanna nätverket från utsidan. Medan `netstat` och `ss` talar om vad som händer inuti servern, visar `nmap` vad en angripare eller en klient faktiskt kan se över nätverket. Det är ett outhärligt verktyg för att verifiera att brandväggsregler och **NAT** (Network Address Translation) fungerar som tänkt. För administratörer som föredrar ett grafiskt gränssnitt (GUI) finns **Zenmap**, vilket är den officiella grafiska fronten för `nmap` och fungerar utmärkt på både Windows och Linux. Med dessa verktyg kan du utföra allt från enkla portskanningar till avancerad tjänsteidentifiering (service discovery) och operativsystemdetektering (OS detection).

Förtydligande (sammanfattning):

- **Windows:** Använd `netstat -ano` för att se alla portar och deras tillhörande PID. Med `netstat -anob` (kräver administratörsrättigheter) kan du även se namnet på den körbara filen direkt. För att matcha ett PID med ett programnamn används `tasklist /FI "PID eq [nummer]"`.
- **Linux:** Använd `ss -tulpen` för att se TCP/UDP-anslutningar, lyssnande portar, processnamn och användar-ID på ett mycket detaljerat sätt.
- **Nmap/Zenmap:** Används för att skanna enheter utifrån för att se vilka portar som faktiskt är nåbara över nätverket. Det bekräftar att brandväggar och portöppningar är korrekt konfigurerade.
- **Tolkning:** Om status är **LISTEN** lokalt på servern men anslutningen misslyckas utifrån, ligger felet i infrastrukturen (brandvägg/routing) mellan klienten och servern.

Exempel: En webbserver svarar inte på anrop från internet trots att servern är igång. Du kör `netstat -ano` lokalt på servern och ser att tjänsten står som **LISTENING** på port 443. Eftersom tjänsten lyssnar korrekt drar du slutsatsen att felet troligen ligger i nätverkets brandvägg eller att en felaktig **DNAT** (Destination NAT) pekar

*trafiken mot fel intern IP-adress. Du verifierar detta genom att köra en nmap-skanning från en extern klient och ser att porten visas som **filtered** eller **closed**.*

3.2.6 nslookup / dig – namnen som allt hänger på

Mer information om felsökning av nätverkstjänster finns i 11.8 Verktyg, övervakning och loggning i boken Datorteknik.

Domännamnssystemet (DNS - Domain Name System) är den ryggrad som nästan alla moderna nätverksapplikationer vilar på. Utan fungerande namnupplösning (name resolution) stannar kommunikationen av, trots att den underliggande IP-anslutningen kan vara felfri. För en administratör räcker det inte med att veta att namnupplösningen fungerar; man måste kunna kontrollera *vilket* svar en specifik server ger och för *vilken* typ av resurs. Verktygen **nslookup** (tillgängligt i både Windows och Linux) och **dig** (Domain Information Groper, standard i Linux) är oundgängliga för att ställa direkta frågor till namnservrar och verifiera att zondata är korrekt publicerad.

En av de största fördelarna med dessa verktyg är möjligheten att frånga datorns standardinställningar och istället fråga en specifik namnserver (resolver) direkt. Detta är avgörande vid felsökning av interna zoner eller när man precis har migrerat en tjänst och vill se om ändringen har propagerat (propagated) till publika servrar som Google (8.8.8.8) eller Cloudflare (1.1.1.1). Administratören kan också specificera posttyper (record types) för att hämta mer än bara vanliga IPv4-adresser (**A-poster**). Det kan handla om IPv6-adresser (**AAAA**), e-postservrar (**MX**) eller de kritiska **SRV-posterna** som används av Active Directory och andra tjänster för att lokalisera domänkontrollanter och specifika nätverksresurser.

För djupare analys i Linux-miljöer erbjuder dig funktionen +trace, vilket gör att verktyget följer hela kedjan från rotzonen (root zone) ner till den auktoritativa namnservern (authoritative name server) för den specifika domänen. Detta avslöjar exakt var i hierarkin ett fel uppstår. Ett vanligt symptom vid drift av komplexa nätverk är att ett namn fungerar utmärkt internt men inte externt, eller tvärtom. Detta tyder ofta på en felaktig konfiguration av **split-horizon DNS** eller problem med vidarebefordrare

(forwarders), där interna förfrågningar inte hittar rätt väg ut till det publika internet.

Förtydligande (sammanfattning):

- **Snabbtest:** Verifiera ett internt namn mot en specifik server med `nslookup files.skolan.local 10.40.0.10` eller ett publikt namn i Linux med `dig A example.com @1.1.1.1`.
- **Fördjupning:** Kontrollera tjänsteposter i AD-miljö med `nslookup -type=SRV _ldap._tcp.skolan.local` eller följ hela uppslagsresan med `dig +trace example.com`.
- **Symptom:** Om namnupplösningen fungerar internt men misslyckas för externa domäner krävs en granskning av DNS-serverns forwarders eller brandväggens regler för port 53 (UDP/TCP).

Exempel: Vid ett test körs `nslookup intranet.skolan.se` och svaret blir en publik IP-adress. Problemet är att användare som befinner sig inne på skolan behöver få den interna DMZ-adressen för att komma åt tjänsten. Denna diskrepans löses antingen genom att skapa en motsvarande intern A-post i skolans lokala DNS-zon (split-brain) eller genom att implementera **hairpin NAT** i brandväggen (se avsnitt 2.4), vilket tillåter interna klienter att nå den publika adressen via brandväggens insida.

3.2.7 arp / route – lager-2-cache och standardvägar

Mer information om lagren i OSI-modellen finns i 9.2 Arkitektur – Lagren och PDU samt praktiska kommandon i 7.7.2 Kommandon i boken Datorteknik.

För att kommunikation ska fungera hela vägen från en applikation till den fysiska kabeln måste datorn kunna översätta logiska adresser till fysiska adresser och veta vilken väg paketen ska ta. Det är här **ARP** (Address Resolution Protocol) och **routingtabellen** (routing table) kommer in i bilden. Medan de verktyg vi tidigare gått igenom (ping, traceroute) testat anslutningen på avstånd, visar dessa verktyg hur datorns interna logik ser ut i detalj.

Inom IPv4 används **ARP** för att mappa en IP-adress till en fysisk **MAC-adress** (Media Access Control address) på det lokala nätverket. Utan ett fungerande ARP-uppslag kan datorn inte kapsla in sina paket i ramar (frames) på lager 2, vilket innebär att ingen trafik kan skickas även om IP-inställningarna är korrekta. I Windows används kommandot `arp -a` för att se den lokala cachen, medan Linux-administratören använder det moderna `ip neigh` (neighbor). För den moderna efterföljaren IPv6 används istället **Neighbor Discovery** (ND), vilket fyller samma funktion men är integrerat i ICMPv6.

När trafiken ska lämna det lokala delnätet (subnet) konsulterar datorn sin **routingtabell**. Denna tabell fungerar som en karta som talar om för operativsystemet vilket gränssnitt (interface) och vilken **standard-gateway** (default gateway) som ska användas för olika mål. Genom att granska tabellen med `route print` i Windows eller `ip route` i Linux kan en administratör upptäcka felaktiga rutter som skär av trafiken eller skapar onödiga omvägar. En särskilt viktig detalj är att kontrollera att det inte finns flera motstridiga standardvägar, vilket ofta händer om en laptop är ansluten till både trådburet nätverk (Ethernet) och trådlöst nätverk (WLAN) samtidigt.

Förtydligande (sammanfattning):

- **ARP/ND:** Om uppslag till gatewayens MAC-adress saknas i cachen tyder det på ett fel i det fysiska nätverket eller felaktig VLAN-konfiguration på switchen (Lager 2-fel).
- **Default-rutter:** Om tabellen visar dubbla default-rutter kan operativsystemet skicka trafik slumpmässigt mellan olika nätverkskort, vilket skapar asymmetrisk trafik (asymmetric routing) och instabila anslutningar.
- **Specifika rutter:** En felaktig statisk rutt med en hög noggrannhet (t.ex. en /32-mask) kan "stjäla" trafik ämnad för ett specifikt mål och skicka den till helt fel gateway.

Exempel: Kommandot `route print` visar att en klient har två aktiva default gateways, en via WLAN och en via Ethernet. På grund av detta skickar klienten ibland trafik via det långsammare trådlösa nätet trots att kabeln är ansluten. Lösningen är att antingen stänga av det ena gränssnittet eller justera gränssnittets **metric** (viktning)

så att det trådbundna nätverket alltid prioriteras av operativsystemet.

3.2.8 Wireshark / tcpdump – när du behöver se paketen

Mer information om paketanalys och diagnostik finns i 11.8 Verktyg, övervakning och loggning samt i 10.2.2 HTTPS – Den krypterade efterföljaren i boken Datorteknik.

När de grundläggande verktygen för nåbarhet och namnupplösning inte räcker till för att identifiera ett problem, krävs en djupare insyn i nätverkstrafiken. Detta görs genom paketfångst (packet capture/sniffing), där administratören spelar in och granskar de faktiska datapaketerna som skickas över nätverkskortet. För detta ändamål är **Wireshark** den ledande lösningen med sitt kraftfulla grafiska gränssnitt (GUI), vilket gör det enkelt att visualisera protokollhierarkier och följa trafikflöden. I miljöer där ett grafiskt gränssnitt saknas, såsom på servrar, brandväggar eller vid fjärradministration via SSH, används standardverktyget **tcpdump**. Det är ett textbaserat terminalverktyg (CLI) som är extremt resurssnålt och effektivt för att snabbt fånga trafik direkt vid källan.

För att undvika att drunkna i informationsmängden är det avgörande att fånga trafik så nära felet som möjligt och använda snäva filter (filtering). Genom att definiera specifika kriterier för värddator (host), port (port) och protokoll (protocol) kan administratören filtrera bort bakgrundsbrus (noise) och fokusera på den relevanta kommunikationen. Vid analys av nätverkstrafik är det fackmässigt att först kontrollera att den grundläggande transporten fungerar korrekt genom att studera treväghandskakningen (three-way handshake). Om analysen visar ett utgående **SYN**-paket som aldrig besvaras med ett **SYN/ACK**, tyder det på att trafiken antingen blockeras av en brandvägg eller att det inte finns någon aktiv tjänst som lyssnar på målporten. Om handskakningen däremot fullbordas men applikationen ändå felar, ligger problemet högre upp i applikationslagret (application layer).

Vid arbete med paketfångst är det av yttersta vikt att beakta sekretess och personlig integritet (privacy/confidentiality). Eftersom dessa verktyg kan fånga känslig information i klartext, såsom användarnamn eller innehåll i okrypterade protokoll, bör de endast användas i felsökningssyfte och på ett sätt som följer verksamhetens säkerhetspolicy. I moderna

krypterade miljöer (HTTPS/TLS) kan innehållet i paketen vara dolt, men administratören kan fortfarande utläsa värdefull metadata om certifikat, krypteringsalgoritmer (ciphers) och anslutningsförsök som ofta är nyckeln till att lösa komplexa anslutningsproblem.

Förtydligande (sammanfattning):

- **tcpdump:** Används i terminalen med syntax som `tcpdump -i eth0 host 10.40.0.50 and port 443` för att fånga trafik på ett specifikt gränssnitt.
- **Wireshark:** Används för grafisk analys med filteruttryck som `ip.addr == 10.40.0.50 && tcp.port == 443`.
- **Tolkning:** En SYN utan efterföljande SYN/ACK indikerar blockering eller saknad lyssnare. En komplett handskakning följt av fel tyder på problem i applikationslagret eller med krypteringen.

Exempel: En klient kan nå en servers port 443, men vid inloggningsförsök stannar processen och "snurrar" oavbrutet. En paketfångst genomförs och visar att trevägshandskakningen lyckas, men att anslutningen därefter bryts på grund av ett **TLS-fel** (Handshake Failure). Vidare analys visar att klienten försöker använda en för gammal eller osäker kryptering (cipher) som serverns hårdade säkerhetspolicy inte längre tillåter. Åtgärden blir att uppdatera klientens mjukvara eller justera serverns TLS-policy för att återigen tillåta säkra anslutningar mellan parterna.

3.2.9 Ett snabbt felsökningsrecept (kedjan från klient till tjänst)

Mer information om metodik finns i 3.1.5 Felsökningsmetodik (lager för lager) samt i 11.8 Verktyg, övervakning och loggning i boken Datorteknik.

Felsökning i ett nätverk handlar om att eliminera felkällor metodiskt genom att röra sig från det lokala och kända utåt mot det fjärranslutna och okända. En erfaren administratör ser felsökningen som en kedja av beroenden; om den första länken brister spelar det ingen roll hur väl resten är konfigurerat. Genom att följa ett fastställt "recept" kan du snabbt ringa in om problemet ligger i klientens mjukvara, nätverkets infrastruktur eller i den slutgiltiga tjänsten. Att

bygga en vana att dokumentera varje steg – vilket kommando som kördes och vad resultatet blev – gör dessutom återkopplingen till användare och kollegor rak och professionell.

Felsökningskedjan: Från länk till applikation

Den logiska ordningen för att härleda ett fel bör alltid följa denna hierarki:

1. **Länk & IP:** Har vi en fysisk eller trådlös anslutning och en giltig adress?
2. **Gateway:** När vi nätverkets utgång och därmed vår lokala infrastruktur?
3. **Extern IP:** Har vi en väg ut till internet (utan att blanda in namnupplösning)?
4. **Namn (DNS):** Kan vi översätta mänskliga namn till IP-adresser?
5. **Väg (Routing):** Tar paketen rätt väg genom nätverket utan att fastna?
6. **Port & Tjänst:** Svarar applikationen på den specifika porten?
7. **Applikation:** Fungerar logiken inuti tjänsten (t.ex. inloggning)?

Genomförande i Windows-miljö

I en Windows-miljö börjar du med att öppna en terminal (PowerShell eller CMD). Du kontrollerar först din adressering med `ipconfig /all` för att se att du inte har en APIPA-adress (`$169.254.x.x$`). Därefter bekräftar du kontakten med din router genom `ping [gateway-ip]`. För att utesluta DNS-problem pingar du en publik adress som `8.8.8.8`. Om det fungerar men `ping google.com` misslyckas, använder du `nslookup` för att granska DNS-svaren. För att kontrollera om en specifik tjänst, till exempel en webbserver, är nåbar på port 443 använder du det kraftfulla PowerShell-kommandot `Test-NetConnection -ComputerName [server] -Port 443`. Detta bekräftar att brandväggar tillåter trafiken och att tjänsten lyssnar.

Genomförande i Linux-miljö

I Linux använder du motsvarande verktyg men med en något annorlunda syntax. Du inleder med `ip addr` för att verifiera ditt gränssnitt (interface) och IP-adress. Kontroll av gateway och internet sker med `ping`, precis som i Windows.

För att granska namnupplösningen är dig det föredragna verktyget eftersom det ger en mer detaljerad teknisk utdata än nslookup. Om du misstänker problem längs vägen kör du mtr [mål-ip] för en levande vy över varje hopp. För att testa om en port är öppen på en fjärrserver kan du använda nc -zv [server] [port] (netcat), vilket snabbt bekräftar om anslutningen (connection) kan etableras. Om allt annat ser bra ut men tjänsten ändå krånglar, kan curl -v [url] användas för att se exakt vad som händer i HTTP-kommunikationen.

Förtydligande (sammanfattning):

- **ipconfig/ip:** Kontrollera om vi har en IP-adress samt rätt VLAN, gateway och DNS-inställningar.
- **ping gateway:** Verifiera lokal nåbarhet och att kabeln/WLAN fungerar.
- **ping extern IP:** Testa internetanslutningen utan beroende av DNS.
- **nslookup/dig:** Verifiera att namnupplösningen fungerar korrekt.
- **tracert/traceroute/mtr:** Identifiera var i nätverket trafiken stannar eller går omvägar.
- **netstat/ss:** Kontrollera lokalt på servern om tjänsten lyssnar på rätt portar.
- **Wireshark/tcpdump:** Används vid behov för djupanalys av protokoll och felaktiga handskakningar.

Exempel: En elev kan inte nå filservern *files.skolan.local*. Du följer receptet:

1. *ipconfig* visar att eleven har en korrekt IP-adress (**OK**).
2. *ping* mot gateway ger svar (**OK**).
3. *ping 10.40.0.50* (filserverns IP) ger svar (**OK**).
4. *nslookup files.skolan.local* returnerar en helt annan IP-adress än den förväntade.

Slutsats: Problemet är varken nätet eller servern, utan en felaktig post i den interna DNS-servern. Genom att uppdatera DNS-posten löser du problemet direkt utan att behöva "röra nätet" eller starta om någon hårdvara.

Kontrollfrågor

1. Du utför en ping mot en server som du vet är igång och som kör en fungerande webbtjänst, men du får 100 % paketförlust (packet loss). Vad är den mest troliga orsaken till detta beteende?
2. Förklara hur fältet TTL (Time to Live) används tekniskt av verktyg som tracert och traceroute för att identifiera varje hopp (hop) längs en nätverkswäg.
3. Vilken specifik fördel erbjuder verktyget mtr jämfört med en vanlig traceroute när man felsöker instabila nätverksförbindelser (intermittent connectivity)?
4. Om kommandot ipconfig visar adressen 169.254.10.12, vad berättar det för dig om klientens kontakt med nätverkets DHCP-server?
5. Varför är det ofta nödvändigt att köra kommandot ipconfig /flushdns efter att man har ändrat en A-post i en DNS-server?
6. Du misstänker att en webbserver inte har startat korrekt. Vilket kommando och vilka växlar använder du i Linux för att se om servern faktiskt lyssnar (listening) på port 443?
7. Beskriv skillnaden i perspektiv mellan att använda netstat lokalt på en server och att använda nmap från en extern klient.
8. Hur använder du nslookup eller dig för att kontrollera en specifik DNS-post mot en extern namnserver (resolver), till exempel 1.1.1.1, istället för att använda datorns standardinställningar?
9. Om en klients ARP-tabell (arp -a / ip neigh) saknar en post för standard-gatewayen (default gateway), vilka slutsatser kan du dra om anslutningen på Lager 2 (Layer 2)?
10. Vid analys i Wireshark ser du att en klient skickar ett SYN-paket men aldrig får något SYN/ACK tillbaka från servern. Ge två exempel på vad som kan orsaka detta avbrott i trevägshandskakningen (three-way handshake).

Fördjupningslänkar

Cloudflare: What is ICMP? (Genomgång av protokollet som driver ping och traceroute) – <https://www.cloudflare.com/learning/network-layer/what-is-icmp/>

PacketLife: Traceroute Internals (Detaljerad visualisering av hur TTL-manipulering fungerar) – <https://packetlife.net/blog/2008/dec/22/how-traceroute-works/>

BitWizard: mtr manual (Officiell dokumentation och avancerade växlar för mtr) – <https://www.bitwizard.nl/mtr/>

Microsoft Learn: ipconfig command reference (Samtliga växlar och funktioner för Windows nätverkskonfiguration) – <https://learn.microsoft.com/en-us/windows-server/administration/windows-commands/ipconfig>

Red Hat: Guide to the ip command (Modern nätverkshantering i Linux med iproute2) – <https://www.redhat.com/sysadmin/linux-ip-command>

Nmap.org: Nmap Network Scanning (Den officiella boken och guiden till nmap och portskanning) – <https://nmap.org/book/man.html>

DigitalOcean: Troubleshooting DNS with Dig (Praktiska exempel på hur man diagnostiserar DNS-problem i terminalen) – <https://www.digitalocean.com/community/tutorials/how-to-use-dig-to-query-dns-servers-on-ubuntu-20-04>

Wireshark: Filter Reference (Komplett lista över filteruttryck för att hitta specifik trafik) – <https://www.wireshark.org/docs/dfref/>

IETF: RFC 792 Internet Control Message Protocol (Den tekniska specifikationen för ICMP) – <https://datatracker.ietf.org/doc/html/rfc792>

Cisco: Troubleshooting TCP/IP (Omfattande guide för metodisk felsökning av nätverksstacken) – <https://www.cisco.com/c/en/us/support/docs/ip/transmission-control-protocol-tcp/10044-8.html>

Egna anteckningar

3.3 Dokumentation, etikett, felsökning

Att drifta ett nätverk handlar inte enbart om att behärska kommandon och protokoll; det handlar i lika hög grad om att skapa ordning och struktur genom noggrann dokumentation (documentation) och professionell etikett (etiquette). Utan en tydlig karta över nätverkets topologi (topology) och en logg över utförda förändringar förvandlas även det mest välbyggda systemet snabbt till en ogenomtränglig labyrinth vid ett framtida driftstopp. Genom att upprätthålla en hög standard för dokumentation skapas spårbarhet (traceability), vilket gör det möjligt för kollegor – eller en själv sex månader senare – att förstå varför en specifik konfiguration genomfördes och hur trafiken faktiskt är tänkt att flöda. Det handlar om att bygga en institutionell kunskap som inte är beroende av en enskild individs minne, vilket minskar sårbarheten i organisationen.

Utöver den rent tekniska dokumentationen krävs en medvetenhet kring arbetsmetodik och etikett, särskilt i miljöer där flera tekniker delar på ansvaret för infrastrukturen. Detta innefattar allt från konsekvent namngivning (naming conventions) och tydlig märkning av fysiska kablar till att följa etablerade rutiner för ändringshantering (change management). När ett fel väl uppstår är det den systematiska felsökningsmetodiken (troubleshooting methodology) som skiljer den professionella administratören från den som agerar reaktivt. Genom att kombinera teknisk insikt med ett metodiskt lugn kan komplexa problem brytas ner i hanterbara delar, vilket minimerar nertid (downtime) och säkerställer att de åtgärder som vidtas är både logiska och långsiktigt hållbara.

3.3.1 Varför dokumentation?

Dokumentation säkerställer att nätverket förblir förutsägbart, överlämningsbart och enkelt att felsöka. Utan uppdaterade kartor, planer för adressering och virtuella nätverk, portmatriser och ändringsloggar riskerar varje tekniskt fel att förvandlas till en tidskrävande gissningslek, medan uppgraderingar blir till osäkra chansningar. En väl genomarbetad dokumentationsstruktur sänker medelreparationstiden (MTTR - Mean Time To Repair), minskar risken för dubbelarbete och gör det möjligt för nya medarbetare i ett team att snabbt bli produktiva och förstå

infrastrukturens uppbyggnad. Genom att skapa en tydlig spårbarhet (traceability) går det att i efterhand visa exakt vad som har ändrats, vid vilken tidpunkt och av vilken anledning, vilket är en förutsättning för effektiv incidenthantering och vid revisioner (audits).

Arbetet med dokumentation handlar i grunden om att omvandla tyst kunskap till en gemensam tillgång som skyddar verksamhetens kontinuitet. När nätverkets logik finns nedtecknad i form av topologidiagram och konfigurationsstandarder, minskar beroendet av enskilda personers minne. Detta skapar en professionell trygghet där driftstörningar kan hanteras metodiskt utifrån fakta snarare än antaganden. För en administratör fungerar dokumentationen som ett kvitto på utfört arbete och som en försäkring mot framtida komplikationer, då den ger svar på hur nätverket var tänkt att fungera innan felet uppstod.

Exempel: Ett arkitektkontor med kontor i två olika städer upplever att den VPN tunneln mellan platserna går ner efter ett strömavbrott. Eftersom kontoret har en uppdaterad ändringslogg ser administratören snabbt att den förra teknikern tvingades byta ut en publik IP-adress för två månader sedan, men glömde att spara den nya konfigurationen permanent i routerns startup-config. Genom att konsultera dokumentationen hittar den nya administratören den korrekta publika adressen och kan återetablera anslutningen utan att behöva kontakta internetleverantören för support.

3.3.2 Vad ska dokumenteras?

För att dokumentationen ska vara till verklig nytta krävs en "minsta gemensamma nämnare" av information som alltid finns tillgänglig. Grundstommen i ett välskött nätverk består av topologidiagram, IP- och VLAN-planer, enhetsinventarier, säkerhetskopior på konfigurationer (config backups), portmatriser, brandväggspolicys och en noggrann ändringslogg. Utöver dessa tekniska dokument bör det även finnas **runbooks**, vilket är steg-för-steg-instruktioner för vanliga åtgärder, samt en driftjournal som beskriver vad som har hänt i nätverket, när det skedde och varför. En gyllene regel är att sträva efter "en sanning" (single source of truth) – en central plats där allt hålls uppdaterat med tydliga ägare för

varje dokument och en versionsmärkning som gör att historiken går att följa bakåt i tiden.

När man dokumenterar nätverkets **topologi** bör fokus ligga på strukturen med kärnnät, distributionsnät och accessnät (core, distribution, access). Det är viktigt att tydliggöra var upplänkar (uplinks) och trunkar (trunks) finns samt hur redundansen är uppbyggd. I **IP- och VLAN-planen** dokumenteras nätverkets prefix, gateways, DHCP-scopes samt inställningar för DNS och tidssynkronisering (NTP). För **enhetsinventariet** är det kritiskt att ha koll på modell, serienummer, fysisk placering, management-IP och aktuell version av operativsystemet. En annan central del är **portmatrisen**, som mappar den fysiska vägen från ett vägguttag via en patchpanel till en specifik switchport och dess tillhörande VLAN.

För att säkerställa säkerheten behövs en tydlig översikt över **brandväggspolicys**, det vill säga vilka trafikflöden som är tillåtna mellan olika zoner i nätverket. Varje förändring som görs bör sedan dokumenteras i en **ändringslogg** som innehåller datum, syfte med ändringen, identifierade risker, vem som utförde arbetet, en plan för att backa bandet (rollback) samt det slutgiltiga resultatet. Genom att ha denna struktur på plats minskar man risken för mänskliga faktorer och skapar ett nätverk som är robust nog att hanteras av flera olika tekniker över tid.

Runbook: Driftsättning av nytt klient-VLAN

En runbook fungerar som ett tekniskt recept som säkerställer att en uppgift utförs på exakt samma sätt oavsett vem i teamet som håller i tangentbordet. Det minskar risken för att man glömmer en kritisk detalj, som att tillåta det nya nätet i brandväggen eller att uppdatera den centrala dokumentationen. Nedan visas ett exempel på hur en sådan instruktion kan se ut för en vanlig administrativ uppgift.

Syfte: Att skapa ett isolerat nätverk för en ny avdelning eller labbmiljö med automatisk tilldelning av IP-adresser.

1. Förberedelser och värden Innan konfigurationen påbörjas ska följande värden fastställas och kontrolleras mot den centrala IP-planen för att undvika krockar:

- **VLAN ID:** 50

- **Namn:** LABB_AK2
- **Subnät:** 10.50.0.0/24
- **Gateway:** 10.50.0.1
- **DHCP-range:** 10.50.0.100 – 10.50.0.250

2. Konfiguration av Core-switch (Layer 3) Skapa det logiska nätverket och definiera dess gateway-gränssnitt (SVI - Switch Virtual Interface). Det är här routing för det nya nätet aktiveras.

- Skapa VLAN 50 och namnge det.
- Konfigurera gränssnittet Vlan50 med IP-adressen 10.50.0.1 och nätmasken 255.255.255.0.
- Aktivera gränssnittet (no shutdown).

3. Inställning av DHCP-server Säkerställ att klienter i det nya nätet får adresser automatiskt. Om DHCP-servern sitter i ett annat VLAN, kom ihåg att lägga till en **IP Helper-adress** på det nya VLAN-gränssnittet i core-switchen.

- Skapa ett nytt scope/pool för 10.50.0.0-nätet.
- Ange gateway (10.50.0.1) och DNS-servrar som ska tilldelas klienterna.

4. Uppdatering av trunkar och access-portar För att trafiken ska nå ut till användarna måste VLAN 50 tillåtas på de länkar (trunks) som går mellan switcharna.

- Identifiera upplänkar till access-switchar och lägg till VLAN 50 i listan över tillåtna nät.
- Gå till aktuell access-switch och tilldela de fysiska portarna till VLAN 50 (access mode).

5. Verifiering och Dokumentation Det sista och viktigaste steget är att kontrollera att allt fungerar och att "den enda sanningen" uppdateras.

- Anslut en testklient till en av de nya portarna och verifiera att den får en IP-adress i rätt spann.

- Utför ett ping-test mot gateway och en extern adress (t.ex. 8.8.8.8).
- **Uppdatera dokumentationen:** Skriv in de nya uppgifterna i IP-planen, portmatrisen och ändringsloggen.

Exempel: När en ny kortterminal ska installeras i en butik används en runbook som anger exakt vilka VLAN-inställningar och brandväggsöppningar som krävs. Detta säkerställer att betaltrafiken isoleras korrekt enligt säkerhetskrav, utan att teknikern behöver improvisera fram lösningar på plats.

3.3.3 Namnstandard, märkning och port-etikett

Ett välstrukturerat nätverk kännetecknas av att det går att "läsa" som en karta. Genom att tillämpa en konsekvent namnstandard kan en administratör omedelbart identifiera en enhets placering och funktion utan att behöva konsultera en manual. En beprövad metod är att namnge enheter utifrån logiken **plats-roll-löpnummer**. En switch placerad i en specifik byggnad med uppgiften att hantera access-lager kan exempelvis få namnet HUS1-ACC-03, medan en central distributionsenhet i samma byggnad benämns HUS1-DIST-A. Denna systematik sparar tid i alla led - från den initiala beställningen och konfigurationen till den dagliga felsökningen och den slutgiltiga städningen efter avslutade projekt.

Lika viktigt som de logiska namnen är den fysiska märkningen (labeling) av infrastrukturen. Det är fackmässigt att märka upp vägguttag, patchpaneler och switchportar så att det finns en obruten kedja av information. Även fiberkablar bör märkas i båda ändar för att undvika förväxlingar vid framtida underhåll. God etikett i nätverksmiljön handlar också om hur man lämnar en arbetsplats efter sig. Det innebär att återställa tillfällig patchning, ta bort labbutrustning som inte längre används och, kanske viktigast av allt, att alltid spara konfigurationen permanent på enheterna innan arbetet avslutas. Utan denna port-etikett riskerar nätverket att med tiden förvandlas till ett svåröverskådligt "nystan" där ingen längre vet vilken kabel som gör vad.

Förtydligande (sammanfattning):

- **Namngivning:** Namnen ska vara korta, meningsfulla och följa en fastställd struktur genom hela organisationen.

- **Portdisciplin:** Dokumentera alltid avsedda ändringar innan kablar dras om och undvik dolda hopkopplingar som inte syns i dokumentationen.
- **Märkning:** Fysiska etiketter på hårdvaran ska alltid stämma överens med den digitala portmatrisen.

Exempel : Vid en genomgång av nätet i en skola identifieras port A3 i switchen HUS2-ACC-04. Tack vare tydlig märkning syns det direkt att porten leder till "Klassrum 214 – skrivare". Genom att kontrollera portmatrisen kan teknikern på två sekunder bekräfta vilket VLAN porten tillhör och om den är konfigurerad med rätt hastighet, vilket gör felsökningen extremt effektiv om utskrifterna plötsligt slutar fungera.

3.3.4 Ärenden och kommunikation – etikett i drift

Ofta skämtar människor om att it-personal lever efter devisen "ingen ticket, ingen hjälp". Detta beror inte på att personalen är ohjälpsam, utan tvärtom; för att kunna hjälpa till på bästa sätt måste de kunna prioritera och tilldela resurser till rätt sak. Felsökning i ett professionellt nätverk sker aldrig i ett vakuum, och för att driften ska fungera smidigt krävs en tydlig ärendeprocess som hanterar allt från de första felanmälningarna till den slutgiltiga lösningen. Det handlar om att bekräfta mottagandet av ett problem, sätta en rimlig prioritet och se till att återkoppla vid viktiga milstolpar. En viktig del i detta är att vara transparent med vad som har mätts och vilka ändringar som har genomförts. Genom att kommunicera löpande, även när en lösning dröjer, minskar man frustrationen hos användarna och skapar ett förtroende för it-avdelningens arbete.

När ett fel väl är åtgärdat är det god etikett att stänga ärendet med en kortfattad analys av grundorsaken, ofta kallat en **RCA** (Root Cause Analysis). I denna analys är det avgörande att arbeta utifrån en kultur som inte söker syndabockar. Istället för att peka ut individer fokuserar man på systemet och processerna: Vad i vår konfiguration eller våra rutiner gjorde att felet kunde uppstå? Genom att vara objektiv och fackmässig i sin analys kan man identifiera hur liknande problem kan undvikas i framtiden. En sådan "blameless" kultur gör att teamet vågar dela med sig av misstag, vilket i slutändan leder till ett mer robust och stabilt nätverk.

Förtydligande (sammanfattning):

- **Prioritering:** Bedöm felets påverkan och brådska för att sätta rätt prioritet, ofta från P1 (kritisk/stopp) till P4 (låg/önskemål).
- **Statusuppdateringar:** Uppdatera ärendet regelbundet. Information om att undersökning pågår är bättre än tystnad för den som väntar på hjälp.
- **RCA light:** Dokumentera kortfattat vad som var den faktiska orsaken, vilken åtgärd som vidtogs och hur nätet kan säkras för att felet inte ska återkomma.

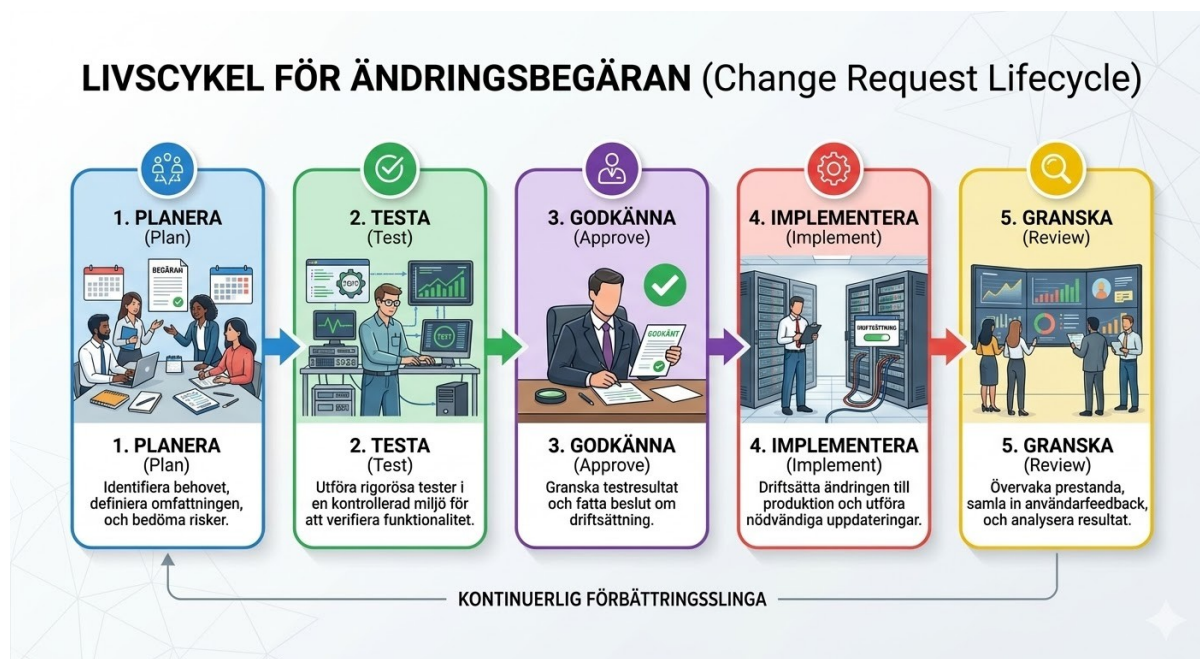
Exempel : En webbshop upplever att vissa kunder inte kan slutföra betalningar, vilket klassas som **P1: Affärskritiskt stopp**. Den tekniska analysen visar att en brandväggsregel blockerade trafik från en ny betaltväxel. Efter att regeln har korrigerats skrivs en RCA som förklarar att den externa leverantören bytt IP-adresser utan förvarning. För att undvika att det händer igen föreslås en automatiserad övervakning av betaltväxels portar så att it-avdelningen får ett larm innan kunderna märker felet nästa gång.

3.3.5 Change management – planera, testa, backa

Förändringar i ett nätverk ska vara förutsägbara och kontrollerade för att minimera risken för oväntade avbrott. Genom att tillämpa en strukturerad process för ändringshantering (change management) säkerställer administratören att varje ingrepp föregås av en ordentlig risk- och effektbedömning. Innan en ändring genomförs i den skarpa miljön bör den alltid utvärderas i en labbmiljö eller genom en pilotinstallation på en mindre, isolerad del av nätverket. Detta gör det möjligt att upptäcka konflikter eller prestandaproblem i en säker miljö innan de påverkar hela verksamheten. En central del i planeringen är att alltid ha en färdig rollback-plan – en tydlig instruktion för hur man snabbt återställer systemet till dess tidigare fungerande skick om något går fel.

Timing är avgörande för att minimera påverkan på användarna. Större ingrepp förläggs därför ofta till ett planerat underhållsfönster (maintenance window), vilket är en förbestämd tidpunkt då nätverket kan ligga nere eller ha begränsad funktion utan att störa den ordinarie verksamheten. Efter att en ändring har genomförts är arbetet inte klart

förrän resultatet har mätts och dokumenterats noggrant. Administratören kontrollerar loggar, larm och prestandavärden som latens och genomströmning för att säkerställa att målet med ändringen har uppnåtts. Genom att använda enkla checklistor minskar risken för de små missar som kan få stora konsekvenser, och man skapar en kvalitet som går att upprepa vid varje framtida ingrepp.



Förtydligande (sammanfattning): En komplett plan bör innehålla syfte, omfattning, riskanalys, testplan och en metod för återställning. Vid val av tidpunkt vägs behovet av tillgänglighet mot risken med ändringen; i en skola undviker man exempelvis kritiska uppdateringar under lektionstid för att inte störa undervisningen. Efterkontrollen innebär att man aktivt letar efter avvikelser i loggar eller prestandamätningar för att validera att allt fungerar som tänkt.

Change Request #2026-042 - Uppgradering av Core-brandvägg

Scenario: Skolans centrala brandvägg behöver en kritisk säkerhetsuppdatering. För att minimera risken för hela nätet genomförs först en identisk uppdatering på en reservenhet i labbet. När labbtesterna visar att trafiken flyter korrekt schemaläggs den skarpa uppdateringen enligt nedan.

Syfte: Den nuvarande mjukvaran i skolans huvudbrandvägg har identifierade säkerhetshål som behöver åtgärdas (security patches). Uppgraderingen kommer även att aktivera stöd för modernare kryptering för distansarbetande personal.

Omfattning: Uppgradering av operativsystemet från version 6.4 till 7.0 på den centrala brandväggen. Detta påverkar allt internetflöde, VPN-anslutningar och trafik mellan interna VLAN.

Risikanalys: Den största risken är ett totalt avbrott i internetförbindelsen för hela skolan om enheten inte startar om korrekt. Det finns även en låg risk för att befintliga brandväggsregler (firewall rules) inte översätts korrekt till den nya versionen.

Testplan: Efter genomförd uppdatering ska följande kontrolleras:

1. Internetåtkomst från både elev- och personalnät.
2. Etablering av VPN-tunnel från en extern klient.
3. Kontroll av systemloggar för att upptäcka eventuella hårdvarufel eller resursbrist.

Rollback-plan (Återställning): Innan start tas en fullständig backup av konfigurationen. Om brandväggen inte är i full drift inom 20 minuter, eller om kritiska fel upptäcks i testplanen, nedgraderas mjukvaran till version 6.4 via konsolkabel och den sparade konfigurationsfilen återläses.

Tidpunkt: Tisdag 24 mars kl. 17:00 (efter lektionstid). Beräknad nertid för användarna är 10 minuter under enhetens omstart.

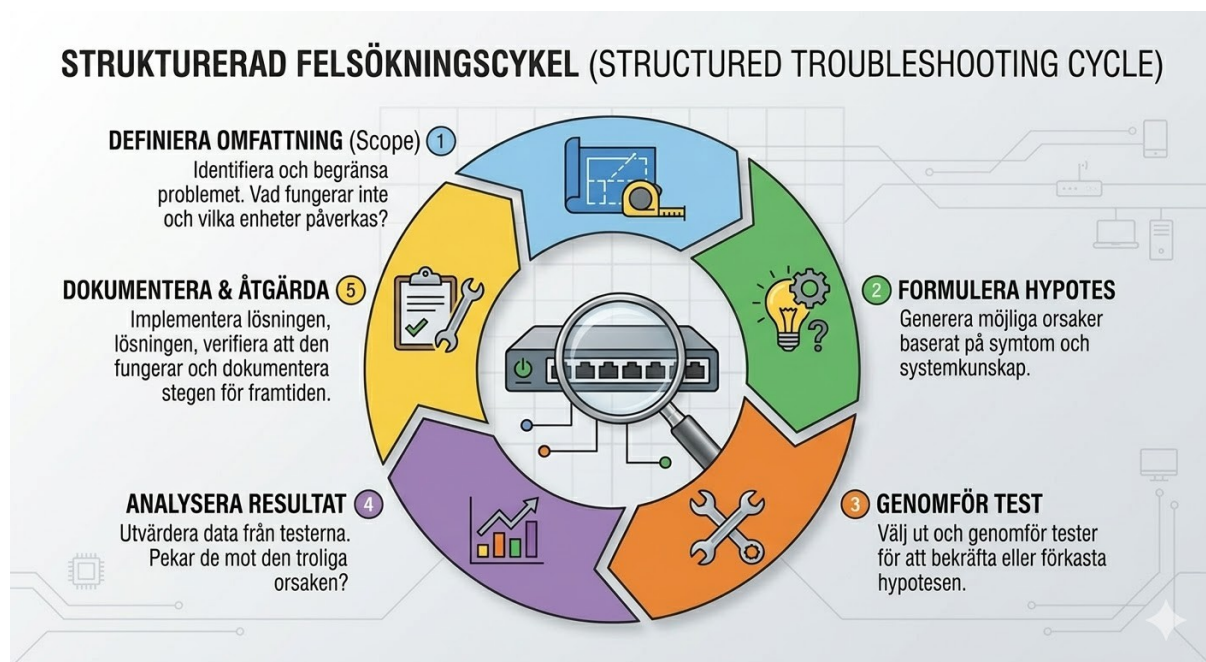
Efterkontroll: Systemet övervakas extra noga via nätverksövervakningen de kommande 24 timmarna. En slutgiltig genomgång av säkerhetsloggarna görs kl. 08:00 nästa morgon för att säkerställa att ingen legitim trafik blockeras.

3.3.6 Felsökningsmetodik – från symptom till rotorsak

Effektiv felsökning (troubleshooting) handlar mer om struktur än om intuition. Det första steget är att fastställa problemets omfattning (scope) genom att ställa kritiska frågor: Drabbar felet en enstaka användare eller en hel avdelning? Är det bara ett specifikt VLAN som är nere eller är hela

*byggnaden drabbad? Genom att tydligt definiera symptomen och jämföra dem med det förväntade beteendet kan administratören snabbt börja utesluta fungerande delar. Arbetet sker mest effektivt genom att arbeta systematiskt lager för lager, en metodik som beskrivs i detalj i avsnitt **3.1** och **3.2**. Varje steg i processen ska verifieras med mätbara tester, och en gyllene regel är att aldrig ändra flera variabler samtidigt; om tre saker ändras på en gång är det omöjligt att veta vilken av dem som faktiskt löste problemet eller förvärrade det.*

Att dokumentera i realtid under pågående felsökning är en av de mest värdefulla vanorna en tekniker kan utveckla. Genom att skriva ned exakt vad som har testats och vilka resultat det gav skapas en logisk kedja av bevis som förhindrar att man går i cirklar. Denna löpande dokumentation fungerar inte bara som ett stöd för det egna minnet, utan är också ovärderlig när ett ärende behöver eskaleras eller lämnas över till en kollega. Istället för att kollegan behöver börja om från början finns en tydlig historik över vilka hypoteser (hypotheses) som redan har prövats och förkastats.



Förtydligande (sammanfattning):

- **Arbetsflöde:** Processen följer stegen Omfattning (Scope) → Hypotes → Test → Resultat → Nästa steg.

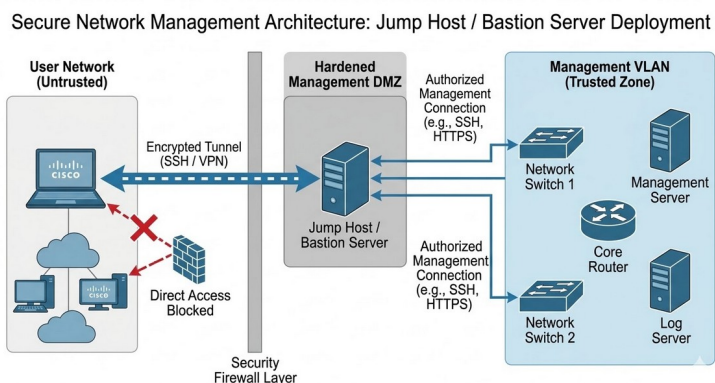
- **Isolering:** För att hitta källan isoleras felet genom att byta ut en variabel i taget, såsom kabel, port eller VLAN-tillhörighet.
- **Bevisning:** Spara alltid kommandon och den utdata (output) som genererades direkt i ärendet för framtida referens och spårbarhet.

Exempel: En trådlös skanner på ett lager tappar kontakten i en specifik zon. Mätningar visar att en nyinstallerad metallvägg skärmar signalen från närmaste accesspunkt. Genom att flytta accesspunkten två meter återfås fri sikt och anslutningen stabiliseras, vilket verifieras med nya mätvärden under ett arbetspass.

3.3.7 Säkerhetsetikett i vardagen

I delade nätverksmiljöer, som exempelvis på en skola, är den främsta regeln för en administratör att aldrig orsaka skada på den befintliga infrastrukturen. Det innebär att man aldrig startar egna adresstjänster som DHCP eller DNS på nätverk där vanliga klienter befinner sig, eftersom detta kan leda till att enheter får felaktig nätverksinformation och tappar sin anslutning. Man bör också undvika att skanna nätverket utan en specifik anledning och aldrig köra osignerade eller okända verktyg på produktionsservrar (production servers). När det krävs paketfångst (packet capture) för felsökning måste man vara medveten om att informationen kan innehålla känsliga personuppgifter. Etiketten föreskriver här att man minimerar fångsten till det absolut nödvändiga, använder snäva filter (filters) och raderar filerna så snart analysen är klar.

För att skydda nätverkets hjärta bör administratörsgränssnitt aldrig nås direkt från ett vanligt klientnät. Istället används ett separat, isolerat nätverk för hantering (management VLAN). Åtkomsten till detta nät sker via en så kallad **jump host** eller **bastion**, vilket är en särskilt härdad



(hardened) mellanserver. En tekniker loggar först in på denna server, ofta med multifaktorautentisering (MFA), och kan därifrån nå utrustningens management-IP. Denna arkitektur säkerställer att ingen direktadministration sker från osäkra användarnät, vilket dramatiskt minskar risken för intrång. En viktig del av säkerhetsetiketten är också att alltid lämna spår efter sig; genom att dokumentera vad som gjorts och varför, kan nästa person som loggar in förstå förändringen och nätverkets aktuella status.

Förtydligande (sammanfattning): Att driva nätverk handlar om att eliminera risker genom att förbjuda otillåtna tjänster (rogue services) på elev- och personalnät. Vid paketfångst gäller principen om minsta möjliga data, säker lagring och snabb radering. All administrativ åtkomst bör ske utifrån principen om minsta behörighet (least privilege) via säkra hopp-servrar.

Exempel: Vid upptäckt av ett okänt DHCP-scope används **DHCP Snooping** för att spåra källan till en elevs privata router i ett vägguttag. Porten stängs omedelbart för att stoppa nätverkskonflikten och händelsen loggas i driftjournalen. Detta visar vikten av att proaktivt förhindra otillåtna tjänster (rogue services) i delade miljöer.

3.3.8 Mallar och checklistor

Standardisering är nyckeln till en stabil och förutsägbar driftsmiljö. Genom att använda mallar (templates) för återkommande arbetsuppgifter, såsom skapandet av nya virtuella maskiner (VM) eller konfiguration av trådlösa nätverk (SSID), minimeras risken för manuella konfigurationsfel. Dessa mallar bör förvaras på en central plats, exempelvis i en Wiki eller ett Git-arkiv, tillsammans med tillhörande checklistor. En checklista fungerar som en validering av arbetet och säkerställer att varje steg når upp till nätverkets fastställda kvalitetskrav, även kallat "Definition of Done". Genom att versionsmärka dessa dokument med datum och ansvarig ägare skapas en tydlig historik som gör det enkelt att se hur standarden har utvecklats över tid och underlättar framtida revisioner.

Exempel på konfigurationsmall för nytt VLAN (Cisco-syntax)

```
vlan [ID]
name [NAMN_AVD]
```

```
interface Vlan [ID]
  description Gateway_for_[NAMN_AVD]
  ip address [IP_ADRESS] [MASK]
  no shutdown
```

Exempel på checklista: Driftsättning av ny access-switch

1. **Grund:** Ange unikt namn och management-IP.
2. **System:** Konfigurera NTP, Syslog och aktivera SSH/AAA för säker inloggning.
3. **Lager 2:** Ställ in STP-prioritet, definiera trunk-VLAN och tillämpa portprofiler.
4. **Slutför:** Märk utrustningen fysiskt, uppdatera portmatrisen och validera anslutningen med en testklient.

Förtydligande (sammanfattning): Effektiv administration bygger på att ha en enda källa för sanningen där både mallar och checklistor förvaras. Varje dokument ska vara tydligt versionsmärkt och ha en utpekad ägare. Genom att definiera exakt vad som ska vara uppfyllt och mätbart vid arbetets slut skapas en jämn kvalitet som är lätt att granska i efterhand.

Exempel: Vid installation av en ny switch i ett access-lager används checklistan för att säkerställa att både NTP-synkronisering och STP-prioritet blir rätt från början. Detta förhindrar att en enskild switch orsakar nätverksloopar eller loggningsfel, vilket sparar timmar av framtida felsökning.

3.3.9 Mätetal för driftkvalitet

För att nätverksdriften ska gå från reaktiv brandsläckning till proaktiv förvaltning krävs mätbara mål, så kallade **SLO:er** (Service Level Objectives). Genom att välja ut ett fåtal men kritiska mätetal, såsom tillgänglighet (uptime) per zon, fördröjning (latens) mellan olika byggnader eller svarstider för DHCP och DNS, kan administratören få en objektiv bild av nätets hälsa. Istället för att lita på magkänsla eller enskilda användares upplevelser används fakta för att identifiera flaskhalsar. Att visualisera dessa mätetal i en **dashboard** med tydliga färgkoder (grönt, gult, rött) gör det möjligt att upptäcka negativa trender tidigt och sätta in resurser där de gör störst nytta för verksamheten.

Exempel på mätetal för servicenivåmål (SLO)

Mätetal (Metric)	Mål (SLO)	Syfte och betydelse
Tillgänglighet (Uptime)	99,9%	Verifierar nätets stabilitet per zon (t.ex. serverhall eller elevnät).
Latens (Latency)	< 5 ms	Indikerar överbelastning i kärnnätet eller sviktande hårdvara.
DHCP-svarstid	< 200 ms	Tidig varningssignal för serverbelastning eller problem med snooping.
DNS-hit-rate	> 95%	Bekräftar fungerande namnupplösning och korrekta klientinställningar.
Reparationstid (MTTR)	< 4 timmar	Mäter it-avdelningens effektivitet vid högprioriterade incidenter.

Förtydligande (sammanfattning):

- **Få men relevanta tal:** Det är bättre att följa fem mätetal som faktiskt leder till handling än trettio som ignoreras; fokusera på det som påverkar användarna mest.
- **Visualisering:** Använd dashboards för att göra statusen begriplig för både tekniker och ledning, där trender och avvikelser syns direkt.
- **Lärdomar:** Varje allvarlig incident (P1) bör ses som en möjlighet att förbättra nätverkets design, administrativa processer eller dokumentation genom en ordentlig analys.

Exempel: Efter återkommande problem med att klienter inte får IP-adresser införs krav på **DHCP Snooping**-kontroll i alla checklistor. Samtidigt läggs en automatisk mätning av DHCP-svarstider till i nätverkets dashboard, vilket gör att teamet kan upptäcka belastningsproblem innan användarna drabbas.

3.3.10 Ett mini-bibliotek (vad och var)

En av de största utmaningarna vid en nätverksincident är inte nödvändigtvis att lösa det tekniska problemet, utan att snabbt hitta den information som krävs för att kunna fatta rätt beslut. En bra startsida, ett "mini-bibliotek", är därför värd sin vikt i guld. Genom att samla länkar till allt från topologidiagram och IP-planer till kontaktlistor och ändringsloggar på en och samma plats, minskar man startsträckan för nya medarbetare och sparar dyrbara minuter när det brinner i systemen. Denna yta fungerar som nätverkets centralstation och säkerställer att alla i teamet vet exakt var man börjar felsökningen.

Exempel på en central dokumentationssida

Nedan visas hur en enkel men effektiv indexsida kan struktureras för att ge administratören omedelbar tillgång till de viktigaste verktygen och dokumenten.

Dokumenttyp	Innehåll och syfte	Länk / Plats
Topologi	Logiska och fysiska kartor över nätverkets struktur.	Wiki/Nät/Karta
IP- & VLAN-plan	Subnät, gateways, VLAN-ID och DHCP-scopes.	SharePoint/IP-Excel
Portmatris	Koppling mellan vägguttag, patchpanel och switchport.	Git/Hårdvara/Portar
Runbooks	Steg-för-steg för vanliga jobb (t.ex. nytt VLAN).	Wiki/Manualer/Run
Ändringslogg	Historik över genomförda ändringar och incidenter.	System/Ärenden/Log
Kontaktlista	Supportnummer till ISP, elektriker och it-ledning.	Pärm/Kriskontakt

Förtydligande (sammanfattning):

- **En källa:** Använd en wiki eller en enkel webbsida som samlar alla länkar på ett ställe.
- **Versioner:** Säkerställ att dokumenten på startsidan alltid är de senaste versionerna.
- **Tillgänglighet:** Dokumentationens måste gå att nå även om det primära nätverket ligger nere (t.ex. via en offline-kopia eller molntjänst via 4G/5G).

Exempel: När en ny tekniker börjar på it-avdelningen räcker det med en enda länk till startsidan för att få en fullständig överblick. Under ett större strömavbrott kan teamet snabbt hitta kontaktuppgifter till elansvarig och samtidigt se i topologin vilka switchar som saknar reservkraft (UPS).

Kontrollfrågor

1. Varför betraktas dokumentation ofta som nätverkets livförsäkring vid ett oväntat driftstopp?
2. Redogör för skillnaden mellan den fysiska vyn och den logiska vyn i ett nätverksdiagram.
3. Vad innebär begreppet en sanning (single source of truth) i samband med förvaltning av dokumentation?
4. Definiera begreppet runbook och beskriv ett scenario där en sådan är direkt nödvändig.
5. Varför är det ur ett driftsperspektiv bättre att använda en namnstandard som plats-roll-löpnummer än mer beskrivande eller personliga namn?
6. Ge tre exempel på vad som innefattas i god port-etikett i ett korskopplingsrum.
7. Förklara hur man inom ärendehantering väger samman påverkan och brådska för att fastställa en prioritetsgrad.
8. Vilket är det primära syftet med att genomföra en grundorsaksanalys (root cause analysis) efter att en incident har lösts?
9. Varför är det riskabelt att påbörja en ändring i nätverket utan att först ha en fastställd rollback-plan?
10. Vid systematisk felsökning är regeln att bara ändra en variabel i taget. Förklara varför denna princip är så viktig för resultatet.

Fördjupningslänkar

Network World (guide för användbar dokumentation) -

<https://www.networkworld.com/article/712398/network-documentation-best-practices.html>

Cisco (topologidiagram enligt industristandard) -

<https://www.cisco.com/c/en/us/support/docs/ip/routing-information-protocol-rip/13788-3.html>

ITIL Foundation (säker ändringshantering) -

<https://www.itilstudy.com/blog/change-management-til-process/>

Atlassian (incidenthantering och kommunikation) -

<https://www.atlassian.com/incident-management/handbook>

The 5 Whys (metod för grundorsaksanalys) -

<https://www.isixsigma.com/tools-templates/cause-effect/determine-the-root-cause-5-whys/>

Netgate (dokumentation och runbooks för administratörer) -

<https://docs.netgate.com/pfsense/en/latest/solutions/reference/documentation-best-practices.html>

SolarWinds (strategier för logisk namngivning) -

<https://thwack.solarwinds.com/resources/b/library/posts/network-naming-conventions>

RIPE NCC (adressplanering i större nätverk) -

https://www.ripe.net/support/training/material/ipv6-for-lirs-training-course/IPv6_addr_plan_v4.pdf

IBM Documentation (formella ändringsärenden) -

<https://www.ibm.com/docs/en/control-desk/7.6.1?topic=management-change-request-process-flow>

Google SRE (kultur kring blameless postmortems) - <https://sre.google/sre-book/postmortem-culture/>

Egna anteckningar

3.4 Introduktion till CLI (inkl. Cisco-kommandon)

När man rör sig från att vara en vanlig användare till att bli en nätverkstekniker är steget från det grafiska gränssnittet till kommandoraden (command line) ett av de mest avgörande. Medan grafiska verktyg är utmärkta för att få en snabb överblick, är det i kommandoradsgränssnittet (command line interface) som den verkliga kontrollen finns. Att behärska detta gränssnitt handlar inte bara om att lära sig kommandon utantill, utan om att förstå logiken bakom hur nätverksutrustning kommunicerar och konfigureras. I detta kapitel läggs grunden för att navigera, administrera och felsöka nätverksenheter med precision, med ett särskilt fokus på industristandarden från Cisco.

3.4.1 Vad är CLI och varför använda det?

Ett kommandoradsgränssnitt (command line interface) är ett textbaserat sätt att administrera datorer och nätverksutrustning där användaren skriver in specifika instruktioner istället för att interagera med knappar och menyer i ett grafiskt användargränssnitt (graphical user interface). För en nätverkstekniker erbjuder detta gränssnitt en nivå av precision, hastighet och reproducerbarhet som ett grafiskt verktyg sällan kan matcha. Genom att arbeta i textform ser man exakt vilka parametrar som konfigureras och man kan enkelt spara ner konfigurationen som textfiler för säkerhetskopiering eller för att använda som mallar (templates) till andra enheter. Detta möjliggör även automatisering genom skript, vilket gör att man kan utföra samma uppgift på hundratals switchar samtidigt med exakt samma resultat varje gång.

Vid felsökning är kommandoraden nästintill oundgänglig. Den ger tillgång till djupgående detaljer som routingtabeller (routing tables), detaljerade felräknare (error counters) och realtidsdiagnostik (debugging) som ofta är dolda eller förenklade i ett grafiskt gränssnitt. Att lära sig kommandoraden är dessutom en investering i yrkesrollen som sträcker sig bortom enskilda märken; även om kommandona kan skilja sig åt mellan olika tillverkare, är de bakomliggande koncepten och logiken ofta slående lika. Det gör att en tekniker som är bekväm med ett kommandoradsgränssnitt snabbt kan sätta sig in i utrustning från nya leverantörer.

Förtydligande (sammanfattning):

- **Precision och kontroll:** Du ser exakt vad du gör utan att verktyget "gissar" eller döljer inställningar.
- **Automatisering:** Möjligheten att använda skript sparar tid och minskar risken för mänskliga faktorer vid stora utrullningar.
- **Djupgående felsökning:** Ger tillgång till rådata och avancerade mätvärden som krävs för att hitta komplexa fel.

Exempel: En instabil länk visar status "up" i ett grafiskt gränssnitt, men vid kontroll med kommandot `show interfaces` syns en duplex-mismatch via specifika felräknare. Genom att läsa rådatan i textformat kan teknikern identifiera och åtgärda felet på några sekunder.

3.4.2 Åtkomst till nätutrustning: console, SSH och sessionshygien

För att kunna skicka kommandon till en nätverksenhet måste man först etablera en anslutning. Det finns två huvudsakliga vägar: lokal åtkomst via **konsol** (console) och fjärråtkomst (remote access) via nätverket. Konsolanslutningen är teknikerns räddningsplanka och kräver en fysisk seriell kabel som kopplas direkt mellan en dator och enhetens konsolport. Detta är den enda vägen in om nätverket ligger nere eller om enheten är helt nykonfigurerad. För vardaglig administration används istället **SSH** (Secure Shell), vilket är en krypterad fjärranslutning som gör att man kan sitta var som helst och administrera utrustningen säkert. Det äldre protokollet Telnet bör undvikas helt eftersom det skickar all information, inklusive lösenord, okrypterat i klartext.

Branschstandard är **SSH v2**, och för att aktivera det krävs att enheten har ett unikt **hostname**, ett **domännamn** och genererade **krypteringsnycklar**. Vi måste också skapa en lokal administratör och konfigurera enhetens virtuella linjer att endast acceptera krypterad trafik och kräva lokal inloggning. I professionella miljöer används ofta terminalklienter som PuTTY eller Windows Terminal för att hantera dessa sessioner.

ip domain-name [namn.se]: Krävs som en del av identiteten för att kunna generera krypteringsnycklar.

crypto key generate rsa modulus 2048: Skapar den digitala nyckel som krypterar din session.

username [namn] secret [lösenord]: Skapar en lokal användare med ett starkt krypterat (hashed) lösenord.

transport input ssh: En kritisk säkerhetsinställning som tillåter SSH men blockerar osäkra protokoll som Telnet.

login local: Talar om för enheten att den ska använda den lokala användardatabasen för inloggning.

För att hålla en hög säkerhet och bra arbetsergonomi vid kommandoraden tillämpar man vad som kallas **sessionshygien**. Det innebär bland annat att man ställer in en timeout (**exec-timeout**) som automatiskt loggar ut inaktiva användare, samt aktiverar synkroniserad loggning (**logging synchronous**). Det senare är en ovärderlig funktion som förhindrar att systemmeddelanden "skräpar ner" mitt i ett kommando du håller på att skriva, genom att flytta ner din påbörjade textrad under meddelandet. All fjärradministration bör ske från ett dedikerat management-VLAN för att isolera kontrolltrafiken från vanliga användare.

exec-timeout [minuter]: Loggar ut sessionen automatiskt efter en tids inaktivitet för att förhindra obehörig åtkomst.

logging synchronous: Håller din kommandorad ren genom att synkronisera systemutskrifter med din inmatning.

Förtydligande (sammanfattning):

- **Console:** Kräver fysisk närvaro och seriell kabel; fungerar även när nätverket är nere (out-of-band).
- **SSH v2:** Standard för säker fjärradministration; kräver hostname, domännamn och RSA-nycklar.
- **Sessionshygien:** Handlar om att säkra sessioner med timeout och underlätta arbetet med logging synchronous.

Exempel: En felkonfigurerad länk bryter fjärranslutningen. Du ansluter en konsolkabel, loggar in lokalt och återställer SSH-åtkomsten genom att korrigera inställningarna på de virtuella linjerna.

```
line vty 0 4
  transport input ssh
  login local
  exec-timeout 10
  logging synchronous
```

3.4.3 Cisco IOS – lägen och hjälp

Operativsystemet i Ciscos nätverksutrustning, **Cisco IOS** (Internetwork Operating System), är uppbyggt kring en hierarkisk struktur av olika lägen (modes). Varje läge ger tillgång till specifika kommandon och skyddar enheten från oavsiktliga ändringar. När man loggar in hamnar man först i **användarläge** (user EXEC mode), vilket indikeras av prompten >.

Härifrån kan man bara utföra enklare läskommandon. För att kunna se djupare systeminformation eller gå vidare till konfiguration krävs **privilegierat läge** (privileged EXEC mode), vilket nås med kommandot `enable` och kännetecknas av prompten #.

För att faktiskt förändra hur enheten fungerar måste man gå in i **globalt konfigurationsläge** (global configuration mode) via kommandot `configure terminal`, vilket ändrar prompten till (config)#. Inifrån detta läge kan man sedan gå vidare ner i olika **underlägen** (sub-modes) för att konfigurera specifika delar av enheten, exempelvis ett fysiskt gränssnitt, en virtuell länk eller ett VLAN. Denna struktur säkerställer att man som administratör alltid är medveten om vilken del av systemet man för tillfället påverkar.

enable: Tar dig från användarläget till det privilegierade läget för att se detaljerad status.

configure terminal: Öppnar dörren till nätverkets hjärta där alla faktiska förändringar utförs.

interface [namn]: Ett underläge där inställningar görs på en specifik fysisk eller virtuell port.

exit: Backar ett steg i hierarkin, från underläge tillbaka till global konfiguration eller privilegierat läge.

Navigering och effektivitet i kommandotolken bygger på ett inbyggt hjälpsystem som gör att man sällan behöver memorera exakt syntax. Genom att skriva ett frågetecken (?) visas alla tillgängliga kommandon i det nuvarande sammanhanget. Om man påbörjar ett ord och trycker på **tabb-tangenten** (tab) kompletteras kommandot automatiskt, och om man skriver en del av ett ord följt av ett frågetecken visas alla alternativ som matchar den inledningen. En annan central funktion är prefixet **no**, som används för att negra eller ta bort en inställning. Om man behöver kontrollera nätverkets status med ett läskommando medan man befinner sig djupt inne i konfigurationsläget, kan man använda prefixet **do** för att slippa backa ut i strukturen.

?: Din bästa vän i CLI; visar vad du kan skriva härnäst i det läge du befinner dig i.

Tab: Sparar tid och stavfel genom att skriva klart kommandona åt dig.

no: Fungerar som ett suddgummi; skriv det före ett kommando för att ta bort en specifik inställning.

do: Gör det möjligt att köra läskommandon (som show) utan att lämna konfigurationsläget.

Exempel: Vid osäkerhet kring kommandot för att säkra en port skrivs *spanning-tree bpdug?* varpå systemet föreslår *bpduguard*. Detta ger en snabb påminnelse om rätt syntax utan att administratören behöver lämna sitt nuvarande läge.

3.4.4 Visa, spara och jämföra konfiguration

När du arbetar i CLI befinner du dig i en miljö där ändringar sker omedelbart i enhetens arbetsminne (RAM). Denna aktiva konfiguration kallas för **running-config**. Om strömmen bryts innan du har sparat, går alla ändringar förlorade. För att göra ändringarna permanenta måste de skrivas över till enhetens icke-flyktiga minne (NVRAM), vilket kallas

startup-config. Det är också här du kan använda smarta filter för att slippa bläddra igenom hundratals rader text; genom att använda "rörledningar" (pipes) kan du filtrera utmatningen så att du bara ser precis det du letar efter, till exempel inställningarna för ett specifikt gränssnitt.

show running-config: Visar den konfiguration som körs i arbetsminnet just nu.

copy running-config startup-config: Sparar dina ändringar permanent till NVRAM så att de överlever en omstart.

show running-config | section [ord]: Filtrerar utmatningen så att du bara ser de delar som rör ett specifikt ämne (t.ex. ett interface).

reload in [minuter]: En "fallskärm" som schemalägger en omstart. Om du gör en ändring som låser ut dig själv, kommer enheten att starta om och återställa den sparade konfigurationen automatiskt.

Förtydligande (sammanfattning): Att hantera konfiguration handlar om att förstå skillnaden mellan det som körs nu och det som sparas för framtiden. Genom att använda kommandon som `copy run start` säkerställer administratören driftkontinuitet. Innan riskfyllda ändringar görs på distans är `reload in` den viktigaste säkerhetsåtgärden för att undvika att behöva åka fysiskt till platsen vid ett misstag.

Exempel: Innan en ändring görs i en fjärrswitch sätts `reload in 10`. Om kontakten bryts på grund av ett fel rullar enheten tillbaka till den säkra konfigurationen efter tio minuter. Fungerar allt som tänkt avbryts omstarten med `reload cancel`.

3.4.5 "Show" du använder varje dag (snabbdiagnos)

För att snabbt kunna ställa en diagnos på ett nätverk använder en tekniker ett antal standardiserade "show-kommandon". Dessa fungerar som nätverkets röntgenbilder. Det vanligaste felet i access-lagret är fysiska problem eller VLAN-fel, och genom att titta på gränssnittens status kan man direkt se om en port är deaktiverad (err-disabled), har fel hastighet eller om det finns duplex-problem. Ett annat kraftfullt verktyg är

att titta på grannskapet via **CDP** eller **LLDP**; dessa protokoll låter enheter "presentera sig" för varandra, vilket gör att du kan se exakt vilken port på grannswitchen du är kopplad till utan att behöva följa kabeln fysiskt.

show interfaces status: En snabb tabellvy över alla portar, deras VLAN-tillhörighet och om de är anslutna eller ej.

show vlan brief: Visar alla skapade VLAN och vilka portar som är tilldelade till dem.

show mac address-table: En karta som visar vilken fysisk enhet (MAC-adress) som sitter på vilken port.

show cdp neighbors: Visar direktanslutna Cisco-enheter, vilket är outhärligt för att rita upp en topologi.

show ip interface brief: Ger en snabb överblick över alla IP-adresser och om gränssnitten är logiskt "upp".

Förtydligande (sammanfattning): Snabbdiagnos handlar om att verifiera lager 1 och lager 2 så tidigt som möjligt. Genom att kontrollera MAC-tabeller och VLAN-status kan administratören avgöra om ett problem är logiskt eller fysiskt. Att se sina grannar via CDP eller LLDP bekräftar att den fysiska infrastrukturen stämmer överens med dokumentationen.

***Exempel:** En användare i sal 215 når inte nätet. show vlan brief visar att porten ligger i VLAN 1 istället för VLAN 20. Teknikern flyttar porten till rätt VLAN och verifierar med en ny show-kontroll.*

3.4.6 Identitet och grundläggande systeminställningar

Innan du börjar konfigurera tekniska funktioner som VLAN eller routing, måste enheten få en unik identitet. Det absolut första steget är att sätta ett **hostname**. Detta namn syns inte bara i din prompt, utan det är också det namn som loggas i centrala loggservrar (**Syslog**). Utan ett tydligt namn är det omöjligt att veta vilken switch ett larm kommer ifrån. Vi behöver också se till att enhetens klocka går rätt, eftersom korrekta tidsstämplar är kritiska när man korrelerar händelser vid en felsökning. Detta görs bäst genom att peka ut en **NTP-server** som synkroniserar tiden automatiskt över nätverket.

För att höja säkerhetsnivån direkt från start bör alla lösenord som sparas lokalt krypteras i konfigurationsfilen. Dessutom bör administratören sätta upp juridiska varningsmeddelanden, så kallade **banners**, som informerar alla som försöker logga in om att systemet är privat och övervakas. Genom att aktivera detaljerade tidsstämplar i loggarna säkerställer man att varje händelse dokumenteras med millisekunders precision, vilket är ovärderligt vid teknisk analys.

hostname [namn]: Ger enheten ett unikt namn som syns i CLI-prompten och i alla loggmeddelanden.

service password-encryption: Krypterar de lösenord som annars visas i klartext när du granskar konfigurationen.

banner login # [text] #: Skapar ett meddelande som visas före inloggning, ofta använt för juridiska varningar.

service timestamps log datetime msec: Formaterar loggarnas tidsstämplar så att de inkluderar datum, tid och millisekunder.

ntp server [ip-adress]: Talar om för enheten var den ska hämta den korrekta tiden ifrån för att hålla loggarna synkroniserade.

logging host [ip-adress]: Skickar en kopia av alla systemloggar till en central server för långtidslagring.

Förtydligande (sammanfattning): Grundinställningar handlar om att göra enheten hanterbar, säker och spårbar. Genom att sätta en identitet och kryptera lösenord läggs den första nivån av ordning. Korrekta tidsstämplar och central loggning är nätverkets "svarta låda" som gör det möjligt att i efterhand pussla ihop vad som hände under ett avbrott eller ett intrångsförsök.

Exempel: Här konfigureras switchen KLR-ACC-01 med korrekta systeminställningar och loggning mot skolans centralserver.

! Sätt identitet och kryptera lösenord

```
hostname KLR-ACC-01
service password-encryption

! Juridisk varningstext
banner login #
*****
VARNING! Endast behörig personal har tillträde.
All inloggning loggas och övervakas.
*****
#

! Synkronisering och loggning
service timestamps log datetime msec
ntp server 10.0.0.50
logging host 10.0.0.60
```

3.4.7 Att arbeta med gränssnitt (Interface)

För att konfigurera en port på en switch eller router måste du först "gå in" i gränssnittet. Kommandot `interface` flyttar dig från den globala konfigurationen till ett specifikt underläge för just den porten. En viktig detalj i Cisco-världen är att gränssnitt på routrar ofta är avstängda som standard (administratively down), medan de på switchar ofta är påslagna direkt. För att aktivera en port används kommandot `no shutdown`. För att underlätta framtida arbete bör varje gränssnitt också få en beskrivning (**description**) som förklarar vart kabeln leder – en direkt koppling till den dokumentation vi diskuterade i kapitel 3.3.

interface [typ/nummer]: Flyttar dig till konfigurationsläget för en specifik port (t.ex. `interface GigabitEthernet 0/1`).

description [text]: Läger till en kommentar på porten som syns i `show run` och underlättar identifiering.

no shutdown: Aktiverar ett gränssnitt som är avstängt (öppnar porten för trafik).

shutdown: Stänger av en port manuellt, till exempel av säkerhetsskäl om den inte används.

interface range [start-port] - [slut-port]: Gör det möjligt att konfigurera många portar samtidigt med exakt samma inställningar.

Förtydligande (sammanfattning): Gränssnittskonfiguration är där den logiska världen möter den fysiska. Genom att använda beskrivningar skapar man en karta direkt i koden. Om du behöver konfigurera hela klassrummets uttag samtidigt är `interface range` det mest tidseffektiva verktyget i en teknikers verktygslåda.

Exempel: *Administratören ska aktivera portarna 1 till 24 på en ny switch. Istället för att gå in i varje port för sig används `interface range gi1/0/1-24` följt av `no shutdown`, vilket öppnar alla portar på en gång.*

3.4.8 Grundläggande VLAN-konfiguration – Router (Router-on-a-Stick)

När en router ska hantera trafik mellan flera olika virtuella nätverk (VLAN) används ofta metoden **Router-on-a-Stick**. Genom att dela upp ett fysiskt gränssnitt i flera logiska undergränssnitt (sub-interfaces) kan routern agera gateway för många nätverk samtidigt över en enda fysisk kabel. För att routern ska veta vilken trafik som hör till vilket nätverk används **inkapsling** (encapsulation) enligt standarden 802.1Q. Det är kritiskt att det fysiska gränssnittet är aktiverat (`no shutdown`), men själva adresseringen sker på de enskilda undergränssnitten som representerar de olika näten.

hostname [namn]: Identifierar routern unikt, vilket är grundläggande för all professionell administration.

interface [typ/nr].[vlan-id]: Skapar ett undergränssnitt som fungerar som en oberoende logisk port för ett specifikt VLAN.

encapsulation dot1Q [vlan-id]: Aktiverar 802.1Q-tagging så att routern kan läsa och skicka paket med rätt VLAN-id.

ip address [ip] [mask]: Sätter den adress som blir nätverkets standard-gateway (default gateway).

description [text]: Beskriver gränssnittets syfte, till exempel vilket VLAN det hanterar eller vilken byggnad det gäller.

Förtydligande (sammanfattning): Logiken bygger på att det fysiska gränssnittet fungerar som en bärare (trunk) medan undergränssnitten sköter routing mellan VLAN. Utan rätt encapsulation-inställning kommer routern inte att kunna kommunicera med de taggade paketen från switchen, och utan en tydlig description blir felsökning onödigt svår.

Exempel: Här konfigureras routern R-GW-CORE för att hantera routing mellan VLAN 10 och VLAN 20. Vi aktiverar det fysiska gränssnittet och skapar därefter två undergränssnitt med korrekta beskrivningar och adresser.

```
! Sätt enhetens namn
hostname R-GW-CORE

! Aktivera den fysiska porten mot switchen
interface gigabitEthernet 0/0
  description TRUNK-TO-SW-ACC-SAL215
  no shutdown

! Skapa undergränssnitt för VLAN 10 (Personal)
interface gigabitEthernet 0/0.10
  description GATEWAY-VLAN10-STAFF
  encapsulation dot1Q 10
  ip address 10.10.0.1 255.255.255.0

! Skapa undergränssnitt för VLAN 20 (Elever)
interface gigabitEthernet 0/0.20
  description GATEWAY-VLAN20-ELEV
  encapsulation dot1Q 20
  ip address 10.20.0.1 255.255.255.0
```

Grundläggande VLAN-konfiguration - Switch

I en switch delas portarna in i två typer: **access-portar** för slutenheter och **trunk-portar** för förbindelser mellan nätverksenheter. En access-port tillhör bara ett VLAN och skickar trafiken "otaggad" till datorn. En trunk-port kan däremot bära många VLAN samtidigt genom att lägga till en liten etikett (tag) på varje paket så att mottagaren vet var det hör hemma. För att säkra access-portar används ofta **Portfast** (för snabb anslutning) och

BPDU Guard (för att förhindra att obehöriga switchar kopplas in och skapar loopar).

hostname [namn]: Ger switchen en unik identitet i nätverket, vilket är avgörande för loggning och fjärradministration.

vlan [id] / name [namn]: Skapar det virtuella nätverket i switchens databas. Utan detta steg kan inga portar tilldelas nätverket.

switchport mode access / switchport access vlan [id]: Konfigurerar en port för en slutenhet och placerar den i rätt VLAN.

switchport mode trunk: Öppnar porten för att bära trafik från alla tillgängliga VLAN, oftast mot en router eller en annan switch.

description [text]: Nätverkets interna karta; beskriver exakt vart kabeln leder eller vilken funktion porten har.

Förtydligande (sammanfattning): Switchkonfiguration handlar om att styra trafikflödet och säkra gränserna. Access-portar är nätverkets slutstationer, medan trunk-portar är motorvägarna som binder ihop infrastrukturen. Genom att namnge VLAN och begränsa trunkar skapas ett nätverk som är både lättöverskådligt och säkert.

Exempel: En elevport konfigureras som access i VLAN 20 med *bpduguard enable*. Om eleven ansluter en egen switch som skickar kontrollpaket (BPDUs) stängs porten omedelbart ner (*err-disable*) för att skydda resten av skolan.

```
! Sätt enhetens namn
hostname SW-ACC-SAL215
```

```
! Skapa VLAN i databasen
vlan 20
  name ELEV
```

```
! Konfigurera upplänk (Trunk) mot routern
interface gigabitEthernet 1/0/48
  description UPLINK-TO-ROUTER-G0/0
  switchport mode trunk
```

```
! Konfigurera användarport (Access)  
interface gigabitEthernet 1/0/12  
  description ELEV-PORT-SAL215  
  switchport mode access  
  switchport access vlan 20  
  spanning-tree portfast  
  spanning-tree bpduguard enable
```

3.4.9 Grundläggande lager-3-konfiguration (SVI och routing)

*En Layer 3-switch kombinerar snabbheten hos en switch med intelligensen hos en router. Genom att skapa ett virtuellt gränssnitt för ett VLAN, en så kallad **SVI** (Switch Virtual Interface), kan switchen fungera som en standard-gateway (default gateway) för alla enheter i det nätverket. För att switchen faktiskt ska börja skicka trafik mellan de olika virtuella näten krävs kommandot `ip routing`. Utöver detta kan man även konfigurera statisk routing för fasta vägar eller använda dynamiska protokoll som **OSPF** (Open Shortest Path First) för att nätverket automatiskt ska lära sig de bästa vägarna i större miljöer.*

interface vlan [id]: Skapar ett logiskt gränssnitt för ett specifikt VLAN där en IP-adress kan tilldelas.

ip routing: Aktiverar enhetens förmåga att dirigera trafik mellan olika nätverk (VLAN).

ip address [ip] [mask]: Tilldelar gränssnittet en adress som fungerar som utgångspunkt för nätverket.

ip route [nät] [mask] [nästa-hopp]: Skapar en statisk väg i nätverket, ofta använd för en förvald väg (default route) mot internet.

router ospf [process-id]: Startar ett dynamiskt routingprotokoll som låter enheter dela information om sina nätverk automatiskt.

network [nät] [wildcard-mask] area [id]: Definierar vilka lokala nätverk som ska annonseras ut till andra routrar via OSPF.

Förtydligande (sammanfattning): Lager 3-konfiguration handlar om att ge switchen en logisk närvaro i varje VLAN. Genom att tilldela IP-adresser till SVI:er och aktivera routingfunktionen kan enheten hantera trafik mellan nätverk internt utan att behöva skicka upp den till en extern router. Statisk routing och OSPF används sedan för att tala om för switchen vart trafik som ska lämna det lokala nätet ska skickas.

Exempel: Här konfigureras lager 3-switchen SW-CORE-01 för att agera gateway för VLAN 20 och 40. Vi aktiverar routing och lägger även till en statisk väg mot nätverkets internetbrandvägg.

```
! Identitet och aktivering av routing
hostname SW-CORE-01
ip routing

! Gateway för elevnätet (VLAN 20)
interface vlan 20
  description GATEWAY-ELEV-VLAN20
  ip address 10.20.0.1 255.255.252.0
  no shutdown

! Gateway för servernätet (VLAN 40)
interface vlan 40
  description GATEWAY-SERVER-VLAN40
  ip address 10.40.0.1 255.255.255.0
  no shutdown

! Statisk väg mot brandväggen (Default Route)
ip route 0.0.0.0 0.0.0.0 10.0.0.1
```

3.4.10 Effektivitet och vanliga fallgropar i CLI

Att arbeta i kommandotolken handlar inte bara om att kunna kommandona, utan om att utveckla vanor som gör arbetet både snabbare och säkrare. En av de mest grundläggande effektivitetshöjarna är att behärska navigeringsgenvägar; istället för att använda piltangenterna för att bläddra tecken för tecken kan man använda kortkommandon för att hoppa direkt till början eller slutet av en rad. När man arbetar med stora mängder data är rörledningar (pipes) och filtrering oumbärliga verktyg för att isolera precis den information man söker i en lång konfiguration. För

att ytterligare spara tid kan man använda gränssnittsintervall (interface range) för att konfigurera dussintals portar samtidigt, samt skapa egna förkortningar, så kallade alias, för de kommandon man använder oftast. En annan viktig vana är att använda prefixet **do** för att kunna köra läskommandon utan att behöva backa ur konfigurationsläget.

Samtidigt som man ökar takten är det lätt att falla i vanliga fällor som kan orsaka stora problem. En av de mest frustrerande upplevelserna för en tekniker är när enheten försöker slå upp (resolve) ett felskrivet kommando via DNS, vilket låser terminalen i 30 sekunder. Detta undviks helt genom att inaktivera domänuppslagningar. En annan klassisk fallgrop är att glömma att spara ändringarna till det permanenta minnet; ett nätverk som fungerar perfekt ända fram till nästa strömavbrott beror oftast på att man missat att skriva över den aktiva konfigurationen till startkonfigurationen. Det är också vanligt med logiska fel, som att en klient hamnar i fel VLAN trots att länken ser ut att vara aktiv, eller att en trunk-länk saknar tillåtelse för ett specifikt VLAN. För att snabbt verifiera vilken hårdvara och mjukvara man faktiskt arbetar med, samt se hur länge enheten varit igång (uptime), används ett specifikt systemkommando som ger en fullständig översikt.

ctrl-a / ctrl-e: Flyttar markören omedelbart till början respektive slutet av den aktuella raden.

show run | section [ord]: Filtreerar konfigurationen så att du bara ser den del som är relevant för tillfället.

interface range [start] - [slut]: Utför konfiguration på en hel grupp av portar samtidigt.

alias exec [kortnamn] [långt-kommando]: Skapar en personlig förkortning för ett återkommande kommando.

no ip domain-lookup: Förhindrar att terminalen fryser vid stavfel genom att stänga av DNS-sökningar på felskrivna ord.

show version: Visar enhetens hårdvaruinformation, serienummer, mjukvaruversion (firmware) och drifttid.

Förtydligande (sammanfattning): Effektivitet i CLI uppnås genom att minimera antalet knapptryckningar och filtrera bort onödigt brus i utmatningen. Genom att använda masskonfigurationer och genvägar minskar risken för mänskliga faktorer. För att undvika de vanligaste fallgroparna bör man alltid inaktivera DNS-uppslagningar i labbmiljöer, regelbundet kontrollera drifttiden via `show version` för att upptäcka oväntade omstarter, och framför allt införa en rutin att alltid spara och verifiera sina ändringar innan arbetet avslutas.

Exempel: Här används `interface range` för att snabbt säkra 24 portar, samtidigt som vi skapar ett alias för att snabbare kunna se nätverkets status och verifierar enhetens drifttid.

```
! Konfigurera 24 portar på 10 sekunder
interface range gigabitEthernet 1/0/1-24
  spanning-tree portfast
  spanning-tree bpduguard enable
```

```
! Skapa ett alias för att snabbt se VLAN-status
alias exec sv show vlan brief
```

```
! Verifiera hur länge enheten varit igång efter en ändring
show version | include uptime
```

Kontrollfrågor

1. Varför anses kommandoraden (CLI) vara mer effektiv än ett grafiskt gränssnitt (GUI) vid storskalig administration?
2. Förklara skillnaden mellan User EXEC mode och Privileged EXEC mode. Hur ser man på prompten vilket läge man befinner sig i?
3. Vad är den tekniska skillnaden mellan running-config och startup-config, och vad händer om man glömmer att synkronisera dem?
4. Varför är kommandot `no ip domain-lookup` så viktigt att konfigurera i en labbmiljö?
5. Beskriv funktionen av logging synchronous. Hur underlättar det arbetet för teknikern?
6. Förklara begreppet Router-on-a-Stick. Vilket kommando är absolut nödvändigt på routerns undergränssnitt för att trafiken ska kunna sorteras rätt?
7. Vad är skillnaden mellan en access-port och en trunk-port?
8. I vilken situation använder man en SVI (Switch Virtual Interface) istället för ett fysiskt gränssnitt på en router?
9. Vilken funktion fyller BPDU Guard och varför ska det aktiveras på portar där datorer är anslutna?
10. Vad visar kommandot `show version` och när har man nytta av den informationen?

Fördjupningslänkar

Cisco (officiell guide för grundläggande CLI-navigering) -
<https://learningnetwork.cisco.com/s/article/cisco-ios-cli-navigation-and-editing-shortcuts>

Packet Tracer Tutorials (praktiska övningar för CLI-kommandon) -
<https://www.netacad.com/courses/packet-tracer>

NetworkLessons (tydlig genomgång av Router-on-a-Stick) -
<https://networklessons.com/cisco/ccna-routing-switching-icnd1-100-105/how-to-configure-router-on-a-stick>

Study CCNA (detaljerad förklaring av VLAN och Trunking) -
<https://www.studyccna.com/vlan-trunking-protocol-vtp/>

Cisco Press (guide för säkring av management-åtkomst) -
<https://www.ciscopress.com/articles/article.asp?p=2164571>

Router Alley (snabbguide för OSPF-konfiguration) -
<http://www.routeralley.com/guides/ospf.pdf>

Flackbox (lista över de 20 vanligaste Cisco-kommandona) -
<https://www.flackbox.com/cisco-ios-show-commands-cheat-sheet>

Practical Networking (visuell guide till Layer 3 Switching) -
<https://www.practicalnetworking.net/layered-model/header-delivery-layer-3-switch/>

Cisco Community (best practice för hostname och namngivning) -
<https://community.cisco.com/t5/networking-knowledge-base/standard-device-naming-convention/ta-p/3111001>

O'Reilly (djupdykning i SSH v2 konfiguration på IOS) -
<https://www.oreilly.com/library/view/cisco-ios-cookbook/0596527225/ch02s05.html>

Egna anteckningar

...

3.5 Windows Server – Domäner och AD

I dagens moderna nätverksinfrastruktur är det sällan en fråga om att välja antingen Linux eller Windows; de flesta professionella miljöer bygger på en hybrid av båda världarna där de olika operativsystemen får glänsa inom sina respektive specialområden. Medan Linux ofta dominerar när det kommer till tunga nätverkstjänster, webbservrar och databaser, har Windows Server cementerat sin roll som det centrala navet för användarhantering och identitet. Tack vare en hög grad av integration och verktyg som är enkla att koppla samman med allt från nätverksutrustning till molntjänster, fungerar Windows Server som den gemensamma nämnaren som binder ihop organisationens digitala resurser. Det är här administratören skapar en centraliserad ordning för vem som har tillgång till vad, vilket gör det möjligt att styra tusentals enheter från en enda plats.

Kärnan i denna struktur är Active Directory, en katalogtjänst (directory service) som fungerar som nätverkets hjärna och logiska telefonbok. För en nätverkstekniker är förståelsen för domäner och katalogtjänster avgörande, då det är här nätverkets säkerhetsgränser och regler definieras. Medan andra delar av utbildningen, exempelvis i ämnet datorteknik, fokuserar mer på den praktiska installationen och handhavandet av operativsystemet, ligger vårt fokus här på hur Windows Server fungerar som en kritisk infrastrukturkomponent. Genom att förstå samspelet mellan servrar, identiteter och nätverksprotokoll kan vi bygga miljöer som är både robusta och skalbara.

3.5.1 Överblick: vad är en domän och Active Directory (AD DS)?

En domän kan liknas vid en administrativ och säkerhetsmässig överenskommelse om identiteter och regler inom ett nätverk. Det handlar om att fastställa vem du är, vad du har rätt att göra (authorization) och hur detta bevisas på ett sätt som alla anslutna datorer i domänen litar på. Active Directory Domain Services (AD DS) är själva tekniken som håller ihop detta – en databas som lagrar information om användarkonton, grupper, datorer och skrivare samt relationerna mellan dem. När en användare loggar in på en domänansluten dator sker en serie händelser i bakgrunden: klienten hittar först en domänkontrollant (domain controller)

via nätverkets DNS, bevisar sin identitet genom protokoll som Kerberos, och hämtar slutligen de regler och resurser som är kopplade till just den identiteten.

I mer komplexa nätverksmiljöer kan katalogtjänsten delas in i flera domäner som samlas under en gemensam skog (forest). Skogen fungerar som den yttersta säkerhetsgränsen och delar ett gemensamt språk – ett schema – som gör att alla domäner i skogen förstår samma typer av objekt och regler. För en skola innebär detta i praktiken att en elevs inloggning och skrivbordsmiljö kan vara identisk oavsett vilken sal eller byggnad hen befinner sig i. Samtidigt kan administrationen delegeras så att lokala tekniker kan hantera sina egna användare och resurser utan att behöva få fullständiga administrativa rättigheter över hela nätverkets hjärta.

Förtydligande (sammanfattning):

- **Domän:** En gemensam säkerhetsgräns där samma regler och identiteter gäller för alla medlemmar.
- **Skog:** Den högsta nivån i AD-hierarkin som kan innehålla flera domäner som litar på varandra.
- **Domänkontrollant (DC):** Den server som ansvarar för att lagra katalogen och verifiera användarnas inloggningar.

Exempel: En elev får tillgång till samma personliga filer och samma skrivare oavsett vilken dator i nätverket hen sätter sig vid. Det är inte slumpen som avgör detta, utan domänens förmåga att känna igen elevens identitet och omedelbart applicera de förinställda reglerna (policies) för just det kontot.

För en djupdykning i hur du praktiskt installerar Windows Server och sätter upp rollen för Active Directory, se kapitel 4 i Datorteknik-boken.

3.5.2 Infrastruktur: Domänkontrollanter, FSMO-roller och platser (Sites)

En stabil domänmiljö bygger på **redundans** (redundancy). För att förstå infrastrukturen måste man först skilja på den fysiska utrustningen och de logiska uppgifterna. En **domänkontrollant** (domain controller, DC) är inte en person eller ett konto, utan en **fysisk eller virtuell server** – en

dator som kör operativsystemet Windows Server och har fått rollen att hantera nätverkets katalogtjänst. På samma sätt som en stad behöver fler än ett sjukhus för att säkerställa vård vid en kris, behöver en domän minst två sådana servrar. Eftersom Active Directory är en databas där ändringar kan ske på vilken server som helst – en så kallad multi-master-databas – måste dessa datorer ständigt kommunicera med varandra genom en process som kallas **replikering** (replication). Om en server går ner för underhåll, finns den andra kvar för att logga in användare och kontrollera rättigheter.

Även om de flesta uppgifter delas mellan alla servrar, finns det fem specifika funktioner som är så känsliga att de bara får utföras av en server i taget för att undvika logiska konflikter. Dessa kallas för **FSMO-roller** (Flexible Single Master Operation). Man kan likna dem vid "specialist-hattar" som servrarna bär. Den mest kritiska i vardagen är **PDC-emulatorn** (PDC Emulator), vilken fungerar som nätverkets högsta tidsreferens. I en miljö som bygger på autentiseringsprotokollet **Kerberos** är exakt tid en kritisk säkerhetsparameter; om klockan på en klient diffar mer än fem minuter från servern kommer inloggningsbiljetterna (tickets) att ogiltigförklaras och användaren nekas tillträde.

De fem **FSMO-rollerna** (Flexible Single Master Operation) är uppdelade i två kategorier: de som bara får finnas en enda gång i hela **skogen** (forest-wide) och de som finns en gång i varje enskild **domän** (domain-wide). Att förstå dessa är som att förstå rollfördelningen i ett kök; alla kan laga mat, men bara en person ansvarar för inköpslistan och en annan för klockan på väggen.

Skogsövergripande roller (Forest-wide)

Dessa två roller är unika för hela din Active Directory-skog. Det spelar ingen roll om skolan har tio olika underdomäner – det finns fortfarande bara en av varje av dessa i hela infrastrukturen.

Schema Master (Schemaansvarig) fungerar som väktaren av nätverkets DNA. Schemat definierar vilka typer av objekt som får finnas i katalogen (exempelvis vad som utgör en "användare" eller en "dator") och vilka attribut de kan ha. Om man installerar en ny tjänst som behöver lägga till information i katalogen, till exempel Microsoft Exchange, måste Schema

Master vara tillgänglig för att godkänna och utföra dessa strukturella ändringar.

Domain Naming Master (Domännamnsansvarig) har till uppgift att hålla ordning på domänernas namn och existens. Denna server är den enda som får lägga till nya domäner i skogen eller ta bort befintliga. Detta förhindrar att två administratörer råkar skapa domäner med samma namn på olika ställen i nätverket samtidigt, vilket skulle skapa ett logiskt kaos.

Domänspecifika roller (Domain-wide)

Dessa tre roller finns lokalt i varje domän. Om skolan har en huvuddomän och en separat domän för en labbmiljö, kommer båda dessa domäner att ha sina egna innehavare av dessa roller.

PDC Emulator (PDC-emulator) är den mest aktiva rollen i vardagen. Den ansvarar för tidssynkronisering (time synchronization) i hela domänen, vilket vi tidigare nämnt är avgörande för att inloggningen ska fungera. Dessutom är det PDC-emulatorn som har sista ordet vid lösenordsändringar. Om en användare skriver in fel lösenord kontrollerar den lokala servern en extra gång mot PDC-emulatorn för att se om lösenordet nyligen har bytts någon annanstans i nätverket innan inloggningen nekas. Den fungerar också som den primära servern för att redigera gruppprinciper (**GPO**).

RID Master (RID-ansvarig) fungerar som en utdelare av serienummer. Varje objekt i en domän (användare, grupp eller dator) får ett unikt ID som kallas **SID** (Security Identifier). För att säkerställa att två servrar inte råkar ge samma ID till två olika användare, delar RID Master ut "block" av nummer (Relative Identifiers) till alla domänkontrollanter. När en server har slut på sina tilldelade nummer ber den RID Master om ett nytt block.

Infrastructure Master (Infrastrukturansvarig) är nätverkets länkbyggare. Dess huvuduppgift är att hålla koll på referenser mellan olika domäner. Om en användare i Domän A läggs till i en grupp i Domän B, ser Infrastructure Master till att gruppmedlemskapet uppdateras

korrekt och att namnen stämmer även om användaren skulle byta namn i sin hemdomän.

Roll (Svenska/Engelska)	Omfattning	Huvuduppgift
Schemaansvarig (Schema Master)	Skog	Kontrollerar katalogens struktur och DNA.
Domännamnsansvarig (Domain Naming Master)	Skog	Hanterar tillägg och borttagning av domäner.
PDC-emulator (PDC Emulator)	Domän	Ansvarar för tid, lösenord och GPO-redigering.
RID-ansvarig (RID Master)	Domän	Delar ut unika ID-nummer för nya objekt.
Infrastrukturansvarig (Infrastructure Master)	Domän	Uppdaterar referenser mellan olika domäner.

När en administratör sätter upp den allra första servern i en ny skog får den servern automatiskt alla fem rollerna. I takt med att nätverket växer flyttas ofta dessa roller ut på flera olika servrar för att sprida belastningen och öka driftsäkerheten.

Exempel på ett enkelt FSMO-nätverk: Tänk dig en skola med två servrar: **SRV-DC01** och **SRV-DC02**. När domänen skapas hamnar alla fem FSMO-roller automatiskt på den första servern (SRV-DC01). Den bär då "hattarna" för att bestämma över schemat, domännamnen och nätverkets tid. Om skolan växer och belastningen ökar kan administratören välja att flytta några av dessa roller till SRV-DC02. Om SRV-DC01 skulle gå sönder permanent, måste en tekniker manuellt tvinga över (seize) rollerna till SRV-DC02 för att nätverket ska förbli fullt fungerande. Det är alltså ett samspel mellan maskiner som ständigt pratar med varandra för att hålla ordning på vem som ansvarar för vad.

När nätverket sträcker sig över flera fysiska platser, till exempel olika skolbyggnader, används **Sites and Services** för att förklara geografin för Active Directory. Genom att koppla specifika **subnät** (subnets) till olika platser (sites) kan systemet styra trafiken intelligent så att en klient i Hus A pratar med servern i samma hus istället för att belasta länken till Hus B. På platser där den fysiska säkerheten är låg kan man installera en **RODC** (Read-Only Domain Controller). Detta är en fysisk server som innehåller en kopia av katalogen som inte går att förändra lokalt. Om någon skulle stjäla en RODC kan de inte använda den för att ändra lösenord eller skapa nya konton i resten av nätverket, vilket gör den till en säker "utpost" i nätverksinfrastrukturen.

Förtydligande (sammanfattning):

Domänkontrollant (DC): En fysisk eller virtuell server (dator) som kör katalogtjänsten.

Redundans: Kravet på minst två servrar för att nätverket ska fungera även om en maskin går sönder.

FSMO-roller: Fem specifika administrativa uppgifter som fördelas mellan servrarna för att undvika konflikter.

Tidssynk: En kedja där PDC-emulatorn sätter tiden för att Kerberos-inloggningar ska fungera.

Sites: Kopplar samman nätverksadresser (subnät) med fysiska platser för att optimera inloggningstrafik.

Exempel: *En elev byter sitt lösenord på morgonen. SRV-DC01 tar emot ändringen och skvallrar omedelbart för SRV-DC02 via replikering. Tack vare att båda servrarna är synkroniserade mot samma tidskälla (PDC-emulatorn) gäller det nya lösenordet i hela skolan inom några minuter, oavsett vilken server eleven råkar logga in mot.*

För en praktisk genomgång av hur du installerar en sekundär domänkontrollant och kontrollerar replikeringsstatus, se kapitel 5 i Datorteknik-boken.

3.5.3 DNS och DHCP i domänmiljö – Domänens nervsystem och logistik

Om Active Directory är nätverkets hjärna, så är **DNS** (Domain Name System) dess nervsystem. Utan en fungerande DNS-tjänst kan Active Directory inte existera, eftersom tekniken förlitar sig på DNS för att hitta resurser. När en klientdator ska logga in i domänen räcker det inte med att den har en nätverksanslutning; den måste kunna fråga DNS-servern efter specifika **SRV-poster** (service records). Dessa poster fungerar som vägvisare som talar om för klienten exakt vilka servrar i nätverket som erbjuder inloggningstjänster (LDAP och Kerberos). Om en klient pekar på fel DNS-server – till exempel en publik server på internet – kommer den aldrig att hitta skolans domänkontrollanter, och användaren får felmeddelandet att domänen inte kan hittas.

För att underlätta administrationen används ofta **AD-integrerade zoner** (AD-integrated zones). Det innebär att DNS-databasen lagras direkt inuti Active Directory och kopieras (replikeras) automatiskt mellan alla domänkontrollanter. Detta ger två stora fördelar: för det första skapas en naturlig redundans då alla domänkontrollanter också blir DNS-servrar, och för det andra möjliggör det **säkra dynamiska uppdateringar** (secure dynamic updates). Det betyder att när en dator får en ny IP-adress kan den själv legitimera sig mot domänen och uppdatera sitt eget namn i DNS-listan utan att en tekniker behöver göra det manuellt.

DHCP (Dynamic Host Configuration Protocol) fungerar i sin tur som nätverkets logistikcenter som delar ut IP-adresser, men i en domänmiljö finns ett extra lager av säkerhet och integration. För att en Windows-baserad DHCP-server ska få dela ut adresser i en domän måste den först genomgå en **auktorisering** (authorization) i Active Directory. Detta är en skyddsmekanism som hindrar obehöriga servrar (rogue DHCP servers) från att störa nätverket genom att dela ut felaktig information. I moderna miljöer konfigureras DHCP ofta med **redundans** (DHCP Failover), där två servrar samarbetar för att dela ut adresser. Om den ena servern slutar svara tar den andra över omedelbart så att inga avbrott uppstår i klassrummen.

Samspelet mellan DHCP och DNS är det som gör att nätverket känns "levande". När en elevdator får en adress via DHCP kan DHCP-servern konfigureras att automatiskt tala om för DNS-servern att "nu bor dator X

på adress Y". Detta säkerställer att tekniker alltid kan nå enheter via deras namn, även om deras IP-adresser ändras ofta, vilket är vanligt i skolmiljöer där många datorer kommer och går.

Förtydligande (sammanfattning):

SRV-poster: DNS-pekare som talar om för klienter var nätverkets tjänster (som inloggning) finns.

AD-integrerad zon: En DNS-zon som är inbyggd i domänen för ökad säkerhet och automatisk kopiering mellan servrar.

Auktorisering: En säkerhetsspärr som kräver att domänen godkänner en DHCP-server innan den får dela ut adresser.

DHCP Failover: En teknik där två servrar delar på ansvaret för adressutdelning för att undvika driftstopp.

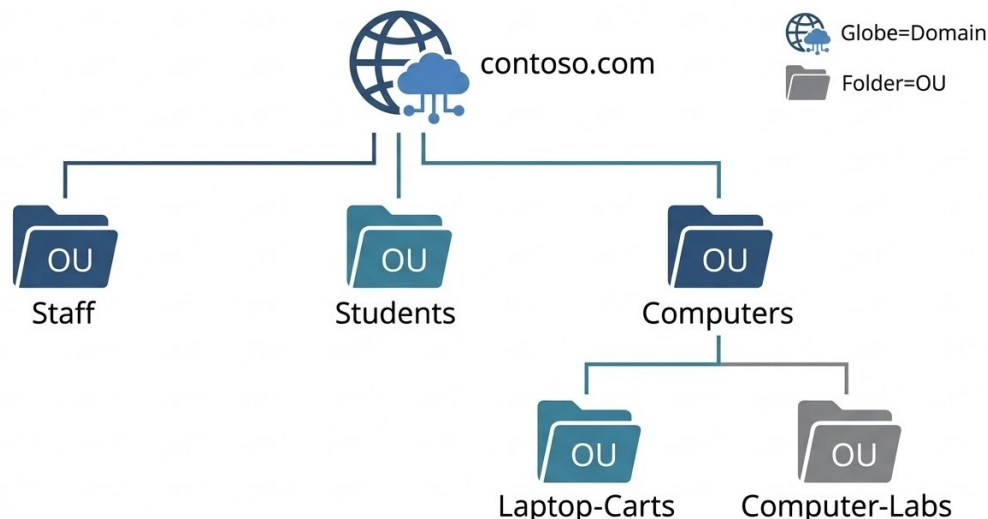
Exempel: En bärbar dator ansluts till skolans nätverk. DHCP-servern ger datorn en IP-adress och uppdaterar samtidigt DNS-servern med datorns namn. När användaren trycker på logga in, frågar datorn DNS efter nätverkets SRV-poster, hittar närmaste domänkontrollant och genomför inloggningen på några sekunder. Allt detta sker tack vare att DNS och DHCP arbetar i symbios med Active Directory.

För en praktisk guide om hur du skapar scopes i DHCP och sätter upp AD-integrerade zoner i DNS-managern, se kapitel 6 i Datorteknik-boken.

3.5.4 Logisk struktur: OU-design, delegation och GPO

När den fysiska infrastrukturen med domänkontrollanter och nätverkstjänster är på plats, behöver administratören skapa en logisk ordning inuti Active Directory. Detta görs genom **organisatoriska enheter** (organizational units, OU), som fungerar som behållare för användare, datorer och grupper. En välplanerad OU-struktur är nätverkets ryggrad för administration; den speglar inte bara hur skolan är organiserad på pappret, utan framför allt hur den drivs tekniskt. Genom att flytta objekt från de osorterade standardbehållarna (standard containers) till specifika OU:er kan man styra exakt vilka inställningar som ska gälla för olika grupper av enheter och människor.

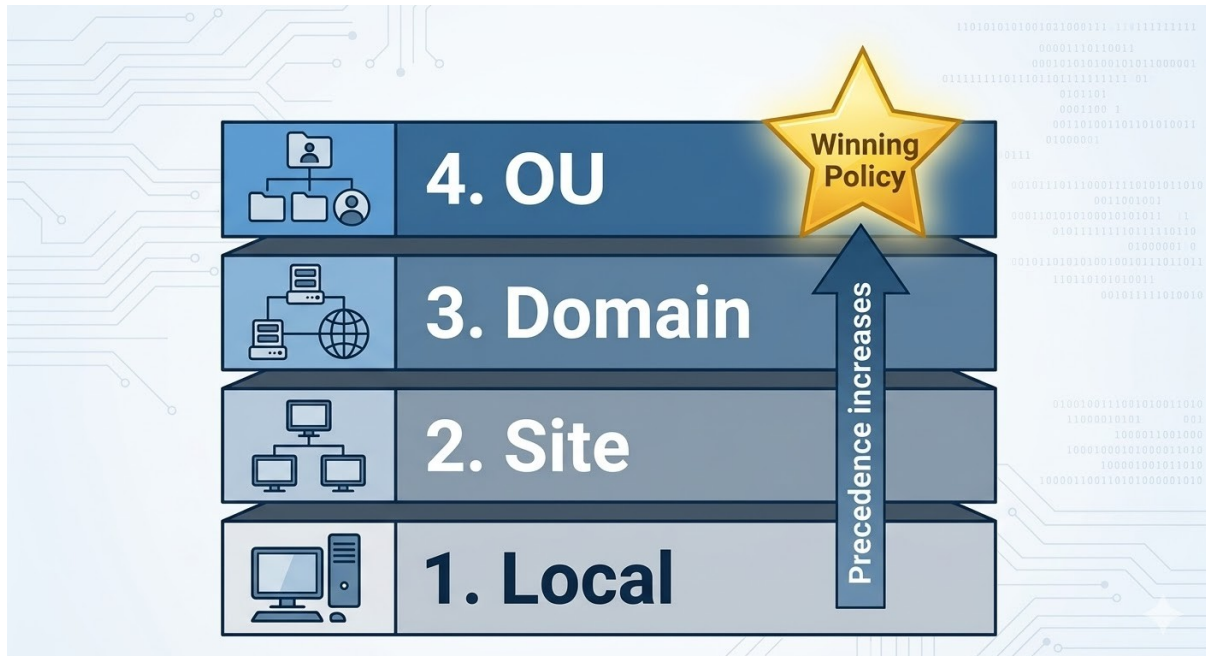
ACTIVE DIRECTORY ORGANIZATIONAL UNIT (OU) STRUCTURE



En av de främsta fördelarna med en genomtänkt OU-design är möjligheten till **delegation** (delegation of control). Istället för att ge en enskild tekniker fullständiga administratörsrättigheter över hela nätverkets hjärta, kan man delegera specifika uppgifter på en begränsad nivå. Exempelvis kan en lärare ges rätten att återställa lösenord för elever inom ett visst program, eller en lokal tekniker få behörighet att ansluta nya datorer till domänen i en specifik byggnad. Detta följer principen om minsta möjliga behörighet (least privilege) och ökar säkerheten genom att begränsa räckvidden av ett eventuellt misstag eller ett kapat administratörskonto.

Group Policy (grupprinciper, GPO) är det verktyg som förvandlar den logiska strukturen till praktisk verklighet. En GPO är en samling inställningar som "länkas" till en plats, domän eller OU för att styra allt från bakgrundsbilder och säkerhetsinställningar till vilka program som ska vara installerade. Inställningarna delas upp i två delar: datorkonfiguration (computer configuration) och användarkonfiguration (user configuration). När en dator startar och en användare loggar in, appliceras dessa regler i en bestämd ordning enligt principen **LSDOU**: lokalt (local), plats (site), domän (domain) och slutligen organisatorisk enhet (OU). Det är den sista policyn som läses in som "vinner" om två inställningar skulle motsäga

varandra, vilket gör att man kan ha generella regler för hela skolan men specifika undantag för enskilda klassrum.



I miljöer som datasalarna på ett gymnasium uppstår ofta ett speciellt behov: man vill att rummet ska styra upplevelsen oavsett vem som loggar in. För detta används **loopback processing**. Normalt sett följer användarinställningarna med eleven (användarobjektet), men med loopback aktiverat på datorns OU tvingas datorns specifika inställningar att gälla även för användaren. Det säkerställer att alla i salen ser samma startmeny och har tillgång till samma specialprogram, oavsett vilka inställningar de har på sina vanliga bärbara datorer. För att ytterligare finjustera vilka enheter som träffas av en policy används säkerhetsfiltrering (security filtering) eller **WMI-filter**, vilket kan begränsa en policy till att bara gälla exempelvis bärbara datorer med en viss version av Windows.

Förtydligande (sammanfattning):

OU-design: Skapar ordning och sätter gränser för var specifika regler och ansvar ska gälla.

Delegation: Tillåter utdelning av administrativa uppgifter på lokal nivå utan att ge bort full kontroll över hela domänen.

GPO (Group Policy Object): Den tekniska motorn som skickar ut inställningar till datorer och användare.

LSDOU: Den hierarkiska ordning (Local, Site, Domain, OU) som avgör vilken policy som har företräde.

Loopback Processing: En funktion som låter datorns placering (OU) styra användarupplevelsen, perfekt för gemensamma datasalar.

Exempel: En skola vill att alla elevdatorer i sal 214 ska ha en specifik skrivare förvald. Administratören skapar en GPO med skrivarinställningen och länkar den till OU:t "Sal-214-Datorer". Genom att aktivera loopback processing säkerställer man att skrivaren dyker upp för alla elever som sätter sig vid datorerna, trots att deras egna användarkonton normalt inte har den skrivaren kopplad.

Logik för GPO-arv:

1. Local Policy (Lägst företräde)
2. Site GPO
3. Domain GPO
4. OU GPO (Högst företräde)

--> Om konflikt uppstår i en inställning, gäller värdet från punkt 4.

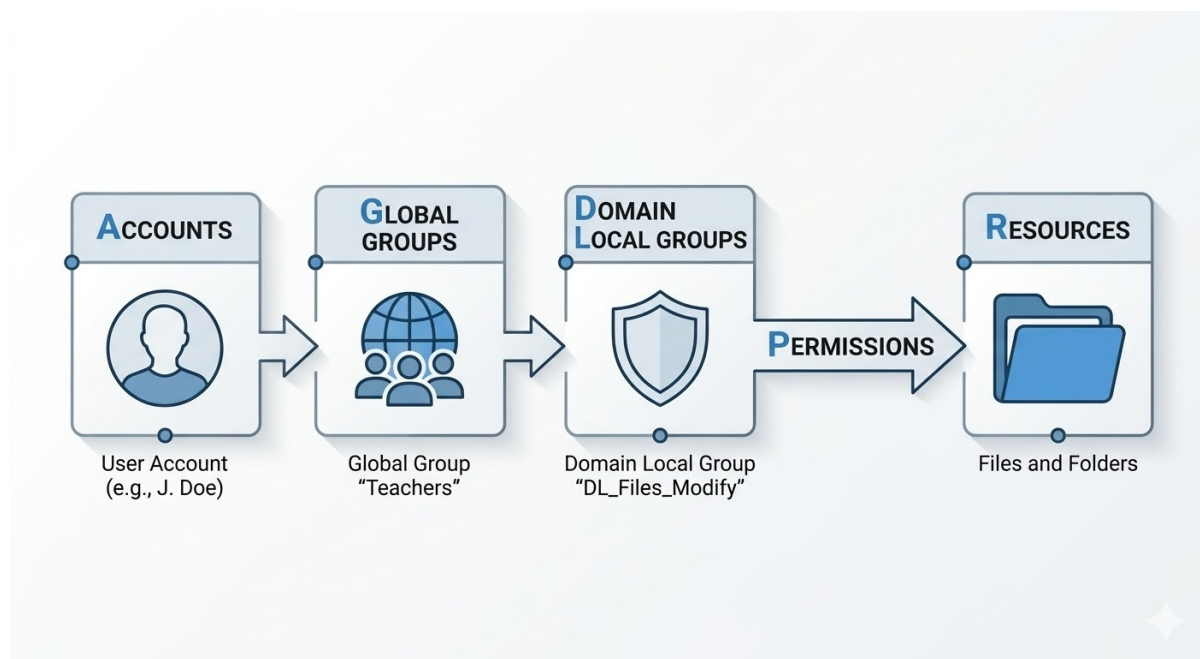
För en steg-för-steg-guide i hur du skapar OU:er, delegerar kontroll genom guiden i AD och skapar din första GPO i Group Policy Management Console, se kapitel 7 i Datorteknik-boken.

3.5.5 Identitet och resurser: AGDLP, rättigheter och fildelning

Identitet i Active Directory blir begriplig först när man skiljer på kontot och gruppen. Kontot beskriver en unik individ eller en tjänst, ofta identifierad genom sitt **UPN-format** (User Principal Name, t.ex. *namn@doman.local*), vilket ger en igenkänningsfaktor från moderna molntjänster. Gruppen däremot beskriver en roll eller en funktion. För att hantera detta på ett professionellt och skalbart sätt används industristandarden **AGDLP**. Det är en förkortning för att vi placerar konton (**A**ccounts) i globala grupper (**G**lobal groups) som representerar en roll (t.ex. lärare eller elever). Dessa grupper läggs i sin tur in i domänlokala grupper (**D**omain **L**ocal groups)

som är knutna till den faktiska resursen och bär behörigheten (**P**ermissions).

Genom att använda AGDLP som en "kopplingsdosa" skapar man en miljö som är nästintill självdokumenterande genom en tydlig namnstandard. Om en domänlokal grupp heter `DL_Share_Personal_Modify` ser man direkt att den ger ändringsrättigheter till mappen "Personal". Denna metod gör att tekniker kan lösa de flesta behörighetsärenden genom att bara flytta användare mellan globala grupper utan att någonsin behöva röra de känsliga rättigheterna på själva filservern. För nätverkets tjänster, som ofta kräver inloggning för att fungera, används **gMSA** (group Managed Service Accounts). Det är en säkrare variant av konton där domänen själv sköter lösenordsbyten och rotation, vilket eliminerar risken med gamla, bortglömda lösenord i klartext.



Det är också viktigt att skilja på var rättigheterna faktiskt appliceras. **Användarrättigheter** (User Rights Assignment) i operativsystemet handlar om vad ett konto får göra mot själva datorn, som att logga in lokalt eller köra en tjänst. Detta är helt separerat från **dataåtkomst**, som styrs genom en kombination av delningsrättigheter (**Share permissions**) och filsystems rättigheter (**NTFS permissions**). Man kan likna delningsrättigheten vid den grova grinden vid ingången till ett hus, medan

NTFS-rättigheten är den finmaskiga styrningen av vilka enskilda dörrar i korridoren du får öppna. Den effektiva rättigheten för en användare blir alltid det mest begränsande snittet av de två.

En modern filserver i en skolmiljö är mer än bara en hårddisk på nätet; den är en scenografi för arbete. Med **ABE** (Access-Based Enumeration) döljs mappar och filer som användaren inte har behörighet till, vilket skapar en renare och säkrare miljö där elever inte ens ser att lärarens mappar existerar. För att öka användarvänligheten används **VSS** (Shadow Copies), vilket låter användare själva återställa tidigare versioner av dokument genom ett enkelt högerklick. Slutligen ser man till att rätt diskar och resurser dyker upp hos rätt personer genom **Group Policy Preferences** (GPP), som automatiskt mappar upp nätverksenheter (t.ex. P: eller S:) baserat på användarens gruppmedlemskap.

Förtydligande (sammanfattning):

Konton vs Grupper: Kontot är individen, gruppen är rollen eller behörighetsbehållaren.

AGDLP: En metod för att separera användare från resurser för att göra nätverket underhållbart (Accounts -> Global -> Domain Local -> Permissions).

NTFS vs Share: Share är ingången till utdelningen, NTFS är den detaljerade kontrollen på filnivå.

ABE & VSS: Tekniker för att dölja oåtkomliga filer och tillåta användare att själva rädda raderade dokument.

gMSA: Automatiserade tjänstekonton som ökar säkerheten genom att domänen sköter lösenordshantering.

Exempel: En ny lärare börjar på skolan. Administratören lägger till lärarens konto i den globala gruppen *G_Larare*. Eftersom denna grupp redan är medlem i den domänlokala gruppen *DL_Share_Personal_Modify* får läraren omedelbart tillgång till personmappen. Dessutom ser en GPP till att mappen automatiskt dyker upp som en nätverkshårddisk på lärarens dator vid nästa inloggning. Ingen behövde ändra på några filer på servern – allt sköttes via gruppmedlemskapet.

För praktiska övningar i hur du skapar fildelningar, konfigurerar NTFS-rättigheter enligt AGDLP-modellen och sätter upp Shadow Copies, se kapitel 8 i Datorteknik-boken.

3.5.6 Nätverkshärdning: Bas-GPO:er och LAPS

Att sätta upp en domän är bara första steget; att göra den motståndskraftig mot angrepp och felkonfigurationer kallas för **härdning** (hardening). I en professionell miljö lämnar man inget åt slumpen eller åt operativsystemets standardinställningar. Istället etablerar administratören ett "golv" av grundläggande säkerhetsinställningar via gruppprinciper som alltid ska gälla. Detta skapar förutsägbarhet i undervisningen och minskar antalet säkerhetsincidenter avsevärt. Genom att använda gruppprinciper för att styra säkerheten kan man vara säker på att en ny dator som ansluts till nätverket omedelbart får samma höga skyddsnivå som de befintliga maskinerna.

En av de viktigaste delarna i detta arbete är hanteringen av uppdateringar. Genom att styra **Windows Update** centralt via en så kallad "ring-modell", kan administratören rulla ut säkerhetsfixar i etapper. Först testas uppdateringarna på ett fåtal maskiner (pilot), och när de visat sig stabila skickas de ut till resten av skolan. Detta förhindrar att en hel datasal slutar fungera samtidigt på grund av en buggig uppdatering. På samma sätt används **Windows-brandväggen** med specifika profiler för domän, privat och offentlig miljö. Det gör att datorn automatiskt blir mer restriktiv så fort den lämnar skolans skyddade nätverk, samtidigt som den tillåter nödvändig trafik när den är ansluten till domänen.

Ett modernt problem i nätverkssäkerhet är "lateral förflyttning", där en angripare tar sig in på en dator och sedan sprider sig vidare genom att använda samma lokala administratörslösenord på nästa maskin. För att stoppa detta används **Windows LAPS** (Local Administrator Password Solution). Istället för att alla datorer har samma lösenord för det lokala administratörskontot, ser domänen till att varje unik maskin får ett eget, slumpmässigt och roterande lösenord som lagras säkert i Active Directory. För fjärradministration via **RDP** (Remote Desktop Protocol) bör man dessutom begränsa åtkomsten så att den endast är tillåten från ett dedikerat management-nätverk, vilket förhindrar att administrativa gränssnitt exponeras i elev- eller personalnätet.

Slutligen är synlighet och spårbarhet avgörande. Genom att aktivera **granskningsspolicyer** (audit policies) kan administratören i efterhand se vem som loggat in, när en policy ändrades eller om någon försökt komma åt filer de inte har behörighet till. Tillsammans med **Group Policy Preferences** (GPP) för att styra användarupplevelsen – som att se till att rätt skrivare och startmeny finns på plats – skapas en miljö som inte bara är säker, utan också effektiv att arbeta i. Att dokumentera och versionera sina policyer är sista pusselbiten för att snabbt kunna backa en ändring om något mot förmodan skulle gå fel.

Förtydligande (sammanfattning):

Härdning (Hardening): Processen att säkra systemet genom att ta bort onödiga tjänster och strama åt regler.

Windows Update (Ring-modell): En kontrollerad utrullning av uppdateringar för att undvika storskaliga driftstopp.

LAPS: Automatiserar unika och roterande lösenord för lokala administratörskonton på varje enskild dator.

Brandväggsprofiler: Anpassar datorns skyddsnivå beroende på vilket nätverk den är ansluten till.

Audit-policyer: Loggar händelser i nätverket för att möjliggöra felsökning och incidenthantering.

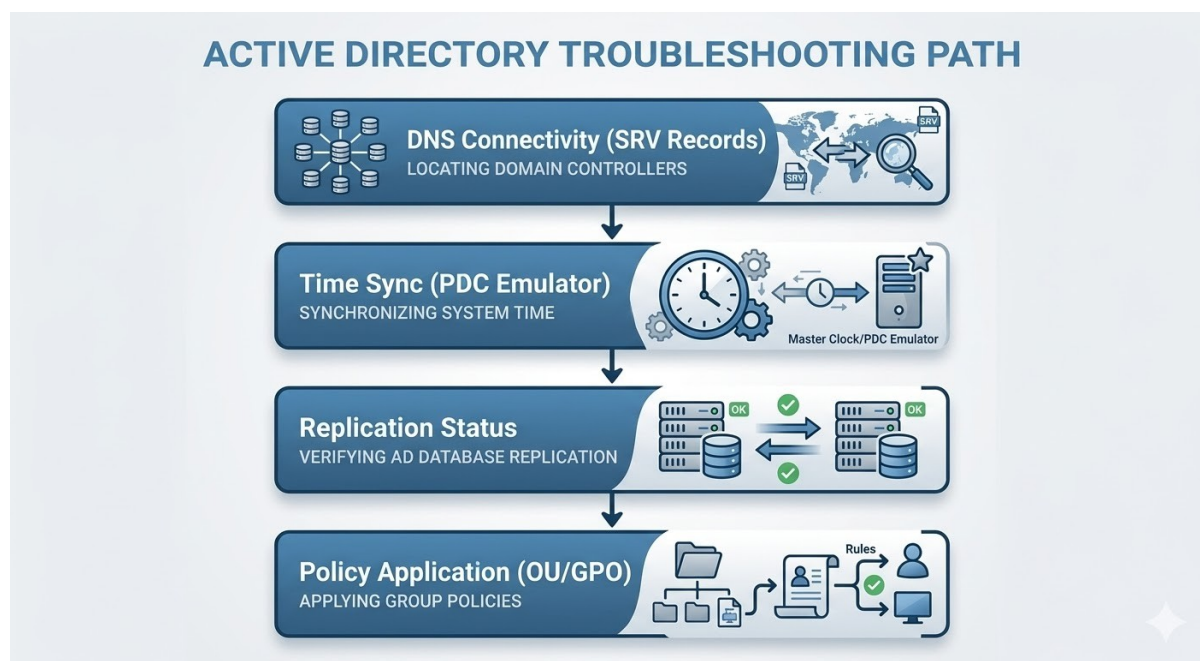
Exempel: En ny datasal utrustas med 30 datorer. Så fort de ansluts till domänen och hamnar i rätt OU, appliceras "Bas-GPO:er" som stänger av osäkra protokoll som Telnet, aktiverar LAPS för unika lösenord och konfigurerar brandväggen. Eleverna möts av en standardiserad startmeny och rätt nätverksdiskar, medan administratören kan sova tryggt i vetskapen om att alla maskiner följer skolans säkerhetsstandard utan manuell handpåläggning.

För en praktisk guide om hur du installerar LAPS, konfigurerar brandvägsregler via GPO och sätter upp loggning av inloggningshändelser, se kapitel 9 i Datorteknik-boken.

3.5.7 Felsökning: DNS, tid och replikering

När ett problem uppstår i en Active Directory-miljö kan det ofta upplevas som komplext och svåröverskådligt, men genom att följa en metodisk felsökningsmodell (troubleshooting model) går det att snabbt ringa in orsaken. Felsökning i AD handlar i grunden om att verifiera de tre fundamenten: att klienten kan hitta sin server, att de talar samma tid och att servrarna är överens om informationen. Genom att börja vid nätverkets grundtjänster och röra sig uppåt i den logiska strukturen kan administratören utesluta felkällor steg för steg.

Det första och absolut vanligaste stoppet vid felsökning är **DNS**. Om en dator kan surfa på internet men inte kan logga in eller hitta domänen, beror det nästan uteslutande på att den inte kan slå upp domänens **SRV-poster** (service records). En tekniker kontrollerar detta genom att verifiera att klienten använder rätt DNS-server – det vill säga en intern domänkontrollant och inte en publik router eller internetleverantörens DNS. Med verktyget nslookup kan man manuellt fråga efter domänens identitet för att se om nätverket svarar med rätt IP-adresser till domänkontrollanterna. Utan detta namnuppslag stannar hela inloggningsprocessen innan den ens har börjat.



När namnuppslaget fungerar är nästa kritiska punkt **tidssynkronisering** (time synchronization). Eftersom protokollet Kerberos använder tidsstämplar för att skydda mot attacker där inloggningsdata spelas in och skickas på nytt (replay attacks), tillåts endast en mycket liten tidsavvikelse. Om klockan på en klientdiffar mer än fem minuter från den domänkontrollant som bär rollen som **PDC-emulator**, kommer inloggningen att nekas med ett kryptiskt felmeddelande. Felsökningen här handlar om att kontrollera att tidskedjan är intakt, från den externa tidskällan till PDC-emulatorn, och vidare ut till alla servrar och klienter i nätverket.

Det tredje steget i en professionell felsökning är att kontrollera **replikering** (replication). I ett nätverk med flera domänkontrollanter måste alla servrar ha exakt samma bild av katalogen. Om en användare har bytt lösenord på en server, men informationen inte når den server som användaren faktiskt loggar in mot, kommer inloggningen att misslyckas. Genom att använda verktyg som repadmin kan administratören se om det finns hinder i nätverket, såsom felaktiga brandväggsregler eller trasiga nätverkslänkar, som hindrar servrarna från att "skvallra" för varandra. Om replikeringen står stilla blir nätverket snabbt fyllt av gamla "sanningar" som skapar förvirring för både användare och administratörer.

Slutligen, om infrastruktur och tjänster fungerar men en dator ändå inte beter sig som förväntat, handlar det ofta om **policytillämpning** (policy application). Det är då viktigt att verifiera att objektet ligger i rätt organisatorisk enhet (**OU**) och att inga blockeringar av arv (inheritance) hindrar gruppprinciperna från att nå målet. Genom att köra diagnostikverktyg som visar den resulterande policyn (Resultant Set of Policy, RSoP) kan man se exakt vilka inställningar som faktiskt har vunnit i den hierarkiska ordningen.

Förtydligande (sammanfattning):

DNS först: Verifiera att klienten kan hitta domänens SRV-poster med rätt interna DNS-inställningar.

Tid sen: Kontrollera att klockan inte diffar mer än fem minuter; Kerberos kräver synk mot PDC-emulatorn.

Replikering: Säkerställ att alla domänkontrollanter delar samma bild av databasen via verktyg som repadmin.

Policy-analys: Kontrollera OU-placering och arvsregler om inställningar inte slår igenom trots att nätverket är uppe.

Exempel: *En elev i Hus B kan inte logga in med sitt nya lösenord, trots att det fungerade i Hus A tio minuter tidigare. Teknikern börjar med att kontrollera DNS (fungerar), kontrollerar tiden (synkad), men ser vid en kontroll av replikeringen att länken mellan husen har legat nere. Lösenordsändringen har helt enkelt inte hunnit kopieras över till servern i Hus B ännu.*

För praktiska exempel på hur du använder kommandon som dcdiag, repadmin och gpresult för att lösa verkliga problem, se kapitel 10 i Datorteknik-boken.

Kontrollfrågor

1. Vad är den främsta anledningen till att man använder Windows Server som navet i användarhanteringen även i miljöer med många Linux-servrar?
2. Förklara skillnaden mellan en domän (domain) och en skog (forest).
3. Varför är det kritiskt för nätverkssäkerheten att ha minst två domänkontrollanter (DC)?
4. Vilken specifik FSMO-roll ansvarar för tidssynkroniseringen i domänen, och varför är tiden så viktig för Kerberos?
5. Vad är syftet med en Read-Only Domain Controller (RODC) och i vilken typ av nätverksmiljö är den lämplig?
6. Hur använder Active Directory DNS för att hjälpa klienter att hitta rätt tjänster, och vad kallas de specifika pekarna som används?
7. Förklara den hierarkiska ordningen för hur grupprinciper (GPO) appliceras (LSDOU). Vilken policy vinner vid en konflikt?
8. Vad innebär AGDLP-modellen och varför anses den vara mer effektiv än att ge rättigheter direkt till enskilda användarkonton?

9. Beskriv skillnaden mellan NTFS-rättigheter och delningsrättigheter (Share permissions).

10. Vilken funktion fyller Windows LAPS i ett härdat nätverk?

Fördjupningslänkar

Microsoft Learn (Grunderna i Active Directory Domain Services) –
<https://learn.microsoft.com/en-us/windows-server/identity/ad-ds/get-started/ad-ds-getting-started>

Cisco Support (Integration av Windows AD med nätverksutrustning via RADIUS) –
<https://www.cisco.com/c/en/us/support/docs/security-vpn/remote-authentication-dial-user-service-radius/212455-Configure-AnyConnect-802-1X-with-Microsoft.html>

ADSecurity.org (Djupdykning i hur Kerberos-biljetter fungerar) –
<https://adsecurity.org/?p=225>

Microsoft Docs (Hantering av FSMO-roller i Windows Server) –
<https://learn.microsoft.com/en-us/troubleshoot/windows-server/identity/fsmo-roles-in-ad-ds>

Group Policy Central (Allt om GPO, arv och loopback-processing) –
<https://www.grouppolicy.biz/>

Petri IT Knowledgebase (Guide till AGDLP-modellen i praktiken) –
<https://www.petri.com/active-directory-group-management-best-practices>

Windows OS Hub (Felsökning av AD-replikering med repadmin) –
<http://woshub.com/troubleshoot-active-directory-replication-repadmin/>

NIST Special Publication (Riktlinjer för härdning av Windows-miljöer) –
<https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-123.pdf>

Technet (Förklaring av DNS SRV-poster för Active Directory) –
<https://social.technet.microsoft.com/wiki/contents/articles/53093.active-directory-dns-srv-records.aspx>

Microsoft Open Source (Windows LAPS dokumentation och best practices) –
<https://learn.microsoft.com/en-us/windows-server/identity/laps/laps-overview>

Egna anteckningar

...

3.6 Ubuntu Server och Linux i nätverk

Linuxkärnan (Linux kernel) är utan tvekan världens mest använda programvarukomponent och utgör den osynliga motorn i vår moderna vardag. Den driver allt från de mest kraftfulla superdatorerna och de molnbaserade servrar (cloud servers) som hanterar global datatrafik, till miljarder mobiltelefoner och smarta hem-enheter (smart home devices) som uppkopplade kylskåp och termostater. Denna enorma spridning beror på systemets öppna källkod (open source), vilket har gjort det möjligt för tekniker över hela världen att optimera och anpassa kärnan för allt från minimala inbäddade system (embedded systems) till komplexa nätverksmiljöer.

Medan Windows är det dominerande operativsystemet på skrivbordsdatorer (desktops), är det Linux som i praktiken har byggt och upprätthåller hela internet. Den stabilitet, säkerhet och transparens som systemet erbjuder har gjort det till förstahandsvalet för webbservrar, databaser och de avancerade brandväggar (firewalls) som skyddar världens digitala infrastruktur. För en nätverkstekniker innebär steget in i Linux-världen en övergång till filosofin att "allt är en fil" (everything is a file). Genom att behärska kommandoraden och de textbaserade konfigurationsfiler som styr systemet, får administratören en nivå av precision och kontroll som krävs för att driva storskaliga nätverkstjänster på ett professionellt och skalbart sätt.

Denna del kommer utgå ifrån att du jobbar i den distribution av Linuxkärnan som heter Debian, mer specifikt Ubuntu, som är den vanligaste distributionen bland användare.

3.6.1 Introduktion: Linux som infrastruktur

I en modern nätverksmiljö fungerar Linux ofta som den stabila grunden i en hybridmiljö (hybrid environment) där olika operativsystem samverkar för att lösa specifika uppgifter. Om Windows Server agerar som nätverkets administrativa nav (administrative hub) för användarhantering och gruppprinciper, är Linux-servern ofta den specialiserade komponenten som hanterar tunga nätverkstjänster (network services), containrar (containers) och webbinfrastruktur. Den största omställningen för en tekniker som är van vid grafiska gränssnitt är den fundamentala Linux-

filosofin att **"allt är en fil"** (everything is a file). Detta innebär att nästan allt från nätverkskort (network interface cards) och hårddiskar till pågående processer och systeminställningar kan nås, läsas och konfigureras via det gemensamma filsystemet (file system).

Att arbeta med Linux innebär i de flesta fall att operativsystemet körs utan ett grafiskt gränssnitt (headless), vilket gör att administratören interagerar med systemet direkt via kommandoraden (command line). Eftersom Linux inte behöver lägga resurser på att rita upp fönster eller menyer kan det köras med extremt hög prestanda på relativt enkel hårdvara, vilket gör det till ett kostnadseffektivt val för skalbara molntjänster (cloud services). För en nätverkstekniker ger denna textbaserade transparens (transparency) en oöverträffad kontroll; genom att läsa en konfigurationsfil ser du exakt hur tjänsten är tänkt att fungera, utan att behöva gissa vad som döljer sig bakom en knapp i ett grafiskt fönster. Denna djupgående insyn i systemets inre delar är en förutsättning för att kunna bygga och underhålla de nätverk som i förlängningen utgör hela internet.

För en fördjupning i hur du praktiskt navigerar i filsystemet och använder de mest grundläggande terminalkommandona, se kapitel 10 i Datorteknik-boken.

3.6.2 Nätverkskonfiguration: Netplan och gränssnitt

Innan en server kan konfigureras måste administratören identifiera de fysiska och logiska nätverkskort (network interfaces) som finns i systemet. I moderna Linux-distributioner används **förutsägbara nätverksnamn** (predictable network interface names), vilket innebär att ett kort behåller sitt namn baserat på sin fysiska placering på moderkortet. För att lista alla tillgängliga gränssnitt används kommandot `ip link show` eller det mer detaljerade `networkctl list`. Ett snabbt sätt att se namnen är även att titta direkt i systemkatalogen `/sys/class/net/`.

Namngivningen följer en logisk standard som gör det enkelt att se skillnad på olika typer av anslutningar. Gränssnitt som börjar på **en** (exempelvis `enp3s0` eller `eno1`) är trådbundna anslutningar (Ethernet), medan namn som börjar på **wl** (exempelvis `wlp2s0`) representerar trådlösa nätverkskort (WLAN). Det

virtuella gränssnittet **lo** (loopback) används av systemet för att kommunicera med sig självt (127.0.0.1) och finns alltid på plats.

Om det finns ett behov av att snabbt ändra status på ett kort i realtid används verktyget **ip**. Detta påverkar kärnans (kernel) nuvarande tillstånd men sparas inte permanent. För att stänga av ett kort används `sudo ip link set [namn] down` och för att sätta på det används `sudo ip link set [namn] up`. Dessa kommandon är användbara vid omedelbar felsökning men ändringarna försvinner vid en omstart.

För att skapa en beständig konfiguration (persistent configuration) som överlever en omstart använder Ubuntu Server verktyget **Netplan**. All konfiguration samlas i **YAML-filer** placerade i katalogen `/etc/netplan/`. Netplan fungerar som en arkitekt som beskriver hur nätverket ska se ut, och skickar sedan instruktionerna vidare till en underliggande motor (renderer), oftast **systemd-networkd**.

YAML-formatet kräver stor noggrannhet med **indentering** (indragning); man får aldrig använda tabbar, utan endast mellanslag (vanligtvis två per nivå). En felaktig indragning innebär att filen inte kan läsas in av systemet. Nedan visas ett exempel på en konfigurationsfil för en server med två nätverkskort, där ett använder DHCP och det andra har en statisk (fast) IP-adress.

YAML

```
network:
  version: 2
  renderer: networkd
  ethernets:
    # Kort 1: Hämtar adress automatiskt via DHCP
    enp3s0:
      dhcp4: true
    # Kort 2: Fast (statisk) IP-adress
    enp4s0:
      addresses:
        - 192.168.10.50/24
      nameservers:
        addresses: [8.8.8.8, 1.1.1.1]
```

```
routes:
  - to: default
    via: 192.168.10.1
```

I professionella miljöer används Netplan även för att konfigurera avancerade funktioner som **VLAN (802.1Q)**, **Bonding** (redundans genom att slå ihop flera kort) och **Bridges** (broar som fungerar som virtuella switchar för containrar eller virtuella maskiner). När en ändring har gjorts i YAML-filen bör administratören först använda `sudo netplan try`. Detta kommando aktiverar inställningarna temporärt och väntar på en bekräftelse; om anslutningen bryts och ingen bekräftelse ges, återställs de gamla inställningarna automatiskt efter två minuter. För att göra ändringarna permanenta direkt används `sudo netplan apply`.

Förtydligande (sammanfattning):

Nätverkskonfiguration i Linux delas upp i nuläge och beständighet. Genom att identifiera kort via deras prefix (en/wl) kan administratören skapa korrekta YAML-ritningar i Netplan. Systemet erbjuder inbyggda säkerhetsfunktioner som `netplan try` för att förhindra att man råkar låsa ut sig själv vid konfigurationsändringar på distans.

Exempel: En tekniker identifierar serverns kort som `enp3s0`. Hen skapar en Netplan-fil som tilldelar kortet en statisk IP-adress i rätt subnät och anger gateway för internetåtkomst. Efter att ha verifierat syntaxen aktiveras nätverket och verifieras med `ip address`.

För en praktisk steg-för-steg-guide i hur du skriver avancerade Netplan-filer för bonding och bryggor, se kapitel 11 i Datorteknik-boken.

Snabbguide: Vanliga kommandon för nätverk i Linux

Syfte	Kommando
Lista alla nätverkskort och status	<code>ip link show</code>
Visa IP-adresser på alla kort	<code>ip address</code>
Visa routingtabell och gateway	<code>ip route</code>

Syfte	Kommando
Sätta på ett nätverkskort (realtid)	<code>sudo ip link set [namn] up</code>
Stänga av ett nätverkskort (realtid)	<code>sudo ip link set [namn] down</code>
Testa en ny Netplan-konfiguration	<code>sudo netplan try</code>
Aktivera Netplan-konfiguration permanent	<code>sudo netplan apply</code>
Visa detaljerad status för länkar	<code>networkctl status</code>

3.6.3 Systemtjänster: DNS och Tid (Nervsystemet)

Om nätverkskortet är serverns lemmar, så är **DNS** (namnuppslag) och **NTP** (tidssynkronisering) dess nervsystem. Utan fungerande namnuppslag är servern i praktiken blind; den kan nå andra enheter via IP-adresser men förstår inte mänskliga namn eller var kritiska tjänster i domänen bor. På samma sätt är exakt tid inte bara en praktisk detalj för loggfiler, utan en absolut förutsättning för säkerhet. En Linux-server som ingår i en hybridmiljö med Windows måste ha en klocka som tickar i exakt takt med domänkontrollanten för att inloggningsbiljetter (Kerberos tickets) ska accepteras.

På moderna Ubuntu-system sköts namnuppslaget av tjänsten **systemd-resolved**. Den fungerar som en lokal mellanhand (stub resolver) som sköter cachelagring av adresser och håller koll på vilken namnserver (nameserver) som ska användas för vilket nätverkskort. Detta möjliggör så kallad **split-horizon DNS**, där servern kan använda en intern DNS för skolans resurser och en publik DNS för trafik mot internet. För att kontrollera hur servern ser på världen och vilka DNS-servrar som faktiskt används, används verktyget `resolvectl`.

resolvectl status: Visar en detaljerad lista över vilka DNS-servrar som är kopplade till respektive nätverkskort.

resolvectl query [domän]: Testar ett specifikt namnuppslag och visar exakt vilken väg informationen tog.

Tidssynkroniseringen hanteras oftast av den lätta klienten **systemd-timesyncd**. Den använder protokollet **NTP** (Network Time Protocol) för att hämta korrekt tid från en tidsserver på nätet eller en lokal domänkontrollant. För en nätverkstekniker är det viktigt att veta att en tidsavvikelse på mer än fem minuter oftast gör det omöjligt att logga in i en Windows-domän, eftersom säkerhetsprotokollet Kerberos tolkar tidsskillnaden som ett potentiellt intrångsförsök. För att hantera systemklockan och tidszoner används verktyget **timedatectl**.

Förtydligande (sammanfattning):

Namnuppslag och tid är de två mest kritiska tjänsterna för att en server ska kunna interagera med resten av nätverket. Genom **systemd-resolved** får systemet en intelligent hantering av DNS-frågor, medan **systemd-timesyncd** säkerställer att klockan alltid är synkroniserad. Att verifiera dessa två tjänster är ofta det första steget vid felsökning av inloggningsproblem eller nätverksfel.

Exempel: En Ubuntu-server kan inte ansluta till en fildelning i domänen. Vid kontroll med **resolvectl status** ser teknikern att servern av misstag pekar på en publik DNS (t.ex. 8.8.8.8) som inte känner till de interna SRV-posterna. Genom att korrigera detta i Netplan och verifiera att **timedatectl** visar "System clock synchronized: yes" blir servern omedelbart en fungerande del av infrastrukturen.

För en djupare genomgång av hur du manuellt konfigurerar mer avancerade NTP-servrar som Chrony eller konfigurerar DNS-search domains, se kapitel 12 i Dator teknik-boken.

Snabbvy: Kommandon för DNS och Tid

Funktion	Kommando
Visa aktuell DNS-status	resolvectl status
Testa ett namnuppslag	resolvectl query google.com

Funktion	Kommando
Visa tid, tidszon och NTP-status	timedatectl status
Lista tillgängliga tidszoner	timedatectl list-timezones
Sätt tidszon	sudo timedatectl set-timezone Europe/Stockholm
Aktivera/Inaktivera NTP-synk	sudo timedatectl set-ntp true/false

3.6.4 Fjärradministration och Härdning: SSH och Brandvägg

Fjärradministration (remote administration) är ryggraden i hanteringen av Linux-servrar, då dessa sällan har en fysisk bildskärm ansluten.

Standardverktyget för detta ändamål är **SSH** (Secure Shell), ett protokoll som skapar en krypterad tunnel mellan administratörens dator och servern. SSH är dock mycket mer än bara en fjärrterminal; det fungerar som ett mångsidigt transportlager som kan bära allt från filöverföringar (SFTP) till säkra tunnlar för andra nätverksprotokoll. Att säkra denna ingång är det enskilt viktigaste steget i att härda (harden) en server mot externa hot.

En av de mest effektiva metoderna för att höja säkerheten är att ersätta lösenordsinloggning med **SSH-nycklar** (SSH keys). Denna metod bygger på asymmetrisk kryptering (asymmetric encryption) där administratören skapar ett nyckelpar bestående av en privat nyckel (private key) och en publik nyckel (public key). Den publika nyckeln placeras på servern, medan den privata nyckeln stannar på administratörens lokala maskin, skyddad av en lösenfras (passphrase). Vid inloggning skickas aldrig något lösenord över nätverket; istället bevisar administratörens dator sin identitet genom en kryptografisk signatur. När nyckelbaserad inloggning är verifierad och fungerar, är det branschstandard att helt avaktivera lösenordsinloggning och förbjuda direktinloggning som root i konfigurationsfilen `/etc/ssh/sshd_config`.

ssh-keygen: Skapar ett nytt par av kryptografiska nycklar.

ssh-copy-id: Kopierar den publika nyckeln till servern på ett säkert sätt.

PermitRootLogin no: En inställning i SSH-konfigurationen som tvingar administratörer att logga in som vanliga användare och sedan använda sudo.

I mer komplexa nätverksmiljöer används ofta en **bastionvärd** (bastion host) eller hoppserver (jump host). Detta är en ensam, extremt härdad server som fungerar som den enda tillåtna ingångsporten från internet till nätverkets interna servrar. Genom att använda en bastionvärd kan administratören stänga alla externa administrativa portar på de interna servrarna och endast tillåta trafik som kommer inifrån det egna nätverket. Detta minskar nätverkets exponerade yta (attack surface) och skapar en central punkt för loggning av vem som ansluter till infrastrukturen.

För att kontrollera vilken trafik som tillåts nå servern lokalt används brandväggen **UFW** (Uncomplicated Firewall). UFW fungerar som ett användarvänligt gränssnitt till de mer komplexa underliggande systemen i Linux-kärnan. En viktig princip vid konfigurering av brandväggar är att alltid tillåta SSH-trafik innan brandväggen aktiveras, för att undvika att administratören låser ut sig själv från servern. Genom att kombinera en strikt brandväggsprofil med nyckelbaserad SSH-inloggning skapas en mycket motståndskraftig miljö som kan stå emot både automatiserade attacker och riktade intrångsförsök.

Förtydligande (sammanfattning):

Fjärradministration i Linux bygger på SSH som säkras genom kryptografiska nycklar istället för traditionella lösenord. Genom att använda en bastionvärd isoleras det interna nätverket från direkt exponering mot internet, och med hjälp av UFW begränsas åtkomsten ytterligare på varje enskild server. Härdning handlar om att systematiskt stänga alla dörrar som inte absolut måste vara öppna och att kräva starka bevis på identitet för de som tillåts passera.

Exempel: En administratör konfigurerar en ny Ubuntu-server. Först genereras ett nyckelpar och den publika nyckeln kopieras till servern. I SSH-konfigurationen stängs lösenordsautentisering av. Därefter körs kommandona `sudo ufw allow ssh` följt av `sudo ufw enable`. Servern är nu skyddad mot lösenordsattacker och tillåter endast inkommande trafik på de portar som administratören uttryckligen har öppnat.

För en praktisk genomgång av hur du konfigurerar SSH-nycklar i Windows Terminal, skapar ProxyJump-inställningar och sätter upp avancerade brandväggsregler i UFW, se kapitel 13 i Datorteknik-boken.

Snabbvy: Kommandon för SSH och Brandvägg

Funktion	Kommando
Generera SSH-nyckelpar	<code>ssh-keygen -t ed25519</code>
Kopiera publik nyckel till server	<code>ssh-copy-id [användare]@[ip]</code>
Starta om SSH-tjänsten (efter ändring)	<code>sudo systemctl restart ssh</code>
Visa status för brandväggen	<code>sudo ufw status</code>
Tillåt specifik trafik (t.ex. SSH)	<code>sudo ufw allow ssh</code>
Aktivera brandväggen	<code>sudo ufw enable</code>
Visa loggar för SSH-inloggningar	<code>sudo journalctl -u ssh</code>

3.6.5 Identitet och rättigheter: UID, GID och ACL

I Linux-världen bygger identitet och behörighet på ett system som är både enkelt och transparent. Varje användarkonto (user account) tilldelas ett unikt siffervärde som kallas **UID** (User Identifier), och varje grupp (group) får ett motsvarande **GID** (Group Identifier). Även om vi som administratörer arbetar med namn som "elev" eller "lärare", ser operativsystemet endast dessa siffervärden när det avgör vem som äger en fil eller en process. Denna sifferbaserade struktur gör det möjligt för

Linux att snabbt och effektivt kontrollera rättigheter i filsystemet genom att jämföra användarens ID mot filens ägare och grupptillhörighet.

Grundbulten i behörighetslogiken är den klassiska triaden av rättigheter: läsa (**r**ead), skriva (**w**rite) och köra (**x**ecute). Dessa appliceras på tre nivåer: ägaren (owner), gruppen (group) och alla andra (others). En viktig detalj för förståelsen av nätverksadministration är hur körrätten (execute) tolkas olika beroende på om det gäller en fil eller en katalog (directory). På en fil innebär ett "x" att den kan startas som ett program, men på en katalog betyder det rätten att "gå in i" eller passera genom katalogen. Utan körrätt på en överliggande mapp kan en användare inte nå sina filer, även om hen har fullständiga läsrättigheter på själva filnivån.

Linux-rättigheter (Permissions)

Filtyp	Ägare (User)	Grupp (Group)	Övriga (Others)	
	v v v	v v v	v v v	
-	r w x	r - x	r - -	
			+ - - - -	Läs (Read)
		+ - - - -		Ingen skriv rätt (-)
		+ - - - -		Kör/Gå in (Execute)
	+ - - - -			Kör/Starta (Execute)
	+ - - - -			Skriv (Write)
	+ - - - -			Läs (Read)

+-- Bindestreck (-) betyder vanlig fil.
Ett "d" betyder katalog (directory).

Vad betyder siffrorna? (Oktala värden)

För att snabbt kunna sätta rättigheter används ofta siffror istället för bokstäver. Varje bokstav har ett fast värde som man adderar ihop:

r (Read) = 4

w (Write) = 2

x (Execute) = 1

Exempel på beräkning:

Ägare: r (4) + w (2) + x (1) = 7

Grupp: r (4) + - (0) + x (1) = 5

Övriga: $r(4) + - (0) + - (0) = 4$

Resultat: `chmod 754 filnamn.txt`

Varför är "x" så viktigt på en katalog?

Som vi nämnde i texten är det här en vanlig fallgrop. Om vi tittar på en mapp med rättigheten `r-- (400)` jämfört med `r-x (500)`:

- **r-- (Endast läs):** Du kan se *namnen* på filerna i mappen om du kör `ls`, men du får inte öppna dem eller se deras storlek/detaljer eftersom du inte får "gå in" (execute) i mappen.
- **r-x (Läs och kör):** Du kan både se filerna och gå in i mappen för att faktiskt arbeta med dem.

För att hantera mer komplexa behov i en delad miljö används speciella attribut som **SetGID** och **Sticky bit**. I en gemensam ämnesmapp kan SetGID aktiveras på katalogen för att säkerställa att alla nya filer som skapas där automatiskt ärver katalogens grupptillhörighet istället för användarens primära grupp. Detta är avgörande för att filer ska förbli läsbara för alla medlemmar i gruppen utan manuell handpåläggning. Sticky bit används ofta i mappar för inlämningar, där alla användare har rätt att skriva filer, men endast filens ägare (eller root) har rätt att ta bort den. Detta förhindrar att elever av misstag eller medvetet raderar varandras arbeten i en gemensam yta.

När de grundläggande rättigheterna inte räcker till för att uttrycka mer finkorniga regler, används **ACL** (Access Control Lists). Med ACL kan administratören ge specifika rättigheter till flera olika användare eller grupper på samma fil eller mapp, utöver den ordinarie ägaren och gruppen. Detta påminner mycket om hur rättigheter hanteras i Windows-världen och är ofta en förutsättning för att en Linux-server ska kunna integreras sömlöst i en hybridmiljö. För att administrera systemet med hög säkerhet används **sudo** (superuser do), vilket tillåter en vanlig användare att utföra specifika administrativa uppgifter med tillfälligt förhöjda rättigheter. Detta skapar en spårbarhet (audit trail) i systemloggarna som visar exakt vem som utfört vilken ändring, till skillnad från att logga in direkt som det allsmäktiga kontot root.

Förtydligande (sammanfattning):

- **UID/GID:** De numeriska identiteter som Linux använder för att hålla reda på användare och grupper bakom kulisserna.
- **Kör rätt (x):** Ger rätten att exekvera filer eller navigera in i kataloger; en förutsättning för all åtkomst.
- **SetGID & Sticky bit:** Specialfunktioner för att styra grupparv och förhindra radering i delade mappar.
- **ACL:** Möjliggör detaljerad behörighetsstyrning för flera användare och grupper på samma objekt.
- **Sudo:** Ett verktyg för kontrollerad och spårbar delegation av administrativa rättigheter.

Exempel: Genom SetGID på kursmappen ärver nya filer automatiskt gruppen "Lärare" (Teachers). Kombinerat med en standard-ACL (Default ACL) får eleverna läsrätt direkt vid uppladdning. Detta automatiserar behörighetsflödet och eliminerar behovet av manuell handpåläggning vid varje ny fil.

För en praktisk genomgång av hur du använder kommandona `chmod`, `chown` och `setfacl` för att bygga säkra mappstrukturer, se kapitel 10 i *Datorteknik-boken*.

3.6.6 Fildelning och integration: SMB, SFTP och rsync

När rättigheterna på filsystemet är säkrade är nästa steg att göra resurserna tillgängliga för användarna. I en hybridmiljö (hybrid environment) handlar fildelning om att välja rätt verktyg för rätt situation. Linux-världen erbjuder flera kraftfulla protokoll som fyller olika funktioner, från den vardagliga åtkomsten för elever till administratörens behov av effektiva säkerhetskopior (backups). Det som binder samman alla dessa tjänster är förmågan att bevara de rättigheter (permissions) och den identitetslogik som vi precis har gått igenom.

För interaktion med Windows-världen är **SMB** (Server Message Block) det absolut viktigaste protokollet. Det är viktigt att förstå att SMB inte bara handlar om mappar och filer; det är ett protokoll för att dela ut resurser (resources) i nätverket. Detta inkluderar allt från skrivare och skannrar till seriella portar och andra enheter som klienterna behöver nå. Eftersom Windows-datorer inte talar Linux eget "språk" direkt, används

programvaran **Samba** som en översättare. Samba gör det möjligt för en Linux-server att agera som en Windows-resursserver, komplett med stöd för gruppbaserad behörighet och integration med Active Directory. Genom att mappa Linux-servers UID/GID mot domänens användare och grupper ser användaren ingen skillnad; de loggar in med sina vanliga uppgifter och når de resurser de har rätt till. Här blir användningen av **ACL** (Access Control Lists) i Linux avgörande, då de tillåter Samba att översätta de mer komplexa rättighetsnivåer som Windows-användare förväntar sig.

För fjärråtkomst och teknisk överföring används ofta **SFTP** (Secure File Transfer Protocol). Det är viktigt att komma ihåg att SFTP inte är en variant av det gamla osäkra FTP-protokollet, utan ett subsystem (subsystem) inuti **SSH**. Detta innebär att exakt samma säkerhet och identitetshantering som skyddar servers terminal (SSH-nycklar och kryptering) även skyddar filöverföringen. För en elev eller lärare som behöver ladda upp filer hemifrån är SFTP ett robust och säkert val som fungerar överallt där port 22 är öppen.

När det kommer till att flytta stora mängder data eller spegla hela katalogträd är **rsync** (Remote Sync) teknikerns främsta verktyg. rsync är känt för sin extrema effektivitet då det endast överför de delar av en fil som faktiskt har ändrats (differential transfer). Det kan köras över SSH för att behålla säkerheten och har förmågan att bevara alla filrättigheter, ägarskap och tidsstämplar under överföringen. Detta gör det till det givna valet för skriptade säkerhetskopior eller för att synkronisera kursmaterial mellan olika servrar i nätverket.

Förtydligande (sammanfattning):

- **Samba (SMB):** En brobyggare som delar ut resurser (filer, skrivare, enheter) så att de fungerar som "vanligt" för Windows-användare.
- **SFTP:** En säker filkanal inbyggd i SSH; perfekt för fjärråtkomst utan att behöva öppna extra portar i brandväggen.
- **rsync:** Teknikerns verktyg för effektiv synkronisering och säkerhetskopiering; sparar bandbredd genom att bara skicka förändrad data.

Exempel: En skola använder en Linux-server för att dela ut en kraftfull storformatsskrivare och gemensamma projektmappar. Tack vare **Samba** ser eleverna både skrivaren och mappen som vanliga

*resurser i sin Windows-miljö. Samtidigt använder administratören **rsync** varje natt för att spegla all data till en annan server för säkerhetskopiering, vilket säkerställer att inga arbeten går förlorade.*

För en praktisk guide om hur du konfigurerar en Samba-share mot Active Directory eller sätter upp automatiska rsync-jobb med cron, se kapitel 14 i Datorteknik-boken.

3.6.7 Metodisk felsökning i Linux

Felsökning i en Linux-miljö kräver ett strukturerat tillvägagångssätt där man utnyttjar systemets transparens (transparency). Eftersom nästan all information finns tillgänglig i textformat handlar det om att veta var man ska leta och vilka verktyg som avslöjar vad som händer under huven. Precis som i Windows-världen börjar en bra felsökning vid fundamenten: nätverksanslutning, namnuppslag och tid.

Det första steget är att verifiera det fysiska och logiska nätverket. Med kommandot `ip address` ser vi om nätverkskortet har fått rätt adress, och med `ip route` kontrollerar vi att servern vet hur den ska nå sin gateway. Om nätverket fungerar men tjänsterna strular är nästa anhalt namnuppslaget via `resolvectl status`. Felaktig DNS är ofta roten till problem där servern upplevs som "långsam" eller inte hittar domänen. Om servern ingår i en domän är det också avgörande att kontrollera tiden med `timedatectl`, då en tidsavvikelse bryter autentiseringen (authentication).

Därefter dyker vi in i systemets minne: loggarna. I moderna Linux-system samlas alla meddelanden i en central databas som kallas journalen (journal). Med verktyget `journalctl` kan administratören läsa av exakt vad som hänt i realtid eller gå tillbaka i tiden för att hitta felmeddelanden från specifika tjänster (t.ex. SSH eller Samba). Genom att läsa loggarna slipper man gissa; servern berättar ofta själv exakt varför en inloggning nekades eller varför en tjänst inte startade. Slutligen kontrolleras rättigheterna i filsystemet; ett vanligt fel är att en tjänst inte har rätt att läsa sin egen konfigurationsfil eller att en användare saknar körrätt (x-bit) på en överliggande katalog.

Förtydligande (sammanfattning):

- **Nätverkskoll:** Verifiera IP, rutter och länkstatus med `ip` och `networkctl`.
- **Tjänsteverifiering:** Kontrollera DNS (`resolvectl`) och tid (`timedatectl`) – fundamenten för nätverkskommunikation.
- **Loggläsning:** Använd `journalctl` för att läsa systemets egna felrapporter och händelseförlopp.
- **Rättighetsanalys:** Kontrollera UID/GID och bitar (`rwX`) för att utesluta åtkomstproblem i filsystemet.

Exempel: En webbtjänst på servern slutade plötsligt svara. Teknikern kör `sudo journalctl -u apache2 -f` för att se loggarna i realtid och upptäcker ett felmeddelande om "Permission denied" på en certifikatfil. Genom att kontrollera filen ser hen att ägarskapet av misstag ändrats, korrigerar det med `chown` och tjänsten hoppar igång omedelbart.

För mer djupgående felsökning med verktyg som `tcpdump` och `Wireshark` i Linux, se kapitel 15 i Dator teknik-boken.

Sammanfattning: Viktiga Linux-kommandon för nätverket

Kategori	Syfte	Kommando
Nätverk	Visa IP-adresser och status	<code>ip address</code>
	Visa routingtabell	<code>ip route</code>
	Aktivera nätverksinställningar	<code>sudo netplan apply</code>
Diagnostik	Kontrollera DNS-inställningar	<code>resolvectl status</code>
	Kontrollera tid och NTP	<code>timedatectl status</code>
	Läsa systemloggar i realtid	<code>sudo journalctl -f</code>
Behörighet	Ändra rättigheter (bitar)	<code>chmod [värde] [fil]</code>
	Ändra ägare och grupp	<code>chown [ägare]:[grupp] [fil]</code>
	Visa utökade rättigheter (ACL)	<code>getfacl [fil]</code>
Säkerhet	Visa brandväggens status	<code>sudo ufw status</code>

Kategori	Syfte	Kommando
	Testa anslutning till annan värd	ping [ip/namn]

Kontrollfrågor

1. Vad innebär filosofin "allt är en fil" (everything is a file) i Linux, och hur påverkar det hur du konfigurerar nätverket?
2. Varför är indentering (indragning) så kritisk när du skriver en konfigurationsfil i Netplan (YAML), och vad händer om du råkar använda en tabb istället för mellanslag?
3. Hur ser du skillnad på ett trådbundet (Ethernet) och ett trådlöst (WLAN) nätverkskort bara genom att titta på namnet i terminalen?
4. Beskriv skillnaden mellan `sudo netplan try` och `sudo netplan apply`. Varför är den första att föredra vid fjärradministration?
5. Varför är tidssynkronisering (NTP) via `systemd-timesyncd` så viktig när en Linux-server ska prata med en Windows-domän?
6. Förklara hur SSH-nycklar (SSH keys) höjer säkerheten jämfört med att använda vanliga lösenord.
7. Vad är den praktiska skillnaden mellan körrätt (execute) på en vanlig fil och på en katalog (directory)?
8. I vilka scenarier är det mer effektivt att använda `rsync` istället för SFTP för att flytta data?
9. Vilken roll spelar programvaran **Samba** i ett nätverk som består av både Linux-servrar och Windows-klienter?
10. Om en tjänst slutar fungera, vilket kommando använder du för att läsa systemets loggar i realtid för att se felmeddelandena?

Fördjupningslänkar

Ubuntu Server Guide (Den officiella dokumentationen för allt från installation till avancerade tjänster) - <https://ubuntu.com/server/docs>

Netplan Examples (Praktiska exempel på hur du konfigurerar allt från enkla IP-adresser till komplexa bryggor och bonding) - <https://netplan.io/examples/>

Systemd-resolved & DNS (Djupdykning i hur Ubuntu hanterar namnuppslag och lokala DNS-cachar) - <https://www.freedesktop.org/software/systemd/man/latest/systemd-resolved.service.html>

OpenSSH Security Best Practices (En guide till hur du härdar din SSH-server och hanterar nycklar på ett säkert sätt) - <https://www.ssh.com/academy/ssh/hardening>

Samba Wiki (AD Integration) (Hur du får din Linux-server att bli en fullvärdig medlem i en Windows-domän) - https://wiki.samba.org/index.php/Setting_up_Samba_as_a_Domain_Member

The rsync Manual (Allt du behöver veta om flaggor och effektiv synkronisering av filer) - <https://download.samba.org/pub/rsync/rsync.html>

Linux Permissions Calculator (Ett interaktivt verktyg för att förstå kopplingen mellan rwx och de numeriska (oktala) värdena) - <https://chmod-calculator.com/>

UFW Guide (DigitalOcean) (En mycket pedagogisk genomgång av hur du konfigurerar brandväggen i Ubuntu) - <https://www.digitalocean.com/community/tutorials/how-to-set-up-a-firewall-with-ufw-on-ubuntu>

Mastering Journalctl (En guide till hur du effektivt filtrerar och söker i Linux systemloggar) - <https://www.loggly.com/ultimate-guide/using-journalctl/>

Linux Foundation (Introduction to Linux) (En kostnadsfri kurs för den som vill förstå kärnan och operativsystemets grunder) - <https://training.linuxfoundation.org/training/introduction-to-linux/>

Egna anteckningar

3.7 MDM och klienthantering

I takt med att arbetsplatser och skolor har lämnat den traditionella modellen där alla datorer står permanent anslutna till ett lokalt nätverk, har behovet av en ny typ av administrativ kontroll uppstått. **MDM** (Mobile Device Management) är den tekniska lösningen som gör det möjligt för en organisation att behålla kommandot över sina enheter oavsett var i världen de befinner sig. Om den traditionella serverdriften handlar om att vårda nätverkets kärna, handlar MDM om att projicera denna makt ända ut till slutanvändarens skärm. Det är en vital del av varje modern organisation eftersom det garanterar att säkerhetspolicys, programuppdateringar och konfigurationer efterlevs, utan att enheten någonsin behöver besöka en fysisk tekniker.

Historiskt sett började resan med ren **MDM** (Mobile Device Management), vilket fokuserade på att kontrollera hårdvaran – att kunna låsa en borttappad telefon, tvinga fram ett lösenord eller konfigurera ett trådlöst nätverk. Men i takt med att behovet av att hantera även applikationer och innehåll växte, utvecklades konceptet till **EMM** (Enterprise Mobility Management). EMM tog steget längre genom att inte bara se till enheten som en pryl, utan även hantera behållare för e-post och företagsdata (**MAM** – Mobile Application Management). Idag pratar vi oftast om **UEM** (Unified Endpoint Management), vilket är den slutgiltiga sammansmältningen. En UEM-plattform är administratörens "enda fönster" där allt från bärbara Windows-datorer och kraftfulla Linux-stationer till iPads och Android-telefoner hanteras i samma gränssnitt och med samma logik. I dagligt tal används framförallt uttrycket MDM, men syftat till UEM.

En av de största utmaningarna för en nätverksadministratör idag är balansen mellan säkerhet och personlig integritet, särskilt inom konceptet **BYOD** (Bring Your Own Device). Här äger organisationen inte hårdvaran, men användaren vill ändå ha tillgång till resurser som e-post och interna system. Genom MDM-teknik kan man skapa en logisk separation på enheten – en krypterad arbets-profil som står under administratörens kontroll, medan användarens privata foton och appar förblir helt privata. Detta gör MDM till ett strategiskt verktyg; det handlar inte bara om teknisk support, utan om att möjliggöra ett flexibelt och säkert arbetssätt

där gränsen mellan privat och professionellt styrs av kod snarare än av fysiska väggar.

Utan en fungerande MDM- eller UEM-infrastruktur blir en organisation snabbt sårbar. Varje enhet som inte är centralt hanterad är en potentiell säkerhetsrisk och en administrativ mardröm vid uppdateringar eller avslutade anställningar. Genom att automatisera onboardingen och låta servern sköta utrullningen av certifikat och rättigheter, kan IT-avdelningen skifta fokus från brandsläckning till proaktiv arkitektur. Det är här nätverksadministrationen på lager 4 och uppåt verkligen visar sitt värde: när infrastrukturen är så välbyggd att klienterna nästan "sköter sig själva" genom de policys som administratören har definierat i molnet.

MDM - Mobile Device Management: Grundläggande fjärradministration av hårdvaruinställningar, säkerhetskrav och enhetsstatus för mobila enheter och datorer.

EMM - Enterprise Mobility Management: En vidareutveckling av MDM som även inkluderar hantering av applikationer, innehåll och identiteter i en mobil miljö.

UEM - Unified Endpoint Management: En modern och enhetlig plattform för att administrera samtliga typer av klienter – mobila, stationära och virtuella – från ett och samma system.

BYOD - Bring Your Own Device: Ett upplägg där användare använder sina privata enheter i arbetet, vilket kräver tekniska lösningar för att separera privat data från organisationens resurser.

3.7.1 Vad är MDM – och varför spelar det roll?

MDM (*Mobile/Modern Device Management*) representerar idén om att styra klienternas beteende och säkerhet centralt, oavsett var de befinner sig fysiskt. Där den traditionella modellen med Active Directory och gruppprinciper (*group policy*, GPO) utgår från att datorn befinner sig "på plats" i det lokala nätverket, bygger MDM på en **internet-first**-filosofi. Det innebär att enheten kan stå var som helst – hemma, i klassrummet eller på ett café – och ändå få rätt inställningar över internet via säkra push-kanaler. Systemet fungerar därmed oberoende av plats och lokala VLAN-strukturer.

Det viktiga för administratörer att förstå är skiftet mot **mätbara tillstånd** (measurable states). Istället för att bara anta att en policy har rullats ut, kontrollerar systemet löpande om specifika krav är uppfyllda, exempelvis om hårddiskkryptering (encryption) är aktiv eller om brandväggen (firewall) blockerar obehörig trafik. Denna logik kallas **efterlevnad** (compliance).

Genom att definiera tydliga krav skapas en **strategisk säkerhet** där enhetens eget tillstånd avgör dess rättigheter i nätverket. Om kraven inte uppfylls begränsas åtkomsten automatiskt tills felet är åtgärdat. På så sätt blir säkerhet en egenskap hos klientens faktiska status, snarare än en fråga om vilket fysiskt nätverk man är ansluten till.

Exempel: En lärardator som uppfyller kraven för kryptering och patchnivå får full åtkomst till skolans resurser. Om användaren inaktiverar brandväggen flaggas enheten som "non-compliant" och nekas åtkomst till e-post tills säkerheten är återställd.

3.7.2 Ekosystemet: Intune, Jamf, Google och Samsung

I valet av plattform för hantering av slutpunkter (endpoint management) domineras marknaden av ett fåtal stora aktörer som specialiserat sig på sina respektive ekosystem. Microsoft Intune är den mest utbredda lösningen för Windows-miljöer och fungerar ofta som en central hubb för att administrera en blandad vagnpark av enheter. För organisationer med ett starkt fokus på Apple är Jamf branschstandard, då plattformen erbjuder en djupgående styrning av macOS och iOS genom specifika konfigurationsprofiler (configuration profiles). Inom utbildningssektorn spelar även Google Endpoint Management, som är en integrerad del av Google Workspace, en avgörande roll för effektiv hantering av Chromebooks och Android-enheter.

Microsoft Intune

Microsoft Intune fungerar som en molnbaserad (cloud-based) hörnsten i Microsofts ekosystem för slutpunktshantering (unified endpoint management, UEM). Verktuget bygger på en direkt integration med Entra ID (tidigare Azure AD) och hanterar främst Windows-enheter, men har även ett brett stöd för iOS/iPadOS, macOS och Android. Den största fördelen är den sömlösa integrationen med Microsoft 365-licenser och funktioner som Autopilot för nollberöringsdistribution (zero-touch

deployment). Möjligheterna är nästintill obegränsade för Windows-miljöer, men en begränsning är att plattformen kan upplevas som komplex att konfigurera på grund av sitt enorma regelverk. Intune är en ren molntjänst, även om den kan samverka med lokala miljöer (on-premise) genom så kallad co-management med Configuration Manager (MECM).

Jamf

Jamf är den erkända branschstandarden när det kommer till dedikerad hantering av Apples ekosystem, inklusive macOS, iOS, iPadOS och tvOS. Verktöget arbetar genom en kombination av Apples inbyggda MDM-ramverk och en egenutvecklad mjukvaruagent (binary agent) som ger en djupare kontroll än vad som är möjligt med generella verktyg. Fördelen är den extremt snabba supporten för nya operativsystemversioner (day-zero support) och möjligheten till detaljerad skriptning på Mac-datorer. Begränsningen ligger i att det är en renodlad Apple-plattform som inte hanterar Windows eller Android. Jamf finns både som en molntjänst (Jamf Cloud) och som en lokal installation (on-premise), vilket ger flexibilitet för organisationer med specifika krav på datalagring.

Google Endpoint Management

Inom Google Workspace-ekosystemet finns Google Endpoint Management, vilket är det primära verktyget för att styra Chromebooks (ChromeOS) och Android-enheter. Det fungerar genom policybaserade inställningar direkt i administratörskonsolen och är känt för sin enkelhet och snabba driftsättning. Fördelen är att det ofta ingår i befintliga skollicenser och kräver minimalt underhåll av infrastrukturen. Möjligheterna är stora för molnfokuserade skolor, men begränsningen blir tydlig när man behöver utföra avancerad konfiguration av Windows- eller Mac-datorer, där verktyget saknar det djup som Intune eller Jamf erbjuder. Detta är en renodlad molntjänst (pure cloud) som inte kräver någon lokal serverdrift.

Samsung Knox

Samsung Knox skiljer sig från mängden genom att vara en säkerhetsplattform som är inbyggd direkt i hårdvaran (hardware-backed security) på Samsungs Android-enheter. Själva hanteringsverktyget heter Knox Manage och utnyttjar denna hårdvarunära arkitektur för att ge administratören kontroll ända ner på chip-nivå, vilket skapar en mycket hög nivå av tillit (root of trust). Fördelarna är en extremt hög säkerhetsnivå och möjligheten till granulär kontroll över enhetens

komponenter, såsom kamera eller USB-portar. Den största begränsningen är att de mest avancerade funktionerna kräver Samsung-hårdvara för att fungera fullt ut. Knox Manage är en molnbaserad tjänst som ofta används som ett komplement till andra MDM-system för att säkra mobila flottor i känsliga miljöer.

3:e-part (exempelvis Miradore)

Tredjepartslösningar som Miradore fungerar som plattformsoberoende alternativ som ofta fokuserar på användarvänlighet och snabb onboarding för en blandad vagnpark av Windows, macOS, iOS och Android. Dessa verktyg arbetar genom att utnyttja operativsystemens standardiserade API:er för hantering (MDM protocol). En stor fördel är att de ofta erbjuder en gratisversion med grundläggande funktioner, vilket gör dem till utmärkta verktyg för mindre organisationer eller för laborationsmiljöer. Möjligheterna inkluderar enkel inventering och distribution av grundläggande policyer, men begränsningen ligger i att de saknar de djupa integrationer med identitetstjänster (identity providers) som de plattformsspecifika jättarna erbjuder. Miradore är främst en molnbaserad lösning, vilket minimerar behovet av lokal infrastruktur vid uppstart.

Sammanfattning av ekosystemet

Valet av verktyg handlar i slutändan om att matcha organisationens behov av kontroll mot den befintliga infrastrukturen. Microsoft Intune dominerar för de som lever i Windows-världen, medan Jamf är ohotad för de som kräver fullständig kontroll över Mac och iPad. Google erbjuder den enklaste vägen för molnfokuserade miljöer, och Samsung Knox ger den högsta säkerheten för mobila Android-enheter. Tredjepartslösningar som Miradore fyller en viktig funktion som en tillgänglig brygga för de som behöver hantera flera olika plattformar utan att låsa sig till en specifik tillverkare eller komplex licensmodell.

Exempel: En kommun kör Intune för Windows-parken och låter Jamf sköta Mac-datorer i estetiska programmet. Båda plattformarna rapporterar efterlevnad till samma identitetstjänst, så åtkomstregler blir konsekventa.

3.7.3 Centralt styrda policyer och säkerhetsinställningar

När en enhet väl är inrullad i ett MDM-system börjar det verkliga arbetet med att styra dess beteende och säkerhet. Detta görs främst genom

konfigurationsprofiler, som fungerar som digitala regelverk som skickas ut till enheterna. En profil kan innehålla allt från enkla inställningar, som vilket bakgrundsbild som ska användas, till kritiska säkerhetskrav som att hårddisken måste vara krypterad eller att kameran ska vara avstängd. Istället för att en tekniker går runt och klickar manuellt på varje dator, "pushas" dessa profiler ut över internet, vilket garanterar att hela vagnparken har exakt samma inställningar inom några sekunder. Exempel på vad konfigurationsprofiler kan innehålla

- Skärmlås
- Lösenordspolicy
- Brandvägg
- Kryptering (BitLocker/FileVault)
- USB-policy (får du sätta in okända usb-enheter)
- WLAN-profiler (SSID, 802.1X-certifikat)
- VPN-profiler
- Webbfilter.

Andra saker som styrs genom konfigurationsprofiler är apphanteringen. Via en MDM-plattform kan du enkelt paketera, distribuera, uppdatera och – vid behov – ta bort appar och operativsystemsuppdateringar. Det ger en möjlighet att kontrollera så att det är säkert och att operativsystem och appar fungerar ihop. I Applevärlden sköts detta via VPP (licenser) och i Android via Managed Google Play. Via konfigurationsprofiler kan du även styra hur användarna får sina program.

- Rättighet att installera själv
- Endast via Store/Play-butik
- Endast pushade ut centralt

En av de viktigaste delarna i en modern nätverksmiljö är att slippa hantera lösenord för trådlösa nätverk. Här kommer **certifikatlivscykeln** (certificate lifecycle) in i bilden. Genom tekniker som **SCEP** (Simple Certificate Enrollment Protocol) eller **PKCS** (Public Key Cryptography Standards) kan MDM-servern automatiskt ansöka om, installera och förnya digitala certifikat på varje klient. Dessa certifikat fungerar som enhetens digitala legitimation. När enheten sedan försöker ansluta till skolans Wi-Fi används **802.1X-autentisering**. Det innebär att nätverket känner igen certifikatet och släpper in enheten automatiskt utan att användaren behöver skriva in ett enda tecken. Det är både säkrare och

mer användarvänligt än traditionella lösenord som lätt kan spridas eller tappas bort.

För att knyta ihop säcken används **villkorad åtkomst** (conditional access). Detta är den intelligenta logik som bestämmer vem som får komma åt vad. Systemet ställer frågor i realtid: "Är användaren inloggad med rätt konto?", "Är enheten krypterad?" och "Finns de senaste säkerhetsuppdateringarna installerade?". Om svaret på någon av dessa frågor är nej, nekas åtkomst till känsliga tjänster som e-post eller molnlagring. På så sätt fungerar MDM-systemet som en dörrvakt som ser till att endast "friska" och godkända enheter får interagera med organisationens data, vilket dramatiskt minskar risken för att skadlig kod sprids i nätverket.

MDM uttrycker säkerhet som mätbara regler. Istället för "har vi rullat ut brandväggen?" frågar du "är brandväggen på och blockerar den inkommande trafik?". Därifrån följer ett antal byggstenar som återkommer oavsett OS: Det här låter stort, men effekten i vardagen är just enkelhet: du behöver inte längre fråga "var står datorn?", utan "uppfyller den kraven?".

Exempel: En elev försöker logga in på skolans portal från en privat dator utan antivirus. Tack vare **villkorad åtkomst** nekas inloggningen omedelbart, och eleven får ett meddelande om att enheten inte uppfyller säkerhetskraven för att hantera skolans data.

3.7.4 Onboarding och teknisk setup: AD, länkar och inrullning

För att en MDM-server ska kunna börja styra enheter krävs en initial teknisk setup som bygger på förtroende (trust). Det första steget är att etablera en säker kommunikationsväg mellan organisationens server och operativsystemens tillverkare. Detta görs genom att installera **push-certifikat** (push certificates), såsom Apples APNs-certifikat. Utan detta certifikat har servern ingen "nyckel" för att kunna skicka kommandon till enheterna över internet. När detta digitala handskak är genomfört kan själva processen med att få in enheterna i systemet börja, vilket kallas för **inrullning** (enrollment).

Det finns flera metoder för **onboarding** beroende på om enheten ägs av skolan eller är en privat enhet (BYOD). Valet av metod avgör hur mycket manuellt arbete som krävs av både tekniker och användare:

- **AD-koppling:** Genom att synkronisera MDM-plattformen med organisationens identitetstjänst, som Active Directory (AD) eller Entra ID, kan användaren logga in med sitt vanliga skolkonto på enheten. Inloggningen triggas då automatiskt inrullningen och hämtar rätt policyer baserat på användarens roll.
- **Länk eller QR-kod:** För enheter som inte är förregistrerade kan administratören skicka ut en unik inrullningslänk (enrollment link) via e-post eller visa en QR-kod. När användaren klickar på länken eller skannar koden laddas hanteringsprofilen ner och enheten ansluts till servern.
- **Zero-touch:** Detta är den mest automatiserade metoden och kallas för **Windows Autopilot** hos Microsoft eller **Automated Device Enrollment (ADE)** hos Apple. Här är enhetens serienummer redan registrerat hos tillverkaren och kopplat till skolans MDM-server.

Vid zero-touch behöver IT-avdelningen aldrig röra enheten fysiskt. Så fort användaren packar upp datorn ur kartongen och ansluter till ett nätverk känner enheten igen att den tillhör skolan. Den kontaktar då automatiskt MDM-servern och påbörjar sin konfiguration. Detta minskar risken för fel och säkerställer att alla enheter är skyddade från första sekunden de används.

Exempel: En skola köper 100 nya iPads. Genom ADE (zero-touch) behöver teknikern bara registrera ordernumret i portalen. När eleverna startar sina iPads hemma, laddas skolans inställningar och appar ner automatiskt utan att någon lärare eller tekniker behövt hjälpa till.

Push-certifikat: Ett digitalt certifikat som skapar en betrodd länk mellan MDM-servern och tillverkaren (t.ex. Apple), vilket krävs för att kunna skicka instruktioner till enheterna.

Inrullning (enrollment): Den tekniska processen där en enhet registreras i MDM-systemet och får sin första uppsättning policyer och konfigurationer.

Zero-touch: En metod där enheter förkonfigureras i molnet så att de automatiskt ansluter till organisationens MDM-server direkt vid första uppstarten, utan manuell hantering.

3.7.5 Mobilitet, BYOD och den personliga integriteten

Begreppet **BYOD** (Bring Your Own Device) innebär att personal eller elever använder sina egna privata mobiler eller datorer för att komma åt skolans nätverksresurser. För en nätverksadministratör innebär detta en utmaning i att balansera organisationens krav på säkerhet mot användarens rätt till personlig integritet (privacy). Lösningen på denna utmaning är teknisk **dataseparering**, där man skapar en tydlig gräns mellan vad som tillhör skolan och vad som är privat på samma fysiska enhet.

På moderna mobila operativsystem uppnås detta genom att skapa en **arbetsprofil** (work profile), vilket ofta liknas vid en krypterad behållare (container). Inuti denna profil bor skolans e-post, filer och appar som administratören har full kontroll över. Utanför profilen lever användarens privata liv med foton, meddelanden och sociala medier. Tack vare denna arkitektur kan MDM-servern styra säkerheten i arbetsprofilen utan att tekniskt kunna se eller komma åt användarens privata innehåll, vilket bygger det förtroende som krävs för att BYOD ska fungera i praktiken.

När en person slutar eller om en privat enhet tappas bort, använder administratören funktionen **selektiv rensning** (selective wipe). Till skillnad från en fullständig fabriksåterställning (full wipe) skickar servern här ett kommando som endast raderar arbetsprofilen och dess tillhörande data. Detta innebär att skolans känsliga information försvinner omedelbart, medan användarens privata semesterbilder och appar förblir helt orörda. Detta gör att "skolan" kan flytta ut från enheten utan att lämna några spår efter sig eller skada användarens egna filer.

Exempel: En vikarie använder sin privata Android-telefon för att nå lärarportalen via en arbetsprofil. När uppdraget tar slut utförs en selektiv rensning som raderar portal-appen och skolans dokument, men lämnar vikariens privata appar och bilder helt intakta.

Begreppsförklaringar

BYOD (Bring Your Own Device): Ett upplägg där användare använder sin privata utrustning i tjänsten, vilket kräver lösningar för att säkra organisationens data utan att kränka integriteten.

Arbetsprofil (work profile/container): En isolerad och krypterad del på en enhet där verksamhetens appar och data hanteras separat från användarens privata innehåll.

Selektiv rensning (selective wipe): Ett kommando från MDM-servern som endast raderar organisationens data och konfigurationer från en enhet utan att påverka användarens privata filer.

3.7.6 Arketyper och Windows Servers roll som nav

I de flesta miljöer fungerar Windows Server med Active Directory (AD DS) som ett nav för identitet, policy och infrastruktur. Det beror både på klientflottans sammansättning (ofta Windows) och på att AD samlar autentisering (Kerberos/NTLM), grupp- och rättighetsmodeller, DNS/DHCP och gruppolicy (GPO) i en sammanhängande helhet. Alltifrån skrivare och filresurser till inloggningsupplevelse kan därför kopplas till samma identitetskälla. Under senare år har många organisationer gått mot hybrida upplägg där ett lokalt AD samverkar med molnidentitet och MDM (t.ex. Entra ID/Intune), så att traditionella GPO:er och moderna, internetförsta policyer kan samexistera.

Ofta inleder man med att ha en Hybrid-AD, en arkitektur där den lokala katalogtjänsten Active Directory (AD DS) synkroniseras med en molnbaserad identitetstjänst, såsom Entra ID. Detta skapar en gemensam hubb för inloggning (identity hub) där användaren har samma användarnamn och lösenord oavsett om hen loggar in på en lokal serverresurs eller en molntjänst.

Exempel: Vid uppsättning av Windows-miljön installeras en server med verktyget Microsoft Entra Connect. Denna läser av de lokala elevkontona från Windows Server och "speglar" upp dem till molnet. Det gör att när en elev startar sin laptop, som vi tidigare nämnde rullas ut via Autopilot, kan hen logga in med sina vanliga uppgifter och omedelbart bli igenkänd av MDM-systemet.

Hur reglerna senare ställs upp anpassas ofta av vart enheten befinner sig. GPO är byggt för enheter som är fast anslutna till den lokala domänen (on-premise), medan MDM är optimerat för internetanslutna enheter som rör sig fritt (modern management).

Exempel: I skolans fasta datasal för CAD eller programmering används GPO för att snabbt mappa upp lokala nätverksenheter och styra inställningar på lager 2 och 3. För bärbara lärardatorer, som vi i kapitel 3.7.3 styrde med konfigurationsprofiler, används istället MDM. Detta säkerställer att säkerhetsreglerna (compliance) följer med datorn även när läraren jobbar hemifrån via sitt privata Wi-Fi.

Även om Windows oftast utgör grunden i en nätverksmiljö fyller alla noder sin funktion. Det är alltid bättre att ha flera specialiserade enheter än bara en central. Linux-serverar används ofta som specialiserade noder för att hantera specifika nätverkstjänster som kräver hög stabilitet och öppna standarder. De fungerar som "arbetshästar" som levererar kritiska funktioner till både Windows- och Apple-klienter. De används ofta för filer och utskrifter (Samba/CUPS), webb och applikationer (NGINX/Apache, applikationsramverk), infrastrukturtjänster (reverse proxy, cache/proxy, databaser, RADIUS, syslog), automatisering (t.ex. Ansible) och containrar (Docker/Kubernetes). Linux kan även bära identitetstjänster (t.ex. Samba-AD-DC eller FreeIPA/IdM) när arkitekturen kräver det

Exempel: För att nätverket ska kunna hantera **802.1X-autentisering**, sätts en Linux-server upp med **FreeRADIUS**. Denna Linux-server fungerar som en mellanhand: när en iPad (hanterad i Jamf) försöker ansluta till Wi-Fi med sitt certifikat, frågar Linux-servern Windows-serverns Hybrid-AD om enheten är godkänd. Om svaret är ja, släpper Linux-servern in enheten på nätverket.

Förmågan hos olika tekniska system att samarbeta sömlöst genom att använda gemensamma protokoll (t.ex. LDAP, Kerberos eller SMB) kallas interoperabilitet. Det är detta som gör att en nätverksmiljö med både Windows, Linux och Apple-enheter kan fungera som en enda enhetlig infrastruktur.

Där Windows-klienter dominerar är AD ofta fortsatt den primära identitetskällan, medan Linux fungerar som stabila tjänstenoder runt navet. Valet handlar mindre om "bäst i test" och mer om arkitektur: AD där sammanhållen klientidentitet och policystyrning är centralt; Linux där

öppna, skalbara servertjänster och plattformar behövs – med interoperabilitet via LDAP/Kerberos/SMB som gör att helheten sitter ihop.

Exempel: En Linux-server kör **Samba**. Denna Linux-nod blir en del av Windows-domänen, vilket gör att en elev på en Mac-dator (inrullad via en länk) kan spara sina filer direkt på Linux-servers lagringsyta med sitt vanliga AD-konto. Allt samverkar: inloggningen sker mot Windows, lagringen sker på Linux och klienten styrs av MDM.

Kontrollfrågor

1. Vad är den fundamentala skillnaden i filosofi mellan att hantera en enhet via gruppprinciper (group policy, GPO) och via modern klienthantering (MDM)?
2. Varför är begreppet efterlevnad (compliance) så centralt när man pratar om säkerhet på lager 4 och uppåt?
3. Förklara hur ett push-certifikat (push certificate) fungerar som en teknisk förutsättning för att MDM-servern ska kunna kommunicera med en enhet.
4. Vilka är de praktiska fördelarna med nollberöringsdistribution (zero-touch deployment) jämfört med att manuellt installera operativsystemet (imaging)?
5. Hur skyddar en arbetsprofil (work profile) användarens personliga integritet i ett BYOD-scenario?
6. Vad innebär det tekniskt att utföra en selektiv rensning (selective wipe) på en enhet, och när är det att föredra framför en fullständig fabriksåterställning?
7. Beskriv rollen för Microsoft Entra Connect i en hybrid arkitektur (hybrid setup).
8. Varför väljer man ofta att sätta upp en Linux-server som en tjänstenod (service node) för RADIUS istället för att köra tjänsten direkt på en domänkontrollant?
9. Ge ett exempel på hur interoperabilitet (interoperability) mellan Windows, Linux och Apple-enheter underlättar vardagen för en användare.
10. I vilka situationer är en tredjepartslösning (3rd-party solution) som Miradore ett bra alternativ till de större plattformarna som Intune eller Jamf?

Fördjupningslänkar

Microsoft Intune Fundamentals (Hantering av enheter, appar och säkerhetspolicyer i molnet) -

<https://learn.microsoft.com/en-us/mem/intune/fundamentals/what-is-intune>

Apple Platform Deployment Guide (Teknisk dokumentation för MDM, ADE och VPP i Apples ekosystem) -

<https://support.apple.com/guide/deployment/welcome/web>

Google Android Enterprise (Hantering av enheter och arbetsprofiler i Android-miljöer) - <https://www.android.com/enterprise/management/>

Samsung Knox for Enterprise (Djupdykning i hårdvarunära skydd (hardware-backed security) för Android-enheter) -

<https://www.samsungknox.com/en/solutions/it-solutions/knox-manage>

Miradore Knowledge Base (Praktiska guider för molnbaserad MDM på flera olika operativsystem) - <https://www.miradore.com/knowledge-base/>

Windows Autopilot Overview (Flödet från fabriksbeställning till färdigkonfigurerad Windows-klient) -

<https://learn.microsoft.com/en-us/mem/autopilot/windows-autopilot>

FreeRADIUS Project (Dokumentation för RADIUS-server och 802.1X-autentisering (authentication)) - <https://freeradius.org/documentation/>

NIST Special Publication 800-124 (Säkerhetsstandarder för hantering av mobila enheter i organisationer) -

<https://csrc.nist.gov/publications/detail/sp/800-124/rev-2/final>

Microsoft Entra Connect (Synkronisering av lokala identiteter till molnet för hybrid-AD) -

<https://learn.microsoft.com/en-us/entra/identity/hybrid/connect/whatis-azure-ad-connect>

Samba Wiki - Domain Member (Integration av Linux-servrar i en Windows-domän för hög interoperabilitet) -

https://wiki.samba.org/index.php/Setting_up_Samba_as_a_Domain_Member

Egna anteckningar

...

4 Säkerhet och övervakning

4.1 Vad är säkerhet

När vi pratar om nätverkssäkerhet (network security) hamnar fokus ofta på dramatiska hackerattacker och sofistikerade intrång. Men för en nätverksadministratör (network administrator) är begreppet betydligt bredare än så. Säkerhet handlar lika mycket om att skydda data från obehöriga ögon som att se till att systemen faktiskt är i drift när de behövs. Ett nätverk som ligger nere på grund av ett tekniskt fel, en strömspik eller en felkonfiguration är, ur användarens perspektiv, precis lika "trasigt" som ett nätverk som blivit kapat. I det här kapitlet ska vi därför utforska hur vi bygger en miljö som är både motståndskraftig mot hot och robust nog att garantera hög tillgänglighet (availability). Vi går från de strategiska principerna för försvar till den praktiska övervakningen som gör att vi kan upptäcka fel innan användarna ens märker att något har hänt.

4.1.1 Säkerhetsprinciper och CIA-modellen

Nätverkssäkerhet (network security) handlar i grunden om att minska konsekvenserna när något går fel, snarare än att bara hoppas på att ingenting någonsin händer. För att navigera i detta arbete används ofta **CIA-triaden** som en kompass. Triaden beskriver tre huvudmål som ramar in hela säkerhetsarbetet: konfidentialitet (confidentiality), integritet (integrity) och tillgänglighet (availability). Varje teknisk kontroll eller policy som införs i nätverket bör kunna motiveras utifrån vilket av dessa tre mål den stöttar.

Konfidentialitet handlar om att säkerställa att endast rätt mottagare kan ta del av informationen. I nätverket uppnås detta genom kryptering (encryption) och strikt behörighetshantering.

Integritet innebär att data är tillförlitlig och oförändrad från det att den skickas till att den tas emot, vilket skyddas genom tekniker som digitala signaturer och kontrollsummor (checksums).

Tillgänglighet är det mål som oftast kopplas till den dagliga driften; det innebär att tjänsterna fungerar och är nåbara när de behövs. Här blir redundans, kapacitetsplanering och skydd mot överbelastningsattacker (DDoS) helt avgörande för att nätverket ska anses vara säkert.

För att uppnå säkerhetsmålen i praktiken används principen om **försvar i flera lager** (defense in depth). Grundtanken är att inget enskilt säkerhetssystem är perfekt, och genom att stapla olika typer av skydd skapas en struktur där ett enskilt fel – som en sårbar klient, ett läckt lösenord eller en missad uppdatering – inte automatiskt leder till ett totalt haveri eller dataintrång. Man kan likna arkitekturen vid ett medeltida slott med vallgrav, yttre murar och ett inre torn; om angriparen tar sig över vallgraven stoppas hen av nästa barriär.

I en nätverksmiljö innebär detta att vi kombinerar fysiskt skydd av serverrum med nätverkssegmentering (segmentation) via VLAN, brandväggar (firewalls), krypterad kommunikation och klientskydd (endpoint protection). Varje lager fungerar som en oberoende kontrollpunkt som tvingar ett hot att forcera flera hinder, vilket ger administratören värdefull tid att upptäcka och isolera händelsen innan den når den mest känsliga informationen.

Ett exempel på detta är om en användare råkar klicka på en skadlig länk i en e-post. Även om klientskyddet missar koden, blockerar nätverkets brandvägg anslutningen till angriparens server, och segmenteringen hindrar koden från att sprida sig sidleds till andra servrar i nätverket.

Detta arbete kompletteras av principen om **minsta möjliga rättighet** (least privilege). Det är en fundamental säkerhetsmodell som innebär att en användare, ett program eller en systemtjänst aldrig ges mer befogenheter i nätverket än vad som är absolut nödvändigt för att utföra sin specifika uppgift. Syftet är att minimera den potentiella skadan vid ett fel eller ett angrepp, något som ofta beskrivs som att begränsa "sprängraden".

Genom att tillämpa strikt åtkomstkontroll (access control) säkerställer administratören att rättigheter delas ut baserat på funktion snarare än personlig status. Om ett konto med begränsade behörigheter skulle bli komprometterat (compromised), blir konsekvenserna för hela organisationen hanterbara eftersom kontot saknar teknisk möjlighet att

utföra administrativa ändringar eller komma åt skyddad information i andra delar av infrastrukturen. Detta kräver en noggrann genomgång av både användarkonton och de tjänstekonton som driver nätverkets olika applikationer.

Om ett tjänstekonto skapas för att en nätverksskrivare ska kunna spara skannade filer på en server. Kontot ges endast skrivrättigheter till en specifik mapp och nekas helt möjligheten att logga in på andra servrar eller läsa filer i andra personers hemkataloger.

Förtydligande (sammanfattning):

- *Lager:* små fel blir inte stora.
- *Rättigheter:* konton/tjänster har bara det de behöver.
- *Synlighet:* utan logg/larm finns ingen säkerhet i praktiken.
- *Prioritering:* segmentering, patchning, stark identitet ger hög "träff per timme".

Exempel: Elevklienter och serversystem ligger i olika VLAN. Även om en elevdator infekteras kan skadlig kod inte prata direkt med servern – brandvägg och routingregler står i vägen. Händelsen syns i loggarna och kan hanteras innan den sprider sig.

4.1.2 Brandväggar och modern hotdetektering

En brandvägg (firewall) fungerar som nätverkets dörrvakt. Dess främsta uppgift är att uttrycka och upprätthålla regler för vilken trafik som får passera mellan olika zoner, exempelvis mellan internet (untrusted) och det interna nätverket (trusted). All trafik som inte uttryckligen är tillåten nekas som standard, en princip som kallas "default deny".

Den mest grundläggande formen av modern brandväggsteknik är **tillståndsbaserad paketfiltrering** (stateful inspection). Den arbetar främst på lager 3 och 4 i OSI-modellen och håller reda på anslutningars tillstånd genom att analysera käll- och mål-IP, portar och protokoll. Genom att förstå om ett paket är en del av en redan etablerad session (exempelvis ett svar på en förfrågan inifrån nätverket) kan brandväggen fatta snabba beslut utan att behöva analysera varje paket isolerat. Detta är mycket effektivt för att hantera kända flöden men räcker inte alltid till

när moderna applikationer döljer sin trafik bakom vanliga portar som TCP/443 (HTTPS).

Här kommer **nästa generations brandväggar** (Next-Generation Firewall, NGFW) in i bilden. En NGFW tar steget upp till lager 7, applikationslagret, vilket ger den förmågan att identifiera faktiska applikationer oavsett vilka portar de använder. Istället för att bara se "trafik på port 443" kan en NGFW se att det rör sig om specifikt Microsoft Update, Facebook eller en osäker fildelningstjänst. Denna **applikationskontroll** (application control) gör det möjligt att skapa regler som är mycket mer träffsäkra och anpassade efter verksamhetens behov.

När det gäller hårdvara och implementation finns det två huvudsakliga vägar att gå. **Ciscos fysiska brandväggar** (exempelvis Firepower-serien) är ofta byggda med dedikerad hårdvara och specialiserade kretsar kallade **ASIC** (Application-Specific Integrated Circuit). Dessa är optimerade för att hantera enorma mängder trafik med extremt låg fördröjning (latency), vilket gör dem lämpliga för datacenter och stora företagsnätverk där prestanda är kritiskt. Å andra sidan har vi mjukvarubaserade lösningar som **OPNsense**. Det är en kraftfull, öppen källkodsbaserad router och brandvägg som körs på vanlig processorhårdvara (x86). OPNsense erbjuder en enorm flexibilitet och är ett fantastiskt verktyg för att lära sig avancerad nätverksteknik, då den ger administratören full tillgång till alla inställningar och moduler utan dyra licenskostnader.

För att förstå skillnaden mellan att bara upptäcka ett hot och att faktiskt stoppa det behöver vi titta närmare på hur dessa system arbetar och var i nätverkshierarkin de placeras. Medan en brandvägg (firewall) främst tittar på "kuvertet" (IP och portar), läser dessa system innehållet i "brevet" för att hitta skadliga mönster.

IDS och IPS

Ett **IDS** fungerar som en passiv observatör eller ett digitalt inbrottslarm. Det placeras oftast "out-of-band", vilket innebär att trafiken inte passerar fysiskt genom systemet. Istället skickas en kopia av trafiken till IDS-noden via en så kallad **SPAN-port** (Switched Port Analyzer) eller en nätverks-**TAP** (Test Access Point). Fördelen med denna placering är att systemet inte påverkar nätverkets prestanda eller fördröjning (latency), och om IDS-

noden skulle krascha fortsätter nätverkstrafiken att flöda som vanligt. Nackdelen är att systemet endast kan varna (larm) efter att ett misstänkt paket redan har passerat in i nätverket. Det är ett utmärkt verktyg för att få djup synlighet (visibility) utan att riskera nätverkets tillgänglighet.

Vi skiljer på två typer av IDS, dels NIDS (Network Intrusion Detection System) vilket är den typ vi precis pratat om. Vi kan också ha en så kallad **HIDS** (Host Intrusion Detection System). Det sitter istället direkt på en enskild server eller klient. Det övervakar vad som händer *inuti* operativsystemet, såsom ändringar i systemfiler, onormala processer eller inloggningsförsök som inte syns på nätverket (exempelvis på grund av kryptering).

Ett **IPS** är den aktiva vidareutvecklingen av ett IDS. Det placeras "inline", vilket betyder att all trafik fysiskt måste passera genom IPS-motorn innan den når sin destination. Detta gör det möjligt för systemet att blockera eller "droppa" skadliga paket i realtid innan de hinner göra skada. Denna placering ger ett betydligt starkare skydd, men ställer också mycket högre krav på hårdvarans prestanda. Om en IPS-motor blir överbelastad kan den bli en flaskhals som saktar ner hela nätverket, och vid ett tekniskt fel kan den i värsta fall bryta hela internetanslutningen.

Var man placerar dessa sensorer beror på vad man vill skydda. Placering vid nätverksskanten, så kallad **North-South**-trafik, fokuserar på att stoppa angrepp som kommer utifrån internet. En sensor som placeras mellan interna nätverkssegment (VLAN), så kallad **East-West**-trafik, har istället som uppgift att upptäcka om en angripare eller skadlig kod försöker sprida sig sidleds inuti organisationen. Att placera en sensor innanför brandväggen är ofta mest effektivt, då den slipper analysera all den trafik som brandväggen ändå skulle ha blockerat, vilket sparar på systemresurserna.

I modern nätverksadministration är gränsen mellan brandvägg och IDS/IPS ofta flytande. **Ciscos Firepower**-serie är ett exempel på en dedikerad hårdvarulösning där IPS-motorn är djupt integrerad i brandväggens operativsystem för att med hög hastighet inspektera trafik på lager 7. På mjukvarusidan är **Snort** och **Suricata** de två mest dominerande motorerna. Snort är den klassiska standarden, medan Suricata är känd för att vara effektivare på modern hårdvara genom att använda flera processorkärnor samtidigt (multithreading). I system som **OPNsense** kan

Suricata enkelt aktiveras som en tjänst, vilket gör att en vanlig PC kan förvandlas till en kraftfull IPS-nod.

Förtydligande (sammanfattning):

- *NIDS/HIDS*: nät- respektive värd-perspektiv; kompletterar varandra.
- *Signatur vs anomali*: känt-snabbt kontra okänt-upptäckbart.
- *Inline vs out-of-band*: blockera vs larma.
- *Samspelet*: NGFW-IPS stoppar vid kanten; NIDS/HIDS ger djup och kontext internt.

Exempel: En serverzon visar plötsliga utgående anslutningar mot ovanliga mål. NIDS (via TAP) larmar för "rare outbound" och HIDS på samma server rapporterar nyskapade processer i ett ovanligt katalogträd. Brandväggen sätter en tillfällig block på utgående trafik från den servern (hög-tillitsregel i IPS-läget), och korrelation med AD-loggar visar ett läckt tjänstkonto som spärras och roteras.

4.1.3 Enhetssäkring och Management Plane

För att ett nätverk ska förbli säkert och tillgängligt räcker det inte med att bara bevaka trafiken som flödar genom det; vi måste även säkra själva "maskinrummet". **Management Plane** är den administrativa trafiknivå som används för att konfigurera, övervaka och styra nätverksenheter. Om en obehörig får åtkomst till detta plan är nätverket i praktiken vidöppet. Enhetssäkring, eller **hardening** (härdning), handlar om att systematiskt stänga ner onödiga ingångar och förstärka de skydd som finns kvar. Att stänga av portar på en switch, eller router, som inte används är en viktig grundregel att utgå ifrån. Ju fler anslutningsmöjligheter desto större attackyta. Förutom det rent fysiska, men som ändå är grundläggande finns ett antal andra viktiga saker att göra.

Att styra en router eller switch på distans är en nödvändighet, men valet av protokoll avgör om administratörens lösenord är säkra eller inte. Tidigare användes ofta **Telnet**, men eftersom det skickar all data i klartext kan vem som helst på nätverket "lyssna" på trafiken och fånga upp inloggningsuppgifter. Branschstandard är istället **SSH** (Secure Shell), vilket skapar en krypterad tunnel mellan administratörens dator och nätverksenheten. Nyttan är dubbel: konfidentialitet (ingen kan läsa

lösenorden) och integritet (ingen kan ändra kommandona under överföringen).

För att sätta upp SSH på en Cisco-enhet krävs ett unikt värddamn (hostname) och ett domännamn (domain-name) för att generera de kryptografiska nycklarna.

```
Router(config)# ip domain-name skolan.se
Router(config)# crypto key generate rsa # Skapar de
digitala nycklarna
Router(config)# line vty 0 4 # Går in i de
virtuella terminalerna
Router(config-line)# transport input ssh # Tillåter ENDAST
SSH, nekar Telnet
```

Ovan är ett exempel på härdning av en router, som automatiskt nekar Telnet, endast krypterad trafik via port 22 (SSH) tillåts, vilket innebär att även om någon avlyssnar nätverket ser de bara oläsligt brus istället för administratörens kommandon.

En grundregel inom säkerhet är att en tjänst som inte används inte heller kan attackeras. Många nätverksenheter levereras med bekvämlighetsfunktioner aktiverade, såsom webbgränssnitt (HTTP) eller automatiska upptäcktsprotokoll, som utgör en onödig risk. Genom att utföra en **hardening** stänger vi dessa digitala bakdörrar.

Lika viktigt för säkerheten är nätverkets förmåga att stå emot fysiska fel, vilket vi kallar för **redundans** (redundancy). Ett nätverk som går ner på grund av en trasig kabel är ett nätverk som har förlorat sin tillgänglighet (availability). I Cisco-miljöer löser vi ofta detta genom **EtherChannel** (LACP). Genom att bunta ihop flera fysiska kablar till en logisk länk får vi både högre bandbredd och automatisk felsäkring; om en kabel går av fortsätter trafiken flöda på de resterande utan avbrott.

```
Switch(config)# interface range gi0/1 - 2
Switch(config-if-range)# channel-group 1 mode active #
Aktiverar LACP
```

Effekt och resultat: Switchen ser nu två fysiska kablar som en enda logisk länk (Port-Channel). Om en elev skulle råka dra ut den ena kabeln kommer nätverket inte att gå ner, och administratören får tid att åtgärda felet under pågående drift.

För att veta hur nätverket mår i realtid använder vi **SNMP** (Simple Network Management Protocol). Protokollet fungerar så att en central övervakningsserver (manager) ställer frågor till nätverkets enheter (agents) om deras aktuella status. Äldre versioner som v1 och v2c skickar sina lösenord, så kallade "community strings", i klartext. Det innebär att en angripare som avlyssnar trafiken enkelt kan ta reda på nätverkets struktur eller till och med ändra inställningar. Genom att implementera **SNMPv3** inför vi stark autentisering (authentication) och kryptering (privacy), vilket gör telemetrin oläslig för obehöriga.

Nytan med SNMPv3 är att vi kan samla in specifik data som direkt påverkar nätverkets tillgänglighet (availability). Vi övervakar exempelvis enhetens **processorlast** (CPU utilization) och **minnesanvändning** (RAM usage) för att upptäcka om en router är nära att krascha på grund av överbelastning. Vi mäter även **bandbreddsnyttjande** (bandwidth utilization) på varje gränssnitt för att se vilka länkar som riskerar att bli flaskhalsar. Särskilt viktigt för nätverksteknikern är att läsa av **felräknare** (error counters) på portarna. Om antalet felaktiga paket (input errors/CRC errors) ökar på en länk, är det ett tidigt tecken på en trasig kabel eller elektromagnetiska störningar som måste åtgärdas innan anslutningen dör helt. Vi kan även hämta miljödata som **temperatur** och fläktstatus för att säkra att hårdvaran i serverrummet inte tar skada av värme.

För att sätta upp en säker SNMPv3-miljö på en Cisco-enhet skapar vi en grupp med högsta säkerhetsnivå (authPriv) och kopplar en användare till den med starka algoritmer som SHA för lösenord och AES för kryptering.

```
# Skapar en grupp med krav på både lösenord och kryptering
Router(config)# snmp-server group G1 v3 priv

# Skapar användaren 'admin' med lösenord och
krypteringsnyckel

Router(config)# snmp-server user admin G1 v3 auth sha
MyPassword123 priv aes 128 MyCryptoKey456
```

Övervakningssystemet kan nu hämta detaljerad statistik om allt från portstatus till temperatur. Om någon försöker tjuvlyssna på nätverket ser de bara krypterat dataflöde. Resultatet är en proaktiv drift där administratören kan byta ut en sviktande kabel eller fläkt innan ett avbrott ens hinner uppstå, samtidigt som nätverkets konfiguration förblir hemlig.

I en mjukvarubaserad router som **OPNsense** är säkerhetstänket inbyggt från start, men det kräver medvetna val av administratören. Till skillnad från en vanlig hemrouter kommer OPNsense ofta med en "default deny"-policy på alla gränssnitt förutom det lokala (LAN). En viktig del av hardening-processen här är att aldrig använda "allow all"-regler. Istället bör administratören definiera exakt vilka IP-adresser och portar som får kommunicera.

En annan kritisk punkt i OPNsense är att begränsa åtkomsten till webbgränssnittet (Web GUI). Genom att ändra standardporten och endast tillåta inloggning från ett specifikt administratörs-VLAN (management VLAN) minskar risken för att en infekterad klient på det vanliga elenvätverket ska kunna försöka logga in i routern. Man bör även se till att tjänster som SSH på routern kräver certifikat eller starka lösenord och att alla oanvända insticksprogram (plugins) avinstalleras för att hålla systemet så rent och snabbt som möjligt.

När den ordinarie mjukvaran eller nätverkskonfigurationen sviker, behövs en sista livlina för att garantera tillgängligheten (availability). Detta kallas för **Out-of-Band (OOB) management**. De flesta professionella servrar är utrustade med en dedikerad hanteringsprocessor (Baseboard Management Controller, BMC) som fungerar oberoende av serverns huvudsakliga operativsystem. Hos Cisco kallas denna lösning för **CIMC** (Cisco Integrated Management Controller) och hos HP för **iLO** (Integrated Lights-Out).

Dessa system har egna fysiska nätverksportar och strömförsörjning, vilket gör att en administratör kan nå serverns hårdvara även om operativsystemet har kraschat eller om servern är helt avstängd. Via dessa gränssnitt kan man utföra kritiska uppgifter som att ändra inställningar i BIOS/UEFI, läsa av hårdvaruloggar eller använda **KVM-over-IP** för att få en virtuell bildskärm och tangentbord direkt i webbläsaren. Det är också möjligt att montera **virtuella media** (virtual media), vilket gör att administratören kan "stoppa i" en ISO-fil på distans för att installera om ett operativsystem utan att behöva vara på plats i serverrummet. För att bibehålla säkerheten är det av yttersta vikt att dessa management-portar placeras i ett strikt isolerat **Management VLAN** som aldrig är åtkomligt för vanliga användare eller elever, då åtkomst till dessa system i praktiken innebär total kontroll över den fysiska hårdvaran.

Att använda SSH för att styra **CIMC** är ofta snabbare än det grafiska webbgränssnittet, särskilt när bandbredden är begränsad eller när du vill automatisera en omstart. När du loggar in via SSH möts du inte av det vanliga IOS-gränssnittet som på en router, utan av ett specifikt kommandospråk för hanteringsprocessorn.

Nedan ett praktiskt exempel på hur en nätverkstekniker utför en tvingad omstart (hard reset) av en Cisco-server när operativsystemet har slutat svara.

```
# Steg 1: Anslut till hanterings-IP via terminalen
ssh admin@10.0.50.10

# Steg 2: Navigera till chassi-nivån
CIMC-server# scope chassis

# Steg 3: Ange önskat strömläge (power-state)
# Alternativ: power-cycle, hard-reset, shut-down
CIMC-server /chassis # set power-state hard-reset

# Steg 4: Bekräfta ändringen (CIMC kräver 'commit' för att
verkställa)
CIMC-server /chassis* # commit
```

Genom att använda denna metod kan administratören bibehålla kontrollen över maskinparken även under kritiska fel. Det understryker vikten av att **Management Plane** inte bara är säkrat, utan också tillgängligt via robusta protokoll som SSH. Detta gör att du som tekniker kan sitta kvar på din arbetsplats och återställa driften på några sekunder, istället för att behöva springa ner till serverrummet med en fysisk monitor och tangentbord.

Exempel: Genom att konfigurera en brandväggsregel i OPNsense som endast tillåter trafik från administratörens IP till routerns port 443, blockeras alla andra enheter i huset från att ens se inloggningssidan. Resultatet blir att angreppsytan krymper från hela nätverket till en enda betrodd dator.

4.1.4 Säkerhet i VLAN-design och Accesslagret

En genomtänkt **VLAN-design** utgör fundamentet för nätverkssäkerheten. En av de mest grundläggande inställningar att genomföra, är att pruna

trunkportar. Det innebär att endast vissa specifika VLAN tillåts på trunkportarna. Genom att segmentera nätverket i mindre, logiska delar begränsas dessutom förmågan för skadlig kod eller en obehörig användare att röra sig sidleds (lateral movement). Istället för att ha ett enda stort, platt nätverk skapas zoner för exempelvis elever, personal, administration och gäster. Varje zon isoleras från de andra, och all kommunikation mellan dessa VLAN måste passera en kontrollpunkt, vanligtvis en router eller brandvägg, där trafikregler (**Access Control Lists, ACLs**) bestämmer vad som är tillåtet.

För att kontrollera den fysiska åtkomsten till switchens portar används **port-säkerhet** (port security). Denna teknik gör det möjligt för administratören att begränsa antalet enheter som får ansluta till en specifik port genom att låsa den till en eller flera unika hårdvaruadresser (MAC-adresser). Om en obehörig enhet ansluts kan switchen konfigureras att vidta olika åtgärder, såsom att stänga ner porten helt (**shutdown**) eller bara blockera trafiken och generera ett larm (**restrict**).

Som ett komplement till detta används **storm-control** (stormkontroll) – nätverkets skydd mot "trafikinfarkter". En broadcast-storm kan uppstå om en nätverksloop bildas eller om ett trasigt nätverkskort börjar skicka ut miljontals felaktiga paket per sekund. Utan begränsning kan sådan trafik snabbt äta upp all bandbredd. Genom att aktivera storm-control sätter administratören en tröskel (threshold) för hur stor del av länkens kapacitet som får användas för broadcast, multicast eller okänd unicast-trafik. Om trafiken går över denna gräns kan switchen antingen filtrera bort (suppress) den överskjutande trafiken eller stänga ner porten helt.

Exempel: En administratör vill skydda nätverket från att en trasig switch eller loop i ett klassrum sänker hela skolan. Man ställer in att broadcast-trafik på en port aldrig får överstiga 10 % av länkens totala bandbredd.

```
# Konfiguration på en Cisco-switch
Switch(config-if)# storm-control broadcast level 10.00 #
Tillåt max 10% broadcast
Switch(config-if)# storm-control action shutdown      #
Stäng av porten om nivån nås
```

Utöver att kontrollera vilka enheter som ansluts, måste nätverkets logiska struktur skyddas mot loopar och obehöriga switchar. **Spanning Tree**

Protocol (STP) säkras genom att använda **PortFast** på klientportar så att de går direkt i drift, vilket kombineras med **BPDU Guard**. Detta ser till att om någon ansluter en obehörig switch som skickar **BPDU**s (Bridge Protocol Data Units) till en klientport, stängs porten omedelbart ner. Samtidigt används **Root Guard** på strategiska portar nära nätverkets kärna för att förhindra att en felkonfigurerad enhet utropar sig till ny rot-switch (root bridge) och tvingar all trafik att ta omvägar.

Den säkra treenigheten: DHCP Snooping, DAI och IP Source Guard

För att skydda mot angrepp som utnyttjar nätverkets grundläggande funktioner implementeras en kedja av tekniker som samarbetar via en gemensam bindningstabell.

1. **DHCP-snooping:** Fungerar som en dörrvakt som avgör vilka portar som får skicka ut DHCP-svar (**trusted**) och vilka som är klientportar (**untrusted**). Detta förhindrar att en obehörig person agerar falsk DHCP-server. När en klient får en adress via en godkänd server skapar switchen en **DHCP-snooping-bindningstabell** (binding table) som mappar MAC-adress, IP-adress och port.
2. **Dynamic ARP Inspection (DAI):** Bygger vidare på tabellen genom att kontrollera ARP-paket. Detta stoppar **ARP-spoofing**, där en angripare försöker utge sig för att vara nätverkets gateway för att kunna avlyssna trafik.
3. **IP Source Guard:** Är tekniken som sätter stopp för **IP-spoofing**, det vill säga när en användare manuellt ställer in en falsk IP-adress för att utge sig för att vara någon annan. Switchen skapar ett hårdvarufilter (**ASIC**) som endast släpper igenom paket om käll-IP och käll-MAC matchar exakt det som står i bindningstabellen.

Exempel: En elev i VLAN 20 tilldelas IP 192.168.20.50 via DHCP. Eleven försöker sedan manuellt ändra sin IP till 192.168.20.1 (gatewayen) för att omdirigera trafik. Eftersom tabellen säger att porten endast får skicka trafik från .50, blockerar IP Source Guard omedelbart all trafik från elevens dator.

```
# Aktivering på en klientport (Cisco)
Switch(config-if)# ip verify source # Kontrollerar att käll-IP
stämmer
```

Identitet och proaktiv detektering

När kravet på säkerhet är ännu högre används **802.1X**, vilket innebär identitetsbaserad åtkomstkontroll. Här krävs att användaren eller enheten autentiserar sig mot en central **RADIUS-server** innan porten ens öppnas för trafik. Detta säkerställer att endast kända och godkända enheter får tillträde, oavsett vilket fysiskt uttag de ansluts till.

Som ett proaktivt komplement används **honeypots** (honungsfällor). En honeypot är en avsiktligt sårbar resurs som inte fyller någon funktion i driften; dess enda syfte är att larma när någon rör den. **Cowrie** är en populär interaktions-honeypot specialiserad på att efterlikna SSH och Telnet. Den lockar angripare till en falsk skalmiljö (**shell**) och loggar varenda kommando de skriver. Detta ger administratören en detaljerad bild av angriparens metoder utan att riktiga servrar exponeras. Cowrie kan med fördel kopplas mot exempelvis **Wazuh** som larmar vid intrångsförsök.

Exempel: I elevernas VLAN placeras en honeypot som ser ut som en gammal filserver. Om en elev använder ett skanningsverktyg och försöker ansluta till denna server, skickas omedelbart ett larm via **SNMP** eller **Syslog**. Genom att analysera larmet kan administratören se exakt vilken switchport försöket kom ifrån och vidta åtgärder innan eleven hittar en riktig server.

Effekt och resultat: Genom att kombinera dessa tekniker skapar vi ett accesslager som är självsanerande. Om en elev försöker fuska med sin IP-adress stoppar **IP Source Guard** trafiken. Om utrustning går sönder och börjar flöda nätverket med skräptrafik, ser **Storm Control** till att problemet isoleras, medan resten av skolan kan fortsätta arbeta som vanligt.

4.1.5 Protokollanalys med Wireshark

Protokollanalys är konsten att svara på frågan vad som faktiskt hände i nätverket när teorin inte stämmer överens med verkligheten. Om brandväggen är dörrvakten och SNMP är hälsopulsen, så är **Wireshark** administratörens mikroskop. Verktöget låter oss se rådata i form av ramar (frames), flaggor och exakta tidslinjer, vilket är helt avgörande för att felsöka både prestandaproblem och misstänkta säkerhetsincidenter.

En framgångsrik analys börjar aldrig med att bara "starta en fångst" och hoppas på det bästa. För att undvika att drunkna i en datasjö av ovidkommande trafik krävs en tydlig metod. Man börjar med ett smalt syfte – till exempel "varför får klienterna ingen IP-adress?" eller "varför är inloggningen långsam?". Man bör sedan fånga trafiken så nära källan som möjligt, antingen direkt på klienten, via en **SPAN-port** (Switched Port Analyzer) på switchen eller direkt på den berörda servern. Genom att använda filter både vid själva fångsten (capture filters) och vid analysen (display filters) kan man isolera de intressanta paketen och snabbare hitta rotorsaken.

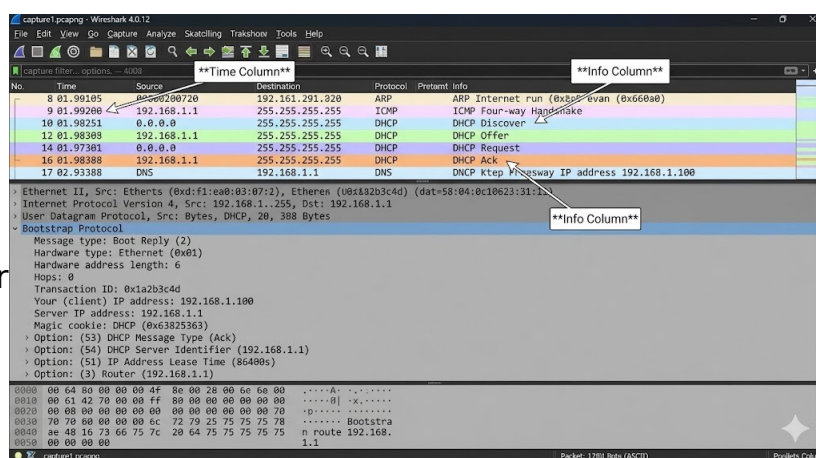
Etik, lag och integritet

Eftersom en protokollanalys innebär att man faktiskt läser den data som skickas över nätverket, spelar etik och juridik en stor roll. Som administratör har du ett ansvar att följa principen om dataminimering. Det innebär att du endast ska fånga den trafik som behövs för att lösa det specifika problemet. Man bör undvika att spara själva innehållet i paketen (payload) om det bara är huvudena (headers) som behövs för analysen. Dessutom är det god sed att kryptera sparade fångstfiler (pcap) och radera allt material så snart felsökningen är avslutad för att skydda användarnas personliga integritet.

Analysverktyg och filter i praktiken

Wireshark erbjuder flera vyer som ger snabb insikt. **Follow Stream** låter dig se en hel TCP- eller UDP-konversation som en sammanhängande text, vilket är ovärderligt för att förstå applikationsflöden. **I/O Graphs** visar trafikmängden över tid, vilket kan avslöja plötsliga spikar eller avbrott, och **Expert Information** flaggar automatiskt för nätverksproblem som retransmissioner eller trasiga kontrollsummor.

För att snabbt ringa in problem använder administratören standardiserade filter:



- **dhcp || bootp**: För att analysera adressdelning.
- **dns**: För att se om namnuppslag fungerar eller tar för lång tid.
- **arp**: För att kontrollera tabeller och upptäcka eventuell ARP-spoofing.
- **icmp**: För att kontrollera nåbarhet och felmeddelanden.
- **tcp.flags.syn == 1 && tcp.flags.ack == 0**: För att se nya anslutningsförsök och upptäcka portskanningar.
- **tcp.analysis.retransmission**: För att hitta störningar på den fysiska länken eller överbelastade noder.
- **tls.handshake**: För att verifiera att krypterade anslutningar upprättas korrekt.

Exempel: Klienter i ett nytt VLAN saknar IP-adress. En fångst (capture) på uplink-porten visar att DHCP DISCOVER skickas men att inga svar returneras då porten inte markerats som betrodd (trusted) i DHCP-snooping. När porten konfigureras som trusted i CLI syns direkt hela sekvensen OFFER, REQUEST och ACK i Wireshark, vilket bekräftar att adresseringen nu fungerar.

Kontrollfrågor

1. Förklara de tre begreppen konfidentialitet (confidentiality), integritet (integrity) och tillgänglighet (availability). Ge ett konkret exempel på en teknisk lösning för vart och ett av dessa mål.
2. Vad innebär principen om försvar i flera lager (defense in depth), och varför är det farligt att förlita sig på en enda stark brandvägg vid nätverkskanten?
3. Beskriv skillnaden mellan en klassisk tillståndsbaserad brandvägg (stateful inspection firewall) och en nästa generations brandvägg (Next-Generation Firewall, NGFW). Vilken roll spelar applikationskontroll (application control) i detta sammanhang?
4. Förklara skillnaden mellan ett detekterande system (IDS) och ett skyddande system (IPS). Hur skiljer sig deras fysiska eller logiska placering i nätverket (inline kontra out-of-band)?
5. Varför betraktas Telnet som en säkerhetsrisk i moderna nätverk, och vilka steg krävs i Ciscos kommandoradsgränssnitt (CLI) för att ersätta det med SSH?
6. Vad är syftet med ett dedikerat Management VLAN, och varför ska administrativ trafik aldrig blandas med vanlig användartrafik?
7. Förklara nyttan med CIMC eller iLO. Ge ett scenario där du som tekniker måste använda dessa gränssnitt istället för att logga in direkt i operativsystemet via SSH.
8. Hur samarbetar DHCP-snooping, Dynamic ARP Inspection (DAI) och IP Source Guard för att förhindra att en angripare utför en "man-in-the-middle"-attack?
9. Vad gör BPDU Guard och Root Guard, och på vilka typer av portar i nätverket bör de konfigureras för att garantera stabilitet?
10. Du misstänker att en obehörig person har satt upp en egen router i nätverket. Vilka filter och metoder skulle du använda i Wireshark för att bevisa att det flödar felaktiga DHCP-svar (rogue DHCP) på en specifik länk?

Fördjupningslänkar

ISC2 (Djupdykning i behörighetskontroll och säker administration) -

<https://www.isc2.org/Insights/2021/05/The-Principle-of-Least-Privilege>

Cisco (Officiell checklista för att säkra nätverksutrustning) -

<https://www.cisco.com/c/en/us/support/docs/ip/access-lists/23608-21.html>

NIST (Teknisk genomgång av IDS- och IPS-arkitekturer) -

<https://csrc.nist.gov/publications/detail/sp/800-94/rev-1/final>

Wireshark (Komplett dokumentation för protokollanalys och filterbygge) -

https://www.wireshark.org/docs/wsug_html_chunked/

Suricata (Information om IDS/IPS-motorn som används i bland annat OPNsense) - <https://suricata.io/features/>

Cisco (Praktisk guide till hur bindningstabellen skapas och används) -

<https://www.cisco.com/c/en/us/support/docs/switches/catalyst-6500-series-switches/200788-Understanding-and-Configuring-DHCP-Snoop.html>

MITRE (En global kunskapsbas över angreppstaktiker för att förstå hotbilden mot nätverk) - <https://attack.mitre.org/>

OWASP (Viktigt för att förstå vad en NGFW faktiskt inspekterar på lager 7) - <https://owasp.org/www-project-top-ten/>

Cisco (Tekniska detaljer för Out-of-Band management på Cisco-serverar) -

https://www.cisco.com/c/en/us/td/docs/unified_computing/ucs/c/sw/gui/config/guide/4-1/b_Cisco_IMC_GUI_Configuration_Guide_41.html

IETF (Den tekniska standarden för hur loggmeddelanden struktureras och skickas) - <https://datatracker.ietf.org/doc/html/rfc5424>

•

Egna anteckningar

4.2 Åtkomstkontroll och behörigheter

Efter att ha säkrat nätverkets infrastruktur och byggt upp de tekniska barriärerna i form av brandväggar och segmentering, skiftar fokus nu från de fysiska kablarna till de digitala identiteterna. **Åtkomstkontroll** (access control) handlar om att definiera vem som har rätt att befinna sig i nätverket och vad de tillåts utföra när de väl är där. I en modern IT-miljö ses identiteten ofta som den nya säkerhetsbarriären (the new perimeter), eftersom traditionella gränser suddas ut när användare ansluter från olika platser och enheter. Genom att tillämpa ramverket för **AAA** – **autentisering** (authentication), **auktorisering** (authorization) och **spårbarhet** (accounting) – skapas en logisk struktur där varje begäran om åtkomst kan verifieras, begränsas och loggas för framtida granskning.

Hanteringen av behörigheter (permissions) kräver en noggrann balansgång mellan hög säkerhet och praktisk användbarhet (usability). En miljö som är för strikt begränsar produktiviteten och skapar frustration, medan en för öppen miljö innebär att skadlig kod kan sprida sig obehindrat eller att känslig information hamnar i fel händer. Här blir principen om **minsta möjliga rättighet** (least privilege) den viktigaste ledstjärnan, där målet är att varje användare, program och systemtjänst endast har exakt de befogenheter som krävs för att utföra sin uppgift. Genom att använda **rollbaserad åtkomstkontroll** (Role-Based Access Control, RBAC) kan administratören hantera dessa rättigheter på gruppnivå istället för att administrera enskilda individer, vilket skapar en mer lätthanterlig och felsäker infrastruktur där rättigheter ärvs logiskt genom systemet.

4.2.1 AAA-modellen och identitet

Inom modern nätverkssäkerhet betraktas identiteten ofta som en form av digital valuta. Det är den centrala punkt kring vilken all åtkomst kretsar, och för att hantera denna process på ett strukturerat sätt används **AAA-modellen**. De tre bokstäverna står för **autentisering** (authentication), **auktorisering** (authorization) och **spårbarhet** (accounting). Tillsammans bildar de ett ramverk som säkerställer att rätt person får tillgång till rätt resurser och att alla åtgärder kan verifieras i efterhand.

Authentication

Autentisering är det första steget och handlar om att bevisa vem man är. Detta sker vanligtvis genom att användaren presenterar något de vet, såsom ett lösenord, något de har, till exempel en säkerhetsnyckel (security key), eller något de är, vilket innefattar biometri (biometrics). I professionella nätverksmiljöer används ofta centraliserade tjänster som **Active Directory** eller **LDAP** för att hantera dessa bevis. När identiteten väl är fastställd tar nästa fas vid: **auktorisering**. Här fattar systemet beslut om vilka rättigheter den identifierade användaren faktiskt ska ha. Det är här principen om minsta möjliga rättighet tillämpas praktiskt genom att användaren tilldelas specifika behörigheter baserat på sin roll i organisationen.

Authorization

För att göra behörighetshanteringen skalbar används ofta **rollbaserad åtkomstkontroll** (Role-Based Access Control, RBAC). Istället för att ge enskilda individer rättigheter till specifika mappar eller tjänster, placeras användarna i grupper som representerar deras yrkesroller. Som ett komplement till detta ser vi allt oftare **attributbaserad åtkomstkontroll** (Attribute-Based Access Control, ABAC), där även kontextuella faktorer vägs in. Det kan handla om att åtkomst nekas om användaren ansluter från en okänd IP-adress eller vid en ovanlig tidpunkt, trots att autentiseringen lyckats.

Accounting

Det sista steget i modellen är **spårbarhet**, vilket ofta benämns som loggföring. Detta innebär att systemet registrerar vad som faktiskt hände under sessionen: vem som utförde en åtgärd, när det skedde och från vilken enhet. Utan denna del saknar nätverket historisk synlighet, vilket gör det omöjligt att utreda incidenter eller bevisa att säkerhetspolicyn efterlevs. I praktiken bärs identiteten ofta med genom hela sessionen i form av en digital biljett, till exempel via protokollet **Kerberos** eller moderna webb-tokens, vilket gör att användaren slipper logga in på nytt för varje enskild resurs.

Exempel: En användare loggar in på skolans nätverk med sitt konto och en engångskod via telefonen (autentisering). Systemet kontrollerar användarens gruppmedlemskap och ser att hen tillhör gruppen för lärare, vilket ger läsrättigheter till betygssystemet men

skrivrättigheter till lektionsmaterial (auktorisering). Under hela arbetsdagen loggar servern alla filer användaren öppnar eller ändrar, vilket skapar en tydlig historik över användandet (spårbarhet).

Sammanfattning

- Autentisering handlar om beviset på identitet.
- Auktorisering handlar om beslutet om rättigheter.
- Spårbarhet handlar om beviset i efterhand via loggar.
- RBAC modellerar roller via grupper, medan ABAC lägger till villkor baserat på miljö och status.

4.2.2 Multifaktorautentisering (MFA) och motstånd mot phishing

Traditionella lösenord är idag nätverkssäkerhetens svagaste länk. Oavsett hur komplext ett lösenord är kan det läckas genom dataintrång, fiskas ut via nätfiske (phishing) eller knäckas genom råstyrkeattacker (brute force). För att motverka detta används **multifaktorautentisering** (multi-factor authentication, MFA), vilket innebär att en användare måste presentera minst två oberoende bevis på sin identitet. Dessa faktorer delas vanligtvis in i tre kategorier: något du vet (knowledge), såsom ett lösenord eller en PIN-kod; något du har (possession), till exempel en fysisk säkerhetsnyckel eller en smartphone; och något du är (inherence), vilket innefattar biometriska data som fingeravtryck eller ansiktsgenkänning. Genom att kräva faktorer från olika kategorier räcker det inte längre för en angripare att bara komma över ett lösenord för att ta över ett konto.

Det är dock viktigt att förstå att alla MFA-metoder inte är lika säkra. Enklare metoder som SMS-koder eller röstsamtal kan kapas genom så kallad SIM-bytessvindel (SIM swapping) eller avlyssning i mobilnätet. Även push-notiser i mobilappar kan utnyttjas genom "MFA-trötthet" (MFA fatigue), där en angripare skickar mängder av förfrågningar i hopp om att användaren till slut godkänner en av dem av misstag eller ren irritation. För att uppnå ett genuint **motstånd mot nätfiske** (phishing resistance) krävs modernare tekniker som bygger på **FIDO2** och **WebAuthn**. Dessa metoder använder asymmetrisk kryptering (asymmetric encryption) och binder inloggningen till den specifika webbadressen man besöker, vilket

gör det tekniskt omöjligt för en falsk webbplats att stjäla och återanvända inloggningskoden.

Dessa koncept fördjupas ytterligare i kapitel **11.3 Autentisering och auktorisering - Vem är du och vad får du göra?** i kurslitteraturen för datorteknik. Där dras paralleller mellan hur operativsystemet hanterar lokala identiteter och hur vi i nätverkstekniken skalar upp dessa principer till att omfatta hela infrastrukturen. Genom att kombinera en stark autentisering med en strikt behörighetspolicy (authorization) skapas en miljö där identiteten blir en pålitlig kontrollpunkt. I praktiken innebär det att vi kan tillämpa riskbaserad åtkomstkontroll, där systemet kräver en starkare faktor, som en hårdvarunyckel, endast när en användare försöker nå särskilt känsliga resurser eller ansluter från en okänd plats.

***Exempel:** En lärare råkar skriva in sitt lösenord på en falsk inloggningssida. Eftersom skolan kräver en fysisk säkerhetsnyckel (FIDO2) stoppas angreppet omedelbart. Angriparen saknar den fysiska faktorn och den falska sidan kan inte generera det kryptografiska svar som krävs för den riktiga domänen.*

4.2.3 Behörighetshantering i praktiken

När teorin om AAA ska omsättas i praktiken behöver administratören verktyg som fungerar i stor skala för att hantera tusentals användare och enheter. I en Windows-miljö är **Group Policy Objects** (GPO) det centrala styrsystemet för att konfigurera och låsa ner operativsystemet. Genom GPO kan man tvinga fram säkerhetsinställningar på alla datorer i en domän samtidigt, allt från att inaktivera osäkra tjänster till att bestämma vilka användargrupper som har rätt att logga in lokalt eller via fjärrskrivbord (Remote Desktop). När det gäller själva informationen används **NTFS-rättigheter** (NTFS permissions) där principen om **arv** (inheritance) är central. Det innebär att de behörigheter som sätts på en huvudmapp automatiskt flödar ner till alla underliggande filer och mappar, vilket gör administrationen förutsägbar och minskar risken för att enskilda filer lämnas oskyddade på grund av manuella missar.

I Linux-miljöer bygger rättighetshanteringen på en modell med ägare (owner), grupp (group) och övriga (others), där varje fil eller katalog har specifika flaggor för att läsa (read), skriva (write) och exekvera (execute). För att hantera administrativa uppgifter på ett fackmässigt sätt används

sudo (superuser do). Istället för att logga in som den obegränsade root-användaren, vilket vore ett brott mot principen om minsta möjliga rättighet, tillåter sudo en vanlig användare att utföra specifika kommandon med förhöjda privilegier (elevated privileges). Detta skapar en viktig spårbarhet (accounting) eftersom systemets loggar registrerar exakt vilken användare som utförde en administrativ ändring, snarare än att bara logga att "root" har gjort något.

Single Sign-On (SSO) är tekniken som knyter ihop dessa olika världar och skapar en sammanhängande användarupplevelse. SSO är en metod där en användare endast behöver logga in en gång för att få åtkomst till flera oberoende system och applikationer. Tekniskt fungerar det genom att en central identitetsleverantör (Identity Provider, IdP), såsom Microsoft Entra ID eller en lokal Active Directory, verifierar användaren och utfärdar en digital biljett eller token (token). Denna token presenteras sedan för olika tjänsteleverantörer (Service Providers) som litar på den centrala källan. För att sätta upp SSO krävs att man etablerar en förtroenderelation (trust relationship) mellan systemen, vilket ofta görs genom utbyte av digitala certifikat och användning av protokoll som **SAML** (Security Assertion Markup Language) eller **OpenID Connect**.



Fördelarna med SSO är betydande då det eliminerar lösenordströtthet (password fatigue) för användaren och förenklar administrationen genom att ett konto kan spärras centralt för alla anslutna tjänster samtidigt. Det finns dock nackdelar att ta hänsyn till; systemet skapar en kritisk felkälla (single point of failure) där en sårbarhet i det centrala kontot ger åtkomst till allt. Detta ställer extremt höga krav på att den centrala inloggningen skyddas med multifaktorautentisering och att sessioner (sessions) hanteras med korta livslängder för att minimera riskfönstret om en token skulle hamna på avvägar.

Exempel: En administratör använder SSO för att låta lärare nå både Windows-filer och Linux-verktyg med en och samma inloggning. Genom GPO styrs säkerhetsnivån på datorerna, medan NTFS-arv och sudo-regler säkerställer att lärarna endast når det material de behöver för sin yrkesroll.

Kontrollfrågor

1. Förklara de tre delarna autentisering (authentication), auktorisering (authorization) och spårbarhet (accounting). Ge ett exempel på hur dessa samverkar när en användare försöker nå en filserver.
2. Varför betraktas identiteten ofta som den nya säkerhetsbarriären i moderna nätverk, och hur påverkar det synen på traditionella brandväggar?
3. Beskriv skillnaden mellan rollbaserad åtkomstkontroll (Role-Based Access Control) och attributbaserad åtkomstkontroll (Attribute-Based Access Control). I vilken situation är ABAC mer effektivt än enbart grupper?
4. Vilka är de tre huvudkategorierna av faktorer som används vid multifaktorautentisering? Ge minst ett exempel på en teknisk lösning för varje kategori.
5. Varför anses metoder som bygger på FIDO2 och WebAuthn vara mer motståndskraftiga mot nätfiske (phishing) än traditionella engångskoder via SMS?
6. Vilken roll spelar GPO (Group Policy Objects) i en Windows-domän när det gäller att upprätthålla säkerhetsinställningar på klientnivå?
7. Förklara hur principen om arv (inheritance) fungerar i Windows filsystem. Vad händer med säkerheten om en administratör väljer att bryta arvet på en specifik undermapp?
8. Hur fungerar modellen med ägare, grupp och övriga i Linux, och varför är det säkrare att använda sudo än att logga in direkt som root-användare?
9. Beskriv den tekniska processen för SSO. Vilken roll spelar en identitetsleverantör (Identity Provider) och digitala tokens i detta flöde?
10. Nämn en betydande nackdel med att använda SSO för alla organisationens tjänster och förklara hur denna risk kan minimeras.

Fördjupningslänkar

Microsoft Learn (Teknisk genomgång av NTFS-behörigheter och hur arv fungerar) - <https://learn.microsoft.com/en-us/windows/security/threat-protection/security-reviews/ntfs-permissions>

NIST (Riktlinjer för identitetshantering och säkra autentiseringsmetoder) - <https://pages.nist.gov/800-63-3/>

FIDO Alliance (Hur FIDO2 och WebAuthn skyddar mot moderna nätfiskeangrepp) - <https://fidoalliance.org/fido2/>

Cloudflare (Förklaring av Single Sign-On och de bakomliggande protokollen) - <https://www.cloudflare.com/learning/access-management/what-is-sso/>

Red Hat (Praktisk hantering av rättigheter och användningen av sudo i Linux) - <https://www.redhat.com/en/topics/linux/what-is-sudo>

Microsoft Learn (Introduktion till gruppolicyer och centraliserad konfiguration) - <https://learn.microsoft.com/en-us/windows-server/identity/ad-ds/get-started/group-policy/group-policy-overview>

Okta (Jämförelse mellan de öppna standarderna SAML och OpenID Connect) - <https://www.okta.com/identity-101/saml-vs-openid-connect/>

CISA (Bästa praxis för implementering av multifaktorautentisering i organisationer) - <https://www.cisa.gov/resources-tools/resources/multi-factor-authentication-mfa>

IBM (Djupdykning i skillnaderna mellan RBAC och ABAC för åtkomststyrning) - <https://www.ibm.com/topics/rbac>

Microsoft Learn (Hantering av avancerade lösenordspolicyer i Active Directory) - <https://learn.microsoft.com/en-us/windows-server/identity/ad-ds/manage/component-updates/fine-grained-password-policies>

Egna anteckningar

4.3 Övervakning och loggning

Övervakning och loggning utgör nätverkets ögon och öron och är helt avgörande för att kunna gå från en reaktiv till en proaktiv drift. Utan systematiska loggar befinner sig administratören i ett mörker där det är omöjligt att veta om nätverket är långsamt på grund av en teknisk flaskhals eller ett pågående säkerhetsintrång. Genom att samla in telemetri (telemetry) – det vill säga mätvärden och händelseströmmar från nätverkets alla hörn – skapas en digital spegelbild av verksamheten som gör det möjligt att upptäcka avvikelser innan de hinner eskalera till ett fullständigt driftstopp.

I detta kapitel skiftar vi fokus till hur vi praktiskt samlar in och tolkar denna data för att få insyn i realtid. Vi undersöker hur loggar från olika källor, såsom switchar, brandväggar och servrar, kan centraliseras och korreleras (correlated) för att ge ett sammanhang åt enskilda händelser. Målet är att bygga ett system där vi inte bara samlar in data för sakens skull, utan där vi skapar en miljö som aktivt larmar vid misstänkta beteenden och ger oss de verktyg som krävs för att snabbt kunna felsöka och säkra nätverket när det oväntade inträffar.

4.3.1 Centraliserad logghantering

Inom nätverksadministration är loggar den enskilt viktigaste källan till sanning. Varför vi loggar handlar i grunden om att skapa synlighet (visibility) och spårbarhet (traceability). Utan en strukturerad loggning är teknikern blind inför händelser som skett i det förflutna. Syftet är att kunna genomföra en forensisk analys efter en incident: vad hände, när skedde det, på vilken enhet och vilken identitet var involverad? Genom att centralisera dessa data kan vi korrelera (correlate) händelser. Ett exempel på detta är när en autentiseringslogg från en server matchas mot en trafikspik i brandväggen; isolerade ser de harmlösa ut, men tillsammans indikerar de ett pågående dataintrång. För att detta ska vara tekniskt möjligt krävs att alla enheter i nätverket är synkroniserade via NTP (Network Time Protocol), då loggar med diffande tidsstämplar är värdelösa vid en kronologisk utredning.

För nätverksutrustning och Linux-baserade system är Syslog (System Logging Protocol) den rådande industristandarden. Protokollet är logiskt

uppbyggt kring två huvudbegrepp: Faciliteter (Facilities), som anger vilken typ av programvara eller process som genererat meddelandet (exempelvis *auth*, *kernel* eller *local7*), och Allvarlighetsgrader (Severity levels). Dessa sträcker sig från 0 (*Emergency*) till 7 (*Debug*). En professionell tekniker konfigurerar ofta sina switchar att skicka loggar via UDP port 514, eller det säkrare TCP/TLS, till en central loggserver. Detta säkerställer att loggarna finns bevarade även om nätverksenheten skulle haverera eller bli manipulerad av en angripare.

I Windows-miljöer hanteras händelsedata genom Event Viewer (Händelsevisaren). Till skillnad från Syslog, som ofta består av ostrukturerad text, använder Windows ett strukturerat format där varje händelse tilldelas ett unikt Event ID. Dessa ID-nummer är ovärderliga vid felsökning; exempelvis indikerar *Event ID 4624* en lyckad inloggning, medan *4625* markerar ett misslyckat försök. Loggarna organiseras i specifika kanaler såsom *System*, *Application* och *Security*. För att hantera Windows-loggar i stor skala använder vi ofta tekniker för att vidarebefordra dessa händelser till en central samlare för analys.

För att knyta samman dessa olika världar – Windows, Linux och nätverkshårdvara – används ofta plattformar som Wazuh. Wazuh fungerar som en modern lösning för säkerhetsövervakning genom att använda så kallade agents installerade på varje server och klient. Dessa agenter samlar in data från både Event Viewer och lokala loggfiler och skickar dem krypterat till en central Wazuh-manager. Systemet agerar som en SIEM (Security Information and Event Management) och kör den insamlade datan mot en kraftfull regelmotor. Om Wazuh identifierar ett mönster av misslyckade inloggningar på en Windows-server (via Event ID) samtidigt som brandväggen rapporterar portscanning (via Syslog), genereras ett larm i realtid. Detta automatiserade analysarbete gör att administratören kan fokusera på faktiska hot istället för att manuellt granska miljontals rader av loggdata.

Exempel: En angripare genomför en brute force-attack mot en server och lyckas slutligen logga in, varpå en ovanligt stor datamängd omedelbart börjar lämna nätverket via brandväggen. Genom att ha alla loggar centraliserade kan teknikern se både de misslyckade inloggningsförsöken och den efterföljande trafikspiken i en gemensam vy.

Visibility: Förmågan att se att inloggningsförsöken och trafikspiken existerar i realtid över olika systemgränser.

Traceability: Möjligheten att följa spåren bakåt för att identifiera vilket specifikt konto som komprometterades och från vilken käll-IP attacken ursprungligen kom.

Correlate: Att logiskt koppla samman de misslyckade inloggningarna i Windows-loggen med de ovanliga trafikflödena i brandväggsens syslog för att förstå att det rör sig om en enhetlig säkerhetsincident.

4.3.2 Övervakning av hälsa och telemetri

Medan logghantering ger oss en historisk tillbakablick, fokuserar övervakning av hälsa och telemetri (telemetry) på nätverkets status i realtid. För en nätverkstekniker fungerar detta som instrumentbrädan i en bil; vi behöver veta om motorn överhettas eller om bränslet håller på att ta slut innan fordonet stannar. Inom nätverk innebär telemetri att vi kontinuerligt mäter och samlar in driftdata från switchar, routrar och brandväggar för att proaktivt identifiera flaskhalsar och begynnande hårdvarufel.

Det absolut dominerande protokollet för att hämta denna data är **SNMPv3**. Protokollet fungerar som en säker budbärare som hämtar specifika värden, så kallade **OID:er** (Object Identifiers), från enhetens interna databas, **MIB:en** (Management Information Base). Till skillnad från sina föregångare erbjuder SNMPv3 både autentisering och kryptering, vilket är ett absolut krav i dagens nätverk för att förhindra att obehöriga kan läsa ut känslig information om vår infrastruktur.

För att garantera nätverkets hälsa fokuserar vi främst på fyra kritiska mätvärden:

CPU-belastning: En switch eller router som ligger nära 100% belastning kommer oundvikligen att börja tappa paket. Genom att övervaka processorn kan vi upptäcka om nätverket utsätts för en broadcast-storm, en DDoS-attack eller om en routing-process har hamnat i en loop.

Temperatur: Nätverksutrustning genererar värme, och om kylningen i ett apparatskåp sviktar förkortas hårdvarans livslängd dramatiskt. Telemetri tillåter oss att sätta larmtrösklar som varnar oss långt innan en enhet stänger ner sig själv för att undvika permanent skada.

Länkfel (Interface Errors/CRC): Detta är teknikerens viktigaste verktyg för att upptäcka fysiska problem. Om antalet fel (Cyclic Redundancy Check-fel) ökar på en specifik port indikerar det nästan alltid en skadad kabel, en smutsig fiberkontakt eller elektromagnetiska störningar.

Uptime: Genom att mäta drifttiden kan vi snabbt identifiera om en enhet har startat om oväntat. Om en switch visar en uptime på endast några minuter trots att inget underhåll utförts, tyder det på problem med strömförsörjningen eller en kritisk systemkrasch.

Men protokollet är bara ena halvan; vi behöver även en **NMS** (Network Management System) – den mjukvara som faktiskt visualiserar datan.

PRTG (Paessler Router Traffic Grapher): Ett av de mest populära verktygen i mindre och medelstora nätverk. Det använder ett "sensor-baserat" system där varje sensor övervakar ett specifikt värde, som till exempel temperaturen i en switch eller CPU-lasten i en brandvägg. Det är känt för sina tydliga "trafikljus" (grönt, gult, rött) som gör det lätt att snabbt se var felet ligger.

Zabbix: En kraftfull open-source-plattform som ofta används i stora enterprise-miljöer och datacenter. Zabbix är extremt skalbart och kan övervaka tusentals enheter samtidigt. Det kräver mer konfiguration än PRTG, men ger teknikern total kontroll över hur data samlas in och hur larmtrösklar ska ställas in.

LibreNMS: Ett verktyg som är speciellt framtaget för nätverkshårdvara. Det är fantastiskt på att automatiskt upptäcka nya switchar och routrar och rita upp topologikartor baserat på den information det hämtar via SNMP. För en tekniker är det guld värt att se exakt hur länkarna mellan byggnader mår utan att behöva konfigurera varje port manuellt.

Grafana & Prometheus: I moderna miljöer används ofta dessa för att skapa estetiska och extremt detaljerade instrumentpaneler. Medan Prometheus samlar in tidsseriedata, förvandlar Grafana den till grafer som kan visa allt från realtidsbelastning på en internetlänk till drifttid (uptime) på hundratals accesspunkter samtidigt.

Exempel: Under en pågående skoldag märker övervakningssystemet i LibreNMS att CPU-belastningen på en access-switch plötsligt stiger till 95%, samtidigt som temperaturen ökar och felräknaren (CRC) på en trunk-port börjar löpa amok.

Visibility: Teknikern ser direkt i NMS-verktygets instrumentpanel att switchen i hus C lyser rött och att CPU-grafen har spikat lodrätt uppåt.

Traceability: Genom att granska historiken i Zabbix ser teknikern att problemen började exakt klockan 10:14, precis när en ny fiberpatch anslöts i serverrummet enligt loggen.

Correlate: Teknikern kopplar samman den höga värmeutvecklingen med de massiva länkfelen och förstår att den nya fibern är smutsig/defekt, vilket tvingar switchens processor att jobba hårt med att försöka tolka korrupta data.

4.3.3 SIEM och larmhantering

Om loggarna är nätverkets minne och telemetrin är dess instrumentbräda, så är **SIEM (Security Information and Event Management)** nätverkets hjärna. Ett SIEM-system har till uppgift att samla in enorma mängder data från brandväggar, switchar och servrar för att sedan analysera informationen i jakt på mönster som en människa aldrig skulle kunna upptäcka manuellt. I en modern driftmiljö används plattformar som **Splunk, Microsoft Sentinel** eller den öppna lösningen **Wazuh** för att centralisera detta arbete.

Att arbeta med SIEM handlar till stor del om att **tolka avvikelser**. Systemet bygger upp en förståelse för vad som är "normalt" i ditt nätverk – en så kallad **baseline**. Om nätverket vanligtvis har låg trafikaktivitet klockan 03:00 på morgonen, men plötsligt uppvisar en massiv

dataöverföring från en klientsida till en extern IP-adress, kommer SIEM-systemet att flagga detta som en anomali. Här används ofta **Microsoft Sentinel** som drar nytta av AI och maskininlärning för att automatiskt identifiera hotbilder baserat på globala trender, medan **Splunk** fungerar som en kraftfull sökmotor där teknikern kan gräva djupt i rådata för att förstå exakt hur en angripare rört sig mellan olika nätverkssegment.

En av de största utmaningarna för en nätverkstekniker är att **minska brus (noise reduction)**. Ett dåligt konfigurerat övervakningssystem kan generera tusentals larm varje dygn, varav de flesta är "falska positiva" (false positives) – händelser som ser misstänkta ut men är helt legitima, såsom en schemalagd backup som ser ut som ett dataintrång. För att undvika "larmtrötthet" (alert fatigue) måste vi finjustera reglerna i systemet. I **Wazuh** görs detta genom att justera tröskelvärden och prioritera larm baserat på deras allvarlighetsgrad. Målet är att filtrera bort det oviktiga bruset så att de kritiska larmen – de som faktiskt kräver omedelbar mänsklig handling – blir tydliga och prioriterade.

Slutligen finns det viktiga **etiska aspekter** att ta hänsyn till. Som it-tekniker har du teknisk möjlighet att se nästan allt som sker i nätverket, men med den makten följer ett stort ansvar och juridiska krav som **GDPR**. Det är en hårfin gräns mellan att övervaka nätverkets hälsa och att kränka användarnas integritet. God etik innebär att vi övervakar system och flöden, inte individer. Vi loggar IP-adresser och trafikmönster för att skydda infrastrukturen, men vi läser inte innehållet i användarnas kommunikation utan starka misstankar och godkännande från ledningen. Dokumenterade policys för hur länge loggar sparas och vem som har behörighet att läsa dem är därför lika viktiga som själva tekniken.

Exempel: Ett SIEM-system upptäcker att ett användarkonto i personal-VLAN:et loggar in från en ovanlig geografisk plats samtidigt som brandväggen blockerar trafikförsök mot kända skadliga servrar.

Visibility: SIEM-plattformen (t.ex. **Wazuh**) presenterar händelserna från både inloggningsservern och brandväggen i en samlad tidslinje på teknikerns skärm.

Traceability: Teknikern kan följa händelseförloppet bakåt för att se att kontot först användes för en lyckad inloggning via VPN, följt av de blockerade utgående anslutningsförsöken.

Correlate: Genom att systemet automatiskt kopplar samman (correlation) den ovanliga geografiska platsen med de misstänkta trafikförsöken förstår teknikern att kontot är kapat, trots att inloggningen i sig såg legitim ut.

Kontrollfrågor

1. Varför är strikt tidssynkronisering via NTP en teknisk förutsättning för att kunna använda begreppet *correlate* vid en incidentutredning?
2. Förklara skillnaden mellan *visibility* och *traceability* i samband med en upptäckt säkerhetsincident i nätverket.
3. Inom syslog-protokollet används *facilities* och *severity levels*. Vad styr dessa två parametrar i ett loggmeddelande?
4. Windows Event Viewer använder unika Event ID:n. Nämn ett exempel på ett ID för en misslyckad inloggning och i vilken kanal (logg) det hittas.
5. Vilka är de tre huvudsakliga säkerhetslagren i SNMPv3 som gör det överlägset äldre versioner som v1 och v2c?
6. Om du ser en snabb ökning av länkfel (CRC-fel) på en specifik switchport i ett NMS-verktyg som LibreNMS, vilka fysiska fel på lager 1 bör du undersöka först?
7. Vad är huvudsyftet med att använda agenter i en SIEM-lösning som Wazuh jämfört med att bara skicka rå syslog-data från en server?
8. Vad innebär begreppet "larmtrötthet" (alert fatigue) och hur kan administratören använda tröskelvärden för att minska bruset i ett SIEM-system?
9. Beskriv begreppet *baseline* och varför det är nödvändigt för att ett system som Microsoft Sentinel ska kunna upptäcka avvikelser.
10. Vilka etiska överväganden och lagkrav (t.ex. GDPR) måste en nätverkstekniker ta hänsyn till när hen konfigurerar loggning av användartrafik?

Fördjupningslänkar

Wazuh (Open source SIEM och XDR för centraliserad logganalys) - <https://wazuh.com/>

Zabbix (Professionell lösning för nätverksövervakning via SNMP) - <https://www.zabbix.com/>

Paessler (Pedagogisk guide till SNMP-övervakning, OID:er och MIB:ar) - <https://www.paessler.com/it-explained/snmp>

LibreNMS (Automatiserad nätverksövervakning optimerad för switchar och routrar) - <https://www.librenms.org/>

Microsoft Learn (Teknisk katalog över Windows Event IDs för säkerhetsloggning) - <https://learn.microsoft.com/en-us/windows-server/identity/ad-ds/plan/appendix-l--events-to-monitor>

Cisco (Teknisk vägledning för konfiguration av syslog-nivåer och faciliteter) - https://www.google.com/search?q=https://www.cisco.com/c/en/us/td/docs/routers/access/software/ios/12_3t/12_3t8/syslog.html

Grafana (Visualisering av telemetri och realtidsdata från nätverket) - <https://www.google.com/search?q=https://grafana.com/solutions/network-monitoring/>

IMY (Integritetsskyddsmyndighetens vägledning om loggning, säkerhet och GDPR) - <https://www.google.com/search?q=https://www.imy.se/verksamhet/dataskydd/det-har-galler-enligt-gdpr/loggning/>

Prometheus (System för insamling av tidsseriedata och avancerad larmhantering) - <https://prometheus.io/>

Splunk (Industristandard för analys av "machine data" och storskalig SIEM) - <https://www.splunk.com/>

Egna anteckningar

4.4 Incidenthantering och backup

I nätverksvärlden finns det en gammal sanning som varje tekniker lär sig förr eller senare: det handlar inte om *om* något kommer att gå sönder, utan om *när*. Trots att vi har lagt ner veckor på att planera snygga kabeldragningar på lager 1 och konfigurerat avancerad routing på lager 3, finns det faktorer vi aldrig helt kan styra över. En grävmaskin kan sätta skopan i en stamfiber, ett åsknedslag kan steka portarna i en switch eller så kan nätverket plötsligt digna under trycket av en riktad överbelastningsattack. Incidenthantering är konsten att behålla lugnet när övervakningssystemet lyser rött. Det handlar om att ha en färdig spelplan så att du inte behöver börja fundera på lösningar först när nätet ligger nere och användarna börjar ringa.

Men att bara släcka bränder räcker inte för en professionell administratör. För att ett nätverk ska vara robust krävs en genomtänkt strategi för backup och redundans. Det innebär att vi inte bara säkerställer att användarnas filer finns sparade, utan att vi även har kontroll på nätverkets "hjärna" – dess konfigurationsfiler. Om en kärnrouter (core router) går sönder på grund av ett hårdvarufel, är det skillnaden mellan tio minuters och tio timmars driftstopp om du har en färsk konfigurationsbackup redo att läsas in i en ersättningsenhet. I det här kapitlet ska vi gå igenom hur vi strukturerat möter både fysiska haverier och logiska attacker, och hur vi bygger upp en säkerhetsmarginal som gör att nätverket kan resa sig igen efter en smäll.

4.4.1 Rutiner och dokumentation

När en kritisk länk går ner eller en core-switch slutar svara, sprider sig stressen snabbt. Som tekniker är din största tillgång i detta läge inte din snabbhet i fingrarna, utan din förmåga att följa en förutbestämd plan. Utan fastställda rutiner blir incidenthantering lätt kaotisk, där flera personer försöker lösa samma problem samtidigt utan samordning, vilket ofta leder till fler fel. Dokumentation är därför inte bara något vi gör för att hålla ordning efteråt; det är ett operativt verktyg som räddar oss medan det smäller.

NIST-modellen – Din karta genom kaoset

För att strukturera arbetet använder vi i branschen ofta ramverk. Ett av de mest respekterade är **NIST-modellen** (från National Institute of Standards and Technology). Den beskriver incidenthantering som en cirkulär livscykel i fyra tydliga faser. För en nätverkstekniker innebär det följande:

1. **Förberedelse (Preparation):** Detta är det viktigaste steget och sker *innan* något går sönder. Här ser vi till att vi har uppdaterade nätverkskartor, att vi har tillgång till switcharnas konsolportar och att vi har fungerande backuper på alla konfigurationsfiler. Vi bestämmer också vem som gör vad när larmet går.
2. **Upptäckt och analys (Detection & Analysis):** Här använder vi våra övervakningsverktyg (som vi lärde oss i kapitel 4.3). Ser vi en avvikelse i telemetrin? Är det ett faktiskt hårdvarufel på lager 1, eller är det en logisk loop på lager 2? Vi analyserar omfattningen: Är det en hel byggnad som är nere eller bara ett enskilt VLAN?
3. **Begränsning, sanering och återställning (Containment, Eradication & Recovery):**
 - **Begränsning:** Vi stoppar skadan. Om en loop sänker nätet stänger vi av den specifika porten.
 - **Sanering:** Vi tar bort orsaken, till exempel genom att byta ut en trasig SFP-modul eller en defekt kabel.
 - **Återställning:** Vi tar systemet i drift igen, ofta genom att läsa in en sparad konfiguration och verifiera att trafiken flyter som den ska på lager 3.
4. **Efterarbete (Post-Incident Activity):** Vi sätter oss ner och ställer frågan: "Varför hände detta, och hur hindrar vi att det händer igen?". Detta steg kallas ofta för *Lessons Learned* och är det som gör nätverket starkare över tid.

Roller vid en incident

För att undvika förvirring fördelar vi roller. Även i ett litet team behöver dessa funktioner fyllas:

- **Incident Commander:** Personen som leder arbetet och fattar de stora besluten (t.ex. "Vi stänger ner hela segment B nu").

- **Technical Lead:** Den seniora teknikern som faktiskt arbetar i CLI (Command Line Interface) eller fysiskt byter ut hårdvara.
- **Communications:** Den som informerar användare eller ledning om läget, så att teknikerna får arbetsro att lösa problemet.

Exempel på en incidentrapport

En incidentrapport ska skrivas så snart läget är stabilt. Den fungerar som en teknisk loggbok och bevisföring. Här är ett exempel på en rapport efter ett problem på lager 3:

INCIDENTRAPPORT #2026-04-02-A

Status: Löst **Ansvarig tekniker:** Johan Nilsson (Technical Lead)

Tidpunkt: 2026-04-02, kl. 09:15 – 10:40

Händelsebeskrivning: Övervakningssystemet larmade kl. 09:15 för avbrott mot Core-Router-01. Samtliga klienter i VLAN 20 (Administration) tappade sin default gateway och därmed all internetåtkomst.

Analys: Vid kontroll konstaterades att en fiberpatch mellan Core-Router och Distributions-Switch i hus A blivit skadad (Lager 1). Inget ljus detekterades i SFP-modulen på router-sidan.

Utförda åtgärder:

1. Kl 09:45: Reservfiber drogs mellan enheterna. Länkstatus gick till "Up/Up".
2. Kl 10:00: Verifiering av routingtabeller (OSPF) genomfördes för att säkerställa att tabellerna uppdaterats korrekt.
3. Kl 10:15: Ping-test från klient i VLAN 20 mot 8.8.8.8 lyckades.

Förslag till framtida åtgärder (Lessons Learned): Den skadade fibern låg oskyddad i ett skåp som ofta öppnas. Vi bör installera täcklock och dragavlastning för att skydda de fysiska länkarna i framtiden.

Genom att arbeta så här strukturerat förvandlar vi ett potentiellt haveri till en lärdom som förbättrar hela it-miljön.

4.4.2 Backup som sista försvarslinje: Infrastruktur och hårdvaruredundans

Om incidenthantering handlar om att släcka bränder när de uppstår, så är backupen den försäkring som ser till att du kan bygga upp hela nätverket igen om det brinner ner till grunden. För oss som arbetar med nätverkets infrastruktur innebär backup något mycket mer än att bara spara kopior av dokument. Det handlar om att säkra nätverkets fysiska och logiska förmåga att transportera data. Om en huvudrouter på lager 3 dör på grund av ett hårdvarufel, spelar det ingen roll att serverns data är intakt om ingen trafik kan hitta fram. Backup på lager 1–3 är din sista försvarslinje för att återställa kommunikationen i hela organisationen.

Fysisk backup och ledningsredundans (Lager 1)

En ofta förbisedd men kritisk del av backup-strategin är de fysiska kopplingarna. Vi kan ha världens bästa mjukvarubackuper, men om den enda fiberkabeln som förbinder två byggnader grävs av vid ett vägarbete, så står nätverket stilla oavsett vad vi gör i mjukvaran. På lager 1 handlar backup om att bygga in fysiska alternativ i infrastrukturen så att det alltid finns en väg för ljuset eller elektronerna att färdas, även när en kabel skadas.

- **Vägredundans:** Att ha två fysiskt olika vägar för kablagen genom en byggnad eller ett område. Om en grävmaskin klipper "väst-kabeln" ska trafiken automatiskt kunna flöda via "öst-kabeln". I nätverksdrift är en fysisk reservväg lika mycket en "backup" som en digital fil på en server.

Hårdvaruredundans: Att ha en "extra motor"

När vi rör oss uppåt i lagren inser vi snabbt att konfigurationsfiler bara är text om vi inte har en fungerande maskin att ladda in dem i. Hårdvara slits ut, fläktar stannar och strömförsörjningsaggregat kan brinna upp. Att ha "backup på hårdvara" innebär att vi planerar för maskinfel genom att ha reservenheter redo. Det är här vi som tekniker måste välja mellan hur snabbt vi behöver få upp nätet igen kontra hur mycket pengar vi kan lägga på reservutrustning.

- **Cold Standby (Reservhårdvara på hyllan):** Detta innebär att du har en extra router eller switch liggandes i kartong på lagret. Om den aktiva enheten dör, måste du fysiskt gå dit, skruva ur den gamla, skruva i den nya och ladda in din konfigurationsbackup. Detta är billigare men ger en högre **RTO** (Recovery Time Objective), eftersom nätverket ligger nere under tiden du utför det fysiska arbetet.
- **Hot Standby (Aktiv redundans):** Här har du två enheter inkopplade och igång samtidigt. Genom att använda tekniker på lager 3 som **VRRP** (Virtual Router Redundancy Protocol) kan dessa två enheter samarbeta och se ut som en enda enhet för resten av nätverket. Om den ena dör tar den andra över trafiken på bråkdelen av en sekund.

Konfigurationsbackup: Nätverkets hjärna

Varje switch och router har en "hjärna" i form av en konfigurationsfil. Denna fil innehåller allt vi har programmerat: VLAN-inställningar på lager 2, IP-adresser och routing-tabeller på lager 3 samt säkerhetsregler. Eftersom dessa inställningar ofta lagras i enhetens flyktiga RAM-minne under drift, är de extremt sårbara. Om enheten startar om utan att vi sparar, eller om hårdvaran går sönder, är alla våra arbetstimmar förlorade om vi inte har en extern kopia av dessa instruktioner.

- **Full backup:** Vi exporterar hela enhetens running-config i sin helhet. Det är den säkraste metoden eftersom du har allt du behöver i en enda fil. Vid ett totalhaveri läser du in denna, och switchen är omedelbart tillbaka i exakt samma tillstånd.
- **Inkrementell backup (Incremental):** Här sparar vi bara de ändringar som gjorts sedan den senaste backupen (oavsett om den var full eller inkrementell). Om du ändrar en enda port på en switch under måndagen, sparar den inkrementella backupen bara just den ändringen. Det sparar utrymme, men kräver att du har hela kedjan av backupar kvar för att kunna återställa allt.
- **Differentiell backup (Differential):** Denna metod sparar alla ändringar som gjorts sedan den senaste *fulla* backupen. Om du tar en full backup på söndagen och gör ändringar varje dag, kommer tisdagens differentiella backup innehålla både måndagens och tisdagens ändringar. Det är smidigare vid återställning än

inkrementell, då du bara behöver den senaste fulla backupen och den senaste differentiella.

- **Automatiserad ändringshantering:** I moderna nätverk använder vi verktyg som kontinuerligt känner av om en tekniker ändrat en enda rad i konfigurationen. Det gör att vi kan "backa bandet" till precis innan en felaktig konfiguration gjordes, vilket sparar enormt med tid vid felsökning.

Strategi 3-2-1: Anpassad för infrastruktur

Den klassiska 3-2-1-regeln för backup är lika viktig för en nätverkstekniker som för en systemadministratör, men vi applicerar den på våra nätverksritningar och kritiska mjukvarubilder. Om vi förvarar alla våra backuper på samma ställe som nätverksutrustningen står, riskerar vi att förlora allt vid en fysisk katastrof som en brand eller en översvämning i serverrummet.

1. **3 kopior:** Du ska ha originalkonfigurationen i enheten, en lokal kopia på en manageringsserver och en kopia på en helt annan fysisk plats.
2. **2 olika medier:** Spara konfigurationerna på olika typer av lagring, till exempel en lokal hårddisk (NAS) och en molntjänst. Detta skyddar mot tekniska fel på själva lagringsmedia.
3. **1 offsite:** Minst en kopia måste finnas på en annan fysisk plats. För oss innebär detta även att ha fysiska reservdelar (lager 1-hårdvara) placerade i en annan byggnad, så att inte hela lagret stryker med vid en lokal incident.

RPO och RTO: Tiden som måttstock

När nätverket ligger nere är tid den enda valuta som räknas. För att vi ska kunna förklara för ledningen eller en kund vad våra backup-rutiner faktiskt innebär, använder vi två standardmått. Dessa mått avgör hur vi designar våra system: om vi behöver dyra "Hot Standby"-lösningar eller om det räcker med en reservenhet i ett förråd.

- **RPO (Recovery Point Objective):** Handlar om **dataförlust i tid**. Om du ändrade en routing-tabell klockan 10:00 men din senaste backup är från 08:00, så är ditt RPO 2 timmar. Du har förlorat de senaste ändringarna och måste göra om det tekniska arbetet manuellt.

- **RTO (Recovery Time Objective):** Handlar om **nertid**. Hur lång tid tar det från att switchen dör tills trafiken flyter igen? Om din RTO är 15 minuter kräver det nästan alltid att du har en extra router färdigkonfigurerad och redo att ta över (Hot Standby).

4.4.3 Redundans och Disaster Recovery

När vi pratar om redundans på lager 1 till 3 handlar det om att eliminera "Single Points of Failure" (SPOF). Om en kabel går av eller en router slutar fungera, ska nätverket vara intelligent nog att själv hitta en annan väg utan att användaren märker något. Disaster Recovery är steget efter: vad gör vi när hela serverrummet är dött? Genom att implementera tekniker som HSRP/VRRP och LACP bygger vi ett nätverk som inte bara är snabbt, utan också extremt tåligt.

Gateway-failover: HSRP och VRRP (Lager 3)

I ett vanligt nätverk har alla klienter en "Default Gateway" (standardport), oftast routerns IP-adress. Om den routern dör, kommer klienterna inte ut på internet eller till andra VLAN, även om det finns en reservrouter i racket bredvid. Klienterna vet helt enkelt inte om att reserven finns.

För att lösa detta använder vi **HSRP** (Cisco-proprietärt) eller **VRRP** (öppen standard). Dessa protokoll skapar en **virtuell IP-adress (VIP)** som delas mellan två fysiska routrar. Klienterna pekar sin gateway mot denna VIP. En router är "Active" och sköter trafiken, medan den andra är "Standby" och lyssnar efter "keepalive"-meddelanden. Om den aktiva routern tystnar, tar standby-routern över den virtuella IP-adressen på några sekunder.

Cisco CLI-exempel (HSRP): Här konfigurerar vi två routrar att dela på den virtuella adressen 192.168.1.1.

```
! På Router A (Primary)
interface GigabitEthernet0/1
 ip address 192.168.1.2 255.255.255.0
 standby 1 ip 192.168.1.1          # Den virtuella gateway-
adressen
 standby 1 priority 110            # Högre prioritet gör
denna till Active
```

```
standby 1 preempt                # Tillåter routern att  
ta tillbaka rollen efter omstart  
  
! På Router B (Secondary)  
interface GigabitEthernet0/1  
ip address 192.168.1.3 255.255.255.0  
standby 1 ip 192.168.1.1  
standby 1 priority 100  
standby 1 preempt
```

LACP: Redundans för kabelfel (Lager 1 & 2)

LACP (Link Aggregation Control Protocol) handlar om att bunta ihop flera fysiska kablar till en enda logisk länk, ofta kallad en *Port Channel* eller *EtherChannel*. Detta ger oss två stora fördelar:

1. **Ökad bandbredd:** Två 1 Gbps-kablar fungerar som en 2 Gbps-länk.
2. **Redundans:** Om en kabel går av (Lager 1-fel), fortsätter trafiken att flöda genom den andra kabeln utan avbrott.

Cisco CLI-exempel (LACP):

```
interface range GigabitEthernet0/1 - 2  
channel-group 1 mode active  
# "active" innebär att vi använder LACP (802.3ad)  
exit  
  
interface Port-channel 1  
switchport mode trunk  
# Nu konfigurerar vi hela bunten som en enhet
```

Redundans i OPNsense

I den öppna brandväggsplattformen **OPNsense** hanteras redundans på ett mycket kraftfullt sätt. Istället för att bara förlita sig på enkla protokoll har OPNsense inbyggt stöd för **High Availability (HA)** via **CARP** (Common Address Redundancy Protocol), vilket motsvarar VRRP.

För att detta ska fungera i praktiken använder man ofta följande funktioner och plugins:

- **CARP (Inbyggt):** Hanterar virtuella IP-adresser så att två brandväggar kan dela på rollen som gateway.

- **os-xmlrpc-sync (Plugin/Funktion):** Detta är "magin" i OPNsense. Det gör att när du ändrar en regel i brandvägg A, kopieras den automatiskt till brandvägg B. Utan detta skulle du behöva göra alla inställningar dubbelt.
- **os-frr (Plugin):** Används för dynamisk routing (OSPF/BGP). Om du har redundanta internetleverantörer ser detta plugin till att trafiken snabbt dirigeras om ifall en ISP går ner.
- **pfsync:** Ser till att brandvägg B vet exakt vilka anslutningar (states) som är igång i brandvägg A. Om brandvägg A dör kan brandvägg B ta över utan att dina SSH-sessioner eller bankinloggningar bryts.

Återställningstester (*Disaster Recovery Testing*)

Den farligaste meningen i ett serverrum är: *"Vi har redundans, så det ska fungera"*. Som tekniker med 15 års erfarenhet kan jag lova dig att redundans som aldrig testats, sällan fungerar när det väl gäller.

Återställningstester innebär att vi medvetet simulerar fel under kontrollerade former:

1. **Failover-test:** Dra ur strömkabeln till den primära routern mitt under en arbetsdag. Tar reserven över? Bryts trafiken?
2. **LACP-test:** Koppla ur en av kablarna i en Port Channel. Tappar vi paket, eller märks det inte ens?
3. **Backup-validering:** Försök att återställa en switch-konfiguration till en helt ny, tom switch. Fungerar tftp-servern? Är lösenorden rätt?

Genom att göra dessa tester regelbundet hittar vi de konfigurationsfel som annars skulle förvandla ett litet hårdvarufel till en total nätverkskatastrof.

Kontrollfrågor

1. Varför anses fasen "Förberedelse" vara den viktigaste delen i en incidenthanteringscykel?
2. Förklara varför det är viktigt att skilja på rollerna Technical Lead och Communications under en pågående kris.
3. Du har ont om lagringsutrymme men behöver ta backup ofta. Vilken metod väljer du av Full, Inkrementell och Differentiell? Motivera.
4. Om ett företag säger att de har en RTO på 15 minuter, vilken typ av hårdvaruredundans (Cold eller Hot Standby) krävs då oftast?
5. Vad innebär "1:an" i 3-2-1-regeln för en nätverkstekniker när det gäller fysisk hårdvara?
6. Vad är syftet med en "Virtuell IP-adress" (VIP) i protokoll som HSRP och VRRP?
7. Vilket kommando används i Cisco CLI för att se till att den primära routern tar tillbaka rollen som "Active" efter att den startat om?
8. Nämn två fördelar med att använda Link Aggregation (LACP) istället för att bara ha en enkel kabel mellan två switchar.
9. Vilken funktion i OPNsense ser till att brandvägg B vet exakt vad brandvägg A gör, så att befintliga anslutningar inte bryts vid en failover?
10. Varför räcker det inte med att bara se att lamporna lyser grönt på reservutrustningen för att garantera att nätverket är säkert?

Fördjupningslänkar

NIST SP 800-61 (Officiell guide för incidenthantering) -

<https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-61r2.pdf>

Cisco HSRP Configuration Guide (En djupdykning i hur Cisco hanterar gateway-redundans) -

https://www.cisco.com/c/en/us/td/docs/switches/lan/catalyst3750x_3560x/software/release/12-2_55_se/configuration/guide/3750xscg/swhsrp.html

OPNsense High Availability Guide (Steg-för-steg hur man sätter upp CARP och synkronisering) - <https://docs.opnsense.org/manual/hacarp.html>

Veeam 3-2-1 Backup Rule (En tydlig förklaring av branschstandarden för säkerhetskopiering) - <https://www.veeam.com/blog/321-backup-rule.html>

IEEE 802.3ad (LACP) Explained (En teknisk genomgång av hur Link Aggregation fungerar på lager 2) - <https://community.fs.com/blog/lacp-definition-features-and-work-principle.html>

CERT-SE (Sveriges nationella CSIRT där du kan se aktuella hot och incidentrapporter) - <https://www.cert.se/>

RPO vs RTO (Video-förklaring av hur man beräknar nertid och dataförlust) - https://www.youtube.com/watch?v=0_u6L0IE_7M

NTP.org (Information om vikten av tidssynkronisering för loggar och incidentanalys) - [misstänkt länk borttagen]

SANS Incident Handler's Handbook (En praktisk lathund för tekniker som arbetar i hetluften) - <https://www.sans.org/white-papers/33901/>

Microsoft Learn: Disaster Recovery Strategies (Hur man planerar för att få upp ett nätverk efter en katastrof) - <https://learn.microsoft.com/en-us/azure/reliability/disaster-recovery-overview>

Egna anteckningar

5 Fördjupning

När de grundläggande principerna för nätverkshårdvara, adressering och segmentering är på plats, är nästa steg att förstå hur dessa tekniker samverkar i komplexa, moderna miljöer. I detta kapitel lyfter vi blicken från enskilda switchar och routrar för att utforska hur nätverket integreras med servrar och klienter på ett sätt som kräver både hög automatisering och precision. Vi rör oss här i gränslandet mellan infrastruktur och systemadministration, där nätverket inte bara fungerar som en passiv transportväg utan som en aktiv, mjukvarustyrd komponent.

Målet med fördjupningen är att ge en förståelse för hur lager 1–4 i OSI-modellen hanteras när kraven på skalbarhet och säkerhet ökar. Vi kommer att titta på hur virtualisering förändrar den fysiska topologin, hur centraliserad policystyrning kan säkra tusentals enheter samtidigt, och hur framtidens nätverk rör sig mot en modell där identitet och mjukvarulogik väger tyngre än de fysiska kablarna. Denna kunskap är avgörande för att kunna designa och underhålla nätverk som är både flexibla och motståndskraftiga mot moderna hotbilder.

5.1 Virtualisering – plattformar och gemensamma mönster

Virtualisering har i grunden förändrat hur vi ser på nätverkstopologier. Genom att köra flera virtuella maskiner på en och samma fysiska hårdvara kan vi utnyttja resurserna mer effektivt och isolera tjänster från varandra, vilket ökar driftsäkerheten markant. För nätverksteknikern innebär detta att den fysiska kabeln som går in i servern ofta bär på dussintals olika logiska nätverk (VLAN), vilket ställer högre krav på förståelsen för hur trafiken separeras och prioriteras direkt i mjukvaran.

I centrum för denna teknik står hypervisorn, som agerar som en mellanhand mellan hårdvaran och de virtuella maskinerna. Hypervisorn skapar en abstraktion av nätverkskortet, vilket gör att vi kan flytta, klonas och återskapa hela nätverksmiljöer på några minuter utan att röra den fysiska infrastrukturen. I detta avsnitt utforskar vi hur nätverket logiskt flyttas in i servern och hur vi använder virtuella switchar och bryggor för att behålla kontrollen över kommunikationen mellan maskinerna.

5.1.1 Virtualiseringens fundament: Hypervisorn och den virtuella switchen

Virtualisering handlar i grunden om att bryta det direkta beroendet mellan mjukvara och fysisk hårdvara. Genom att introducera en **hypervisor** skapas ett abstraktionslager som gör det möjligt att köra flera oberoende operativsystem på en och samma fysiska server. Hypervisorns främsta uppgift är att fördela resurser som processor (CPU) och arbetsminne (RAM), men för nätverksteknikern är dess viktigaste funktion skapandet av det virtuella nätverket. För en genomgång av de hårdvarumässiga förutsättningarna och grundläggande virtualiseringskoncept, se boken *Datorteknik* och dess kapitel om virtualisering.

Inuti hypervisorn skapas en **virtuell switch (vSwitch)**, även kallad en brygga (bridge). Denna fungerar i praktiken som en Lager 2-enhet och är den logiska bryggan mellan de virtuella maskinerna (VM) och nätverkskortet i den fysiska värddatorn. Precis som de fysiska switchar vi gick igenom i kapitel 1.2, hanterar den virtuella switchen trafik baserat på MAC-adresser. Varje virtuell maskin tilldelas ett eller flera virtuella nätverkskort (vNIC), som var och ett har en unik MAC-adress. När en VM skickar en ethernetram går den först till den virtuella switchen, som i sin tur avgör om ramen ska skickas vidare till en annan VM på samma fysiska server eller ut på det fysiska nätverket.

Genom att den virtuella switchen i många fall har fullt stöd för **VLAN (802.1Q)**, abstraheras nätverket ytterligare ett steg. I en traditionell miljö krävdes en fysisk kabel och en 5.1.2 *Nätverksmönster i virtuella miljöer: Bridge, NAT och Trunkingspecifik port* i en switch för varje nätverk. Med en VLAN-medveten vSwitch kan vi istället använda en enda fysisk nätverkskabel som en **trunk** (se kapitel 2.2) mellan den fysiska switchen och servern. Hypervisorn tar sedan hand om att sortera trafiken och leverera rätt VLAN till rätt virtuell maskin baserat på mjukvaruinställningar.

Detta mjukvarustyrda lager löser flera historiska problem. Tidigare tvingades man ofta köra en tjänst per fysisk server för att undvika konflikter, vilket ledde till att dyr hårdvara stod outnyttjad. Genom den virtuella switchen kan vi nu isolera tjänster i olika virtuella maskiner men låta dem kommunicera på samma Lager 2-nivå med extremt låg latens, eftersom trafiken aldrig behöver lämna serverns interna minne.

Abstraktionen gör också att nätverket blir mobilt; eftersom switchen är definierad i mjukvara kan en virtuell maskin flyttas mellan olika fysiska servrar utan att dess nätverksidentitet eller anslutning bryts.

5.1.2 Nätverksmönster i virtuella miljöer: Bridge, NAT och Trunking

Oavsett vilken virtualiseringsplattform man arbetar med återkommer tre huvudsakliga mönster för att hantera nätverkstrafiken på lager 2 och 3: Bridge, NAT och Trunking. Dessa mönster avgör hur de virtuella maskinerna kommunicerar med varandra och hur de ansluter till den fysiska infrastrukturen.

Bridge och Trunking (Lager 2)

Det vanligaste sättet att ansluta en virtuell maskin till ett befintligt nätverk är genom en **bridge** (brygga). Som vi lärde oss i kapitel 2.2 krävs det ofta att nätverk segmenteras med hjälp av VLAN för att öka säkerheten och minska onödig trafik. Genom att använda en **VLAN-aware bridge** kan serverns fysiska nätverkskort konfigureras som en trunk-port. Detta innebär att nätverkskabeln mellan den fysiska switchen och servern bär på trafik från flera olika VLAN samtidigt. Den virtuella switchen i hypervisorn läser av 802.1Q-taggar i ethernetramarna och ser till att rätt trafik hamnar hos rätt virtuell maskin. Detta mönster kallas ofta för att man "trankar hela vägen in" till hypervisorn. Det ger nätverksteknikern full flexibilitet att tilldela VLAN-ID direkt i mjukvaran utan att behöva röra den fysiska kableringen.

NAT och isolerade nät (Lager 3)

I vissa scenarier, som i skolans labbmiljöer, vill man att virtuella maskiner ska kunna nå internet men vara helt dolda från resten av det lokala nätverket. Här tillämpar vi teorin om routing från kapitel 2.3 genom att använda **NAT (Network Address Translation)**. I detta mönster skapas en virtuell switch som inte har en direkt koppling till det fysiska nätverket på lager 2. Istället fungerar hypervisorn som en lager 3-gateway (router) för de virtuella maskinerna. Detta är en idealisk lösning för labbar där elever behöver köra egna tjänster, som exempelvis DHCP, utan att riskera att störa skolans ordinarie nätverksdrift. Trafiken översätts när den lämnar den virtuella miljön, vilket innebär att alla maskiner i labbet utåt sett ser ut att ha samma IP-adress som den fysiska servern.

Genom att kombinera dessa mönster kan man skapa avancerade topologier. En webbserver i en DMZ-zon kan exempelvis kopplas via en bridge till VLAN 70 för att vara åtkomlig utifrån, medan en bakomliggande databasserver kan placeras i ett helt isolerat virtuellt nätverk som endast är nåbart för webbservern. Detta skapar en säkerhetszonering som är svår och dyr att uppnå med enbart fysisk hårdvara.

5.1.3 Ekosystem och plattformsval: Egenskaper i Lager 2/3

Valet av virtualiseringsplattform handlar för en nätverkstekniker inte bara om vilket användargränssnitt man föredrar, utan om hur plattformen implementerar nätverksstacken på lager 2 och 3. De olika ekosystemen har olika filosofier kring hur trafiken ska styras, vilket påverkar allt från prestanda till hur man skalar upp miljön. Genom att förstå de tekniska egenskaperna i lager 2 och 3 kan man välja rätt plattform utifrån verksamhetens nätverkskrav.

Gemensamt för samtliga dessa plattformar (Proxmox/KVM, ESXi, Hyper-V och XCP-ng) är att de klassificeras som **bare-metal** hypervisorer (Typ 1). Detta innebär att hypervisorn installeras direkt på den fysiska hårdvaran utan ett underliggande generellt operativsystem som tar upp resurser. Ur ett nätverksperspektiv är detta kritiskt eftersom det ger hypervisorn direkt kontroll över nätverkskortet (NIC), vilket minimerar fördröjningar (latens) och möjliggör avancerade funktioner på lager 2 som trunking och hårdvaruaccelererad switchning.

Proxmox VE

Proxmox bygger på Debian Linux och använder sig i grunden av standardiserade Linux-komponenter för nätverk. Proxmox bygger på **KVM** (Kernel-based Virtual Machine) för fullvirtualisering, vilket innebär att Linux-kärnan själv agerar hypervisor. Den primära metoden är **Linux Bridge**, som agerar som en virtuell switch på lager 2. För mer avancerade behov stödjer Proxmox även **Open vSwitch (OVS)**, en mjukvarubaserad switch som är designad för massiv automatisering och stödjer protokoll som VXLAN för att skapa virtuella nätverk över lager 3-gränser. Styrkan här är transparensen; eftersom det är standardiserad Linux-teknik kan en tekniker använda välkända verktyg som `ip` `addr` och `brctl` (se kapitel 3 för Linux-verktyg) för att felsöka nätverket direkt i terminalen.

Begränsningen är att det kräver en högre grad av Linux-kunskap för att konfigurera mer komplexa nätverksmönster manuellt.

VMware vSphere

VMware är marknadsledande inom företagssegmentet, och deras nätverksarkitektur är högt automatiserad genom **vSphere Distributed Switch (VDS)**. Till skillnad från en vanlig virtuell switch, som lever på en enskild server, spänner en distribuerad switch över flera fysiska värdar. För nätverksteknikern innebär detta att lager 2-konfigurationen (som VLAN-id och port-säkerhet) hanteras centralt för hela klustret. VMware har även ett mycket moget stöd för **vMotion**, vilket tillåter att en virtuell maskin flyttas mellan fysiska servrar utan att tappa sin nätverksanslutning. Detta kräver dock en noggrann lager 3-planering så att de fysiska serverna har åtkomst till samma subnät och gateways. Styrkan är den extrema skalbarheten och kontrollen, medan begränsningen ligger i de proprietära licenserna och att tekniken är låst till VMwares egna ekosystem.

Microsoft Hyper-V

Hyper-V använder en **Virtual Switch** som är djupt integrerad i Windows nätverksstack. Switchen kan konfigureras i tre lägen: *Extern* (kopplad till fysiskt NIC), *Intern* (kommunikation mellan VM:ar och värden) och *Privat* (endast mellan VM:ar). Hyper-V utmärker sig genom sin hantering av **NIC Teaming** (LACP) direkt i operativsystemet, vilket gör det enkelt att skapa redundans på lager 1 och 2. Det är ett naturligt val i miljöer som är tunga på Windows Server och Active Directory, men det kan upplevas som mindre flexibelt än Linux-baserade lösningar när det kommer till att köra lätta containrar eller avancerade open-source brandväggar.

XCP-ng

XCP-ng bygger på **Xen-hypervisorn**, en teknik som har sitt ursprung vid University of Cambridge under sena 1990-talet. Xen utvecklades ursprungligen som ett forskningsprojekt för att möjliggöra effektiv virtualisering genom så kallad paravirtualisering, där operativsystemet är medvetet om att det körs på en hypervisor för att optimera prestanda. Idag är Xen en mogen open source-plattform som hanteras av Linux Foundation. XCP-ng erbjuder ett kraftfullt alternativ som liknar VMwares struktur. Nätverkshanteringen sker genom **Networks**, där man definierar lager 2-bryggor som kan taggas med VLAN. Genom hanteringsverktyget

Xen Orchestra får man en bra grafisk överblick av lager 2-topologin och kan enkelt sätta upp nätverksreplikering. Det är en balansgång mellan Proxmox flexibilitet och VMwares strukturerade hantering.

När man väljer plattform utifrån nätverkets egenskaper bör man väga behovet av standardisering (Proxmox/KVM) mot behovet av centraliserad abstraktion (VMware). För en mindre miljö eller en skollabb ger Proxmox en djupare förståelse för hur lager 2 fungerar i praktiken, medan en större organisation ofta prioriterar de distribuerade funktionerna i vSphere för att minska risken för manuella konfigurationsfel på enskilda switchportar.

5.1.4 Driftsäkerhet och nätverkssegmentering: Kluster och Management-nät

När flera fysiska servrar kopplas samman för att samverka som en enda enhet skapas ett **kluster**. Syftet är oftast att uppnå **High Availability (HA)**, vilket innebär att om en fysisk server (en nod) slutar fungera, kan de virtuella maskinerna automatiskt starta om på en annan frisk nod i klustret. För att detta ska fungera pålitligt krävs en robust nätverksdesign där olika typer av trafik hålls strikt separerade. Att blanda administrativ trafik, lagringsdata och vanlig användartrafik på samma länk skapar inte bara säkerhetsrisker utan även prestandaproblem.

En professionell nätverksdesign i en virtualiserad miljö bygger på att dela upp trafiken i olika isolerade flöden, ofta genom separata VLAN eller dedikerade fysiska nätverkskort:

Management-nät: Detta nät används enbart för att administrera själva hypervisorerna och för klustrets interna kommunikation. Det bör aldrig vara nåbart direkt från det vanliga användarnätet. Genom att isolera management-trafiken säkerställer man att en överbelastning i användarnätet inte hindrar administratörer från att nå servrarna i ett kritiskt läge.

Lagrings-nät (Storage): Om servrarna använder delad lagring (exempelvis via protokollen iSCSI eller NFS) skickas enorma mängder data mellan server och lagringsenhet. Genom att lägga denna trafik på ett eget VLAN, eller helst på egna fysiska interface, garanteras låg latens och hög bandbredd utan att det stör övrig kommunikation.

Produktions-nät: Här flödar den trafik som faktiskt rör tjänsterna och användarna. Det är i detta nät som de virtuella maskinernas vanliga nätverkskort (vNIC) är anslutna.

I denna struktur är det viktigt att tillämpa en **Zero Trust-modell**, som fördjupas i boken *Datorteknik kapitel 4*. Inom nätverksteknik innebär Zero Trust att vi aldrig utgår från att "insidan" av nätverket är säker. Även om en användare befinner sig på det lokala kontoret ska hen inte ha direkt access till servrarnas administrativa gränssnitt. Istället används ofta en så kallad **bastionvärd** eller *jump host* — en säkrad dator som agerar den enda vägen in till management-nätet.

Exempel: En tekniker som ska uppdatera en server loggar först in på en bastionvärd med flerfaktorsautentisering (MFA). Först därifrån kan teknikern nå hypervisorns webbgränssnitt. Om en vanlig användares dator i produktionsnätet skulle bli infekterad med skadlig kod, finns det ingen nätverksväg som tillåter koden att sprida sig till de underliggande styrsystemen eller lagringen, eftersom dessa ligger i helt andra, isolerade VLAN.

5.1.5 Resursallokering och nätverksprestanda: vNIC, VirtIO och licensiering

När nätverket har flyttat in i mjukvaran blir prestandan inte längre bara beroende av den fysiska kabelns hastighet, utan även av hur effektivt hypervisorn kan förmedla data mellan den virtuella maskinen och den fysiska hårdvaran. Varje virtuell maskin kommunicerar via ett **vNIC (virtual Network Interface Card)**, vilket är en mjukvaru-emulering av ett fysiskt nätverkskort. Hur detta vNIC presenteras för operativsystemet har en direkt påverkan på nätverkets genomströmning (throughput) och serverns processorbelastning.

VirtIO och paravirtualiserade drivrutiner I de tidiga stadierna av virtualisering emulerade hypervisorer ofta äldre, välkända fysiska nätverkskort (som Intel E1000) för att säkerställa kompatibilitet. Detta krävde dock mycket processorkraft eftersom varje nätverkspaket behövde översättas genom ett tungt emuleringslager. Idag används istället **VirtIO** (i KVM/Proxmox) eller liknande paravirtualiserade drivrutiner (i VMware och Hyper-V). Här är operativsystemet medvetet om att det är virtuellt och skickar data via en optimerad genväg som minimerar CPU-

användningen och maximerar genomströmningen. Genom att använda VirtIO kan en virtuell maskin nå nätverkshastigheter som ligger mycket nära den fysiska hårdvarans teoretiska gräns.

Resursplanering och risken med overcommit Vid allokering av nätverksresurser är det viktigt att vara konservativ i början och mäta den faktiska förbrukningen. Begreppet **overcommit** innebär att man lovar bort mer resurser än vad som fysiskt finns i servern. Medan det ofta fungerar bra för CPU, är det riskfyllt för nätverkstrafik och RAM. Om nätverkskortet blir överbelastade uppstår köer som drastiskt ökar latensen för alla maskiner på samma fysiska länk. En god nätverksdesign innebär därför att man mäter den faktiska belastningen och använder tekniker som **LACP/Bonding** (se kapitel 1.2) för att slå ihop flera fysiska nätverkskort till en logisk länk med högre kapacitet för att undvika flaskhalsar.

Licensieringens påverkan på nätverksdesign och placering Även licensregler påverkar hur vi bygger våra nätverk och var vi placerar våra arbetslaster. Microsofts licensmodell för Windows Server är ett tydligt exempel: en **Standard-licens** tillåter endast två virtuella Windows-maskiner per fysisk server, medan **Datacenter-licensen** tillåter ett obegränsat antal virtuella maskiner på den licensierade hårdvaran. För en nätverkstekniker innebär detta att man ofta väljer att koncentrera så många Windows-maskiner som möjligt till specifika "Datacenter-noder" för att optimera resursutnyttjandet.

Exempel: Om en organisation har tio fysiska servrar men bara har licensierat två av dem med Datacenter-utgåvan, kommer dessa två servrar att husera huvuddelen av de virtuella maskinerna. Detta skapar en ojämн nätverksbelastning i klustret. Teknikern måste då se till att just dessa noder har kraftfullare nätverksupplänkar (exempelvis 10 Gbit/s eller dubbla trunkade länkar) jämfört med de andra servrarna. Att placera rätt arbetslast på rätt licensierad hårdvara är alltså inte bara en ekonomisk fråga, utan styr direkt hur vi måste dimensionera nätverkets fysiska och logiska infrastruktur.

5.1.6 Flexibilitet och säkerhet med virtuella maskiner

En av de största fördelarna med att arbeta med virtuella maskiner är den enorma flexibiliteten i hur vi kan skydda, återställa och flytta data jämfört med fysiska servrar. Genom att en virtuell maskin i grunden består av filer

på en disk (ofta i format som .qcow2, .raw eller .vhdx), kan vi hantera hela serverns tillstånd som ett objekt. Detta möjliggör tre kritiska funktioner: snapshots, backuper och migrering.

Snapshots: Den kortsiktiga säkerhetslinan

En snapshot är en ögonblicksbild av en virtuell maskins tillstånd, inklusive dess disk, konfiguration och ibland även innehållet i arbetsminnet (RAM). Tekniskt sett fungerar en snapshot genom att hypervisorn "fryser" den ordinarie diskfilen och börjar skriva alla nya förändringar till en separat differensfil (delta-fil). Om en uppdatering misslyckas eller om en konfiguration i nätverket blir felaktig, kan man på några sekunder "rulla tillbaka" till exakt det läge maskinen var i när ögonblicksbilden togs.

Det är dock livsviktigt att förstå att en snapshot **inte** är en backup. Eftersom en snapshot är beroende av den ursprungliga diskfilen, innebär en fysisk skada på lagringen att även snapshotten går förlorad. Dessutom kan för många eller för gamla snapshots drastiskt försämra serverns prestanda och fylla upp lagringsutrymmet, då differensfilerna växer över tid. Snapshots ska därför ses som en kortsiktig säkerhetsåtgärd inför stora förändringar, inte som långtidslagring.

Backup: Den externa räddningen

En backup är en fullständig och oberoende kopia av den virtuella maskinen som lagras på en separat fysisk enhet eller plats. Till skillnad från snapshots, skyddar backuper mot allvarliga händelser som hårdvarufel, brand eller ransomware-attacker. I en nätverkskontext är det här vi knyter an till avsnitt 5.1.4 om nätverkssegmentering; backuptrafik bör alltid gå via ett dedikerat backup-VLAN. Detta görs för att de stora datamängder som kopieras varje natt inte ska mätta produktionsnätet och påverka användarnas upplevelse.

En god rutin följer ofta **3-2-1-regeln**: Tre kopior av datan, på två olika typer av media, varav en kopia förvaras på en helt annan fysisk plats (off-site). Inom modern virtualisering används ofta tekniker som *immutability* (oföränderlighet), vilket innebär att backuperna låses så att de inte kan raderas eller krypteras av skadlig kod under en viss tid. Att regelbundet testa att en återställning faktiskt fungerar i ett isolerat virtuellt nätverk är lika viktigt som att utföra själva backupen.

Mobilitet och migrering

Utöver ren säkerhet ger virtualisering en unik rörlighet. Möjligheten att flytta en virtuell maskin mellan olika fysiska servrar eller till och med mellan olika virtualiseringsplattformar kallas för **migrering**. Inom ett kluster används ofta tekniken *Live Migration* (i VMware kallat *vMotion*). Det innebär att hela den virtuella maskinen flyttas från en fysisk värd till en annan under pågående drift, utan att användarna märker något avbrott.

För att detta ska fungera krävs att nätverket är identiskt konfigurerat på båda serverna; de VLAN som maskinen använder måste finnas tillgängliga på båda ställena, och serverna behöver ofta ha tillgång till samma delade lagring (Storage). Det finns två huvudtyper av flyttar som en nätverkstekniker behöver ha koll på:

- **Värd-till-värd (Host-to-Host):** Används vid planerat underhåll. Om en fysisk server behöver uppdateras eller lagas, flyttas alla maskiner till en annan server i klustret. När flytten sker behåller maskinen sin IP-adress och sina nätverksanslutningar eftersom den virtuella switchen på den nya servern tar över trafiken sömlöst.
- **Plattform-till-plattform (V2V - Virtual to Virtual):** Detta innebär att man flyttar en maskin från exempelvis VMware till Proxmox eller från Hyper-V till molnet. Detta är en mer komplex process som ofta kräver att man konverterar diskfilen och byter ut de virtuella nätverksdrivrutinerna (som VirtIO, se 5.1.5). Här är det kritiskt att nätverksinställningarna dokumenteras noga så att maskinen hamnar i rätt VLAN och får rätt brandväggsregler på den nya plattformen.

Sammanfattningsvis gör dessa funktioner infrastrukturen extremt flexibel. Det tillåter oss att balansera belastningen i nätverket och säkerställa att kritiska tjänster alltid är igång, oavsett om en enskild fysisk server behöver stängas av. Förutsättningen för att detta ska fungera "sömlöst" är en genomtänkt lager 2- och lager 3-design där nätverksmiljön är enhetlig över hela klustret.

Exempel: Innan du uppgraderar operativsystemet på en kritisk DNS-server tar du en snapshot. Om uppgraderingen kraschar nätverket kan du återställa servern omedelbart. Samtidigt körs en fullständig backup varje natt till en extern lagringsserver i en annan byggnad.

Om serverhallen drabbas av ett strömavbrott som förstör diskarna, kan du använda backupen för att återställa DNS-tjänsten på en helt ny fysisk server. Om den ordinarie servern behöver mer minne, kan du genom Live Migration flytta DNS-servern till en mer kraftfull värd i klustret utan att någon på skolan tappar sin internetanslutning under tiden.

Kontrollfrågor

1. Ge tre skäl till att virtualisera som direkt påverkar tillgänglighet och arbetstakt.
2. Förklara skillnaden mellan **snapshot** och **backup** och när respektive används.
3. Varför bör management, lagring, backup och produktion separeras nätmässigt?
4. När väljer du Proxmox framför vSphere – och tvärtom?
5. Vad talar för Hyper-V i en Windows-tung skolmiljö?
6. Hur sätter du en VM på rätt VLAN i vSphere, Hyper-V respektive Proxmox (kort steg för steg)?
7. Vad innebär **overcommit** och vilka risker finns för RAM och I/O?
8. Nämn två sätt att skapa ett **isolerat NAT-labb** utan att påverka produktion.
9. Vilka komponenter behövs för **HA + live migration** i praktiken?
10. Hur skulle du licensiera Windows-tunga VM-värdar kostnadseffektivt (Standard vs Datacenter)?

Fördjupningslänkar

- Proxmox VE docs: <https://pve.proxmox.com/pve-docs/>
- VMware vSphere docs: <https://docs.vmware.com/>
- Microsoft Hyper-V docs: <https://learn.microsoft.com/windows-server/virtualization/hyper-v/>
- XCP-ng & XO: <https://xcp-ng.org/> och <https://xen-orchestra.com/>
- libvirt/KVM: <https://libvirt.org/> och <https://qemu.readthedocs.io/>
- Linux bridge/OVS grunder:
<https://wiki.linuxfoundation.org/networking/bridge>

Egna anteckningar

...

5.2 Fördjupning: Windows Server och GPO

5.2.1 Vad är GPO – och var börjar man?

Tänk **Group Policy (GPO)** som en central **regibok** för Windows-klienter och -servrar. Du beskriver hur datorer och användarsessioner ska se ut och bete sig, och **domänen** ser till att det händer — automatiskt och konsekvent. Det kan vara allt från skrivbordsdetaljer och genvägar till säkerhet, kryptering, brandvägg, programvaror och uppdateringar. Vitsen är **förutsägbart**, **spårbarhet** och **fart**: samma in, samma ut, utan handpåläggning på varje maskin.

Du arbetar i **Group Policy Management Console (GPMC)**. Där ser du skog → domän → **Group Policy Objects** (själva “dokumenten”) och **OU:er** (behållare där du **länkar** dem). Öppnar du en GPO möts du av två huvuddelar:

- **Computer Configuration** (gäller datorn oavsett vem som loggar in).
- **User Configuration** (gäller användarsessionen på den datorn).

En bra start är att skapa en enkel GPO (t.ex. standardbakgrund), **länka** den till en test-OU och se hur inställningen rullar ut. För att alla admins ska se samma policyspråk och versioner lägger man **administrativa mallar** (ADMX/ADML) i en **Central Store** i SYSVOL. Då blir det lättare att samarbeta och undvika “det såg inte ut så på min dator”.

Exempel: Skapa “GPO-Desktop-Standard”. Sätt ett neutralt skrivbordsunderlägg och dölja vissa kontrollpaneler. Länka till en test-OU med några datorer. Vid nästa uppdatering får användarna samma skrivbordsmiljö — inga manuella klick.

5.2.2 Hierarki och utvärdering – vem “vinner” när policies krockar?

Windows tillämpar policy i ordningen **Lokal → Site → Domain → OU** (L-S-D-O-U). Varje steg kan lägga till eller skriva över inställningar. I en och samma behållare styr även **Link Order**: länken som ligger **sist** kör **sist** — och vinner vid krock.

Två “brytare” påverkar arvet:

- **Enforced (No override):** tvingar igenom en länkad GPO nedåt även om någon längre ner försöker skriva över den.
- **Block Inheritance:** stoppar arv från ovan — men **Enforced** går igenom ändå.

Särfallet **Loopback Processing** gör att **användarinställningar** kan tas från **datorns OU**. Det är ovärderligt för delade maskiner, kiosker och VDI.

- **Merge:** användarpolicyn kombineras; datorns OU kan överstyra detaljer.
- **Replace:** användarpolicyn kommer enbart från datorns OU.

Exempel: En domänlänkad GPO “Säkerhetsbas” sätter standardregler. I en viss server-OU finns “RDP-Tilldelning”. Om de krockar vinner OU-länken — om inte “Säkerhetsbas” är **Enforced**.

Bakgrundnot: Klienter uppdaterar i bakgrunden med jämna mellanrum. Viss funktionalitet (t.ex. programvaruinstallation) sker dock vid **start/inloggning**. gpupdate /force triggar en omedelbar uppdatering, men kräver ibland omstart för att allt ska slå igenom.

5.2.3 Från lätt till tungt – vad kan du styra med GPO?

Börja med **det synliga** — det ger snabb bekräftelse på att kopplingar och arv fungerar — och bygg sedan upp mot säkerhet och drift.

- **Administrative Templates:** färdiga policyer som i grunden skriver välkända registernycklar. Exempel: skrivbordsunderlägg, kontrollpaneler, webbläsarpolicyer, Onedrive/Office-inställningar.
- **Group Policy Preferences (GPP):** lägger till filer, mappar, genvägar, drivmappar och skrivare. Med **Item-Level Targeting** kan du villkora per **grupp, OU, OS-version, plats, IP-område, datornamn** m.m. Resultatet blir samma precision som skript — men mer robust.
- **Security Filtering:** låt bara en viss **säkerhetsgrupp** påverkas av GPO:t.

- **WMI-filter:** träffa t.ex. bara klient-OS, eller bara bärbara (har batteri). Använd sparsamt — många eller tunga filter ökar inloggningstid.

Exempel: En GPP-drivmapp syns bara för gruppen “Ekonomi” **och** endast på datorer i “OU=Ekonomi”. Ingen annan påverkas, även om GPO:t är länkat högre upp.

5.2.4 Skript via GPO – när är Startup/Logon rätt, och när är Schemalagd uppgift bättre?

Startup/Shutdown körs i **dator-kontext** (före/efter inloggning) som **Local System**. Perfekt för maskinnära saker: registry, tjänster, protokoll, filkopior som kräver admin.

Logon/Logoff körs i **användarsession** och passar profilnära uppgifter: genvägar, små städjobb, användar-specifika registerjusteringar.

Vill du ha **retry**, **fördröjning**, **villkor**, **tidsgränser** och bättre loggning använder du **Scheduled Tasks via GPP**. Kör som **SYSTEM** eller **tjänstkonto**, trigga “At startup”, “At logon” eller vid schema. Lägg skript i **SYSVOL** (gemensamt, versionsstyrt) och styr **Execution Policy**/kodsignering i GPO. Tänk på att nätverksresurser kanske inte finns tillgängliga precis i uppstart — vill du garantera det aktiverar du policyn *Always wait for the network at computer startup and logon* i en bas-GPO.

Exempel: Ett Startup-skript stänger av SMBv1 och sätter NTP-källa. En Schemalagd uppgift (SYSTEM) kör varje natt och rensar temporära mappar; resultatet loggas i Event Viewer.

Litet skript-exempel (Startup, körs som SYSTEM)

Exempel: Lägg i `\\<domän>\SYSVOL\<domän>\Scripts\Startup\Hardening.ps1` och kör via **Computer Configuration → Policies → Windows Settings → Scripts (Startup/Shutdown)** eller som **Scheduled Task (At startup)**.

```
# Hardening.ps1 - körs som Local System vid startup
```

```
# 1) Stäng av SMBv1 (klientkomponent)
Try { Disable-WindowsOptionalFeature -Online -FeatureName SMB1Protocol -
NoRestart -ErrorAction Stop } Catch { }
```

```
# 2) Stäng av SMBv1 (serversidan)
```

```
Try { Set-SmbServerConfiguration -EnableSMB1Protocol $false -Force } Catch  
{ }
```

```
# 3) Sätt NTP-källa (ersätt med er tidskälla)  
$ntp = 'time.example.org,0x8'  
w32tm /config /syncfromflags:manual /manualpeerlist:$ntp /update | Out-Null  
w32tm /resync /force | Out-Null
```

```
# 4) Logga enkel status  
$msg = "Hardening klar: SMBv1 avstängd, NTP=$ntp"  
Write-EventLog -LogName Application -Source "Windows PowerShell" -EntryType  
Information -EventId 1001 -Message $msg
```

5.2.5 Programvarudistribution – strategier, val och fallgropar

Utmaningen: Program kommer som **MSI**, **EXE** eller **MSIX**. De har olika tysta switchar, kräver olika **detektion** (hur vet vi att appen redan finns?), har beroenden (drivrutiner, VC++-omfördelningsbara), och vissa kräver **omstart**. En bra strategi ger: spårbarhet (loggar/status), **piloter** (testgrupper), **ringar** (stegvis utrullning) och **rollback**.

A) GPO + MSI (enkelt och stabilt)

Med **Software Installation** i GPO kan du **Assign to Computer** ett **MSI**. Installationen sker vid start, innan inloggning, vilket är robust. Lägg källan på **UNC-share** med läsrätt för datorer. Anpassa via **MST** (transform). Begränsningar: främst MSI, svagt för EXE, enkel rapportering.

B) GPO + skript/GPP (för EXE och specialfall)

Kör EXE med tyst flagga (/quiet, /S, /VERYSILENT) via **Startup-skript** eller **Scheduled Task** (SYSTEM). Bygg in **detektion** (kontrollera fil/reg-nyckel, kör inte igen om det redan finns) och **logga** exit-koder. Bra för mindre/medel miljö — men standardisera loggformat, uninstall och rollback.

C) Intune (MDM) - ringar, självbetjäning och internetnoder

Paketering till **.intunewin**, definiera **install/uninstall**-kommandon, **detektionsregler** och **beroenden**. Kör mot användare eller enheter, gör **Required** (tvingad) eller **Available** i portal. Styrka: ringar, självbetjäning, funkar utanför LAN.

D) Configuration Manager (SCCM/MECM) - djup kontroll on-prem

Full kontroll över **applikationer**, **samlingar**, **phased deployments**,

detaljerad status och rapporter. Passar större on-prem-miljöer med höga krav på precision och uppföljning.

Gemensamma råd:

- Ha en **paketstandard** (katalognamn, logg, detektion, uninstall).
- Skilj **installationskonto** från **detektionslogik**; detection ska vara snabb och pålitlig.
- Undvik “eviga” installationsförsök genom att **bryta** på tydliga exit-koder.
- Planera **omstarter** (t.ex. via deadlines eller nattfönster).
- Kör **pilot** först; utvärdera innan du breddar.

Hands-on: MSI via GPO (Assigned to Computer)

1. Lägg App.msi (ev. App.mst) på \\fileserversoftware\App\ och ge **Read** för *Domain Computers*.
2. Skapa GPO “GPO-Apps-App”.
3. Öppna GPO → **Computer Configuration** → **Policies** → **Software Settings** → **Software Installation** → *New* → *Package...* → peka på **UNC** → **Assigned**.
4. (Valfritt) Lägg App.mst under *Modifications*.
5. **Länka** GPO:t till mål-OU (datorer).
6. Sätt *Always wait for the network at computer startup and logon* = **Enabled** i bas-GPO.
7. Testa: gpupdate /force, **omstart**, verifiera i loggar/Program och funktioner.
8. Uppgradera via *Upgrades*-fliken, eller ta bort med **Remove** (ev. *Immediately uninstall*).

Exempel: “GPO-Apps-Reader” installerar Adobe Reader tyst från \\fileserversoftware\Reader\AcroRead.msi, med AcroRead.mst för språk och avstängd auto-update.

5.2.6 Centrala uppdateringar – WUfB, WSUS och ringar

Målet är att få en trygg rytm: små säkra steg ofta, och stora steg när du är redo — med **ringar** som fångar fel tidigt.

- **Windows Update for Business (WUfB):** Klienter hämtar från Microsoft Update, men du styr **deferral** (hur länge de väntar), **deadlines** (när de måste installera/boota) och **ringar** (Pilot → Bred → Sen). Fungerar utmärkt för enheter som inte alltid är på LAN.
- **WSUS:** Du **godkänner** uppdateringar centralt och styr vilka **datorgrupper** som får vad — när. Bra när du behöver lokal cache, strikt kontroll eller avskilda nät.

Tänk i två flöden:

- **Quality Updates** (månatliga säkerhetsfixar): rulla snabbt genom ringar med kort deferral.
- **Feature Updates** (större OS-lyft): längre deferral och tydligare pilot.

Glöm inte **Delivery Optimization** så klienter delar paket i LAN; det sparar internetkapacitet. Bestäm hur du hanterar **drivrutiner**, **.NET** och **Microsoft 365 Apps** (de kan ha egna kanaler).

Hands-on: WUfB ring-setup (Pilot → Bred → Sen)

A. Grupper och GPO:n

1. Skapa tre **säkerhetsgrupper**: WUfB-Pilot, WUfB-Bred, WUfB-Sen.
2. Skapa tre **GPO:n**: “WUfB-Pilot”, “WUfB-Bred”, “WUfB-Sen”. Länka dem högt upp, men låt **Security Filtering** styra att rätt grupp får rätt GPO.

B. Policys (Computer Configuration → Administrative Templates → Windows Update)

1. **Select when Feature Updates are received = Enabled** →
Deferral days: Pilot **0**, Bred **7-14**, Sen **21-28**.
2. **Select when Quality Updates are received = Enabled** →
Deferral days: Pilot **0**, Bred **2-7**, Sen **10-14**.

3. **Specify deadlines for automatic updates and restarts = Enabled** (sätt rimliga deadlines och grace).
4. **Configure Automatic Updates = Enabled** (auto-nedladdning och schemalagd install).
5. **Delivery Optimization**: sätt läge för LAN-delning.
6. Har ni haft **WSUS** tidigare? Säkerställ att intranät-WSUS **inte** längre pekas ut i dessa GPO:n (WUfB och WSUS samtidigt skapar konflikt).

C. Rulla

1. gpupdate /force på Pilotenheter. Kontrollera att inställningarna syns på Windows Update-sidan.
2. Följ upp status med dina verktyg (oavsett om det är Intune, SCCM eller egen inventering).
3. När piloten är grön: flytta fler enheter in i **Bred**. Upprepa mot **Sen**.

Exempel: Pilot har 0 dagars deferral och 7 dagars deadline. Bred har 14/14. Sen har 28/21. Om något fallerar i Piloten pausar du ringen eller höjer deferral i Bred/Sen tills fix finns.

5.2.7 Felsökning – se vad som faktiskt gäller

Börja alltid med **vad klienten tror är sant**:

- gpresult /h rapport.html visar **vilka GPO:er** som tillämpats (för dator och användare), vilka som filtrerats bort (säkerhet/WMI) och varför.
- **RSOP.msc** ger en snabb översikt av “Resultant Set of Policy”.
- **Event Viewer → Microsoft → Windows → GroupPolicy/Operational** avslöjar timing och fel.
- I **GPMC** kan du **modellera** förväntat utfall (vilka GPO som *bör* gälla) och jämföra med **Results** från en faktisk dator.

Vanliga bovar:

- **Fel OU/ingen länk** → klienten nås inte av GPO:t.
- **Security Filtering** träffar inte datorn/användaren.

- **WMI-filter** för snävt eller långsamt.
- **Nätverk inte klart vid boot** → programvaruinstall missar (aktivera “Always wait for the network...”).
- **AD-replikering** släpar → testa på DC där klienten hämtar policy.
- **SYSVOL-behörigheter**/delningsproblem → MSI-källa når inte fram.

Exempel: *En skrivare mappas inte. gpresult visar att GPO:t filtrerats bort av ett WMI-filter för “Bärbar”. Datoren är stationär. Lösning: flytta villkoret till **Item-Level Targeting** (OU/grupp) eller justera WMI-villkoret.*

Kontrollfrågor

1. Vad vinner du på att styra Windows via GPO istället för att manuellt konfigurera varje maskin?
2. Förklara **L-S-D-O-U** och hur **Enforced** respektive **Block Inheritance** påverkar utfallet.
3. När väljer du **Loopback (Merge/Replace)** och hur påverkar det användarpolicyn?
4. Ge ett exempel där du kombinerar **Administrative Templates**, **GPP** och **Item-Level Targeting**.
5. När är **Startup/Logon-skript** rätt val, och när är **Scheduled Task via GPP** bättre?
6. Beskriv ett komplett **MSI via GPO**-flöde och hur du planerar **uppgradering/rollback**.
7. Hur hanterar du **EXE-appar** centralt med detektion och loggning — utan tredjepartsverktyg?
8. Vad skiljer **WUfB** från **WSUS**, och när väljer du det ena framför det andra?
9. Hur designar du **ringar** (Pilot → Bred → Sen) för både **Quality** och **Feature** Updates?
10. Bygg en enkel felsökningskedja när ett GPO inte tillämpas (vilka verktyg, i vilken ordning, och varför).

Fördjupningslänkar

Group Policy – översikt:

<https://learn.microsoft.com/windows-server/identity/ad-ds/manage/group-policy/group-policy-overview>

Group Policy processing (LSDOU, Enforced/Block):

<https://learn.microsoft.com/windows-server/identity/ad-ds/manage/group-policy/group-policy-processing>

Loopback processing: <https://learn.microsoft.com/troubleshoot/windows-server/group-policy/loopback-processing-of-group-policy>

Group Policy Preferences & Item-Level Targeting:

<https://learn.microsoft.com/windows-server/identity/ad-ds/manage/group-policy/group-policy-preferences>

WMI-filter – skapa och använda: [https://learn.microsoft.com/previous-versions/windows/it-pro/windows-server-2012-r2-and-2012/jj717288\(v=ws.11\)](https://learn.microsoft.com/previous-versions/windows/it-pro/windows-server-2012-r2-and-2012/jj717288(v=ws.11))

Installera program via GPO (MSI):

<https://learn.microsoft.com/troubleshoot/windows-server/group-policy/use-group-policy-to-install-software>

Intune – Win32 app management:

<https://learn.microsoft.com/intune/intune-service/apps/apps-win32-app-management>

Windows Update for Business – konfigurera:

<https://learn.microsoft.com/windows/deployment/update/waas-configure-wufb>

WSUS – kom igång:

<https://learn.microsoft.com/windows-server/administration/windows-server-update-services/get-started/windows-server-update-services-wsus>

gpresult – visa effektiv policy: <https://learn.microsoft.com/windows-server/administration/windows-commands/gpresult>

Central Store för ADMX/ADML:

<https://learn.microsoft.com/troubleshoot/windows-client/group-policy/create-and-manage-central-store>

Egna anteckningar

...

5.3 Fördjupning: Linux-server och policy-styrning

5.3.1 Varför Linux på server?

Om man vill bygga stabila, resurssnåla och flexibla serversystem är Linux ofta ett naturligt val. Man får stor kontroll över vilka tjänster som körs, kan automatisera nästan allt och skala från en liten VM till ett helt kluster. Licensmodellen (öppen källkod eller abonnemang) gör också att ekonomi och support kan anpassas efter verksamheten. Tanken är att beskriva hur systemet **ska** se ut och uppföra sig, och sedan låta verktygen upprepa det — varje gång, på varje server.

Exempel: En gemensam “bas” med samma SSH-policy, brandvägg och loggning används för flera servrar. Varje maskin får sedan sin roll (webb, fil, proxy), men grundbeteendet är likadant och enkelt att återskapa.

5.3.2 Grunden: distribution, paket, konton och baspolicy

Om man är ny och ovan vid att sätta upp Linuxservrar är det lämpligt att välja en **LTS-distribution** (t.ex. Ubuntu Server LTS eller Debian Stable; Rocky/Alma om man vill ha RHEL-kompatibilitet). LTS brukar innebära längre säkerhetsstöd och lugnare förändringstakt. Paket hanteras typiskt via **APT** (Debian/Ubuntu) eller **DNF** (RHEL-familjen); många upplever att det blir tryggast att i första hand hålla sig till distributionens egna arkiv.

När det gäller **konton och rättigheter** börjar många med vanliga användarkonton och låter administratörsrätt gå via **sudo**. Det kan kännas tryggt att logga förhöjda kommandon, så att man i efterhand kan se vad som körts och av vem. Åtkomsten sker ofta via **SSH**; då väljer många att använda **nycklar** i stället för lösenord och, där riskbilden motiverar det, komplettera med tvåfaktor via PAM.

Baspolicy i praktiken: tidsynk (chrony eller systemd-timesyncd), en enkel brandvägg (ufw eller firewalld), ett MAC-skydd (AppArmor eller SELinux), central loggning (rsyslog) och förutsägbar loggrotation. Det är inte komplicerat — men gör stor skillnad över tid.

Exempel: En nysatt server får ett administratörskonto med sudo, SSH-nyckelinloggning, brandvägg som bara öppnar 22/80/443, tidsynk mot intern NTP, och loggar som skickas till en central loggtjänst.

5.3.3 Nätverk på Linux: IP, VLAN, bond och brygga

När du ska konfigurera nätverket i en Linuxserver är det viktigt att man håller sig till **ett** konfigurationssätt (netplan/systemd-networkd, NetworkManager eller distributionsspecifika filer) och inte blandar. Det underlättar felsökning och minskar risken för konflikter där flera komponenter försöker göra samma sak. I serverdrift handlar nätet ofta om några återkommande byggstenar: **statisk IP** med gateway/DNS, **VLAN** för segmentering, **bonding/LACP** för redundans/kapacitet, och ibland en **bridge** om värden ska husera KVM-VM:er eller containrar.

Exempel: En webb-VM i DMZ (VLAN 70) pratar bara med reverse-proxyn. Backuptrafik går på ett andra interface i VLAN 99 till backupservern — skilt från produktionstrafiken.

Så här kan en minimal netplan-definition se ut (statisk IP + VLAN):

```
# /etc/netplan/01-server.yaml
network:
  version: 2
  ethernet:
    eno1:
      dhcp4: no
  vlans:
    vlan40:
      id: 40
      link: eno1
      addresses: [192.0.2.10/24]
      gateway4: 192.0.2.1
      nameservers:
        addresses: [192.0.2.53, 192.0.2.54]
```

5.3.4 Vanliga tjänster: SSH/SFTP, SMB/NFS, webb och certifikat

I många miljöer börjar man med **SSH/SFTP** för administration och filöverföring. Det är vanligt att stänga av lösenordsinloggning, styra vilka användare eller grupper som får logga in och låta åtkomst gå via en bastionvärd i stället för direkt från internet. För filresurser väljer många

Samba/SMB när Windows-klienter ska ansluta och **NFSv4** när det är Linux-till-Linux — båda kan använda POSIX-ACL för finmaskiga rättigheter. För webb är **Nginx** ett vanligt val, ofta som **reverse proxy** framför applikationer, med automatiserad hantering av TLS-certifikat (t.ex. via ACME/certbot) så att förnyelser sköts utan handpåläggning.

Exempel: En publik webbapp kör bakom Nginx. Endast 80/443 är öppna i brandväggen. SSH är åtkomligt enbart från bastion, inte från internet i stort.

5.3.5 Central identitet och “policy”: AD/SSSD, FreeIPA och polkit

Om man vill slippa separata lokala konton på varje server går det fint att koppla Linux till **central identitet**. Med **realmd/SSSD** kan system autentisera mot **AD** och tilldela sudo-rättigheter till vissa grupper. I Linuxvärlden finns också **FreeIPA**, som kombinerar LDAP, Kerberos, certifikat och host-baserad åtkomstkontroll. Lokalt på varje maskin handlar det ofta om **sudoers** och **polkit** (vad en användare får göra när en tjänst ber om höjda privilegier), samt **PAM** för hur inloggningar ska gå till.

Exempel: Gruppen “ops-admin” i AD får sudo på alla Linux-servrar (styrt via SSSD och en enkel fil i /etc/sudoers.d/). Gruppen “ops-ro” får bara läsa journalen och köra ett fåtal diagnostikkommandon.

5.3.6 “Policy som kod”: Ansible som Linux-motsvarighet till GPO

När man vill att servrar ska se likadana ut över tid, oavsett vem som satt dem, brukar **Ansible** vara ett lättillgängligt verktyg. Idén är att beskriva önskat tillstånd i **YAML**; körningen är **idempotent**, så bara det som faktiskt behöver ändras ändras. Många börjar med att standardisera paket, tjänster, sshd-inställningar, brandvägg, användare och filrättigheter — och lägger allt i ett versionshanterat repo.

Så här kan en liten bas-playbook se ut (illustration):

```
- hosts: linuxservrar
  become: true
  tasks:
    - name: Installera baspaket
      package:
        name: [vim, curl, htop, fail2ban]
```

```
state: present

- name: Hårda sshd (stäng lösenord)
  lineinfile:
    path: /etc/ssh/sshd_config
    regexp: '^PasswordAuthentication'
    line: 'PasswordAuthentication no'
  notify: restart ssh

- name: Enkelt brandväggsläge
  ufw:
    state: enabled
    rule: allow
    port: "{{ item }}"
  loop: [22, 80, 443]

handlers:
- name: restart ssh
  service:
    name: ssh
    state: restarted
```

Exempel: En nattlig körning applicerar "Bas-Linux" på alla servrar. Om någon råkat ändra en konfigfil för hand, återställs den till överenskommet läge.

5.3.7 Programvarudistribution och uppdateringar på Linux

För själva programvaran upplever många att det blir smidigast att låta distributionens pakethanterare (APT eller DNF) göra jobbet. Det kan vara hjälpsamt att tänka i **ringar** även här: en liten pilotgrupp först, därefter bredare utrullning, och sist de mest känsliga systemen.

Säkerhetsuppdateringar tenderar att rullas snabbare, medan funktionsuppdateringar ofta får passera en testperiod. På Debian/Ubuntu kan **unattended-upgrades** sköta säkerhetsfixar automatiskt; på RHEL-familjen finns **dnf-automatic**. Kärnuppdateringar kan kräva omstart, så planerade fönster gör livet enklare — i vissa miljöer använder man livepatch för att minska antalet omstarter.

Så här kan ett första steg mot automatiska säkerhetsfixar se ut (illustration):

```
# Debian/Ubuntu
sudo apt install unattended-upgrades
```

```
sudo dpkg-reconfigure unattended-upgrades
```

```
# RHEL/Rocky/Alma
sudo dnf install dnf-automatic
sudo systemctl enable --now dnf-automatic.timer
```

Exempel: Tre inventariegrupper (pilot → bred → sen). Säkerhetsfixar går direkt till alla; större förändringar provar man en vecka i pilot innan de släpps vidare.

5.3.8 Lokala paketförråd (repos) – försörjningskedjans säkerhet i praktiken

När många servrar ska uppdateras regelbundet vill man ofta ha **en egen, kontrollerad källa**. Ett lokalt repo — som spegel, caching-proxy eller **kuraterad källa** — gör att man kan granska och signera innehåll, frysa en viss versionsmängd och “promota” den från **dev → test → prod**. Det minskar exponeringen mot internet, ger förutsägbara byggen och gör rollback enklare.

Exempel: Produktion pekar bara mot .../prod/ och kräver signerat index. Test pekar mot .../test/ och får nya paket en vecka tidigare. Upptäcks en regression i test, stannar promotion — produktion påverkas inte.

Så här kan det se ut i Debian/Ubuntu-spår (översikt):

- repo-servern har en egen GPG-nyckel; indexfiler signeras.
- klienterna installerar **din** keyring och pekas mot **din** URL (med signed-by=).
- pinning kan låsa så att inget annat arkiv används av misstag.

Och i RHEL-spår:

- metadata genereras (t.ex. med createrepo_c), signeras och exponeras via HTTPS,
- klienterna pekar på ditt baseurl och kräver signatur/verifiering,
- externa .repo-filer avaktiveras och egress kan stängas i brandvägg.

5.3.9 Loggning och övervakning

Om man vill kunna förstå vad som hänt i efterhand — eller upptäcka avvikelser i tid — blir central loggning och egna mätvärden värdefulla. I Linux börjar det i **systemd-journald**; sedan skickas loggar ofta vidare till

t.ex. rsyslog och en central loggtjänst. För mätvärden använder många en liten agent (t.ex. node_exporter) och ett system som kan larma när värden avviker från det normala.

Exempel: SSH-inloggningar, sudo-försök och tjänstestarter skickas till en central vy. Ett enkelt larm reagerar om flera misslyckade inloggningar sker på kort tid.

5.3.10 Backup och återställning

När man pratar backup handlar det i grunden om att bestämma **RPO** (hur mycket data man får förlora) och **RTO** (hur snabbt det måste gå att komma tillbaka). På Linux är verktyg som **restic** eller **borg** vanliga för filnivåbackup (deduplicering och kryptering), medan hypervisorn ofta tar **VM-backup**. Det brukar bli mest robust att kombinera båda: en bild av hela VM:n **och** en applikationskonsistent backup av själva datan i gästen. Snapshots (t.ex. ZFS/LVM) kan hjälpa vid förändringar — men ses inte som backup i sig.

Exempel: Varje natt kör restic till ett S3-kompatibelt objektlager (med immutabilitet). En gång i veckan återställs en verklig mapp till en "karantän-server" och jämförs med checksumma.

5.3.11 Säkerhetsbaslinje – hur "bra" brukar se ut

Många utgår från några återkommande byggstenar: **SSH med nycklar** (och gärna avstängd lösenordsinloggning), **default-deny** i brandväggen, **fail2ban** eller motsvarande för att dämpa bruteforce, ett **MAC-skydd** (AppArmor/SELinux) som faktiskt är påslaget, och enkel **filintegritetskontroll** på känsliga system. Principen "minsta privilegium" gäller i sudo/polkit — tillräckligt för att lösa uppgiften, men inte mer.

Exempel: En bastion får samma bas som övriga, men med extra hårdning: enbart nyckel-SSH, tvåfaktor via PAM, stram polkit och realtidsloggning till SIEM.

Kontrollfrågor

1. Vilka fördelar får man av att beskriva Linux-servrar “som kod”, och hur påverkar det felsökning och återställning?
2. När kan en LTS-distribution vara ett klokt val, och vad vinner man på att främst använda standardarkiv?
3. Hur skulle du motivera nyckelinloggning i SSH och loggning av förhöjda kommandon för en ny admin?
4. Varför blir det lättare att felsöka om man håller sig till **ett** nätkonfigurationssätt, och när kommer VLAN/bond/bridge in i bilden?
5. I vilka lägen känns Samba/SMB respektive NFS mest naturligt, och hur påverkar valen rättighetshanteringen?
6. Vad kan AD/SSSD eller FreeIPA lösa i central identitet, och vad behöver fortfarande styras lokalt (sudo/polkit)?
7. Hur hjälper Ansible dig att hålla en baspolicy intakt, och vad betyder idempotens i vardagen?
8. Hur kan man lägga upp uppdateringar i ringar för Linux, och varför brukar säkerhetsfixar gå snabbare?
9. Vad vinner man på lokala repos (spegel, proxy eller kuraterat), och hur hänger signering och promotion ihop med rollback?
10. Hur kombinerar du VM-backup och gäst-backup för att nå RPO/RTO, och hur visar du att återställning faktiskt fungerar?

Fördjupningslänkar

- Ubuntu Server Guide: <https://ubuntu.com/server/docs>
- Debian Admin Handbook: <https://debian-handbook.info/>
- Red Hat Enterprise Linux – Security & Hardening:
https://access.redhat.com/documentation/en-us/red_hat_enterprise_linux/
- Ansible Documentation: <https://docs.ansible.com/>
- Netplan Examples: <https://netplan.io/examples>
- OpenSSH manpages: https://man.openbsd.org/ssh_config
- Samba Documentation: <https://www.samba.org/samba/docs/>
- FreeIPA Documentation:
<https://www.freeipa.org/page/Documentation>
- restic (backup): <https://restic.net/>
- Prometheus node_exporter:
<https://prometheus.io/docs/guides/node-exporter/>

Egna anteckningar

...

5.4 MDM i praktiken

5.4.1 Vad är MDM, EMM och UDM/UEM — likheter och skillnader

Om man vill styra datorer och mobila enheter centralt möter man tre begrepp: **MDM**, **EMM** och **UDM/UEM**. De bygger på samma idé men har olika räckvidd.

- **MDM (Mobile Device Management)** fokuserar på *enheten*: registrera den, tilldela profiler (WLAN, VPN, certifikat), slå på kryptering/brandvägg, rulla ut appar och vid behov fjärr-låsa eller rensa.
- **EMM (Enterprise Mobility Management)** bygger vidare med app-hantering och dataskydd i mobila appar (t.ex. separera jobbdata från privat).
- **UEM/UDM (Unified Endpoint Management/Device Management)** samlar *alla slutpunkter* (Windows, macOS, iOS/iPadOS, Android — och ofta även Linux/särskilda enheter) i ett och samma system och knyter ihop **identitet, compliance, villkorad åtkomst, patch** och ibland **endpoint-skydd**.

I vardagen används orden ibland blandat; viktigast är vad plattformen faktiskt klarar och hur djupt den når i varje operativsystem.

Exempel (Exempel:) *En verksamhet börjar med MDM för att få kryptering och WLAN-profiler på bärbara. När behoven växer (app-livscykel, villkorad åtkomst, patch) landar man i en UEM-plattform som knyts till identitetstjänsten.*

5.4.2 Varför MDM/UEM behövs — automatisering, förutsägbarhet och säkerhet

Om man ska hantera många enheter över tid fungerar inte manuellt pill. MDM/UEM gör konfiguration **upprepad och spårbar**: nya enheter går från kartong till drift på minuter, förändringar rullas ut kontrollerat och säkerhetskrav kan **bevisas** (t.ex. “alla bärbara är krypterade, brandväggen är på, och OS är patchat”). Utan MDM/UEM uppstår ofta

driftsspridning, bortglömda uppdateringar, delade WLAN-lösenord och osäker onboarding/offboarding.

Exempel (Exempel:) *En borttappad laptop med MDM är krypterad, spärras omedelbart och kan fjärr-rensas. Utan MDM vet ingen om den var krypterad eller vad den fortfarande når.*

5.4.3 Enrollment — manuell, automatisk och “zero-touch”

Enrollment kopplar enheten till plattformen så att policyer och appar börjar gälla.

- **Manuell enrollment** passar BYOD: användaren installerar “företagsportalen”, loggar in och får en arbetsprofil (Android) eller hanteras via enhetens hanteringsvy (iOS/macOS).
- **Automatisk/zero-touch** passar företagsägda enheter: serienummer/hårdvaru-ID registreras i förväg hos återförsäljaren. När enheten startar vet den vem som äger den och låser sig till rätt MDM.
 - **Windows:** Autopilot (användardrivet, self-deploying eller pre-provisioned).
 - **Apple:** Automated Device Enrollment via Apple Business Manager (MDM-tvingning).
 - **Android:** Android Enterprise med zero-touch/QR/KNOX Mobile Enrollment. **Knox** är **Samsungägt/styrt** och ger extra djup på Samsung-enheter.
 - **Chromebook:** enrollment via Google Admin.
 - **Linux:** ingen universell MDM-kanal; man använder agent-/policyverktyg i en UEM-lösning (compliance, skript, patch).

Exempel (Exempel:) *En företagsägd Android startas, känner igen sin ägare via zero-touch och skapar en arbetsprofil. Policyer och företagsappar landar utan att IT rört enheten.*

5.4.4 Microsoft Intune — arkitektur, förutsättningar och koppling till Windows AD

*Om man vill ha en modern, molnburen plattform som kan styra många olika operativsystem är **Microsoft Intune** ett naturligt val. I praktiken fungerar Intune som ett **styrplan**: tjänsten pratar den inbyggda **MDM-kanalen** mot varje klient (Windows, iOS/iPadOS, macOS, Android och i viss mån Linux) och ser till att profiler, policyer, appar och uppdateringsregler landar där de ska. Intune krokar djupt i **Microsoft Entra** (identitet och grupper), i **villkorad åtkomst** (bara “gröna” enheter får nå känsliga appar) och samspelar ofta med **Microsoft Defender** för att koppla säkerhetsstatus till åtkomstbeslut. I kontrast till äldre, lokala lösningar (som enbart **GPO** eller en fristående **SCCM/MECM**) är Intune designad för att fungera var enheten än befinner sig — på kontoret, i hemmet eller på resa — så länge den når internet.*

För att Intune ska kunna arbeta behöver några grundbultar vara på plats. Själva **tenanten** måste ha licenser som ger Intune/UEM-funktionalitet, och man sätter **MDM-auktoriteten** till Intune så att plattformen verkligen är “ägaren” av MDM-kanalen. Därefter kopplar man varje plattform till sitt respektive ekosystem: Apple-enheter kräver ett **APNs-certifikat** och blir starkast när de hanteras via **Apple Business Manager** (där både **ADE** för automatisk enhetsregistrering och **VPP** för app-licenser finns). Android-sidan kopplas via **Android Enterprise-bindning**, och i företagsägda scenarier använder man gärna **zero-touch**; på Samsung-enheter finns dessutom **Knox-stödet** som ger extra djup. Windows-datorer registreras smidigast via **Autopilot**, där enhetens hårdvaru-ID (hash) gör att rätt profil och rätt appar rullar in redan vid första uppstarten.

Så fort man vill använda **certifikat** för saker som **WLAN (EAP-TLS)** eller **VPN** behöver man också en **certifikatkedja**. Det löses vanligtvis med en **Intune Certificate Connector** på en server **on-prem** som pratar med organisationens **AD CS**. För Windows-klienter kan man antingen dela ut cert direkt (**PKCS**) eller via **SCEP/NDES** för mer skalbarhet och automation. Poängen är att Intune bestämmer *vem* som ska få vilket certifikat och *när*, medan AD CS faktiskt **utfärdar** det; klienterna kan därefter ansluta till rätt **WLAN** (operativsystemet kallar det ofta “Wi-Fi”,

men i boken använder vi begreppet **WLAN**) och rätt VPN utan delade lösenord.

Hur Intune hänger ihop med ett befintligt **Windows AD** beror på vilken väg man väljer. I ett **moln-först-upplägg** blir enheten **Entra-joined** (d.v.s. ansluten till molnets katalog) och styrs helt via Intune; GPO minimeras och identitet/åtkomst villkoras i molnet. Det här brukar vara enklast att förstå för nya klienter eftersom all styrning kommer från samma håll, och enheten fungerar lika bra på kontoret som utanför. I ett **hybrid-join-upplägg** lever enheten samtidigt i **on-prem AD** och i **Entra**. Då kan vissa historiska GPO:er finnas kvar medan Intune tar över det mesta av klientstyrningen (MDM-policy, appar, uppdateringar). Om man dessutom vill köra **Autopilot Hybrid Join** behövs en liten komponent — **Intune Connector for Active Directory** — som kan skapa en “offline domain join”-blob mot en DC under första uppstarten. Slutligen finns **co-management**, där en miljö med **SCCM/MECM** gradvis delar på ansvaret med Intune; till exempel kan **Windows Update for Business** och compliance flyttas till Intune medan vissa appar fortsatt levereras via SCCM. Oavsett väg är grundidén densamma: låt Intune vara **policy-motorn**, låt **Entra** vara **identitetslagret**, och använd **AD CS** som **tillitsfabrik** för certifikat.

När allt detta sitter på plats blir flödena begripliga. En ny bärbar dator som är registrerad i **Autopilot** startar upp, känner igen sin ägare, **enrollerar** sig själv i Intune och hämtar **basprofiler**: kryptering (BitLocker), brandvägg, uppdateringsring, certifikat och **WLAN-profil**. **Compliance** sätter ribban för vad som är “grönt”, och **villkorad åtkomst** använder den statusen när användaren försöker nå exempelvis e-post eller affärssystem. Om datorn istället ärver några kvarvarande **GPO** (i ett hybridläge) gäller det att hålla ansvarsfördelningen tydlig så man undviker policykrockar: det som ligger i molnet bör i första hand också styras från molnet, medan servernära eller speciella arv kan få leva kvar i GPO under en övergångsperiod.

Exempel: *En organisation med AD on-prem synkar användare och grupper till Entra för att slippa dubbla kataloger. Nya Windows-klienter körs som **moln-först** med Entra-join och Intune för all klientstyrning. Certifikat delas ut via SCEP-connectorn, och klienterna ansluter till skolans **WLAN** med **EAP-TLS**. Några särskilda labbmiljöer ligger kvar i en **hybrid-OU** där ett fåtal GPO:er används — men allt som rör klienternas*

säkerhetsbas, appar och uppdateringar ligger i Intune. Resultatet blir snabb onboarding, jämn säkerhetsnivå och tydlig spårbarhet.

5.4.5 Intune på djupet — bygg en bra grund (varför, hur, och vad som händer om man låter bli)

Gruppdesign (dynamiska grupper)

Varför: policyer/ appar ska landa rätt automatiskt (t.ex. “alla Windows-laptops”).

Hur: använd attribut (plattform, enhetstyp/ägande, taggar). Håll grupperna få och begripliga.

Om man låter bli: manuell målning → felträffar, glapp i säkerhet, långsam utrullning.

Basprofiler för nät & certifikat (WLAN/VPN + PKCS/SCEP)

Varför: EAP-TLS ersätter delade lösenord, minskar läckage och support.

Hur: sätt upp certifikatconnector + AD CS; skapa cert- och WLAN-profiler för aktuella SSID.

Om man låter bli: delade PSK sprids, byten blir stökiga, obehöriga kan “läcka in”.

Compliance + villkorad åtkomst

Varför: endast “gröna” enheter (kryptering, brandvägg, OS-nivå) får nå känsliga appar.

Hur: definiera compliance per plattform; lås kritiska appar bakom villkorad åtkomst.

Om man låter bli: stulna/eftersatta klienter når data; svårt att bevisa efterlevnad.

Uppdateringsringar (WUfB)

Varför: kvalitetsuppdateringar snabbt; funktionsuppgraderingar i ringar (Pilot → Bred → Sen).

Hur: skapa ringar med rimliga deferrals/deadlines; följ upp regelefterlevnad.

Om man låter bli: ojämn patch-nivå, fler incidenter och “stora hopp” som skapar driftstopp.

App-livscykel (Win32-paket, detektion, beroenden)

Varför: rätt version till rätt grupp; repeterbar installation/avinstallation.

Hur: definiera install/uninstall-kommandon, **detektionsregler** och

beroenden.

Om man låter bli: "zombie-appar", dubbletter/konflikter, oklar klientstatus.

Autopilot + Enrollment Status Page (ESP)

Varför: "kartong → färdig" utan handpåläggning; ESP blockerar inloggning tills basen är klar.

Hur: registrera enheter i Autopilot; välj scenario (user-driven/self-deploying/pre-provisioned); aktivera ESP.

Om man låter bli: lång onboarding, fler manuella fel, användare börjar på halvfärdig klient.

RBAC och förändringsstyrning

Varför: minska risken för fel i stor skala och skapa spårbarhet.

Hur: rollstyr åtkomst (minsta privilegium), inför dev/test/prod-tagging och change-fönster.

Om man låter bli: svår root-cause, oavsiktliga ändringar, större påverkan vid misstag.

Skript & proaktiv remediering

Varför: hitta/laga små driftsavvikelser automatiskt, innan de blir ärenden.

Hur: PowerShell (Windows) / shell (macOS/Linux där stött); kör schemalagda kontroller + korrigeringar.

Om man låter bli: små fel byggs upp och skapar störningar och manuellt efterarbete.

Exempel (Exempel:) *"Bas-Windows": dynamisk grupp för laptops, BitLocker-policy + compliance, Defender + ASR, WUfB-ringar, cert-/WLAN-profiler, Autopilot med ESP, samt kärnappar. Resten läggs på i lager.*

5.4.6 Konfigurationsexempel — utförliga typfall

Windows (företagsägt) — Autopilot user-driven

Flöde: klienten startas, kopplar till tenant, ESP visar förlopp; BitLocker, brandvägg och Defender/ASR slås på; WUfB-ring och cert/WLAN-profiler landar; Office + affärsappar installeras.

Resultat: användaren loggar in på en färdig, krypterad klient.

Utan detta: manuell "gör i ordning"-process, osäker säkerhetsnivå och ojämnheter mellan klienter.

macOS (företagsägt) — ADE + FileVault

Flöde: enheten binds i ABM, enrolleras automatiskt, FileVault krypterar,

systeminställningar låses där det behövs; cert/WLAN och appar (VPP/pkg) landar.

Resultat: enhetliga Apple-klienter; återställningsnycklar hanteras enligt policy.

Utan detta: kryptering hoppas över, lokala "hemfixar", svår support och efterlevnad.

iOS/iPadOS — ADE, låst layout och selektiv wipe

Flöde: auto-enrollment, PIN-krav, WLAN, e-postprofil; appar pushas; hemskärm layoutas för arbetsflöde; vid förlust → fjärr-rensa.

Resultat: förutsägbart gränssnitt och låg supportbelastning.

Utan detta: spretig användarupplevelse, appar saknas, data blandas med privat.

Android — Work Profile (BYOD) och COPE/COBO (företagsägt)

Flöde: BYOD får arbetsprofil där jobbdata hålls separerat; företagsägt kan vara låst (COBO) eller blandat (COPE). Zero-touch/Knox underlättar.

Resultat: balans mellan kontroll och integritet.

Utan detta: jobbdata blandas med privat; svårt att rensa vid avslut.

Linux (utvalda distron) — agentbaserad compliance

Flöde: agent registrerar enheten; compliance kräver kryptering, brandvägg och minsta OS-nivå; skript åtgärdar avvikelser.

Resultat: Linux-klienter uppfyller samma "grönt-krav" innan de når resurser.

Utan detta: parallell värld med oklara regler och högre risk.

WLAN via certifikat (EAP-TLS)

Flöde: Intune delar ut klientcertifikat; en WLAN-profil ansluter mot SSID som kräver cert; tar man bort behörighet tappar enheten nätåtkomst utan lösenordsbyte i infrastrukturen.

Resultat: inga delade PSK; enkel offboarding; hög säkerhet.

Utan detta: PSK sprids och lever för länge, byten blir stökiga, onödig exponering.

5.4.7 Säkerhetsaspekter vid fjärrhantering — tre saker som ger störst effekt

1. **Kryptering + skärmlås** på alla bärbara/telefoner.

2. **Compliance + villkorad åtkomst** så att bara “gröna” enheter når känsliga appar.
3. **Certifikat + segmentering** (EAP-TLS för WLAN/VPN, rätt VLAN/roll i nätet).

Lägg till **RBAC** för MDM-admins, tydlig **loggning** av fjärråtgärder och **sekretess** för BYOD (MDM ska inte kunna läsa privat data). Det bygger förtroende utan att sänka säkerheten.

Exempel (Exempel:) *En hittad men misstänkt komprometterad telefon blockeras direkt, jobbprofilen rensas, cert spärras. Åtkomst öppnas först efter ny enrollment och grön compliance.*

5.4.8 Jämförelse i korthet — Intune, Jamf, Workspace ONE, Miradore och Linux-vänliga alternativ

- **Microsoft Intune:** bred täckning (Windows, iOS/iPadOS, macOS, Android; visst Linux-stöd), stark koppling till Entra och villkorad åtkomst, Autopilot.
- **Jamf Pro:** djup och snabb för Apple-plattformar; kan samspela med Intune för compliance.
- **VMware Workspace ONE (AirWatch):** stark mobilhantering, brett OS-stöd, mogna appflöden.
- **Miradore/Hexnode/Kandji/Meraki SM:** lättare insteg; funktionellt djup varierar. **Miradore** är ett praktiskt multi-OS-alternativ.
- **Linux-inriktat:** JumpCloud, Canonical Landscape, SUSE Manager, Red Hat Satellite (agent/policy/patch/inventering i UEM-anda).

Exempel (Exempel:) *En blandad miljö kör Intune som “nav” (compliance, villkorad åtkomst, Autopilot). Apple-delen hanteras i Jamf men rapporterar compliance till Intune.*

5.4.9 Vanliga misstag — och hur man undviker dem

- Starta för brett: börja med **bas** (kryptering, skärmlås, uppdateringar, cert-WLAN).
- Otydliga grupper: bygg **dynamiska** målgrupper och testa på pilot.

- Blanda företagsägt och BYOD: håll isär; **selektiv wipe** för BYOD.
- Lösenords-WLAN i produktion: gå tidigt till **EAP-TLS**.
- Ingen återställningsplan: öva fjärr-wipe, återställning och om-enrollment.

Exempel (Exempel:) *En pilotgrupp får "Bas 1.0". När den är stabil lägger man på appar och finjusteringar. Färre överraskningar och färre supportärenden.*

Kontrollfrågor

1. Förklara skillnaden mellan **MDM**, **EMM** och **UEM/UDM** och när man typiskt växlar upp.
2. Vad vinner man på att koppla **compliance** i MDM till **villkorad åtkomst**?
3. När passar **manuell** enrollment bäst, och när är **zero-touch** att föredra?
4. Vad är **Samsung Knox**, och varför spelar det roll för Android-enheter från Samsung?
5. Vilka **förutsättningar** behöver ofta vara på plats innan Intune kan användas fullt ut (per plattform)?
6. Hur samverkar **Intune** med **Windows AD** i **moln-först**, **hybrid-join** och **co-management**?
7. Varför är **EAP-TLS** för WLAN ett lyft jämfört med delade lösenord, och vad händer om man inte inför det?
8. Beskriv en **WUfB-ringdesign** (Pilot → Bred → Sen) och vad som kan gå fel utan ringar.
9. Hur hjälper **dynamiska grupper**, **RBAC** och **ESP** till att göra utrollningar robusta?
10. Välj ett av konfigurationsexemplen och förklara vad som händer "om man låter bli" det steget.

Fördjupningslänkar

- Microsoft Intune (översikt och guider):
<https://learn.microsoft.com/mem/intune/>
- Windows Autopilot (översikt):
<https://learn.microsoft.com/mem/autopilot/>
- Apple Business Manager (ADE):
<https://support.apple.com/guide/apple-business-manager/welcome/web>
- Android Enterprise (enrollment-alternativ):
<https://developer.android.com/work>
- Samsung Knox (KME/Knox Suite): <https://www.samsungknox.com/>
- Jamf Pro: <https://learn.jamf.com/>
- VMware Workspace ONE:
<https://www.vmware.com/products/workspace-one.html>
- Miradore: <https://www.miradore.com/>
- JumpCloud (device + identity): <https://jumpcloud.com/>
- Google Admin — ChromeOS enhetsregistrering:
<https://support.google.com/chrome/a/answer/1360534>

Egna anteckningar

5.5 Framtiden för nätverk

5.5.1 SDN – från lågnivåkonfiguration till intent och API:er

I dag bygger mycket nätverksdrift fortfarande på att tekniker konfigurerar enheter var för sig: port för port, ACL för ACL. **Software-Defined Networking (SDN)** byter perspektiv. Styrplanet flyttas upp i en **kontroller** som förstår hela topologin, och nätet styrs via **API:er** och deklarativa policys: “det här ska vara tillåtet mellan de här tjänsterna, med den här säkerhetsnivån”. Kontrollen fördelas sedan ned till hårdvaran (switchar, routrar, brandväggar) som ren dataplanslogik. I datacenter är **EVPN/VXLAN** och fabric-kontrollers vardag; i WAN är **SD-WAN** redan brett infört; i campus kommer “intentbaserad” segmentering och identitetsstyrd åtkomst steg för steg.

Konsekvensen är att **feltyperna förändras**. I ett klassiskt nät var fel ofta fysiska (länk ned, loop). I ett SDN blir fel oftare **logiska**: en policy träffar fel grupp, ett objekt saknar tagg, ett API-samtal misslyckas. Det ställer nya krav på **observability** (telemetri, flödesdata, modellinsyn) och på hur vi arbetar: versionering av förändringar, **Git som sanning**, **CI/CD** för nät, och **“pre-change”-tester** i sandlåda eller digital tvilling.

För användare märks skiftet som **snabbare förändring** (nya VLAN/WLAN, ny QoS, ny site) och **jämnare kvalitet**. För driftteamet innebär det att **CLI-hantverket** kompletteras – inte försvinner – av **automation**: YANG/NETCONF/RESTCONF, gNMI, Ansible/Terraform, Python. Rollen glider mot **policy-, data- och livscykelarbete** snarare än “klicka i rätt ruta”. Den som kan läsa logik, förstå modellering och skriva tester blir lika viktig som den som känner varje kommando utantill.

Exempel (Exempel:) * En ny applikation ska ut i tre zoner. I stället för att skapa ACL:er och VLAN på femton switchar beskriver du i kontrollern vilka roller som får prata med vilka, och kontrollerna skapar underliggande flöden. En rollfelmappning gör att bara testzonen får trafik; telemetri och “intent verification” visar exakt var policyn stoppar.**

5.5.2 Zero Trust – identitet först, nätet som transport

Zero Trust utgår från antagandet att **inget** är pålitligt per automatik – inte ens “insidan”. Åtkomst beslutas **varje gång**, baserat på **identitet**, **enhetens skick (compliance)**, **kontext** och **risk**. Nätet blir mer av **transport**, medan kontrollen sker i kombination av **identitetstjänst**, **MDM/UEM**, **applikationsproxy** och **mikrosegmentering**. I campus/edge innebär det att **VLAN som grov kniv** ersätts av **finmaskiga policys** nära klient eller arbetsbelastning. I datahall och moln betyder det att arbetslaster isoleras per **tjänst/roll** snarare än per IP-subnät.

För användare betyder Zero Trust fler **tysta kontroller** (enhetens status, plats, beteende) och färre “stora” lösenordshinder – men **MFA** och **device posture** blir vardag. För driftteamet betyder det **färre platta nät**, **fler policys** och **mer samarbete** med identitetsteam och applikationsägare. Den praktiska utmaningen är **komplexitet**: policyexplosion om man inte modellerar roller rätt, och **friktion** om kraven inte är realistiskt satta. Nyckeln blir **enkla basregler**, tydliga undantagsprocesser och **telemetri** som visar *varför* något nekas.

Exempel (Exempel:) * En klient med gammalt OS tappar åtkomst till HR-systemet för att compliance-regeln kräver uppdaterad kernel och kryptering. Användaren får guidning i klientportalen; access öppnas igen när enheten är grön.**

5.5.3 IPv6-implementering – från dual-stack till IPv6-only

IPv6 har länge “varit på gång”, men de senaste åren har förutsättningarna mognat: operatörer kör IPv6 i ryggraden, stora innehållsleverantörer exponerar tjänster i IPv6, och klient-OS har stabil **privacy**-logik. Nästa steg är **IPv6-only** i allt fler miljöer, med **NAT64/DNS64** eller 464XLAT för det som ännu saknar IPv6. Vinsten är **enklare adressering i stor skala**, **färre NAT-lager** och möjlighet till **renare end-to-end-säkerhet**. Samtidigt blir **säkerhetsmodellen tydligare**: NAT var aldrig en brandvägg – IPv6 kräver **uttalade filter**, god **ICMPv6-hygien** (annars dör funktioner), och skydd mot **RA-spoofing** och felkonfigurerad **SLAAC/DHCPv6**.

För användare märks lite – oftast **bättre reachability** och färre märkliga fel. För driftteamet märks mycket: **adressplanering i /64-tänk**, **brandväggsregler per tjänst**, uppdaterad **övervakning** (flow/NetFlow/IPFIX som förstår v6), och **felsökning** där man komfortabelt läser **Neighbor Discovery** och **ICMPv6**. Övergången kräver **dual-stack-period**, **piloter**, tydlig **inventering av v4-beroenden**, och planerad **avveckling** av gamla NAT-antaganden. I campus och WLAN handlar det om att RA-guard och DHCPv6-snooping fungerar lika bra som IPv4-kontrollerna. I datacenter och moln om att **säkerhetspolicys** (SG/NSG/ACL) migreras utan att varken över- eller under-skyddas.

Exempel (Exempel:) * Ett elev-VLAN blir IPv6-only. DNS64/NAT64 ger fortfarande åtkomst till IPv4-webb. Loggarna visar lägre felsökningsbelastning, men brandväggen fick expanderade v6-regler för att ersätta gamla "NAT-skyddet".**

5.5.4 AI och automatisering – från NMS till beslutssystem

Automatisering har länge handlat om skript och verktyg som Ansible. Nästa steg är **AI-stödd drift**: system som **förutser** fel, **förklarar** avvikelser och i kontrollerade fall **åtgärdar** dem. I WLAN ser vi redan självlärande **radio-optimering** (kanal/effekt), i WAN **policy-styrd ruttval/QoS**, i DC **intent-verifiering** som säkerställer att dataplansflöden matchar avsikten. Med **AIOps** blir brusiga larm till **händelser med sammanhang**: "allt pekar på att DHCP-svar i VLAN 20 stoppas på denna trunk". Med **LLM-stöd** kan runbooks bli **konversationsbara**: "visa sannolik rotorsak och föreslå återställning".

Vinsten är **hastighet och kvalitet**. Risken är **överförtroende**: en modell kan "hallucinera", hamna i **feedbackloopar** eller agera på **partiska data**. Därför blir **skyddsräcken** centrala: förändringar ska gå genom **pull-requests**, testas i **sandlåda**, **stegvis rullas** och **mätas** efteråt. Data måste hanteras med **integritet**: nättelemetri kan innehålla personuppgifter eller känsliga mönster. Rollen för nätverksteknikern blir mer **ingenjör/mätare**: skriva **policys**, definiera **SLO:er** (t.ex. anslutningslatens i WLAN), designa **mätningar** och granska **maskinens förslag**. För användare kan AI betyda **stabilare upplevelse** – om man samtidigt accepterar **mindre "magi"** och **mer transparens**: varför fattades ett beslut, och hur ångras det?

Exempel (Exempel:)*)** Ett AIOps-system ser mönstret “AP-roaming tar >300 ms” i en byggnad, kopplar det till en ny störkälla i 2,4 GHz och föreslår kanalplanering/effektjustering. Förslaget körs i nattfönster mot en testzon och sprids först efter mätbar förbättring.

5.5.5 Etik och cybersäkerhet – data, styrka och ansvar

När nätet blir mer **mätbart** och mer **automatiserat** ökar både möjligheten och frestelsen att **se mer**. Telemetri, loggar och flöden kan avslöja arbetsmönster, platser och beteenden. **Integritet** måste därför vara en **första-klass-fråga**: samla bara det som behövs, **pseudonymisera** där det går, begränsa **åtkomst** och **lagringstid**, och var **öppen** med vad som samlas. I skola och offentlig sektor blir detta särskilt viktigt. På säkerhetssidan fortsätter **segmentering**, **Zero Trust-tänk**, **hårdning** och **snabb patchning** att vara grund. Till det kommer **leverantörskedja** (firmware, SBOM, signering), **out-of-band-hantering** (så att man kan rädda även vid ransomware), och uppmärksamhet på **kryptoframtid** (post-kvant kommer påverka hur vi ser på nycklar och livslängd).

Yrkesrollen rör sig också etiskt: fler beslut om **övervakningens gränser**, **modellers skälighet**, **vendor-låsning** och **hållbarhet** (energi, värme, livslängd). Teknikern blir inte bara den som “får det att fungera”, utan den som **formar hur det bör fungera** – i balans mellan nytta, risk och rättigheter.

Sammanfattning att ta med sig

- **SDN** gör nät till **policy + API**; hantverk blir livscykel, test och modell.
- **Zero Trust** flyttar förtroendet till **identitet och enhetens skick**; nätet segmenteras finmaskigt.
- **IPv6** går från “parallellt” till **IPv6-only**; NAT ersätts av **tydlig filtrering** och god ICMPv6-hygien.
- **AI/AIOps** höjer hastighet och kvalitet – men kräver **skyddsräcken**, **mätning** och **mänsklig granskning**.

- **Etik & säkerhet** blir inbyggt: minst data, tydliga roller, snabb patch, stark kedja från firmware till policy.

Frågor att fundera på

- Vilken **minsta uppsättning mätvärden** behöver vi för att kunna lita på ett AI-förslag i *vårt* nät?
- Var går gränsen mellan **nödvändig telemetri** och **onödig övervakning** i en skolmiljö?
- Om vi gör **IPv6-only** i ett delnät, vilka **tio saker** måste vara på plats för att det ska bli en förbättring – inte en risk?
- Vilka **färdigheter** (språk, verktyg, metoder) behöver en nätverkstekniker lägga till de kommande två åren för att må bra i SDN/Zero Trust-världen?
- Hur beskriver vi **intentionen** för nätet så att den är testbar före och efter varje förändring?

Kontrollfrågor

1. När ett nät styrs via en SDN-kontroller och “intent”, vilka nya feltyper uppstår jämfört med klassisk CLI-konfiguration – och hur märker man dem i drift?
2. Hur skulle du beskriva skillnaden mellan **mikrosegmentering** i ett Zero Trust-tänk och traditionell segmentering med VLAN/ACL – ur både säkerhets- och förvaltningsperspektiv?
3. Om ni inför **Zero Trust**: vilka tre datakällor (identitet, enhet, kontext) måste vara pålitliga för att besluten ska bli rimliga – och hur validerar ni dem?
4. Du får uppdraget att göra ett **IPv6-pilot** i ett campus-VLAN. Vilka tio praktiska kontroller vill du se “gröna” innan pilotens användare märker något negativt?
5. Vid **IPv6-only** med NAT64/DNS64: vilka applikationstyper brukar strula, och vilka mätetal sätter du upp för att snabbt fånga fel?
6. Nämn tre **skyddsräcken** du vill ha på plats innan ni låter automation/AI göra förändringar i nät (tänk kodflöde, test, övervakning).
7. Vilka **SLO:er** (service level objectives) är mest meningsfulla i ett modernt WLAN – och hur kopplas de till åtgärder i radionät/klientpolicy?
8. Vad är ett bra **minsta datalager** för AIOps (telemetri, flöden, loggar) så att analys blir möjlig utan att tumma på integriteten?
9. Hur förändras nätverksteknikerns **kompetensprofil** när SDN, Zero Trust och automatisering blir vardag – och hur tränar man det effektivt i skolan?
10. Välj en realistisk **etikfråga** (t.ex. klienttelemetri i skolmiljö). Var går gränsen mellan nödvändig mätning och onödig övervakning, och hur förklarar du det för användarna?

Fördjupningslänkar

- ONF – *Software-Defined Networking (SDN) Definition*:
<https://opennetworking.org/sdn-definition/>
- RFC 7348 – *VXLAN*: <https://datatracker.ietf.org/doc/rfc7348/>
- RFC 7432 – *BGP EVPN*:
<https://datatracker.ietf.org/doc/rfc7432/>
- NIST SP 800-207 – *Zero Trust Architecture*:
<https://csrc.nist.gov/publications/detail/sp/800-207/final>
- RFC 8200 – *Internet Protocol, Version 6 (IPv6) Specification*:
<https://datatracker.ietf.org/doc/rfc8200/>
- RFC 4861 / RFC 4862 – *Neighbor Discovery / SLAAC*:
<https://datatracker.ietf.org/doc/rfc4861/> |
<https://datatracker.ietf.org/doc/rfc4862/>
- RFC 4443 – *ICMPv6*:
<https://datatracker.ietf.org/doc/rfc4443/>
- RFC 6146 / RFC 6147 – *NAT64 / DNS64*:
<https://datatracker.ietf.org/doc/rfc6146/> |
<https://datatracker.ietf.org/doc/rfc6147/>
- Google SRE – *Service Level Objectives*: <https://sre.google/sre-book/service-level-objectives/>
- NIST AI RMF 1.0 – *AI Risk Management Framework*:
<https://www.nist.gov/itl/ai-risk-management-framework>

Egna anteckningar

...

Appendix I – Begrepp

Not: I brödtexten använder vi termen "WLAN". "Wi-Fi" syftar här på certifieringsprogrammet/varumärket och används i denna begreppsruta.

ABM (Apple Business Manager) – Portal för enhets- och appinköp; grunden för automatisk Apple-enrollment (ADE) och VPP-licenser.

ACL – Åtkomstlista som avgör vilken trafik som tillåts eller nekas.

ACME – Protokoll för automatisk hantering av TLS-certifikat (t.ex. certbot).

AD (Windows Active Directory) – Katalogtjänst för identiteter, datorer och policyer.

AD CS – Certifikatinfrastruktur (CA) i Windowsmiljö.

AIDE – Filintegritetskontroll som larmar när filer ändras otillåtet.

AIOps – AI-stött drift som korrelerar larm, hittar orsaker och föreslår åtgärder.

Air-gap – System eller kopior som är fysiskt/logiskt isolerade från nätet.

Android zero-touch – Förregistrering av företagsägda Android-enheter för "zero-touch".

Ansible – Automationsverktyg som beskriver systemtillstånd i YAML och kör idempotent.

Anycast – Samma IP annonseras från flera platser; trafik hamnar hos närmaste nod.

API – Programmeringsgränssnitt; i SDN/UEM används för styrning och automation.

AP (Accesspunkt) – Trådlös basstation för klientanslutning i ett **WLAN** (OS kallar ofta "Wi-Fi").

APIPA – Automatisk IPv4-adress 169.254.0.0/16 när DHCP saknas.

AppArmor – MAC-säkerhet i Linux som begränsar programrättigheter.

ARP – Protokoll som översätter IPv4-adresser till MAC-adresser i L2.

Autentisering – Bevis av identitet (t.ex. lösenord, certifikat, MFA).

Autopilot (Windows) – Zero-touch-registrering och uppsättning av Windows-klienter.

Backup – Säkerhetskopia av data för återställning vid fel eller incident.

Bandbredd – Maximal datamängd per tidsenhet på en länk.

Bastion/Jump host – Härdad mellanvärd för säker SSH/RDP-åtkomst.

BGP – Externt routingprotokoll som utbyter vägar mellan autonoma system.

BGP community – Attribut som styr policyn för hur rutter hanteras.

BGP ECMP – Lastdelning över flera likvärdiga vägar i BGP.

BGP route reflector – Minskar krav på full-mesh i iBGP.

BNC – Bajonettkontakt för koaxialkabel (historiskt/nischnät).

Bonjour/mDNS – Lokal namn- och tjänsteupptäckt i LAN.

BPDU – STP-kontrollramar som används för loopdetektering.

Bridge (brygga) – L2-koppling mellan segment; i servervärd: virtuell switch.

BSSID – MAC-adress för accesspunktens radiosändare (en SSID-cell).

BYOD – “Bring Your Own Device”; privatägd enhet i arbetsmiljö.

CA (Certificate Authority) – Utfärdare av digitala certifikat.

CAPA – Korrigerande och förebyggande åtgärder efter incident (Corrective/Preventive).

CASB – Cloud Access Security Broker; policy och kontroll för SaaS-åtkomst.

Cat6A (kabel) – Vanlig kopparkabel för 10G-Ethernet upp till 100 m.

Ceph – Distribuerad objekt/blocks/fil-lagring; vanligt i hypervisor-kluster.

Certificate pinning – Låsning av klient till specifik certifikatnyckel.

Certifikat – Kryptografisk legitimation kopplad till identitet/nyckel.

CIDR – Notation/metod för prefixlängd (t.ex. /24) och aggregering.

CI/CD – Automatiserat bygg, test och driftsättning (även för nätkonfig).

CLI – Kommandorad för konfiguration och felsökning.

Cloud-join (Entra-join) – Windows-klient ansluten direkt till Microsoft Entra ID.

Co-management – Samdrift av MECM/SCCM och Intune i samma klientpark.

COBO – Företagsägd, företagsanvänd Android (låst läge).

COPE – Företagsägd, personligen använd (balans jobb/privat).

Container – Isolerad applikationsmiljö (t.ex. Docker/LXC).

Controller (WLAN) – Central styrning/övervakning av många accesspunkter.

CoS (802.1p) – L2-prioritering i VLAN-taggen.

CRC – Felkontrollsumma för att upptäcka bitfel.

DAC-kabel – Direktkopplingskabel för SFP+/QSFP-port till korta avstånd.

DAD (Duplicate Address Detection) – IPv6-kontroll för att undvika adresskollision.

DAI (Dynamic ARP Inspection) – Skydd mot ARP-förfalskning i L2.

Data plane – Där paket faktiskt forwardas (jfr control/management

plane).

Deduplicering – Eliminering av dublettdata i backup/lagring.

Default route – Standardväg när ingen mer specifik rutt matchar.

DFS (WLAN) – Dynamic Frequency Selection; kanalreglering i 5/6 GHz.

DFS (Windows) – Distributed File System; virtuella filnamnrymder/pekare/replikering.

DHCP – Dynamisk tilldelning av IP-konfiguration till klienter.

DHCP relay – Vidarebefordrar DHCP mellan VLAN/subnät.

DHCP snooping – Switchskydd som stoppar falska DHCP-servrar.

DHCPv6 – IPv6-varianten av DHCP (komplement till SLAAC).

DNS – Namnuppslagning mellan domännamn och IP-adresser.

DNSSEC – Signerad DNS för att skydda mot förfalskning.

DoH/DoT – DNS över HTTPS/TLS för integritet.

Docker – Containerplattform för att paketera och köra applikationer.

Domän – Administrativ enhet i AD eller en DNS-zon.

DPI (Deep Packet Inspection) – Klassning/inspektion på applikationsnivå.

DTP – Cisco-protokoll för att förhandla trunk/access (bör undvikas i produktion).

DR (Disaster Recovery) – Återstart av kritiska system efter större avbrott.

DSCP – QoS-märkning i IP-huvud för prioritering.

EAP – Ramverk för nätverksautentisering (802.1X).

EAP-PEAP – 802.1X-metod som kapslar lösenord i TLS-tunnel.

EAP-TLS – Certifikatbaserad 802.1X-autentisering (WLAN/VPN).

EAP-TTLS – 802.1X-metod med tunnlad autentisering utan klientcert.

Edge – Nätets ytterkant (site/klient-nära lager).

EIGRP – Cisco-routingprotokoll (dynamiskt, hybridmodell).

ESP (Enrollment Status Page) – Intune-sida som blockerar tills baskrav är uppfyllda.

ESP (IPsec) – Encapsulating Security Payload; kryptering/integritet i IPsec.

EVPN – BGP-baserad L2/L3-VPN, ofta med VXLAN i datacenter.

Fail2ban – Blockerar upprepade felaktiga inloggningsförsök.

Failover – Automatisk övertagning när primär komponent faller.

FHRP – Protokoll för redundant gateway (t.ex. VRRP/HSRP/GLBP).

FileVault – Diskkryptering i macOS.

Firewall (tillståndsfull) – Brandvägg som förstår och spårar

sessionsflöden.

FreeIPA – Linuxidentitet (LDAP/Kerberos/CA/HBAC), likt AD.

FTP/SFTP – Filöverföring; SFTP går över SSH och är säkert.

Gateway – Standardväg ut ur ett subnät (L3-gräns).

Grafana – Visualisering av tidsserier (dashboards).

GRE – Tunnelprotokoll för inkapsling av trafik.

GFS (Grandfather-Father-Son) – Backup-retentionsmodell dag/vecka/månad.

Git – Versionshantering; grund för GitOps även i nät.

GNS3/EVE-NG – Emulering/virtualisering för nätlabbar.

GPO – Group Policy Objects; Windows-policyer kopplade till AD.

GPO-länkning – Att koppla GPO till Site/Domain/OU i AD-trädet.

GUA – Globala IPv6-adresser som routas på internet.

HA (High Availability) – Design för att undvika enkel felpunkt.

HBA (Host Bus Adapter) – Gränssnitt mellan server och SAN/lagring.

HCI (Hyper-Converged Infrastructure) – Kluster som förenar compute/lagring/nät.

Helm/Kubernetes – Orkestrering av containeriserade applikationer (paket/utrullning).

Honeypot – Avsiktligt lockbete för att observera angripare.

Hop – Ett routingsteg mellan källa och destination.

Host – Slutenhet/servrar/klienter som använder nätet.

Host-ID – Värddelen av en IP-adress (allt utanför Net-ID).

HSRP – Ciscos redundanta gateway (FHRP), liknande VRRP.

HTTP/HTTPS – Webprotokoll; HTTPS säkrat över TLS.

Hypervisor – Mjukvara som kör och isolerar virtuella maskiner.

IBN (Intent-Based Networking) – Nät styrt av deklarerad avsikt/policy.

ICMP – Kontrollprotokoll (ping, felmeddelanden m.m.).

ICMPv6 – IPv6-kontroll: Neighbor Discovery, fel, MTU m.m.

IdP (Identity Provider) – Identitetsleverantör (t.ex. Entra, ADFS).

IDS/IPS – Intrångsdetektering/-prevention i nätverk.

IGP/EGP – Routing inom ett AS (OSPF/IS-IS) respektive mellan AS (BGP).

Immutability (backup) – Skrivskyddad retention; skydd mot ransomware.

iSCSI – Blocklagring över IP (SAN).

Intune – Microsofts UEM/MDM-plattform.

IP – Internet Protocol; adressering och leverans av paket.

IP-adress – Identifierar ett nätverksinterface i IP-nät.

IP Source Guard – Skydd mot IP-spoofing på accessportar.
IPFIX/NetFlow – Flödesmätning (käll/dest, portar, bytes).
IPsec – Krypterade tunnlar (site-to-site/klient-VPN).
IPv4 – 32-bitars IP; begränsat adressutrymme.
IPv6 – 128-bitars IP; stort adressutrymme och modern funktionalitet.
IS-IS – Länkstatusrouting (IGP), alternativ till OSPF.
JSON/YAML – Dataformat vanliga i API/automation.
Journald – systemd-loggtjänst på Linux.
JumpCloud – Moln-identitet/UEM för bl.a. Linux/Windows/macOS.
Kerberos – Biljettbaserad autentisering (AD/FreeIPA).
Kernel – Operativsystemets kärna.
Keyring (GPG) – Nyckelring för signering/validering av paket/repo.
Knox (Samsung) – Samsungägt ramverk/portal som ger extra MDM-djup.
KVM – Linux-hypervisor (KVM/QEMU) för VM-drift.
L2-switch – Växlar ramar via MAC-tabell inom samma broadcastdomän.
L3-switch – Switch med routingfunktioner mellan VLAN/subnät.
LACP – Länksammanlagning (802.3ad) för redundans/kapacitet.
LACP fast rate – Snabbare LACP-handtryck (~1 s intervall).
LAPS – Hantering av unika lokala adminlösenord (Windows).
Latens – Fördröjning från sändare till mottagare.
Layer 2/3/7 – OSI-lager: Datalänk/Network/Applikation.
LC/SC/MPO – Vanliga fiberkontakter (duplex/multifiber).
Link-local – Lokalt giltig adress (IPv4 APIPA; IPv6 fe80::/10).
Linux bridge – Programvarubrygga; virtuell switch i Linux.
Live migration – Flytt av VM mellan värdar utan driftstopp.
LLDP/CDP – Protokoll som visar grannar och portinformation.
LLM (språkmodell) – AI-modell som kan assistera felsökning/runbooks.
LTS – Långtidsstödd version av ett OS.
LUKS – Linux-diskkryptering (dm-crypt).
MAC-adress – Unik datalänksidentifierare (L2).
MAC-flapping – MAC växlar port ofta; tyder på loop/länkproblem.
MACsec – L2-kryptering på länknivå (802.1AE).
MAN – Metropolitan Area Network; stadsnät.
Management plane – API/CLI/SNMP-kanal för styrning/konfig.
MAB (MAC Authentication Bypass) – 802.1X-alternativ för “dumma” klienter.
MECM/SCCM – Microsofts klienthantering lokalt (f.d. SCCM).
MFA – Multifaktorautentisering (t.ex. lösenord + app).

Mesh (topologi) – Nät med flera redundanta vägar mellan noder.

Microsegmentering – Finmaskig policy per applikation/roll i stället för grova VLAN.

MIMO/MU-MIMO – Flerantennteknik i WLAN (flera simultana strömmar).

MPLS – Etikettbaserad forwarding; VPN/QoS i operatörsnät.

MSTP – STP-variant med multipla instanser per VLAN-grupp.

MTTR – Genomsnittlig återställningstid (Mean Time To Repair/Restore).

MTU – Maximal ramstorlek på länknivå.

MSS – Maximal TCP-nyttolast utan fragmentering.

NAC – Nätverksåtkomstkontroll (802.1X/MAB/posture).

NAT – Adressöversättning mellan privata och publika adresser.

NAT64/DNS64 – Överbryggar IPv6-klienter till IPv4-resurser.

NAT-T – IPsec över UDP för att passera NAT.

ND (Neighbor Discovery) – IPv6-grannprotokoll (ersätter ARP).

NDES – Microsofts SCEP-endpoint för certifikat via Intune.

Net-ID – Nätverksdelen i en IP-adress (prefix/mask).

NETCONF/RESTCONF – Modellbaserad konfig via YANG (SSH/HTTP).

Netplan – YAML-baserad nätverkskonfiguration i Ubuntu.

NetworkManager – Linux-tjänst/verktyg för nätverkskonfiguration.

NFS – Filresursprotokoll, främst Linux↔Linux.

Nginx – Webbserver/reverse proxy.

Node exporter – Agent som exponerar Linux-mätvärden för Prometheus.

NPTv6 – Prefixöversättning i IPv6 (utan portöversättning).

NTP/chrony – Tidssynkronisering i nät/operativsystem.

NTP-stratum – Nivå i tidssynkhierarki (avstånd till referens).

OOB (Out-of-Band) – Separat hanteringsnät, används även vid kris.

OpenSCAP – Efterlevnadskontroll mot säkerhetsprofiler.

OpenTelemetry – Standard för insamling av spår/logg/metrics.

Overlay/Underlay – Logiskt nät ovanpå fysisk fabric (t.ex. VXLAN över IP).

OSPF – Länkstatusrouting (IGP) i interna nät.

OSPF area – Hierarkisk uppdelning; Area 0 är ryggraden.

OSPF cost – OSPF-metric; baseras ofta på bandbredd.

OT (Operational Technology) – Produktions-/styrsystemsnet.

Packer – Bygger automatiserade OS-avbilder.

PAM – Pluggbar autentiseringsram i Linux/Unix.

PAT – Portbaserad NAT (många interna → en publik).

PE/CE – Provider Edge/Customer Edge i operatörsnät.

PFsense/OPNsense – BSD-brandväggar för routing/NAT/VPN.

PHY – Fysiska lagret: kablage, modulering, optik.

PKI – Infrastruktur för nycklar/certifikat (CA, CRL, kedjor).

PKCS (Intune) – Direkt certifikatutfärdande till enheter/användare.

Polkit – Policy för privilegier i Linux (höjda rättigheter).

Port (L2) – Fysisk/logisk anslutning på switch/router.

Port forwarding – Vidarebefordran av portar från utsida till insida.

Portfast – STP-optimering för accessportar (snabb upp).

PoE – Ström över Ethernet till t.ex. AP/kameror.

PPPoE – Inkoppling över Ethernet (vanligt på WAN/operatör).

Prefixdelegation (PD) – Automatisk IPv6-prefixutdelning (CPE→LAN).

Private VLAN (PVLAN) – L2-isolering inom samma VLAN.

Prometheus/Alertmanager – Mätinsamlare och larmmotor.

Proxmox VE – Debian-baserad hypervisor (KVM/LXC) med ZFS-stöd.

Proxmox Backup Server (PBS) – Backup-lösning i Proxmox-ekosystemet.

Proxy (reverse) – Front som tar emot klienttrafik och vidarebefordrar till backend.

PXE – Nätboot av klienter (DHCP/TFTP).

QoS – Prioritering/klassning av trafik för bättre upplevelse.

QoS shaping/policing – Buffra jämnt vs hårt strypa trafikflöden.

RA (Router Advertisement) – IPv6-utdelning av prefix/standardväg.

RA-guard – Skydd mot falska IPv6-RA på access.

RADIUS – Central autentisering/auktorisering (t.ex. 802.1X).

RBAC – Rollbaserad behörighetskontroll för administratörer.

REST API – Webbaserat API (HTTP/JSON) för automation/integration.

Restic – Krypterande, deduplicerande backupklient.

RJ45 (8P8C) – Vanlig kontakt för koppar-Ethernet (twisted pair).

RMM (Remote Monitoring & Management) – Fjärrövervakning/klienthantering.

RPO – Hur mycket data som får förloras vid återställning.

RRM (Radio Resource Management) – Automatisk kanal/effekt-optimering i WLAN.

RTO – Hur snabbt system ska vara uppe efter fel.

RIP – Enkelt distans-vektor-routingprotokoll (mindre nät).

RSTP – Snabbare spanning-tree (802.1w).

Route summarization – Aggregera prefix för mindre routingtabeller.

Routing – Framledning av paket mellan nät/subnät (L3).

Router – Enhet som kopplar samman flera nät (L3-gräns).

RSSI/SNR – Signalstyrka och brusförhållande i trådlöst.

SASE/SSE – Säker nätarkitektur där skydd/åtkomst läggs i molnkant.

SBOM – Förteckning över programvarans komponenter (supply-chain).

SCEP – Förenklat protokoll för certifikatutfärdande (via NDES/Intune).

SDA (Software-Defined Access) – SDN i campus; policyn vid kanten.

SDN – Programvarustyrt nät; policy/intention via controller och API.

SD-WAN – Policy-styrt WAN över flera länkar/operatörer.

Security baseline – Minimikrav för konfiguration och säkerhet.

SELinux – MAC-säkerhet i Linux (RHEL-familjen).

sFlow – Provtagning av trafik för flödesanalys (alternativ till NetFlow).

SIEM – Central logg/säkerhetsanalys (korrelation och larm).

SLA – Service Level Agreement; avtalad tjänstenivå.

SLAAC – Automatisk IPv6-adressättning via RA (stateless).

SLO – Mätbar målnivå för en tjänst (t.ex. latens/anslutning).

SFP/SFP+/QSFP – Modulplatser/optik i switchar/routrar.

SMF/MMF (OS2/OM3/OM4) – Single/Multi-mode-fiber och deras klassningar.

SNAT/DNAT – Käll- och destinations-NAT.

SNMP – Övervakningsprotokoll för status/konfiguration.

Snapshot – Ögonblicksbild av data/VM.

SPAN (port mirroring) – Kopiera trafik till analysport (t.ex. Wireshark).

Spanning-tree guards – BPDU-/root-/loop-guard mot L2-fel.

SR-IOV – Delar upp NIC i virtuella funktioner för hög prestanda.

SSH – Krypterad fjärrterminal och filöverföring (scp/sftp).

SSL/TLS – Kryptering av applikationsprotokoll (t.ex. HTTPS).

SSID – Nätverksnamn som en AP annonserar i ett **WLAN**.

Stateful/stateless brandvägg – Med eller utan sessionskännetecken.

Sticky MAC – Lås MAC-adress till en accessport.

Storm control – Begränsa broadcast/multicast-stormar.

Subnät – Del av nät med eget prefix/mask.

Switch – L2-enhet som kopplar ihop värdar i samma LAN.

Syslog – Standard för loggtransport (UDP/TCP/TLS).

Systemd – Init-/tjänstehanterare i moderna Linux.

TACACS+ – Autentisering/auktorisering för nätverksenheter.

Taggning (802.1Q) – VLAN-identifiering i Ethernet-ramar.

Terraform – “Infrastructure as Code” för moln och nät.

TFTP – Enkel filöverföring (UDP); används bl.a. vid PXE.

Throughput/Goodput – Total genomströmning vs faktisk nyttolast.

TLS – Krypteringsprotokoll för säker kommunikation.

TPM/UEFI Secure Boot – Plattformsskydd och nyckelmodul i klient/servrar.

TOTP – Engångslösenord i MFA-appar.

Trunkport – Switchport som bär flera VLAN (802.1Q-tagging).

Trunk pruning – Tillåt endast nödvändiga VLAN på en trunk.

TWT (Target Wake Time) – Strömsparfunktion i 802.11ax (WLAN).

UDLD – Upptäcker enkelriktade länkar för att undvika loopar.

UDP – O-tillförlitligt transportprotokoll (låg overhead, realtid).

UDM/UEM – Samlad enhetshantering för alla plattformar (endpoint-fokus).

UEFI Secure Boot – Startskydd som verifierar bootkomponenter.

UFW – Enkel brandväggsfrontend i Ubuntu/Debian.

ULA – Unika lokala IPv6-adresser (ej internet-routade).

Uplink – Överordnad anslutning (access→distribution→core).

UTM/NGFW – “Nästa generations” brandvägg (applikationsmedveten).

VLAN – Logisk L2-segmentering i ett LAN.

VM – Virtuellt maskin (OS i hypervisor).

VPN – Krypterad tunnel mellan site/klient och nät.

VRP – Standardiserat FHRP för redundant gateway.

vSwitch – Virtuellt switch på hypervisor/värd.

VSS (Windows) – Volymkuggkopior; app-konsistens vid backup.

VXLAN – L2-overlay över IP/UDP (24-bitars VNI).

WAF – Web Application Firewall (lager-7-skydd).

WAN – Wide Area Network; nät över större avstånd.

Wazuh – Öppen plattform för säkerhet/övervakning/EDR.

WDS/MDT – Windows-avbildning/utrullning lokalt (om inte Autopilot används).

Wi-Fi (Wi-Fi Alliance) – Certifieringsprogram/varumärke för interoperabilitet enligt IEEE 802.11. Många operativsystem benämner trådlösa nätverksgränssnitt som “Wi-Fi”. I boken använder vi WLAN för själva tekniken (802.11) och reserverar “Wi-Fi” för certifieringen.

WLAN – Trådlöst lokalt nät (OS kallar ofta gränssnittet “Wi-Fi”).

WMI – Windows Management Instrumentation.

WMI-filter – GPO-filtrering baserat på systemegenskaper.

WPA2/WPA3 – Säkerhetsstandarder för **WLAN**-kryptering.

WPA2-Enterprise – 802.1X/EAP-baserad **WLAN**-autentisering.

WRED – Köhantering som droppar tidigt för att förebygga kollaps.

WUfB – Windows Update for Business; ringstyrda uppdateringar.

WireGuard – Modernt VPN-protokoll (snabbt, enkelt).

Wireshark – Protokollanalysator för nätverkstrafik.

ZFS – Avancerat filsystem/volymanhantering (checksumma, snapshot, replika).

Zero Touch/ZTP – Automatisk provisionering/registrering utan handpåläggning.

Zero Trust – Åtkomst beslutas per begäran utifrån identitet, enhetens skick och kontext.

Zigbee/Z-Wave – Lågströmsprotokoll för IoT (hem/byggnad).

Appendix II - CLI-REFERENS

Kodsnippets att använda

Sammanfattning

När vi jobbar med nätverkskonfiguration handlar mycket av arbetet om planering, kontroll och felsökning. Är det något som inte fungerar, kontrollera paketets väg, från början till slut för att se var det har blivit fel.

Ta ALLTID för vana att placera ut saker organiserat och döp allting med exempelvis ip-adresser eller korrekta hostnames. Även det kommer göra felsökningen mycket enklare.

Många av de kommandon som används skrivs sällan ut som hela kommandon, då du kan använda förkortningar istället. Ofta när du söker hjälp på google m.m. så kommer förkortningarna upp. Exempel på detta är när du ansluter till en port eller skall sätta på en port. I de fallen skrivs bara förkortningar till kommandot ut

Den första fråga du får när du ansluter till enheten är: *Would you like to enter the initial configuration dialog? [yes/no]*, där svarar vi ALLTID no.

Att använda sig av ett frågetecken, ?, listar alla tänkbara kommandon som finns på nivån eller som fortsättning på kommandot du matat in. Det är bra att använda sig av om du är osäker.

För att avbryta pågående kommando trycker du **Ctrl+Shift+6**

Det sista tipset är att alltid ansluta en dator till enheten som du skall konfigurera. Detta för att minimera risken att du råkar göra ändringar i portinställningarna på enheten.

Lycka till

Grundläggande CLI-kommandon

enable

Används för att logga in i enheten som administratör. Om du har konfigurerat användarnamn och lösenord får du fylla i det efteråt.

syntax:

enable

Exempel:

Router>**enable**

reload

Används för att starta om en enhet. Tänk på att alla kommandon du har lagt in kommer att nollställas, om du inte har sparat dem.

syntax:

reload

Exempel:

Router>**reload**

show

Är ett "grundkommando" som används för att visa inställningar och/eller anslutna enheter till både switch och router.

syntax:

Exempel:

Router>show [vad du vill visa]

Exempel:

Router>show interfaces status

Exempel:

Router>show running-config

exit

Används för att backa ur den nivå du befinner dig i. Om du har konfigurerat en port, och vill logga ut från det läget är detta kommandot du använder dig av

syntax:

exit

Exempel:

Router (config-if)# **exit**

configure terminal

Ändrar enhetens konfiguration, exempelvis portar m.m. Här behöver du vara för att göra alla inställningar.

syntax:

configure terminal eller
conf t

Exempel:

router#**configure terminal** eller
Router#**conf t**

hostname

Vi arbetar med många enheter i nätverket och något som är viktigt är att ha koll på vad de heter. Generellt döps routrar till R, och nummer i ordningen, till exempel: R1. Switchar döps på samma sätt och heter SW istället

syntax:

hostname [namn]

Exempel:

Router (config)#**hostname** R1

interface

Används för att ansluta till en port och göra inställningar, brukar också skrivas **int**. Här är det vanligt att använda förkortningar för porten istället för portens hela namn.

syntax:

interface [portnamn]

Exempel:

Router (config)#**interface** GigabitEthernet0/0

eller:

Router (config)#**int** g0/0

shutdown

Används för att stänga av en port. Skrivs no framför blir det omvänt, dvs no shutdown blir: sätt på porten

syntax:

shutdown eller **sh**

Exempel för att stänga av:

Router (config-if)#**shutdown**

eller:

Router (config-if)# **sh**

Exempel för att sätta på:

Router (config-if)#**no shutdown**

eller:

Router (config-if)# **no sh**

ip address

Används för att tilldela en IP-adress till porten. Vanligt att förkorta

syntax:

ip address [ip-adress subnätmask]

eller

ip ad [ip-adress subnätmask]

Exempel:

Router (config-if)#**ip address** 192.168.0.1 255.255.255.0

eller:

Router (config-if)#**ip ad** 192.168.0.1 255.255.255.0

ip route

Används för att stänga skapa en statisk route, det vill säga hur trafiken går från en routers nätverk till en annan routers nätverk. Tänk på att du oftast måste göra en statisk route åt båda håll så att datorerna i det andra nätverket kan svara också.

syntax:

ip route [nätverksadress subnätmask next_hop]

Exempel:

Router (config)#**ip route** 192.168.0.0 255.255.255.0 10.0.0.1

dhcp

Används för att skapa en dhcp-utdelning av ip-adresser. Den skapas i flera steg. Det första du anger är poolens namn. En pool är alla ip-adresser i ett subnät. Det andra du anger är nät-id för subnätet samt nätmask. Tredje saken är vilken gateway som nätverket skall ha. Avslutningsvis kan du (men är inget krav), lägga in DNS-serverns adress

syntax:

```
Router (config-if)#ip dhcp pool [poolnamn]
Router(dhcp-config)#network [nät-id] [subnätmask]
Router(dhcp-config)#default-router [gateway-ip]
Router(dhcp-config)#dns-server [dns ip]
```

Exempel:

```
Router (config-if)#ip dhcp pool polen
Router(dhcp-config)#network 192.168.0.0 255.255.255.0
Router(dhcp-config)#default-router 192.168.1.1
Router(dhcp-config)#dns-server 8.8.8.8
```

Precis som med ip-adresser, i ett VLAN, på en multilayerswitch, går det även att upprätta dhcp på en L3 switch. Kommandona är de samma, men du måste vara i VLAN-config-läge (se mer under rubriken VLAN)

dhcp-avancerat - Ranges

Om du använder dig av DHCP kan du vilja ta bort vissa adresser eller ranges

syntax:

```
Router(dhcp-config)# ip dhcp excluded-address [start-IP] [slut-IP]
```

Exempel:

```
Router(dhcp-config)# ip dhcp excluded-address 192.168.0.2 192.168.0.9
```

dhcp-avancerat - MAC-reservation

Om du använder dig av DHCP kan du vilja låsa en specifik mac-adress mot en viss ip

syntax:

```
Router(dhcp-config)# host [IP-adress] [subnetmask]
Router(dhcp-config)# client-identifier [MAC-adress]
```

Exempel:

```
Router(dhcp-config)# host 192.168.0.1 255.255.255.0  
Router(dhcp-config)# client-identifier 0100.1A2B.3C4D.5E6F
```

DYNAMISKA ROUTINPROTOKOLL

Rip v2

```
router rip  
v 2  
network [IP för anslutning]  
network [IP för LAN som delas ut]  
no auto-summary
```

Exempel:

```
Router(config)# router rip  
Router(config-router)# v 2  
Router(config-router)# network 10.0.0.1  
Router(config-router)# network 192.168.0.0  
Router(config-router)# no auto-summary
```

OSPF

```
router ospf [process-id]  
network [IP för anslutning] [Wildcardmask] area [nummer]  
network [IP för anslutning] [Wildcardmask] area [nummer]
```

Exempel:

```
Router(config)# router ospf 1  
Router(config-router)# network 192.168.0.0 0.0.0.255 area 0  
Router(config-router)# network 200.0.0.0 0.0.0.3 area 0
```

Du kan styra cost för länken. Detta görs exempelvis i anslutande switch

```
interface [Port]  
ip ospf cost [Värde]
```

Exempel:

```
Switch(config)# interface g0/0  
Switch(config-if)# ip ospf cost 1000
```

VLAN

Skapar ett virtuellt nätverk på en port, så kallade sub-portar. VLAN standard är att de döps till jämnt 10-tal

syntax:

```
int [portnamn.vlan-id]  
encapsulation dot1q [vlanid]
```

Exempel:

```
Router (config)#int g0/0.10  
Router(config-subif)# encap dot1q 10
```

Om du utför detta på en multilayerswitch behöver du inte skapa vare sig subport eller encapsulation. Där räcker det med att du skapar VLAN och tilldelar portar

syntax:

```
[vlan-id]  
name [vlan-name]
```

Exempel:

```
Switch(config)#vlan 10  
Switch(config-vlan)# name vlan10
```

Därefter måste du fortfarande tilldela portar (enligt nedan), ip adresser, dhcp m.m.

switchport

Används på switch för att hantera VLAN. Porten ställs antingen som access eller trunk. En trunk-port har hand om all trafik från samtliga VLAN och är ofta den port som leder tillbaka till routern. Access är den port som enheter ansluter till.

syntax:

`switchport mode [typ av anslutning]`

Exempel trunk:

`Switch(config-if)#switchport mode trunk`

Exempel access:

`Switch(config-if)#switchport mode access`

`Switch(config-if)#switchport access vlan 10`

Skall du göra flera portar samtidigt kan du använda kommandot `int range [startport-slutport]`

ip routing

Vill du låta din multilayerswitch hantera trafiken måste du aktivera intern routing i den
syntax:

`ip routing`

Exempel:

`Switch(config)#ip routing`

Kommandon i Windows

`ipconfig`

Kommando för att se dina nätverksanslutningar i datorn. Den visar alla olika, ethernet, wlan, bluetooth m.m. Alla med en enkel översikt över ip adress m.m.

`ipconfig /all`

Kommando för att se dina nätverksanslutningar i datorn men, **all information**. Den visar alla olika, ethernet, wlan, bluetooth m.m. Alla med en enkel översikt över ip adress m.m.

`ipconfig /release`

Kommando för att släppa den ip-adress du har blivit tilldelad

`ipconfig /renew`

Kommando för att begära ny ip-adress

`ipconfig /displaydns`

Visar innehållet i DNS-cache

`ipconfig /flushdns`

Töm innehållet i DNS-cache

`nslookup`

Ger information om domännamn och ip-adresser

Syntax:

```
nslookup google.com
```

`ping`

Skickar ett ICMP-paket till en specifik ip adress, i syfte att kontrollera anslutningen

Syntax:

```
ping 192.168.0.1
```

tracert

Visar hur nätverkstrafiken routras till en viss ip. Du kan även (om du är ansluten till en DNS) skriva in en URL (webbadress)

Syntax:

tracert 192.168.0.1

Kommandon i Linux (Debian)

`ip -brief addr`

Kommando för att snabbt se aktiva nätverksgränssnitt och deras IP-adresser (kompakt översikt).

`ip addr show`

Kommando för att se dina nätverksgränssnitt i detalj: IPv4/IPv6-adresser, prefix, flags m.m.

`ip link show`

Visar länkstatus, MAC-adresser och MTU per interface (lager 2-vy).

`ip route show`

Visar routningstabellen (standardgateway, statiska rutter m.m.).

Syntax:

```
ip route show
```

```
ip route get 192.168.0.1
```

`dhclient -r`

Kommando för att släppa (release) din DHCP-lease på ett interface.

Syntax:

```
sudo dhclient -r -v eth0
```

`dhclient`

Kommando för att begära en ny DHCP-lease (renew) på ett interface.

Syntax:

```
sudo dhclient -v eth0
```

`resolvectl status`

Visar DNS-resolvrar och cache-statistik om **systemd-resolved** används.

Syntax:

```
resolvectl status
```

```
resolvectl query google.com
```

`resolvectl flush-caches`

Tömmer DNS-cache (gäller systemd-resolved; annars töm respektive

cache-tjänst).

dig

Ger detaljerad information om domännamn och IP-adresser (ersätter ofta nslookup).

Syntax:

```
dig +short google.com
```

```
dig -x 192.168.0.1 +short
```

ping

Skickar ICMP-ekon till en adress för att testa anslutning och rundtid.

tracpath

Visar vägen (hop för hop) som paket tar till en destination, utan root-krav.

mtr

Kombinerar ping och traceroute i realtid (fördröjning/förluster per hop).

Kan behöva installeras (`sudo apt install mtr`).

Syntax:

```
mtr google.com
```

```
mtr -6 2607:f8b0:4003:c00::6a
```